

课程讲义

集成电路基础（1）
下册

（限课内使用）

吴行军

清华大学 微电子与纳电子学系

2020 年 12 月

目 录

第 7 章 高速 CMOS 逻辑电路设计	1
第一节 驱动大电容负载设计	1
7.1.1 反相器链的设计	1
7.1.2 驱动大电容负载的设计	6
第二节 组合电路中的性能优化	11
7.2.1 逻辑努力	11
7.2.2 组合逻辑的性能优化	14
第 8 章 CMOS 逻辑电路的高级技术	18
第一节 镜像电路	18
第二节 有比逻辑	20
8.2.1 伪 nMOS 电路	20
8.2.2 差分串联电压开关逻辑 (DCVSL)	22
第三节 动态 CMOS 设计	25
8.3.1 动态逻辑：基本原理	25
8.3.2 动态逻辑的速度和功耗	27
8.3.3 动态设计中的信号完整性问题	29
8.3.4 动态门级连	35
第 9 章 组合逻辑电路设计	42
第一节 组合逻辑电路简介	42
第二节 组合逻辑电路分析	43
第三节 组合逻辑电路设计步骤	45
第四节 数值比较器	49
第五节 译码器	50
第六节 编码器	54
第七节 多路选择器	56
第八节 加法器设计	62
9.8.1 二进制加法器	62
9.8.2 全加器：电路设计考虑	67
9.8.3 二进制加法器：逻辑设计考虑	72
第九节 十进制加法器	86

第十节 二进制减法器	88
9.10.1 补码	88
9.10.2 带符号的二进制数	90
9.10.3 二进制减法器	93
第 10 章 时序逻辑电路设计	96
第一节 时序电路基本概念	96
10.1.1 存储单元的分类	98
10.1.2 时序电路的时间参数	99
第二节 存储单元电路：静态锁存器和寄存器	100
10.2.1 双稳态原理	100
10.2.2 静态锁存器	102
10.2.3 主从边沿触发寄存器	108
10.2.4 其他触发器	115
第三节 存储单元电路：动态锁存器和寄存器	117
10.3.1 动态传输门边沿触发寄存器	118
10.3.2 C2MOS —— 一种对时钟重叠不敏感的方法	120
第四节 钟控时序电路分析	122
第五节 时序电路的设计过程	132
第六节 寄存器和计数器设计	139
10.6.1 寄存器设计	139
10.6.2 移位寄存器	140
10.6.3 行波计数器	146
10.6.4 同步计数器	150
10.6.5 其他计数器	155
第七节 流水线概念和同步时序原理	159
10.7.1 流水线	159
10.7.2 同步时序原理	160
10.7.3 时钟不确定性*	161
10.7.4 偏差和抖动的来源*	167

参考书:

1. 超大规模集成电路与系统导论 (Introduction to VLSI Circuits and Systems) . John P. Uyemura 著,

周润德译. 电子工业出版社, 2004 年 1 月

2. 数字集成电路—电路、系统与设计（第二版）(Digital Integrated Circuits—A Design Perspective, Second Edition). Jan M. Rabaey 等著, 周润德译. 2017 年 1 月
3. 数字设计与 Verilog 实现（第五版）(Digital Design: With an Introduction to Verilog HDL, Fifth Edition). M. Morris Mano, Michael D. Ciletti 著, 徐志军等译. 电子工业出版社, 2015 年 1 月
4. 超大规模集成电路设计（第四版）(CMOS VLSI Design: A Circuits and Systems Perspective, Fourth Edition). Neil H. E. Weste, David Money Harris 著, 周润德译. 电子工业出版社, 2012 年 7 月

第 7 章 高速 CMOS 逻辑电路设计

MOSFET 的沟道宽度 W 确定了它的宽长比 (W/L)，这是决定逻辑电路电气特性的关键参数。在 VLSI 中，是否达到系统要求的时序目标与逻辑电路的开关速度有着紧密的联系。如果电路不能满足时序要求，则可能需要调整逻辑。

本章研究高速系统设计及选择晶体管尺寸的技术，这些方法无论对单元库的建设还是对全定制设计都十分有用。

第一节 驱动大电容负载设计

高速设计的许多要点可以通过对反相器电路延时特性的研究来获得。这些分析为几种熟知的设计技术奠定了基础，它们可以推广用于任意逻辑门。

在上一章中我们分析了单个反相器尺寸对性能的影响。在该分析中，我们假设一个对称反相器，即它的 NMOS 和 PMOS 尺寸使上升和下降延时相同。并将反相器的负载电容分为本征和外部两个部分，即 $C_L = C_{int} + C_{ext}$ 。 C_{int} 代表反相器的自载即本征输出电容，它与 NMOS 和 PMOS 管的扩散电容以及栅漏覆盖（密勒）电容有关。 C_{ext} 是外部负载电容，它来自扇出和导线电容。一个尺寸系数为 S 的反相器（以一个最小尺寸的反相器为参考），其本征电容为 $C_{int} = SC_{iref}$ (C_{iref} 为参考反相器的本征电容)，其传播延时可以表示为：

$$t_p = t_{p0} (1 + C_{ext}/C_{int}) = t_{p0} (1 + C_{ext}/(SC_{iref})) \quad (7.1)$$

其中， t_{p0} 为反相器的本征延时，与门的尺寸无关，而只取决于工艺及反相器的版图，增加 S 可以使传播延时减小。

7.1.1 反相器链的设计

确定反相器链的尺寸

虽然加大反相器的尺寸可以减小它的延时，但这也加大了它的输入电容。如孤立地确定门的尺寸而不考虑它对前级门延时的影响，则纯粹是一种脱离实际的研究。由此一个比较相关的问题是当一个门处在实际环境中时如何确定它的最优尺寸。一个简单的反相器链则是最好的研究起点。

为了决定输入的负载效应，必须建立起反相器的输入栅电容 C_g 与本征输出电容 C_{int} 之间的关系。这两个电容均正比于门的尺寸。因此，下列关系成立而与门的尺寸无关：

$$C_{int} = \gamma C_g \quad (7.2)$$

式中 γ 是比例系数，它只与工艺有关，并且对于大多数的亚微米工艺接近于 1。重新写出公式（7.1），我们得到：

$$t_p = t_{p0} (1 + C_{ext}/(\gamma C_g)) = t_{p0} (1 + f/\gamma) \quad (7.3)$$

其中， $f = C_{ext}/C_g$ 。上式表明，反相器的延时只取决于它的外部负载电容与输入电容间的比值，这一比值称为等效扇出 f 。

考虑图 7.1 的反相器链电路。我们的目的是要使通过反相器链的延时最小，其中第一个反相器（通常为最小尺寸的门）的输入电容为 C_{g1} ，而反相器链末端为一个固定的负载电容 C_L 。

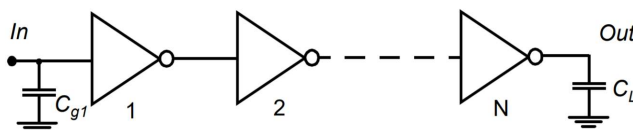


图 7.1 由 N 个反相器组成的具有固定输入和输出电容的反相器链

反相器链中第 j 级反相器的延时可表示为：

$$t_{p,j} = t_{p0} \left(1 + \frac{C_{g,j+1}}{\gamma C_{g,j}} \right) = t_{p0} \left(1 + \frac{f_j}{\gamma} \right) \quad (7.4)$$

因此，可以推导出反相器链的总延时为：

$$t_p = \sum_{j=1}^N t_{p,j} = t_{p0} \sum_{j=1}^N \left(1 + \frac{C_{g,j+1}}{\gamma C_{g,j}} \right), \text{ 其中 } C_{g,N+1} = C_L \quad (7.5)$$

这个方程含有 $N-1$ 个未知数，即 $C_{g,2}, C_{g,3}, \dots, C_{g,N}$ 。通过求 $N-1$ 次偏微分并令它们都等于 0，即 $\partial t_p / \partial C_{g,j} = 0$ ，可以求得最小延时。由此得到了一组约束条件：

$$C_{g,j+1} / C_{g,j} = C_{g,j} / C_{g,j-1} \quad \text{其中 } (j = 2 \dots N) \quad (7.6)$$

换言之，每一个反相器的最优尺寸是与它相邻的前后两个反相器尺寸的几何平均数：

$$C_{g,j} = \sqrt{C_{g,j-1} C_{g,j+1}} \quad (7.7)$$

这意味着每个反相器的尺寸都相对于它前面反相器的尺寸放大相同的倍数 f ，即每个反相器都具有相同的等效扇出 ($f_j = f$)，因而也就具有相同的延时。当 $C_{g,1}$ 和 C_L 给定时，我们可以推导出尺寸系数为：

$$f = \sqrt[N]{C_L / C_{g,1}} = \sqrt[N]{F} \quad (7.8)$$

以及通过该反相器链的最小延时：

$$t_p = N t_{p0} (1 + \sqrt[N]{F} / \gamma) \quad (7.9)$$

式中, F 代表该电路的总等效扇出, 它等于 C_L/C_{g1} 。注意 t_p 和 F 之间的关系是如何与反相器链的级数密切相关的。正如可以预见到的, 当只存在一级时, 这是一个线性关系, 加入第二级将使它变为平方根关系, 依次类推。现在明显的问题是对于给定的 F 值如何选定级数使延时最短。

反相器链的级数设计

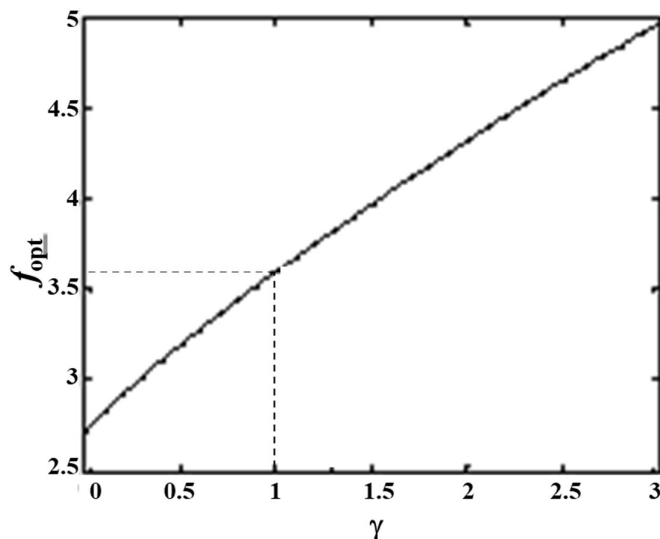
从对公式 (7.9) 的求值中可以看出对于给定的 $F (=f^N)$ 在选择级数时需要综合考虑。当级数 N 太大时, 公式的第一部分 (它代表了反相器级的本征延时) 将占主导地位。而如果级数太少, 则每一级的等效扇出变大, 使公式的第二部分占主导地位。通过求最小延时表达式对级数 N 的导数并令它为 0, 可以求得最优值。我们得到:

$$\gamma + \sqrt[N]{F} - \frac{\sqrt[N]{F} \ln F}{N} = 0$$

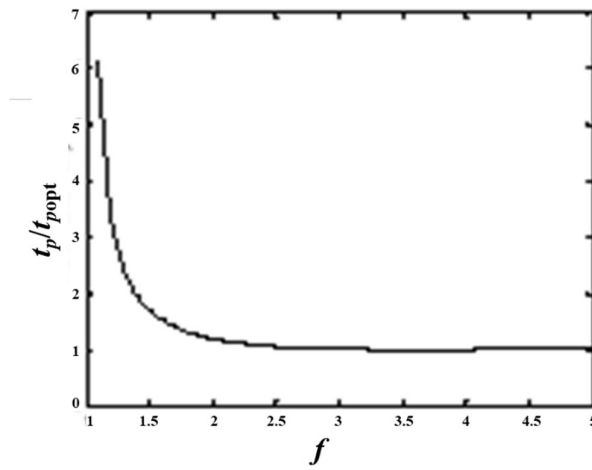
这相当于:

$$f = e^{(1 + \gamma/f)} \quad (7.10)$$

方程 (7.10) 只有一个收敛解, 即 $\gamma=0$ 时的解——此时忽略自载, 因此负载电容只由扇出构成。在这些简化条件下可以得到最优的级数为 $N=\ln(F)$, 且每一级的等效扇出为 $f = e = 2.71828$ 。这一优化的缓冲器设计以一种指数形式逐级放大各级尺寸, 并且因此称为指数锥形。当包括自载时, 方程 (7.10) 只能求数值解, 其结果画在图 7.2 (a) 中。对于 $\gamma=1$ 的典型情形, 最优的锥形系数将接近于 3.6。图 7.2 (b) 画出了 $\gamma=1$ 时反相器链的传播延时 (归一化) 与等效扇出的关系。选择扇出值大于最优值并不会过多地影响延时, 但能减少所要求的缓冲器级数和实现面积。一个通常的做法是选择最优的扇出为 4。反之, 采用过多的级数 (即 $f < f_{opt}$) 对延时会有明显的负面影响, 因而应当避免。



(a) 反相器链中最优的等效扇出 f 与自载系数 γ 的关系



(b) $\gamma=1$ 时归一化传播延时($t_p/(t_{popt})$)与等效扇出 f 的关系

图 7.2 优化反相器链的级数

例 7.1 假设需要驱动一个 $C_L=10\text{pF}$ 的负载电容。输入级的电容为 $C_1 = 20\text{fF}$ 。使延时最小所需要的级数 N 可计算为（假设 $\gamma=1$ ）:

$$N = \ln F / \ln f = \ln(10\text{pF}/20\text{fF}) / \ln(3.6) = \ln(500) / \ln(3.6) = 4.85$$

把 N 近似为一个整数值，取 $N=4$ 或 $N=5$ 。当 $N=4$ 时，实际的尺寸放大因子（等效扇出） $f = F^{1/4} = (500)^{1/4} = 4.73$ ，延时 $t_p = 4 \times t_{p0} \times (1 + 4.73) = 22.92t_{p0}$ 。当 $N=5$ 时，实际的 $f = F^{1/5} = (500)^{1/5} = 3.47$ ，延时 $t_p = 5 \times t_{p0} \times (1 + 3.47) = 22.35t_{p0}$ 。

作为对比，表 7.1 列出了 N 分别取值为 2, 3, 4, 5 和 6 时对应的延时及各级反相器的尺寸系数（相对于第 1 级反相器的尺寸）。

表 7.1 例 7.1 中采用不同的反相器级数时对应的设计结果

N	2	3	4	5	6
f	22.4	7.94	4.73	3.47	2.82
t_p/t_{p0}	46.8	26.81	22.92	22.35	22.90
反相器链中	s_2 : 22.4	s_2 : 7.94	s_2 : 4.73	s_2 : 3.47	s_2 : 2.82
各反相器尺		s_3 : 63.0	s_3 : 22.4	s_3 : 12.0	s_3 : 7.95
寸系数（相			s_4 : 106	s_4 : 41.8	s_4 : 22.4
对于第 1 级				s_5 : 145	s_5 : 63.2
反相器）					s_6 : 178

由表 7.1 可见，反相器的尺寸在接近输出级时增长很快。比较合理的级数是 $N=4$ 或 $N=5$ 。当 $N=6$ 时，虽然延时比较小，但级数较多，反相器总的尺寸太大，从而功耗也较大。实际设计中，当需要较小面积和功耗时，也可以选择 $N=3$ （延时比最小值略有增加）。

$N=2$ 时虽然面积最小，但延时太大，而且信号波形变化慢，会产生较大的短路功耗。

例 7.2 引入缓冲器级的影响

表 7.2 列出了无缓冲器的设计、两级反相器以及优化的反相器链对于不同的 F 值所对应的 $t_{p,opt}/t_{p0}$ 值 ($\gamma=1$)。我们注意到在驱动非常大的电容负载时，采用串联的反相器可以达到非常明显的加速。

表 7.2 不同驱动器结构的 $t_{p,opt}/t_{p0}$ 与 F 的关系

F	无缓冲器	两级反相器	反相器链
10	11	8.3	8.3
100	101	22	16.5
1000	1001	65	24.8
10000	10001	202	33.1

以上分析可以延伸为不仅包括反相器链，而且也包括含实际扇出的反相器网络。一个这样的例子显示在图 7.3 中。我们只需要调整 C_{ext} 的表达式以考虑附加的扇出系数。

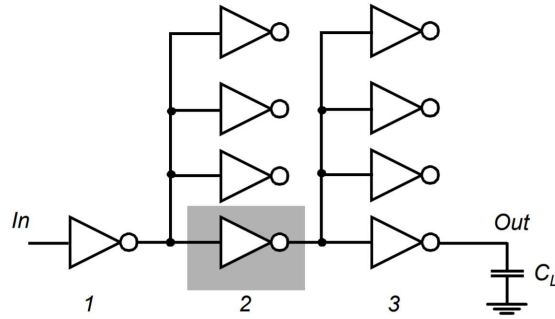


图 7.3 反相器网络。其中每个门的扇出都为 4 个门，把一个输入以树结构的形式分配给 16 个输出信号

为了确定图 7.3 电路中反相器的尺寸，使在节点 Out 和 In 之间的延时最小（设 $C_L=64C_{g,1}$ ）。首先确定使延时最小的各个器件之间的比。将式 (7.5) 分别对图中 3 级的输入电容求偏导数（注意第 1 级和第 2 级有附加的扇出），得到以下的关系：

$$\frac{4C_{g,2}}{C_{g,1}} = \frac{4C_{g,3}}{C_{g,2}} = \frac{C_L}{C_{g,3}}$$

同时有 $C_L=64C_{g,1}$ 。求解这一方程组，得到 3 级反相器的尺寸系数为： $s_1 = 1$ ； $s_2 = 2^{4/3} = 2.52$ ； $s_3 = 2^{8/3} = 6.35$ 。由此得到在输入和输出之间的最小延时为 $33.24t_{p0}$ （假设 $\gamma=1$ ）。如果直接按反相器链的确定反相器的尺寸而不考虑额外的扇出，将得到尺寸系数为 4 而不是 2.52。

7.1.2 驱动大电容负载的设计

互连电容值的增加，特别是全局连线电容值的增加，使得更需要能以足够快的速度对电容进行充（放）电的高效驱动电路。这一需要由于以下事实变得更为突出，即在复杂的设计中单个门常常需要驱动很大的扇出，因而具有很大的电容负载。片上大负载的典型例子有总线、时钟网络以及控制线。后者包括复位（*reset*）和置位（*set*）信号。这些信号控制着大量门的操作，所以扇出通常很大。其他大扇出的例子可以在存储器中遇到，那里有大量的存储单元连接到一小组控制线和数据线上。这些节点的电容很容易达到几个 pF 的范围。最坏情况发生在当信号送出芯片的时候。这时负载由封装导线、印刷电路板导线以及所连接的 IC 或部件的输入电容组成。片外负载可以大至 50pF，这比一个标准的片上负载要大数千倍。因此能以足够快的速度驱动这些节点成为最关键的设计问题之一。

有效驱动大电容负载的方法已在前面介绍，它们可以归纳为以下两个主要概念：

- 合适确定晶体管的尺寸有助于解决大负载的问题。
- 把驱动器划分成逐渐增大的缓冲器链有助于解决大扇出系数的问题。

为了清楚起见，从分析中得到的其他一些重要结论也在此重复提出：

- 在优化性能时，多级驱动器的延时应当平均分配到所有各级中。
- 如果可以自由选择级数，那么对于目前的半导体工艺，每级扇出（尺寸）系数大约为 4 时可以使延时最小。选择再稍大一些的扇出系数不会对性能有多大的影响，但会大大节省面积。

下面进一步仔细研究这一问题，我们将特别集中在非常大的电容的驱动问题上，如那些在片外遇到的电容，此外还将介绍一些特别的和非常有用的驱动器电路。

驱动片外电容—实例研究

正如在第 3 章中已说明的，当今集成电路复杂性的增加意味着需要大量增加输入输出引线（pin）。具有 1000 个以上引线的封装已成为必需品。这就抗噪声能力而言对压焊块的设计提出了难度很高的要求。许多压焊块同时切换，而每一个又都驱动一个大电容，从而造成很大的瞬态电流，并引起电源线和地线上的电压波动。这降低了噪声容限，影响了信号的完整性。

与此同时，工艺尺寸的缩小减小了片上的内部电容，但片外电容仍大致保持不变，一般为 10 至 20pF。其结果是，随着工艺尺寸的缩小，一个输出引线驱动器总的等效扇出系数 F 出现了净增长。在一个协调一致的系统设计方法中，我们希望片外的传播延时与片上的延时能以同样的方式缩小。这就使压焊块缓冲器的设计变得更加困难。压焊块驱动器设计所面临的挑战可以通过以下一个简单的实例研究来说明。

例 7.3 输出缓冲器设计

考虑一个片上最小尺寸的反相器需要驱动 20pF 的片外电容 C_L 的情形。在 0.25 μm CMOS 工艺中一个标准门的 C_i 约等于 2.5fF，这相当于 t_{p0} 近似为 30ps。总的等效扇出 F (C_L 与 C_i 之比) 等于 8000，这无疑要求多级缓冲器设计。根据公式 (7.9)，并假设 $\gamma=1$ ，我们得到接近最优的设计为 7 级，其尺寸放大系数 f 等于 3.6，总传播延时等于 0.97ns。对于 PMOS 与 NMOS 之比为 1.9（这是例 6.x 中标准工艺参数下推导得到的最优比率）和最小尺寸为 0.25 μm 的情形，可以推导出前后各级反相器中 NMOS 和 PMOS 晶体管的宽度，如表 7.3 所示。

表 7.3 最优尺寸串连缓冲器的晶体管尺寸

级数	1	2	3	4	5	6	7
W_n (μm)	0.375	1.35	4.86	17.5	63	226.8	816.5
W_p (μm)	0.71	2.56	9.2	33.1	119.2	429.3	1545.5

显然这一结果需要一些栅宽大至 1.5mm 的超大晶体管！这一缓冲器的总体尺寸是最小尺寸反相器的数千倍，这是完全无法接受的，因为一个复杂的芯片需要许多这样的驱动器。这一结论清楚地说明要得到最优延时需要付出巨大的代价。

用性能换取面积和能量的减少

幸运的是，在大多数情况下并不需要达到最优的缓冲器延时。片外通信常常能以片上时钟速度的几分之一来进行。放宽延时要求仍然可以使片外时钟速度超过 100MHz，但却大大降低了对缓冲的要求。这一重新定义的缓冲器设计问题如下：

给出一个最大传播延迟时间 $t_{p,\max}$ ，确定使总面积最小的缓冲器的级数 N 和所需要的尺寸放大系数 f 。这相当于找出能使 t_p 尽可能接近 $t_{p,\max}$ 的解决办法。

现在最优化的问题可以重新叙述成在满足公式 (7.11) 的约束条件下求 N 的最小整数值的问题：

$$\frac{t_{p,\max}}{t_{p0}} \geq \ln(F) \frac{1+f/\gamma}{\ln(f)} = N \times \left(1 + \frac{F^{1/N}}{\gamma} \right) \quad (7.11)$$

这一复杂的优化问题可以通过运用一个小计算机程序或利用数学软件工具（如 MATLAB）来解决。图 7.4 画出了对于某些 F 值 t_p/t_{p0} 与 N 的关系曲线。对于一个给定的总等效扇出 F 和延时的最大值 ($t_{p,\max}/t_{p0}$) 查找合适的曲线可得到缓冲器的最少数目 N 。

如以晶体管的总宽度为衡量标准，我们可以设计具有较大尺寸系数的电路以节省面积。假设最小反相器的面积等于 $A_{\min}$ ，晶体管的尺寸放大系数为 f 时其面积也增加同样的倍数，由此可以推导出驱动器的面积与 f 的关系：

$$A_{driver} = (1 + f + f^2 + \dots + f^{N-1})A_{min} = \frac{f^N - 1}{f - 1}A_{min} = \frac{F - 1}{f - 1}A_{min} \quad (7.12)$$

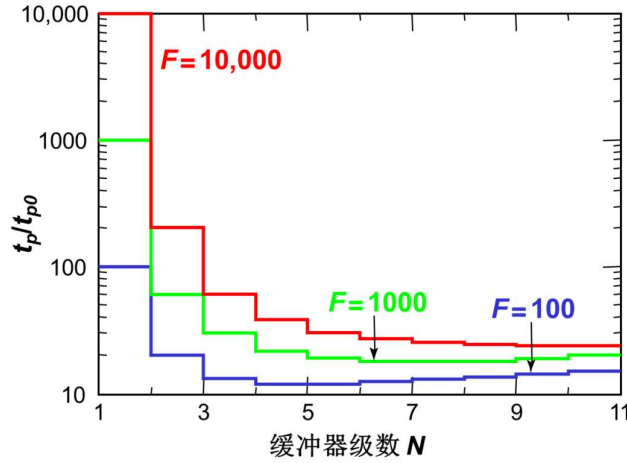


图 7.4 总扇出 F 取不同值时 t_p/t_{p0} 与缓冲器级数的关系

简言之，驱动器的面积近似地与锥形（逐级增大）的尺寸系数 f 成反比。选择较大的 f 值有助于显著降低面积成本。

还有一点值得提及，即以指数形式增大的缓冲器对能量消耗的影响。虽然只需消耗等于 $C_L V_{DD}^2$ 的能量就能切换负载电容，但驱动器本身还需要消耗能量以切换它的内部电容（忽略短路能耗）。驱动器的额外能量消耗可近似表示成下列表达式：

$$E_{driver} = (1 + f + f^2 + \dots + f^{N-1})(1 + \gamma)C_i V_{DD}^2 = \frac{F - 1}{f - 1}(1 + \gamma)C_i V_{DD}^2 \approx \frac{2C_L}{f - 1}V_{DD}^2 \quad (7.13)$$

这意味着快速驱动大电容（即采用 3.6 的最优锥形系数）较未缓冲的情况多消耗 80% 的能量。对于大的负载电容来说，这是一个很大的开销。降低这一最优的锥形系数有助于减少这一额外的消耗。

例 7.4 输出驱动器设计——再次考虑

把这些结果应用到压焊块驱动器问题中并设定 $t_{p,max} = 2ns$ ，则有 $N = 3$ ， $f = 20$ ， $t_p = 1.89ns$ 。所要求的晶体管尺寸总结在表 7.4 中。

表 7.4 重新设计的串连缓冲器的晶体管尺寸

级数	1	2	3
W_n (μm)	0.375	7.5	150
W_p (μm)	0.71	14.4	284

这一设计的总面积大约比最优设计小 7.5 倍而它的速度降低不到 2 倍。每次切换翻转因缓冲器本征电容引起的额外能量消耗减少到了几乎可以忽略不计的程度。缓冲器和负载的总能耗降低了 24%，对于给定的负载电容大小，这是一个相当大的数量。显然，把一个电路设计成合适的速度而不是最高的速度是值得的！

设计考虑——宽晶体管的实现

即使是这一重新设计的缓冲器也需要较宽的晶体管，在设计这些器件时必须十分小心，因为大的 W 意味着非常长的栅连线。较长的多晶线通常有较高的电阻，从而降低了开关性能。这个问题可以用以下办法来解决：一个宽的晶体管可以通过并联许多较小的晶体管（也称为“指状晶体管”）来构成（见图 7.5）。

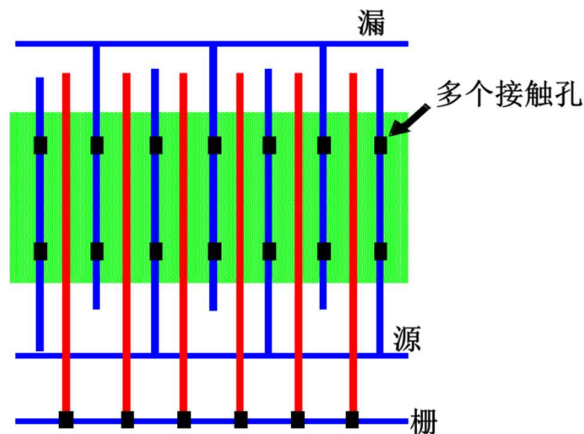


图 7.5 将许多晶体管（或称“指状”晶体管）并行排列来实现非常宽的晶体管

采用低电阻的金属线旁路连接较短的多晶部分可以降低栅的电阻。图 7.6 是采用这些技术设计的一个压焊块驱动器的例子。当试图决定一个很宽的晶体管是否应当划分成多个较小的器件时，采用下列经验法则：多晶栅的 RC 延时应当总是大大小于整个数字门的开关延时。

一些有意义的驱动器电路

至此，我们讨论的大多数驱动器电路都是简单的反相器。但有时还需要它们有其他一些功能。三态缓冲器即是不同类型驱动器的一个例子。在大多数数字系统中总线非常重要——总线是一束共用的导线，它们连接一组发送和接收器件，如处理器、存储器、磁盘以及输入输出器件。当一个器件通过总线发送信息时，所有其他的发送器件都应当与总线断开。这可以通过把这些器件的输出缓冲器置于高阻态来实现，它有效地使逻辑门与输出线断开。这样的缓冲器具有三种可能的状态 0、1 和 Z（高阻），因而称为三态缓冲器。

在 CMOS 中实现一个三态反相器很容易。同时关断 NMOS 和 PMOS 管时产生一个浮空的输出节点，它断开了与反相器输入的联系。图 7.7 显示了一个三态缓冲器两种可能的实现。第一个较简单，第二个则较适合于驱动大的电容。应当避免在输出级上堆叠

晶体管因为这会消耗大量的面积。

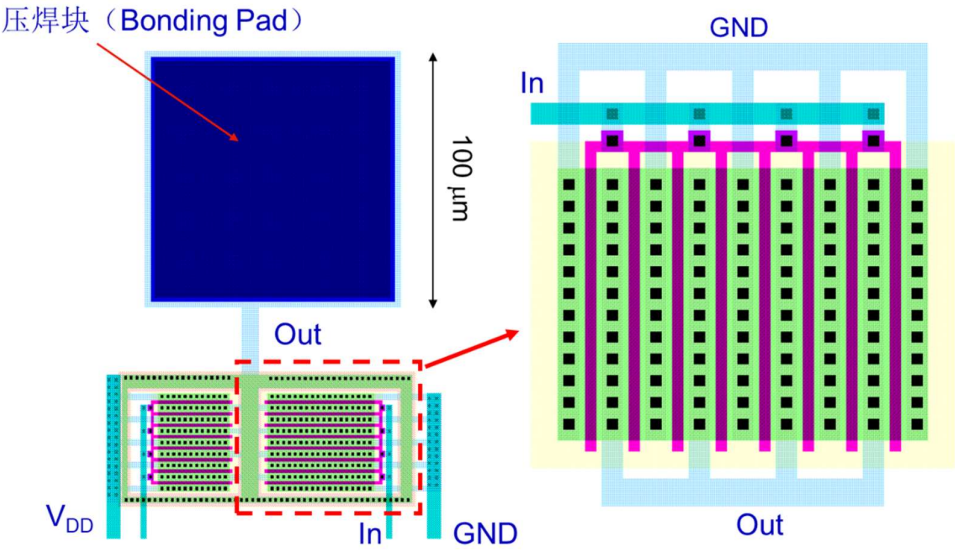


图 7.6 压焊块驱动器最后一级的版图。右图是一连接在 GND 和 Out 之间的 NMOS 管的放大图

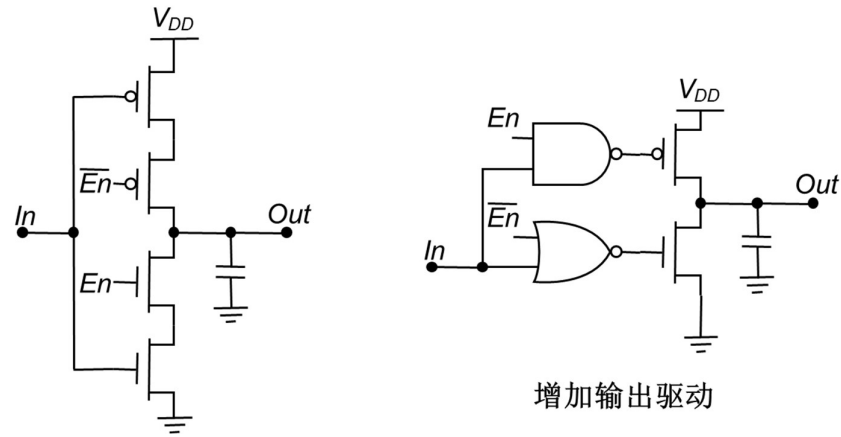


图 7.7 一个三态缓冲器的两种可能的实现方式。En=1 使该缓冲器工作

第二节 组合电路中的性能优化

7.2.1 逻辑努力

前面已经说明,使一个孤立的门的传播延时最小是一个纯粹脱离实际的努力,器件的尺寸应当在其具体环境中确定。自数字 MOS/VLSI 电路出现以来,确定逻辑链的每级尺寸一直是一种主要的技术。它用来作为设计快速逻辑链的指南,并且提供许多定性的特点可用于常用的电路。在上一节中已经建立了一种针对反相器这样做的方法。我们也已经知道,对于一个驱动负载 C_L 的反相器链它的最优扇出为 $(C_L/C_{in})^{1/N}$, 这里 N 是反相器链的级数,而 C_{in} 是该链中第一个门的输入电容。如果有机会选择级数我们已发现希望使每一级的扇出保持在 4 左右。

这一结果能否延伸到确定任何组合路径的尺寸以达到最小的延时?通过把前面的方法延伸来解决复杂逻辑电路,我们发现这确实是可能的。Sutherland 等重新形成了包含在对逐级变化分析中的概念并且把它用来开发称为“逻辑努力”的一般化技术。逻辑努力描述逻辑门的特性以及它们如何在逻辑链中相互作用,并且提供使延时最小的技术。它使这个理论除应用于标准的逻辑门如 NAND 和 NOR 之外,还可扩展到包括复杂的逻辑门电路。在本节中,将考察这一方法的基础,学习如何用它来设计高速逻辑链。

为此我们把第 1 节中介绍过的反相器基本延时公式 (7.3)

$$t_p = t_{p0} (1 + C_{ext}/(\gamma C_g)) = t_{p0} (1 + f/\gamma)$$

修改为:

$$t_p = t_{p0} (p + gf/\gamma) \quad (7.14)$$

式中, t_{p0} 仍代表反相器的本征延时, f 为等效扇出,它定义为该门的外部负载和输入电容之间的比。在这里 f 又称为电气努力 (*electrical effort*), 而 p 代表该复合门和简单反相器的本征 (即无负载) 延时的比,它与门的拓扑结构以及版图样式有关。多输入门比较复杂的结构会使它的本征延时比反相器的大。表 7.5 列出了一些标准门的 p 值,这里假设具有简单的版图样式并忽略如内部节点电容等二次效应。

表 7.5 不同逻辑类型本征延时的估计, 假设具有简单的版图样式及固定的 PMOS-NMOS 比

门的类型	p
反相器	1
n 输入的 NAND	n
n 输入的 NOR	n
n 路多路开关	$2n$
XOR, XNOR	$n2^{n-1}$

系数 g 称为逻辑努力 (*logical effort*)，它代表了这样一个事实，即对于一给定负载，复合门必须比反相器更“努力”工作才能得到类似的响应。换言之，一个逻辑门的逻辑努力说明，当假定这个逻辑门的每一个输入具有与一个反相器相同的输入电容时，在产生输出电流方面它比这个反相器差了多少。或相当于说，逻辑努力表示一个门与一个反相器提供相同的输出电流时它所表现出的输入电容比反相器大多少。逻辑努力是一个很有用的参数，因为它只与电路的拓扑结构有关。

基本 CMOS 门的逻辑努力

分析逻辑努力时以一个反相器作为参照门，如图 7.8 所示为一个用最小尺寸设计的对称反相器参照电路，晶体管旁边列出了宽长比的相对值 (1 和 r)。这里假设为了得到对称的响应，反相器中器件的宽长比关系为：

$$\left(\frac{W}{L}\right)_p = r \left(\frac{W}{L}\right)_n$$

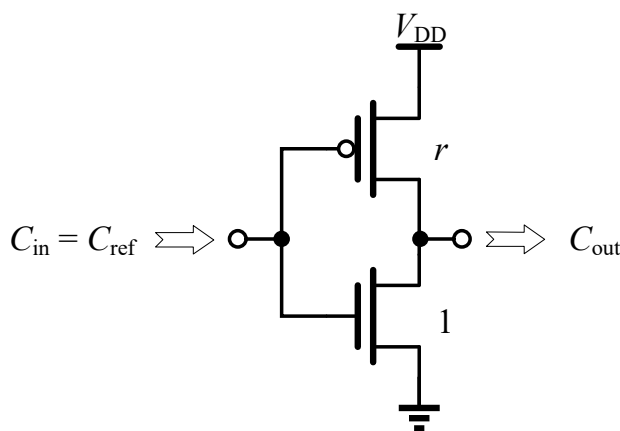


图 7.8 用于计算逻辑努力的参照反相器

所有逻辑门逻辑努力的计算都参照这个参照反相器，其输入电容为 C_{ref} ，晶体管电阻为 R_{ref} 。对于逻辑门来说，最简单的设计是保持对称的设计，也就是 $R_n = R_p = R_{ref}$ 。这要求我们调整串联晶体管的尺寸。

图 7.9 (a) 显示一个对称的 NAND2 门。PMOS 的尺寸仍为 r ，因为最坏情形下从输出至电源的路径与反相器是相同的。然而 NMOS 必需是反相器值的 2 倍，因为它们串联在一起的；它们的相对值因此表示为 2。对每个输入，输入电容于是为：

$$C_{in} = C_{ref} \frac{2+r}{1+r}$$

所以 NAND2 门的逻辑努力是：

$$g_{\text{NAND2}} = \frac{C_{\text{in}}}{C_{\text{ref}}} = \frac{2+r}{1+r} \quad (7.15)$$

图 7.9 (b) 的 NOR2 电路可用相同方式分析。并联 NMOS 的相对尺寸与反相器一样为 1，而选择 PMOS 的尺寸为 $2r$ ，使 R_p 与 R_{ref} 相同。因此输入电容为：

$$C_{\text{in}} = C_{\text{ref}} \frac{1+2r}{1+r}$$

所以这个门的逻辑努力是

$$g_{\text{NOR2}} = \frac{C_{\text{in}}}{C_{\text{ref}}} = \frac{1+2r}{1+r} \quad (7.16)$$

因此， g 的值与比率 r 有关。

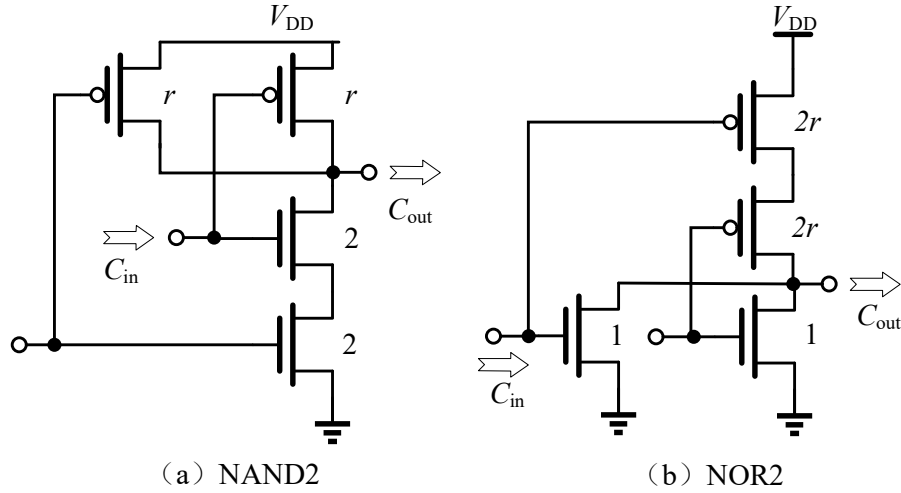


图 7.9 对称 NAND 和 NOR 门

这些结果可以一般化到扇入更多的门。一个 n 个输入的 NAND 门具有尺寸为 r 的 n 个 PMOS 并联及尺寸为 n 的 n 个 NMOS 串联。在输入端看到的电容为：

$$C_{\text{in}} = C_{\text{ref}} \frac{n+r}{1+r}$$

所以逻辑努力为

$$g_{\text{NAND}} = \frac{n+r}{1+r} \quad (7.17)$$

一个 n 个输入 NOR 门的逻辑努力为

$$g_{\text{NOR}} = \frac{1+nr}{1+r} \quad (7.18)$$

它可以用相同的方法验证。很容易看到任何基本的 CMOS 门都可以用逻辑努力 g 来描

述它的特性。表 7.6 列出了一些常用逻辑门的逻辑努力，假设参考反相器 PMOS-NMOS 的尺寸比为 $r=2$ 。

表 7.6 常用逻辑门的逻辑努力，假设 PMOS-NMOS 的尺寸比为 2

门的类型	输入端的数目			
	1	2	3	n
反相器	1			
NAND		4/3	5/3	$(n+2)/3$
NOR		5/3	7/3	$(2n+1)/3$
多路开关		2	2	2
XOR		4	12	

公式 (7.13) 中表示的一个逻辑门的延时模型是一个简单的线性关系。图 7.10 为这一关系的图示：图中画出了一个反相器及一个两输入 NAND 门的延时与扇出的关系。直线的斜率就是该门的逻辑努力，它与纵轴的交点就是本征延时。该图表明我们可以通过调整等效扇出（通过调整晶体管的尺寸）或通过选择具有不同逻辑努力的逻辑门来调整延时。同时注意到扇出和逻辑努力是以类似的方式来影响延时的。我们称这两者的积 $h = fg$ 为门努力（*gate effort*）。

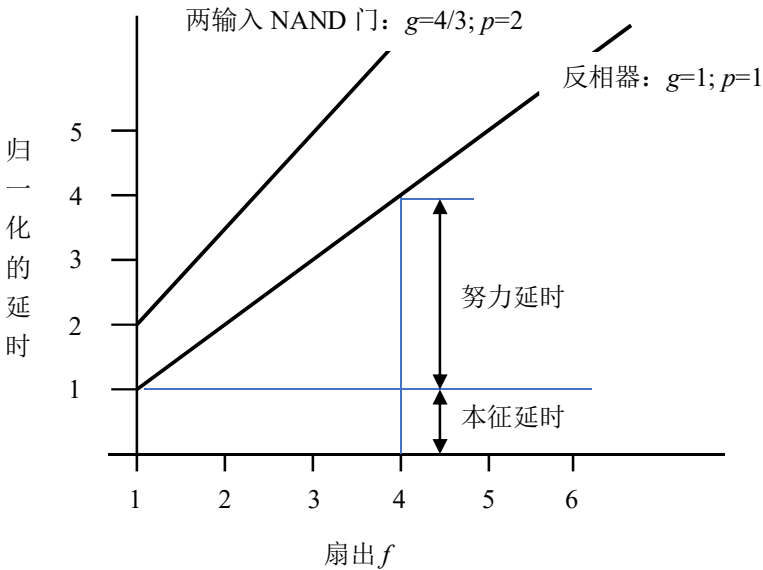


图 7.10 反相器及两输入 NAND 门的延时与扇出的关系

7.2.2 组合逻辑的性能优化

现在一条通过组合逻辑块的路径的总延时可以表示成：

$$t_p = \sum_{j=1}^N t_{p,j} = t_{p0} \sum_{j=1}^N \left(p_j + \frac{f_j g_j}{\gamma} \right) \quad (7.19)$$

我们用上一节中对反相器所采用的类似步骤来决定这条路径的最小延时。求 $N-1$ 个偏导数并令它们为零，可以发现每一级应当具有相同的门努力：

$$f_1 g_1 = f_2 g_2 = \dots = f_N g_N \quad (7.20)$$

沿电路中一条路径的总逻辑努力可以通过把这条路径上所有门的逻辑努力相乘求得，由此得到了路径逻辑努力 (*path logical effort*) G ：

$$G = \prod_{i=1}^N g_i \quad (7.21)$$

我们也可以定义路径的有效扇出 (或电气努力) F ，建立路径中最后一级负载电容与第一级输入电容之间的关系：

$$F = \frac{C_L}{g_1} \quad (7.22)$$

为了把 F 和各个门的等效扇出联系起来，必须引入另一个系数来考虑电路内部的逻辑扇出。当一个节点的输出上有扇出时，总驱动电流中的一部分沿我们正在分析的路径流动，而有些则离开这条路径。我们把给定路径上一个逻辑门的分支努力 (*branching effort*) b 定义为：

$$b = \frac{C_{on-path} + C_{off-path}}{C_{on-path}} \quad (7.23)$$

式中， $C_{on-path}$ 是该门沿我们正在分析的路径上的负载电容，而 $C_{off-path}$ 是离开这条路径的连线上电容。注意，如果路径没有分支 (如在逻辑门链中的情形)，则分支努力为 1。路径分支努力 (*path branching effort*) 定义为该路径上每一级分支努力的积，即

$$B = \prod_{i=1}^N b_i \quad (7.24)$$

于是路径电气努力现在就可以与各级的电气努力和分支努力联系起来：

$$F = \prod_{i=1}^N \frac{f_i}{b_i} = \frac{\prod f_i}{B} \quad (7.25)$$

最后，可以确定总路径努力 H 。由公式 (7.25) 可以写出：

$$H = \prod_{i=1}^N h_i = \prod_{i=1}^N g_i f_i = GFB \quad (7.26)$$

由此时起，分析过程可以沿与反相器链相同的路线进行。使路径延时最小的门努力为：

$$h = \sqrt[N]{H} \quad (7.27)$$

而通过该路径的最小延时为：

$$D = t_{p0} \left(\sum_{j=1}^N p_j + \frac{N(\sqrt[N]{H})}{\gamma} \right) \quad (7.28)$$

注意，路径本征延时与路径中逻辑门的类型有关，而与它们的尺寸无关。于是在逻辑链中各个门的尺寸系数 s_i 可以通过由前至后（或由后至前）计算求得。我们假定一个单位尺寸的门具有与一个最小尺寸反相器相同的驱动能力。根据逻辑努力的定义，这意味着它的输入电容为参照反相器输入电容 C_{ref} 的 g 倍。若逻辑链中第一个门的尺寸系数为 s_1 ，则该链的输入电容 C_{g1} 等于 $g_1 s_1 C_{\text{ref}}$ 。如果包括分支努力，我们得知第二个门的输入电容是这个值的 (f_1/b_1) 倍，即：

$$g_2 s_2 C_{\text{ref}} = \left(\frac{f_1}{b_1} \right) g_1 s_1 C_{\text{ref}} \quad (7.29)$$

对于该链中的第 i 个门，可以得到：

$$s_i = \left(\frac{g_1 s_1}{g_i} \right) \prod_{j=1}^{i-1} \left(\frac{f_j}{b_j} \right) \quad (7.30)$$

例 7.5 确定组合逻辑延时最小时的尺寸

考虑图 7.11 的逻辑电路，它可以代表一个较复杂逻辑块中的关键路径。这个电路输出端的负载是一个电容，它为第一级最小尺寸反相器输入电容的 5 倍。因此该路径的等效扇出为 $F = C_L/C_{g1} = 5$ 。参照表 7.6 中的有关项，我们得到路径逻辑努力如下：

$$G = 1 \times \frac{5}{3} \times \frac{5}{3} \times 1 = \frac{25}{9}$$

由于这里没有分支，所以 $B=1$ 。因此， $H=GFB=125/9$ ，于是最优的每个门的努力 h 为 $\sqrt[4]{H} = 1.93$ 。根据门的类型，我们得到如下的扇出系数： $f_1=1.93$ ； $f_2=1.93 \times (3/5)=1.16$ ； $f_3=1.16$ ； $f_4=1.93$ 。注意两个反相器比更复杂的其余两个门分配了较大的值，这是为了使它们能更好地驱动负载。

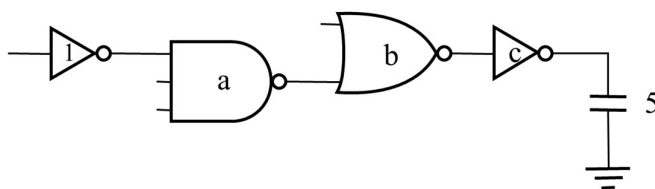


图 7.11 组合电路中的关键路径

最后，我们利用公式 (7.30) 计算出门的尺寸（相对于同一类型中最小尺寸的门）。于是得到如下值： $a = f_1 g_1 / g_2 = 1.16$ ； $b = f_1 f_2 g_1 / g_3 = 1.34$ ；及 $c = f_1 f_2 f_3 g_1 / g_4 = 2.60$ 。

这些计算不必非常精确。正如在第 6 章中讨论过的，使这个门的尺寸再放大或缩小 1.5 倍得到的电路延时仍然在最小延时的 5% 以内。因此利用这一技术进行手工“草算”是很有效的。

例 7.6 重新考虑图 7.3 的反相器网络，用分支努力的方法确定每级反相器电路的尺寸。

考虑分支的影响： $b_1=4$ ， $b_2=4$ ， $b_3=1$ ，所以 $B=16$ 。路径的等效扇出 $F=64$ 。路径逻辑努力 $G=1$ 。于是总的路径努力 $H=GFB=1024$ 。使延时最小的门的努力为 $h = H^{1/3} = 10.079$ 。 $C_{in3} = 64C_{g,1}/10.079 = 6.35C_{g,1}$ ， $C_{in2} = 4C_{in3}/10.079 = 2.52C_{g,1}$ 。由此得到 $s_1 = 1$ ， $s_2 = 2.52$ ， $s_3 = 6.35$ 。

第 8 章 CMOS 逻辑电路的高级技术

各种各样的 CMOS 电路设计风格已为大家熟知，它们在高速 VLSI 电路设计中非常有用。所有这些都是基于简单的逻辑门，但却以不同的方式工作。大多数高级技术开发出来用于克服在实际应用中出现的一个或多个问题。有些问题是非常一般的，而另外一些则只用于特殊的情形。本章将有选择地解释一些运用在 VLSI 中的现代 CMOS 电路技术，将为后面的应用打下基础。

第一节 镜像电路

镜像电路基于串-并联逻辑门，但通常较快并具有较为一致的版图。一个镜像的基本概念可以从图 8.1 的 XOR 真值表中看出。输出为 0 意味着一个 NMOS 链导通接地，而输出为 1 意味着一个 PMOS 从电源得到了电流。这一观察的重要一点是产生 0 和 1 的输入组合数目相同。

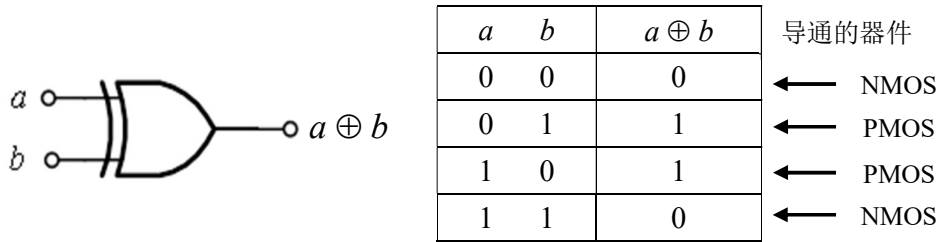


图 8.1 XOR 功能表

一个镜像电路对 NMOS 和 PMOS 采用相同的拓扑连接晶体管。这个概念应用于 XOR 功能，产生图 8.2 的电路。该图显示每个分支的输入组合。“镜像”效应可以通过把一面镜子沿输出线面向上或面向下放置来理解。在镜子中看到的将是该电路的另一边。PMOS 的尺寸比 NMOS 大以补偿它较小的工艺互导 (k') 值。

镜像电路的优点是有比较对称的版图以及较短的上升和下降时间。后一个优点可以通过图 8.3 的 RC 开关模型来理解。在输出和电源线或地线之间的每条路径由两个电阻和一个 MOS 间的寄生电容构成。它的 Elmore 时间常数为：

$$\tau_x = C_{\text{out}} (2R_x) + C_x R_x \quad (8.1)$$

式中下标 x 或者是 n 或者是 p 取决于所指的路径。若把输出电压近似为指数变化则得到如下的上升和下降时间的表达式：

$$t_r \approx 2.2 \tau_p$$

$$t_f \approx 2.2 \tau_n$$

虽然这个式子与 AOI 电路的相同，但上升时间较小因为寄生电容 C_p 较小。这是由于镜像电路只有两个 PMOS 构成 C_p ，而 AOI 电路在该节点上有 4 个晶体管。

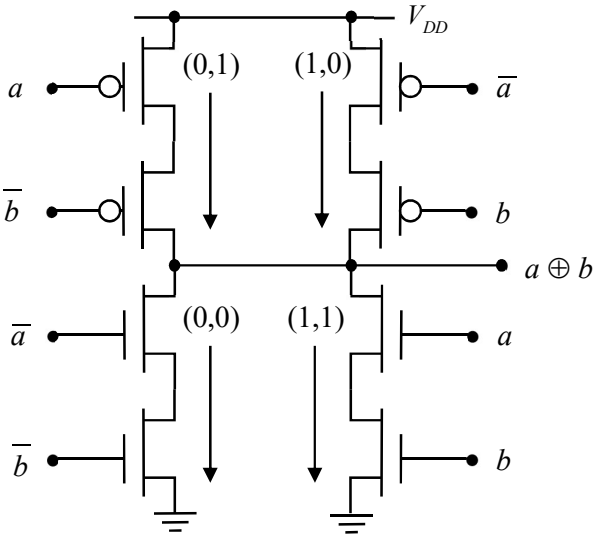


图 8.2 XOR 镜像电路

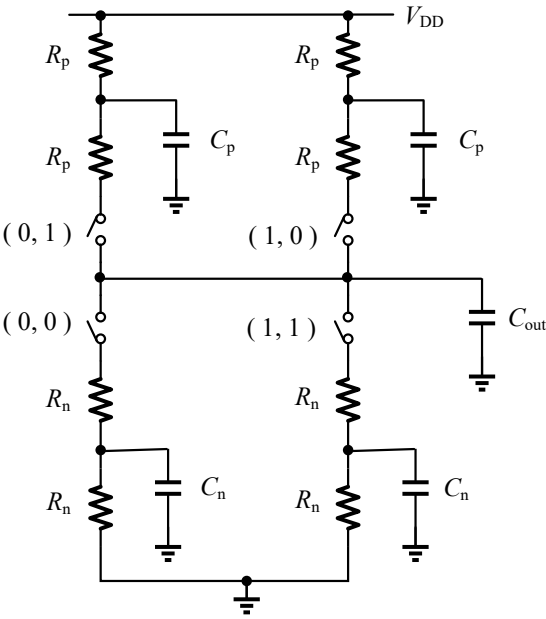


图 8.3 计算过渡过程的开关模型

这一概念可以很容易用来构成图 8.4 的 XNOR 电路。它具有与 XOR 相同的基本结构，只需要把输入 a 和 $\bar{a}$ 互换一下。其他镜像电路将在后面章节讨论特定应用例如加法器电路时再介绍。

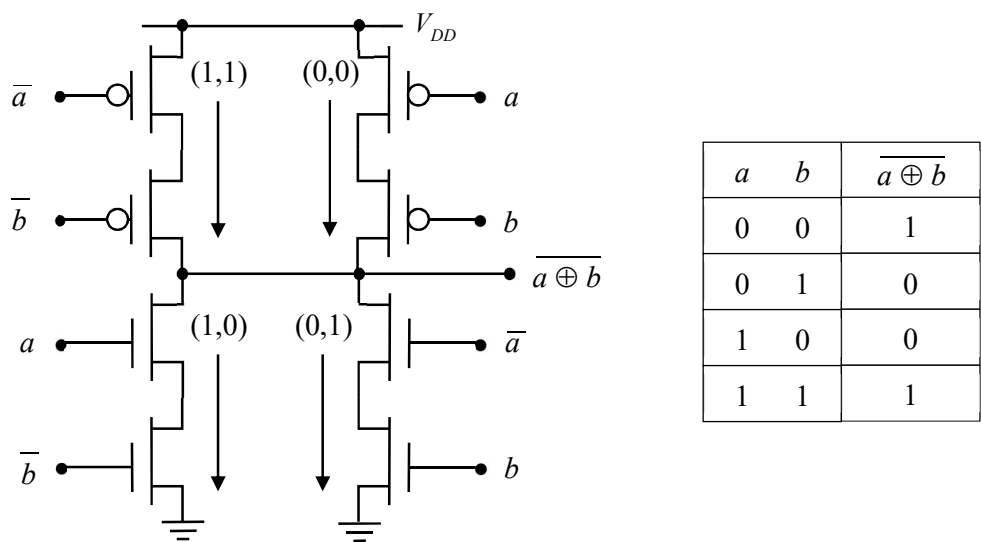


图 8.4 XNOR 镜像电路

第二节 有比逻辑

8.2.1 伪 nMOS 电路

有比逻辑试图减少实现一个给定逻辑功能所需要的晶体管数目,但它经常以降低稳定性和付出额外功耗为代价。在互补 CMOS 中, PUN 的目的是当 PDN 关断时在 V_{DD} 和输出之间提供一条有条件的通路。在有比逻辑中, 整个 PUN 被一个无条件的负载器件所替代, 它上拉输出以得到一个高电平输出, 如图 8.5 (a) 所示。这样的门不是采用有源的下拉和上拉两路的组合, 而是由一个实现逻辑功能的 NMOS 下拉网络和一个简单的负载器件组成。图 8.5 (b) 是有比逻辑的一个例子, 它采用一个栅极接地的 PMOS 负载, 这样的门称为伪 nMOS 门 (有时也称为准 nMOS)。

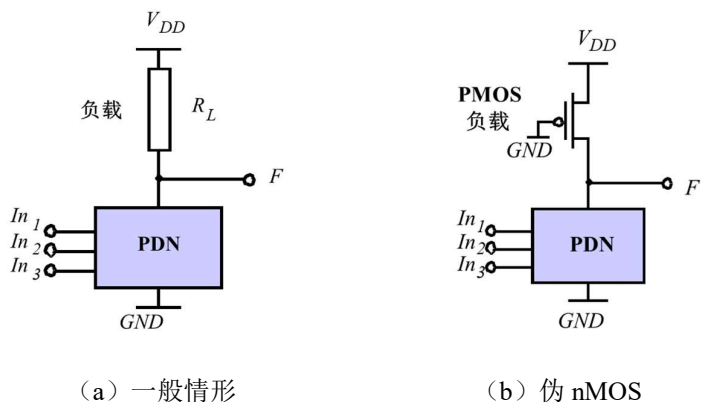


图 8.5 有比逻辑门

伪 nMOS 门的显著优点是减少了晶体管的数目（由互补 CMOS 中的 $2N$ 减少为 $N+1$ ）。该门额定输出高电压（ V_{OH} ）为 V_{DD} ，因为当输出拉高时（假设 V_{OL} 低于 V_{Tn} ）下拉器件关断。但额定输出低电压不是 $0V$ ，因为在 PDN 器件和栅极接地的 PMOS 负载器件之间存在通路。这降低了噪声容限，但更重要的是引起了静态功耗。负载器件相对于下拉器件的尺寸可以用来调整诸如噪声容限、传播延时和功耗等参数。由于输出端的电压摆幅及门的总体功能取决于 NMOS 和 PMOS 的尺寸比，所以该电路称为有比电路。这不同于像互补 CMOS 这样的无比逻辑类型，后者的高低电平与晶体管的尺寸无关。

计算伪 nMOS DC 传输特性的方法类似于计算相应互补 CMOS 的方法。为了求得 V_{OL} 的值，可使在 $V_{in}=V_{DD}$ 时通过驱动器和负载器件的电流相等。此时可以合理地假设 NMOS 器件处于线性工作模式（因为理想上，输出应当接近 $0V$ ），而 PMOS 负载处于饱和状态：

$$k_n \left((V_{DD} - V_{Tn}) V_{OL} - \frac{V_{OL}^2}{2} \right) + k_p \left((-V_{DD} - V_{Tp}) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right) = 0 \quad (8.2)$$

假设 V_{OL} 相对于栅驱动电压 ($V_{DD}-V_T$) 很小，而 V_{Tn} 与 V_{Tp} 在数值上相等，因此 V_{OL} 可近似为：

$$V_{OL} \approx \frac{k_p (V_{DD} + V_{Tp}) \cdot V_{DSATp}}{k_n (V_{DD} - V_{Tn})} = \frac{\mu_p \cdot W_p}{\mu_n \cdot W_n} \cdot V_{DSATp} \quad (8.3)$$

为了使 V_{OL} 尽可能地小，PMOS 器件的尺寸应当明显小于 NMOS 下拉器件的尺寸。遗憾的是，这会对充电输出节点的传播延时产生负面影响，因为它限制了 PMOS 器件能够提供的电流。

伪 nMOS 门的一个主要缺点是当输出为低时，通过存在于 V_{DD} 和 GND 之间的直接电流通路会引起静态功耗。低电平输出模式时的静态功耗很容易推导：

$$P_{low} = V_{DD} \cdot I_{low} \approx V_{DD} \cdot \left| k_p \left((-V_{DD} - V_{Tp}) \cdot V_{DSATp} - \frac{V_{DSATp}^2}{2} \right) \right| \quad (8.4)$$

例 8.1 伪 nMOS 反相器

考虑一个简单的伪 nMOS 反相器（此时图 8.5 中的 PDN 网络简化为单个晶体管），NMOS 的尺寸为 $0.5\mu m/0.25\mu m$ 。在本例中我们研究缩小 PMOS 器件尺寸的效果以说明其对各种参数的影响。接地 PMOS 的 $W-L$ 比值从 4、2、1、0.5 变化到 0.25。 $W-L$ 比小于 1 的器件通过使其长度大于宽度来实现。不同尺寸的电压传输曲线见图 8.6。

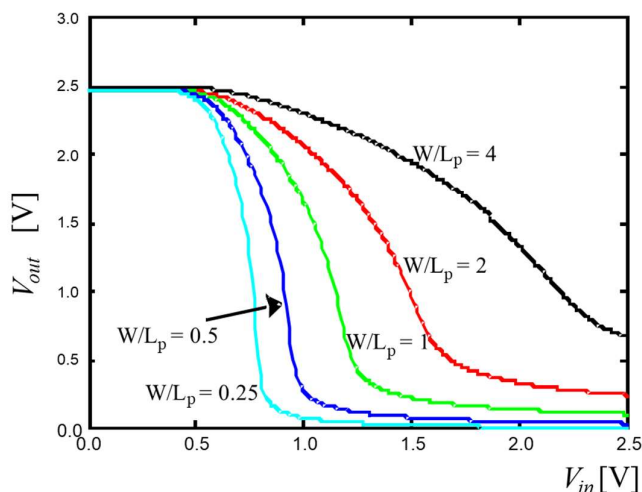


图 8.6 伪 nMOS 反相器的电压传输曲线与 PMOS 尺寸的关系

表 8.1 概括了它们的额定输出电压 (V_{OL})、静态功耗以及由低至高的传播延时。根据定义, 由低至高的延时为输出从 V_{OL} (对于该反相器 V_{OL} 不是 0V) 到达 1.25V 时所需要的时间。静态和动态性能之间的相互制约是很明显的。一个较大的上拉器件不仅提高了性能, 同时也由于增加了 V_{OL} 而使静态功耗增加和噪声容限减小。

表 8.1 伪 nMOS 反相器的性能

PMOS 尺寸(W/L)	V_{OL} (V)	静态功耗 (μW)	t_{pLH} (ps)
4	0.693	564	14
2	0.273	298	56
1	0.133	160	123
0.5	0.064	80	268
0.25	0.031	41	569

我们注意到用来预测 V_{OL} 的简单一阶模型是非常有效的。对于一个 $W-L$ 比为 4 的 PMOS 而言, V_{OL} 为 $(30/115)(4)(0.63V) = 0.66V$ 。

伪 nMOS 的静态功耗限制了它的应用。然而当面积是最重要的因素时, 与互补 CMOS 相比它能减少晶体管的数目, 这一点还是非常吸引人的。因此仍然可以看到伪 nMOS 有时应用在大扇入电路中。图 8.7 是伪 nMOS 中 NOR 和 NAND 门的电路图。

8.2.2 差分串联电压开关逻辑 (DCVSL)

建立一个能够完全消除静态电流并提供从电源轨线至轨线电压摆幅的有比逻辑方式是可能的。这样的门同时利用了两个概念: 差分逻辑和正反馈。一个差分门要求

每一个输入都具有互补的形式，同时它也产生互补的输出，反馈机制保证了在不需要负载器件时将其关断。这样的逻辑系列称为差分串联电压开关逻辑（Differential Cascode Voltage Switch Logic，即 DCVSL），这样一个例子的概念如图 8.8（a）所示。

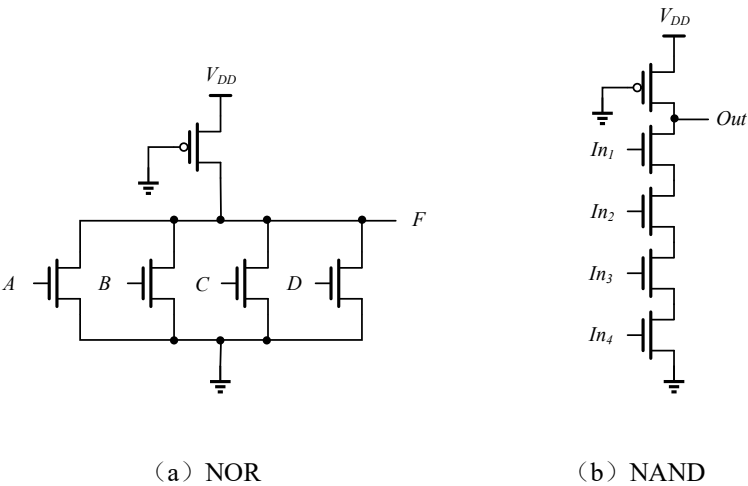


图 8.7 四输入伪 nMOS NOR 和 NAND 门

下拉网络 PDN1 和 PDN2 采用 NMOS 器件，并且两者是互相排斥的，也就是当 PDN1 导通时 PDN2 关断，当 PDN1 关断时 PDN2 导通，这样同时实现了所需要的功能及其相反的功能。现在假设对于给定的一组输入，使 PDN1 导通而 PDN2 不导通，而 *Out* 和 $\overline{Out}$ 最初分别为高电平和低电平。使 PDN1 导通，则引起 *Out* 下拉，虽然在 M_1 和 PDN1 之间仍存在竞争，由于 M_2 和 PDN2 都关断， $\overline{Out}$ 处于高阻抗状态，PDN1 必须足够强使 *Out* 低于 $V_{DD}-|V_{Tp}|$ ，此时 M_2 导通并开始对 $\overline{Out}$ 充电至 V_{DD} ，最终将 M_1 关断。这又使 *Out* 放电至 GND。图 8.8（b）是一个 XOR-XNOR 门的例子。注意在两个下拉网络之间有可能共用晶体管，从而降低了实现的面积开销。

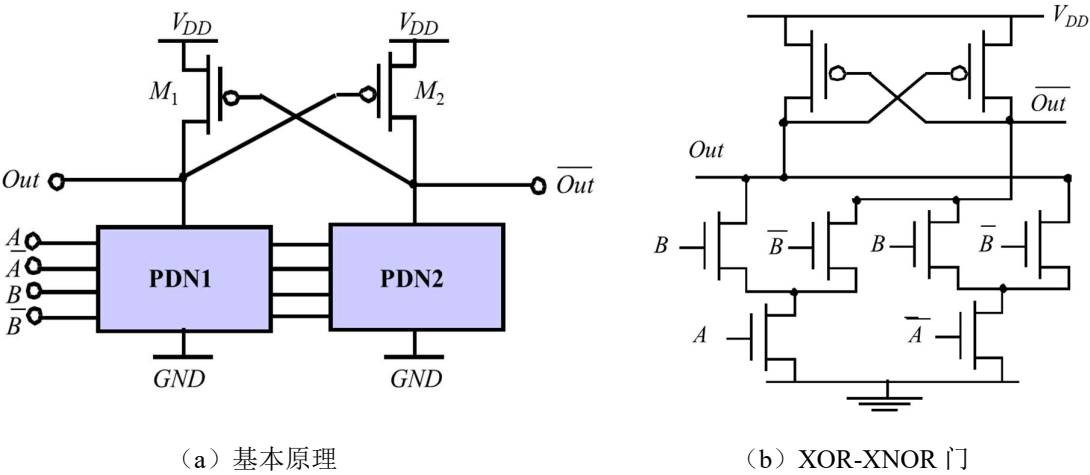


图 8.8 DCVSL 逻辑门

所得到的电路具有从电源轨线至轨线电压的摆幅并消除了静态功耗：在稳定状态，任何一边由 NMOS 管堆叠的下拉网络和相应的 PMOS 负载器件不会同时导通。然而这一电路仍然是有比的，因为 PMOS 器件相对于下拉器件的尺寸不仅对电路的性能很重要，而且对于它的功能也是非常关键的。除了会增加设计的复杂性，这一电路类型还有因渡越电流所引起的功耗问题。在翻转期间 PMOS 和 PDN 会同时导通一段时间，从而产生一条短路路径。

例 8.2 DCVSL 瞬态响应

图 8.9 是 DCVSL 的一个 AND/NAND 门瞬态响应的例子， M_1 和 M_2 为 $1.0\mu\text{m}/0.25\mu\text{m}$ ， M_3 和 M_4 为 $0.5\mu\text{m}/0.25\mu\text{m}$ ，交叉合的 PMOS 器件为 $1.5\mu\text{m}/0.25\mu\text{m}$ 。注意当 Out 被下拉至 $V_{DD}-|V_{Tp}|$ 时， $\overline{Out}$ 开始迅速充电至 V_{DD} 。从输入到 Out 和 $\overline{Out}$ 的延时分别为 197ps 和 321ps。一个 CMOS 静态 AND 门（即 NAND 门后面跟一个反相器）的延时为 200ps。

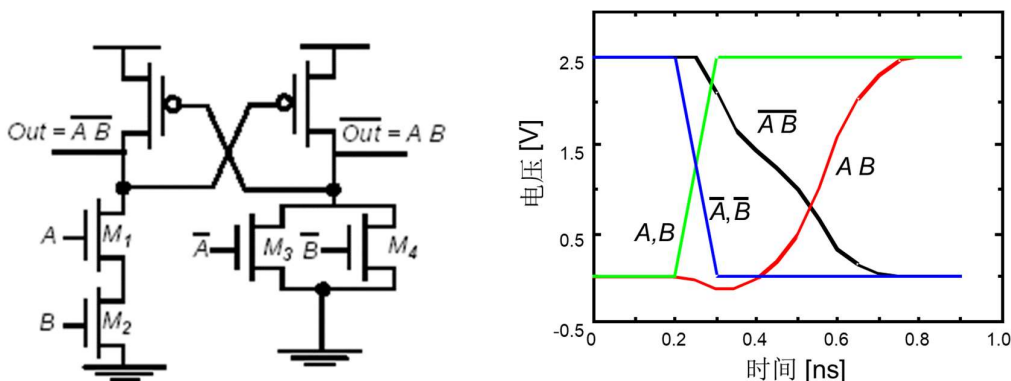


图 8.9 一个简单的 AND/NAND DCVSL 门的瞬态响应

设计考虑单端门与差分门

DCVSL 门提供差分（或互补）输出。输出信号（ V_{out1} ）和它的反信号（ V_{out2} ）同时可用。这是一个非常独特的优点，因为节省了一个额外的反相器来产生互补信号。我们已经观察到，用差分电路实现一个复杂功能可以使所需要的晶体管的数目减少一半！在关键时序路径上门的数目也常常可以减少。最后，这一方法避免了由于增加反相器引起的时差问题。例如，在逻辑设计中经常出现同时需要一个信号和它的互补信号的情况。当用反相器产生该互补信号时，这一反信号常常迟于原来的信号[见图 8.10 (a)]。这就引起了时序问题，特别是在高速设计中这一问题更加明显。具有差分输出能力的逻辑系列在很大程度上避免了这一问题[见图 8.10 (b)]。

既然有了所有这些优点，为什么不总是使用差分逻辑呢？原因是差分的特点实际上使需要布置的导线数量加倍，因而除了在实现每个门时需要额外的开销外，还常常使所设计的电路十分复杂。同时，它的动态功耗也较高。

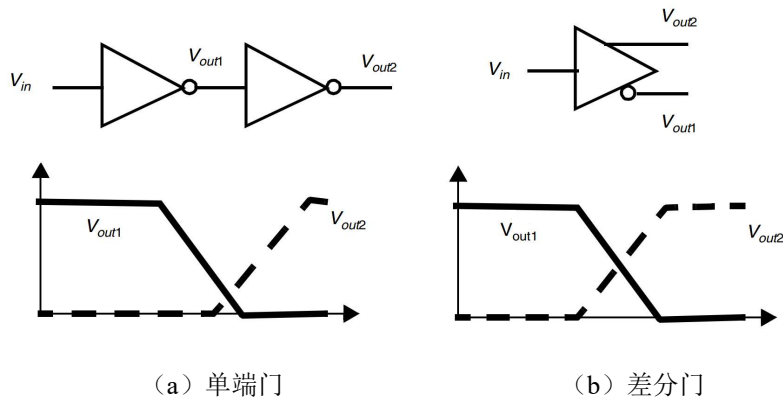


图 8.10 差分门优于单端门之处

第三节 动态 CMOS 设计

我们在前面已注意到具有 N 个扇入的静态 CMOS 门要求 $2N$ 个器件，而且还介绍了各种方法来减少实现一个指定逻辑功能所需要的晶体管数，包括伪 NMOS、传输管逻辑等。伪 nMOS 逻辑类型只需要 $N+1$ 个晶体管就可以实现一个 N 个输入的逻辑门，但可惜的是它具有静态功耗。在本节中将介绍另一种称为动态逻辑的逻辑类型，它有类似的效果却避免了静态功耗。通过增加一个时钟输入，它可以相继完成预充电和条件求值两个阶段。

8.3.1 动态逻辑：基本原理

图 8.11 (a) 是一个 (n 型) 动态逻辑门的基本结构。它的 PDN (下拉网络) 的构成完全与互补 CMOS 一样。这一电路的工作可以分为两个主要阶段：预充电和求值。处于何种工作模式由时钟信号 CLK 决定。

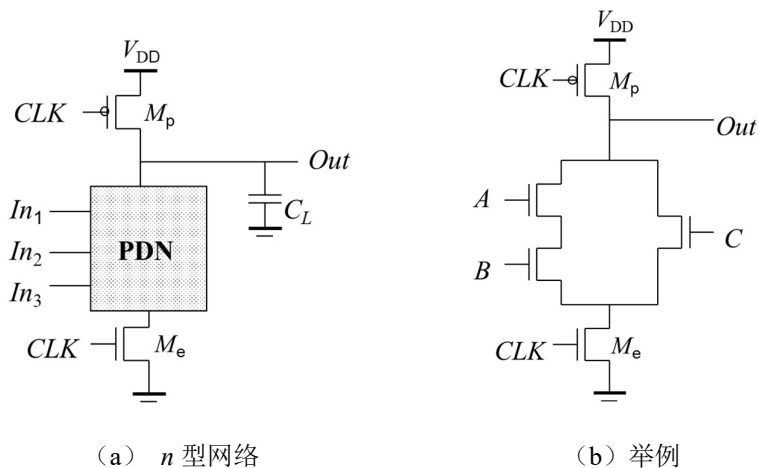


图 8.11 动态门的基本原理

预充电

当 $CLK=0$ 时输出节点 Out 被 PMOS 管 M_p 预充电至 V_{DD} 。在此期间，NMOS 求值管 M_e 关断，所以下拉路径不工作。求值 FET 消除了在预充电期间可能发生的任何静态功耗（即如果没有求值 FET，那么当下拉器件和预充电器件同时导通时，电源线间将有静态电流流过）。

求值

当 $CLK=1$ 时预充电管 M_p 关断，求值管 M_e 导通。输出根据输入值和下拉拓扑结构的情况有条件地放电。如果输入使 PDN 导通，则在 Out 和 GND 之间存在低阻通路，因此输出放电至 GND。如果 PDN 关断，预充电值维持存放在输出电容 C_L 上， C_L 是结电容、布线电容以及扇出门输入电容的组合。在求值阶段，输出节点和电源线之间惟一可能的路径是连接到 GND。因此，一旦 Out 放电就不可能再充电，直到进行下一次预充电。因此门的输入在求值期间最多只能有一次变化。注意，在求值期间如果下拉网络关断，则输出有可能处于高阻抗状态。这一特点与相应的静态门的情况截然不同，静态门在输出和其中一条电源线之间总会存在一条低阻抗通路。

作为一个例子，考虑图 8.11 (b) 的电路，在预充电阶段 ($CLK=0$) 由于求值器件关断，所以无论输入为何值输出都预充电至 V_{DD} 。在求值期间 ($CLK=1$)，如果（仅仅是如果） $AB+C$ 为真，则在 Out 和 GND 之间建立起一条导电通路。否则，输出维持在等于 V_{DD} 的预充电状态。因此可实现以下功能：

$$Out = \overline{CLK} + \overline{(A \cdot B + C)} \cdot CLK \quad (8.5)$$

对于动态逻辑门可以推导出许多重要特性：

- 逻辑功能由 NMOS 下拉网络实现。构成 PDN 的过程与静态 CMOS 完全一样。
- 晶体管的数目（对于复杂门）明显少于静态情况：为 $N+2$ 而不是 $2N$ 。
- 是无比的逻辑门。PMOS 预充电器件的尺寸对于实现门的正确功能并不重要。预充电器件的尺寸可以较大以改善低至高的翻转时间（当然，这是以由高至低的翻转时间为代价的），然而这里还有一个权衡功耗的问题，因为较大的预充电器件会直接增加时钟的功耗。
- 动态逻辑门只有动态功耗。理想情况下在 V_{DD} 和 GND 之间从不存在任何静态电流路径。但是它的总功耗还是可以明显高于静态逻辑门。
- 动态逻辑门具有较快的开关速度，主要有两个原因。第一个（明显的）原因是由于减少了每个门晶体管的数目，并且每个扇入对前级只表现为一个负载晶体管，因而降低了负载电容，这相当于降低了逻辑努力。例如，一个两输入动态 NOR 门的逻辑努力是 $2/3$ ，远小于相应静态 CMOS 门的 $5/3$ 。第二个原因是动态门没有短路电流，并且由下拉器件提供的所有电流都用来对负载电容放电。

低和高输出电平 V_{OL} 和 V_{OH} 很容易被确定为 GND 和 V_{DD} 。并且它们与晶体管的尺寸无关。其他 VTC 参数与静态门差别很大。静态门的噪声容限和开关阈值曾被定义为是静态量，与时间无关。而为了使一个动态门正确工作，要求有一系列周期性的预充电和求值。因此，纯粹的静态分析不再适用。在求值周期，一个动态反相器的下拉网络在输入信号超过 NMOS 下拉管的阈值电压 (V_{Tn}) 时就开始导通。因此把这个门的开关阈值 (V_M) 以及 V_{IL} 和 V_{IH} 都设为等于 V_{Tn} 是合理的。这相当于 NM_L 的值较小。

设计考虑

用对偶的方法来实现另一形态 (p 型) 的动态逻辑也是可能的。这时输出节点通过一个预充电 NMOS 管与 GND 连接，而求值 PUN 网络用 PMOS 实现。工作情况与上面介绍的很类似：在预充电阶段，输出节点放电至 GND ；在求值阶段，输出有条件地被充电至 V_{DD} 。 p 型动态门的缺点是比 n 型动态门慢，因为 PMOS 管的驱动电流较小。

8.3.2 动态逻辑的速度和功耗

动态逻辑的主要优点是提高了速度和减少了实现面积。用较少的器件来实现一个指定的逻辑功能意味着总的负载电容要小得多。对这个门开关特性进行分析，发现它有一些有趣的特点。在预充电阶段之后，输出为高电平。对于低电平输入信号，没有任何另外的切换发生。结果 $t_{pLH}=0$ ！反之，由高至低的翻转要求通过下拉网络放电输出电容。因此， t_{pHL} 与 C_L 以及下拉网络的吸电流能力成正比。求值管的存在多少会减慢门的速度，因为它相当于一个额外的串联电阻。去掉这个晶体管虽然对功能无妨碍，但可能会造成静态功耗及潜在的性能损失。

上面的分析多少有点不公正，因为它忽略了预充电时间对门的开关速度的影响。预充电时间是由通过 PMOS 预充电管充电 C_L 所需要的时间决定的。在这一期间，不能使用门中的逻辑。然而常常可以在设计整个数字系统时把预充电时间与其他系统功能统一起来。例如，一个微处理器中运算单元的预充电可以与指令译码同时进行。设计者必须清楚在动态逻辑中的这一“死区”，因此应当仔细考虑使用它的优点和避免它的缺点，全盘考虑整个系统的要求。

例 8.3 一个四输入的动态 NAND 门

图 8.12 是用动态电路形式设计的一个四输入 NAND 门的例子。由于该门的动态本质，推导电压传输特性时与传统的方法不同。正如前面讨论过的，我们假设这个门的开关阈值等于 NMOS 下拉管的阈值。这导致了不对称的噪声容限，如表 8.2 所示。

这个门的动态特性是用 SPICE 来模拟的，假设时钟变为高电平时所有的输入都置于高电平。在时钟的上升沿，输出节点放电。所形成的瞬态响应画在图 8.12 中，而它的传播延时总结在表 8.2 中。预充电周期的时间可以通过改变 PMOS 预充电管的尺寸来调整，然而应当避免 PMOS 太大，因为它会降低门的速度并增加时钟线上的电容负载。对于较大的设计，后一个因素可能会成为一个主要的设计考虑，因为时钟负载可能会变得

太大而难以驱动。

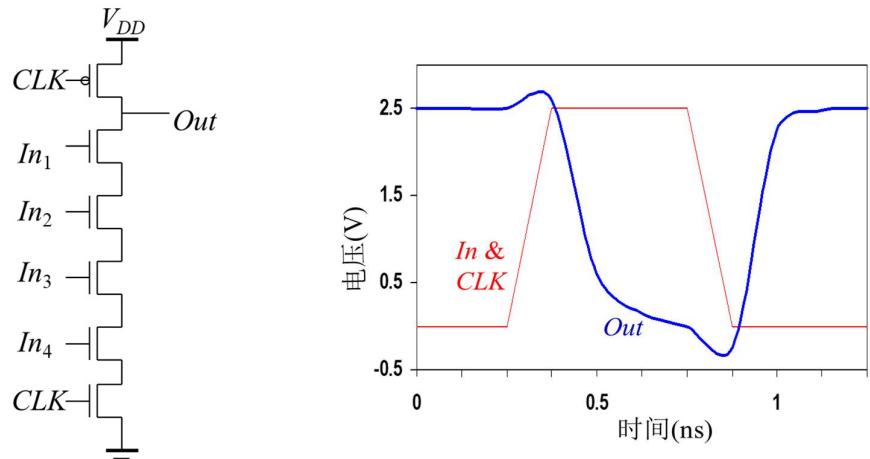


图 8.12 四输入动态 NAND 门的电路图和瞬态响应

表 8.2 四输入动态 NAND 的 DC 和 AC 参数

V_{OH}	V_{OL}	V_M	NM_H	NM_L	t_{pHL}	t_{pLH}	t_{pre}
2.5V	0V	V_{TN}	$2.5 - V_{TN}$	V_{TN}	110ps	0	83ps

正如前面提到的，这些静态参数与时间有关。为了说明这一点，考虑一个四输入 NAND 门，把所有输入都连在一起，并完成一个由低至高的翻转。图 8.13 是对三个不同输入翻转（由 0 分别至 0.45V、0.50V 和 0.55V）的输出电压的瞬态模拟结果。

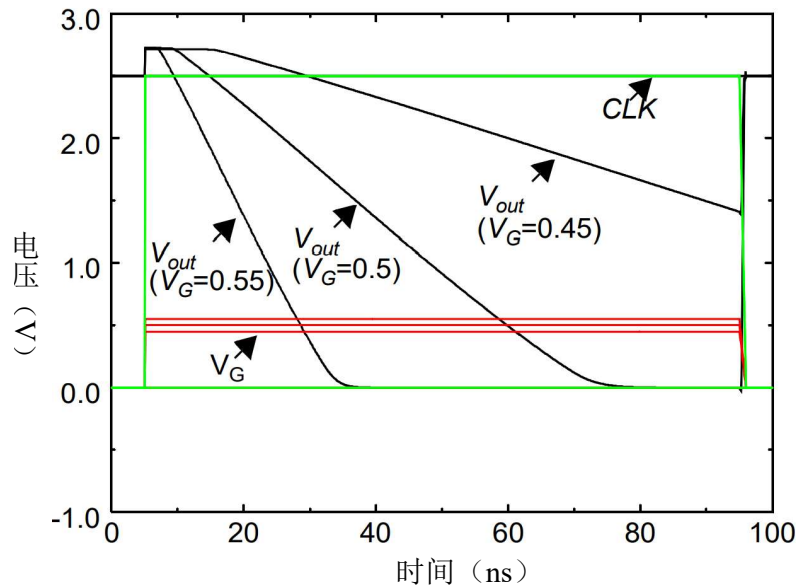


图 8.13 输入毛刺对输出的影响

（开关阈值取决于求值时间，如果求值阶段很短则可以允许有较大的毛刺）

在前面的讨论中我们已经定义了动态门的开关阈值即为器件的阈值。然而应注意，输出电压下降的数量与输入电压以及允许的求值时间密切相关。如果求值时间很短，那么噪声电压必须很大才会破坏信号，换言之，开关阈值确实与时间相关。

看来动态逻辑在功耗方面有明显的优势，这主要有三个原因。第一，它的实际电容较小，因为动态逻辑使用较少的晶体管来实现指定的功能。同时从每个扇出看到的负载是一个而不是两个晶体管。第二，由于动态门的结构使得每个时钟周期最多只能翻转一次。毛刺（或动态故障）在动态逻辑中并不发生。最后，动态门不存在短路功耗，因为它在求值时上拉路径不导通。

虽然这些论点一般都正确，但它们也为其他一些考虑所抵消：（1）动态逻辑的时钟功耗可能很大，特别是因为时钟节点在每一个时钟周期都肯定有一个翻转；（2）晶体管的数目大于实现该逻辑所要求的最小一组晶体管；（3）当增加抗漏电器件时（如将要进一步讨论的）可能会有短路功耗存在；（4）最重要的是，由于周期性的预充电和放电操作，动态逻辑通常表现出较高的开关活动性。前面谈到，静态门的翻转概率为 $P_0 P_1 = P_0(1 - P_0)$ ，对于动态逻辑，它的输出翻转概率不取决于输入的状态（历史情况），而是取决于信号概率。对于一个 n 型树动态门，只有当输出在上一个求值阶段中被放电时，它才会在预充电阶段发生 $0 \rightarrow 1$ 的翻转。因此， n 型动态门进行 $0 \rightarrow 1$ 翻转的概率为：

$$\alpha_{0 \rightarrow 1} = P_0 \quad (8.6)$$

式中， P_0 是输出为零的概率。这一数字总是大于或等于 $P_0 P_1$ 。对于均匀分布的输入， N 个输入门的翻转概率为：

$$\alpha_{0 \rightarrow 1} = \frac{N_0}{2^N} \quad (8.7)$$

式中， N_0 是逻辑功能真值表中输出为零的项数。

例 8.4 动态逻辑的活动性估计

为了说明动态门会增加活动性，再次考虑两输入的 NOR 门。图 8.14 显示了它的 n 型树动态实现以及对应的静态电路。对于概率相同的输入，动态门输出节点在预充电之后立刻放电的概率为 75%，这意味着这样一个门的活动性等于 0.75（即 $P_{\text{NOR}} = 0.75 C_L V_{\text{DD}}^2 f_{\text{CLK}}$ ）。而对于静态电路，其对应的活动性要低得多，为 3/16。对于一个动态 NAND 门，翻转概率为 1/4（因为输出将被放电的概率为 25%），而静态 NAND 的概率为 3/16。虽然这些例子说明动态逻辑的开关活动性一般较高，但也应注意动态逻辑的实际电容较小。在分析动态功耗时这两个因素都必须考虑。

8.3.3 动态设计中的信号完整性问题

显然，动态逻辑比静态电路能获得更高的性能。然而如果要使动态电路能够正确工作，还有几个重要问题必须考虑。这些问题包括电荷泄漏、电荷分享、电容耦合以及时钟馈通。下面我们详细地讨论这些问题。

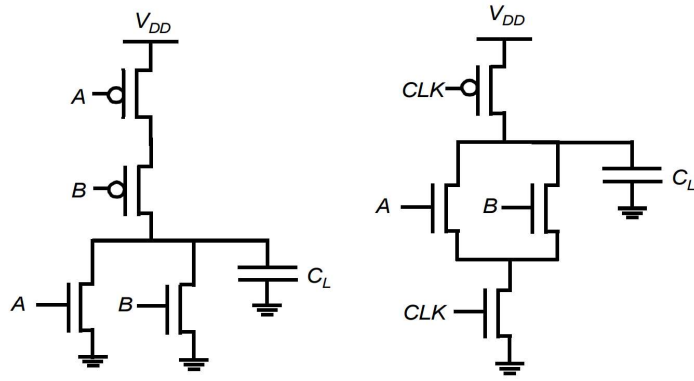
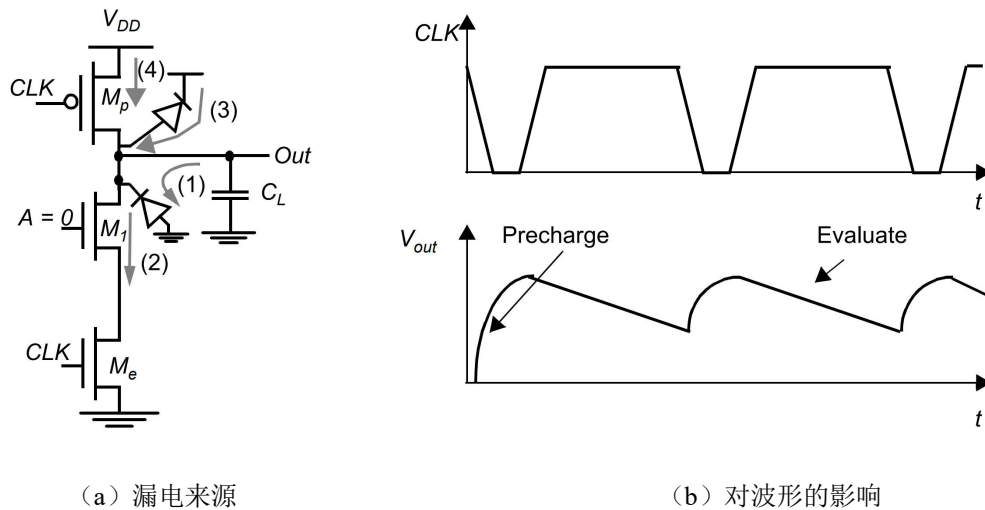


图 8.14 静态 NOR 和 n 型动态 NOR 门的对比

电荷泄漏

一个动态门的工作取决于输出值在电容上的动态存储。如果下拉网络关断，那么理想情况下，输出在求值阶段应当维持在预充电状态的 V_{DD} 。然而由于存在漏电流，这一电荷将逐渐泄漏掉，最终会使这个门的工作出错。图 8.15 (a) 显示了基本动态反相器电路的漏电来源。

来源 1 和来源 2 分别为 NMOS 下拉器件 M_1 反偏二极管和亚阈值漏电。存储在 C_L 上的电荷将通过这些漏电渠道慢慢泄漏掉，使高电平变差[见图 8.15 (b)]。因此动态电路要求一个最低的时钟频率，一般在几千赫兹左右。这使得动态技术的使用对于诸如手表等低性能产品或使用有条件时钟(它不能保证最小的时钟频率)的处理器缺乏吸引力。注意，由于反偏二极管(漏电来源 3)和亚阈值导电(漏电来源 4)，PMOS 预充电器件也引起一些漏电流。在某种程度上，PMOS 的漏电流抵消了下拉路径的漏电。因此，输出电压将由下拉和上拉路径构成的电阻分压器来确定。



(a) 漏电来源

(b) 对波形的影响

图 8.15 动态电路的漏电问题

例 8.5 动态电路中的漏电

考虑所有器件都为 $0.5\mu\text{m}/0.25\mu\text{m}$ 的简单反相器。假设输入在求值期间处于低电平。理想情况下输出应当维持在预充电状态的 V_{DD} 。然而可以从图 8.16 中看到，输出电压下降了。一旦输出降至低于扇出逻辑门的开关阈值，这个输出就被认为是一个低电压。注意，由于 PMOS 上拉提供的漏电流，输出稳定在一个中间电压上。

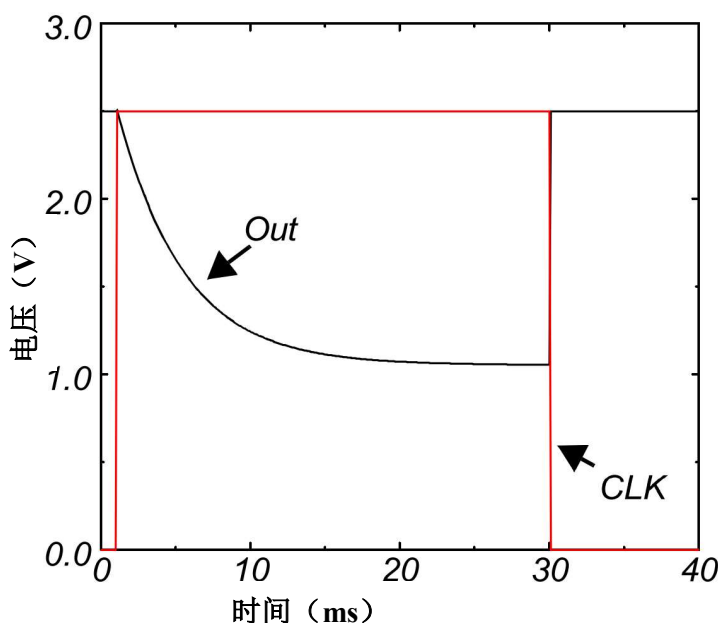


图 8.16 电荷泄漏的影响

（输出稳定在由下拉和上拉器件组成的电阻分压器决定的一个中间电压上）

在求值期间，当下拉路径关断时由于输出节点处于高阻状态，因此引起了漏电。漏电问题可以通过降低求值期间输出节点上的输出阻抗来解决。通常是增加一个泄漏晶体管，如图 8.17（a）所示。该泄漏器为一个伪 NMOS 型的上拉器件，用来补偿由于下拉漏电路径造成的电荷损失。为了避免出现与这类电路相关的尺寸比问题及相关的静态功耗，可以使泄漏器的电阻较高（也就是使器件尺寸较小）。这就使（强）下拉器件能够下拉节点 *Out* 的电压，使它充分低于下一级门的开关阈值。通常泄漏器以反馈形式实现，以同时消除静态功耗[见图 8.17（b）]。

电荷分享

在动态逻辑中需要考虑的另一个重要问题是电荷分享的影响。考虑图 8.18 的电路。在预充电阶段，输出节点被预充电至 V_{DD} 。假设在预充电期间，所有的输入被置为 0，并且 C_a 已放电。同时还假设在求值期间输入 *B* 维持在 0，而输入 *A* 由 $0 \rightarrow 1$ 翻转，使晶体管 M_a 导通。原本存储在电容 C_L 上的电荷就在 C_L 和 C_a 间重新分配，这就造成了输出电压有所下降。由于电路的动态本质，这一下降不能恢复。

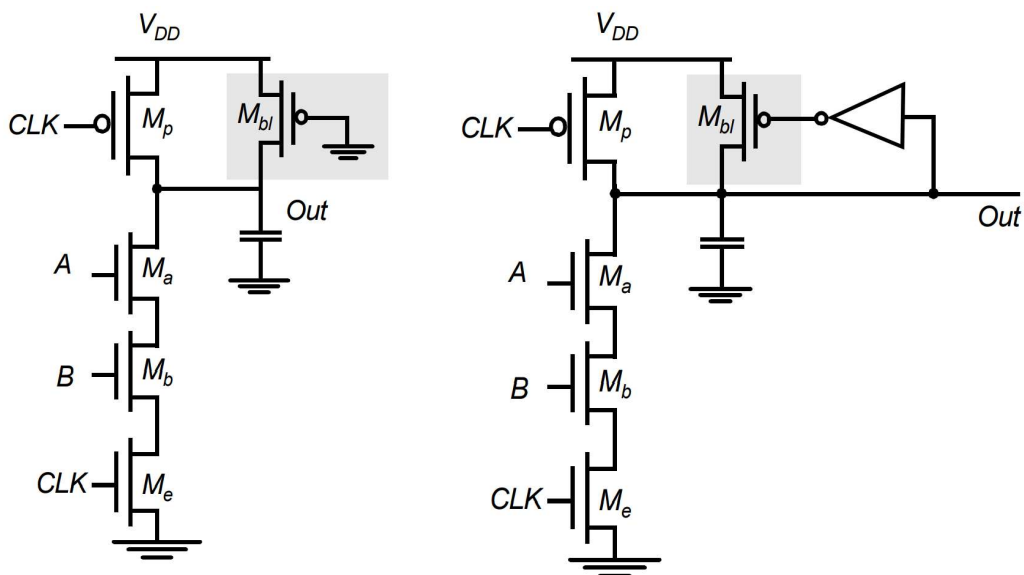


图 8.17 静态泄漏器补偿电荷泄漏

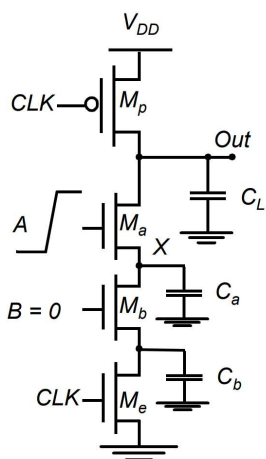


图 8.18 动态电路中的电荷分享

这种对输出电压的影响很容易计算，在前面假设的条件下，以下起始条件成立： $V_{out}(t=0) = V_{DD}$ 和 $V_X(t=0) = 0$ 。因此有两种可能的情形必须考虑：

1. $\Delta V_{out} < V_{Tn}$ 。在这一情形中， V_X 的终值等于 $V_{DD} - V_{Tn}(V_X)$ 。于是由电荷守恒得到：

$$C_L V_{DD} = C_L V_{out}(t) + C_a [V_{DD} - V_{Tn}(V_X)]$$

或

$$\Delta V_{out} = V_{out}(t) - V_{DD} = -(C_a / C_L) [V_{DD} - V_{Tn}(V_X)] \quad (8.8)$$

2. $\Delta V_{out} > V_{Tn}$ 。此时 V_X 与 V_{out} 达到同一值：

$$\Delta V_{out} = -V_{DD} [C_a / (C_a + C_L)] \quad (8.9)$$

由电容比我们可以确定上面哪一种情形成立。两种情形的边界条件可以通过使公式(8.9)中的 ΔV_{out} 等于 V_{Tn} 来确定,从而得到:

$$\frac{C_a}{C_L} = \frac{V_{Tn}}{V_{DD} - V_{Tn}} \quad (8.10)$$

当 (C_a/C_L) 之比小于公式(8.10)确定的条件时情形1成立,否则公式(8.9)成立。总的来说,希望保持 ΔV_{out} 的值在 $|V_{Tp}|$ 以下。动态门的输出可能会连到一个静态反相器上,在这种情况下 V_{out} 过低会引起静态功耗。要注意如果输出电压低于它所驱动的门的开关阈值,电路工作就会出错。

例 8.6 电荷分享

让我们考虑电荷分享对图 8.19 所示动态逻辑门的影响。这个门实现了一个三输入 EXOR 门的功能 $y = A \oplus B \oplus C$ 。第一个要解决的问题是在什么条件下会造成节点 y 上电压降的最坏情况。为了简化分析,忽略负载反相器,并假设在预充电工作期间所有输入位于低电平,所有被隔离的内部节点(V_a, V_b, V_c 和 V_d)开始时为 0V。

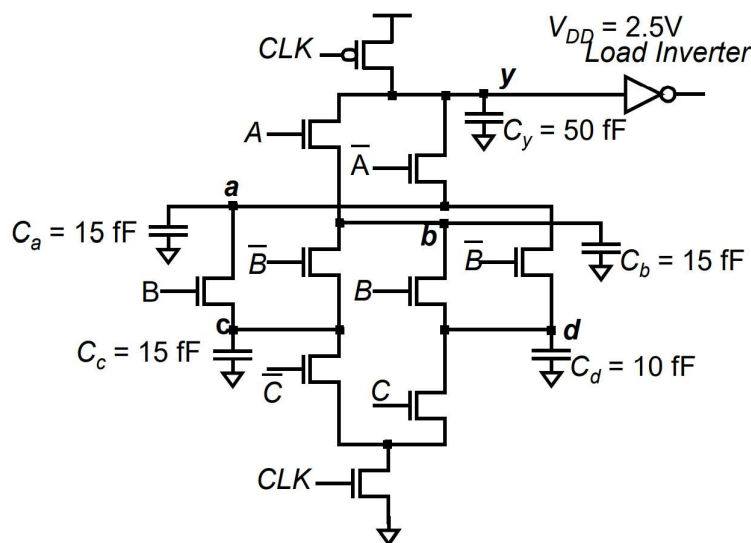


图 8.19 动态逻辑中电荷分享影响的例子

检查这一具体逻辑功能的真值表显示:输出在 8 种情形中有 4 种处于高电平。最坏情形的输出变化发生在求值阶段有最多数目的内部电容加到输出节点上。这发生在 $\bar{A}BC$ 或 $A\bar{B}C$ 时。于是电压的变化可以通过如公式(8.9)那样使开始的电荷等于最终的电荷而得到,由此求得最坏情况的电压变化为 $30/(30+50)*2.5V=0.94V$ 。为了保证电路正确工作,连接反相器的开关阈值应当在 $2.5 - 0.94 = 1.56V$ 以下。

解决电荷再分布最常用也是最有效的办法是对关键的内部节点预充电,如图 8.20 所示。由于在预充电期间内部节点被充电至 V_{DD} ,电荷分享不再发生。这一方法显然是

以增加面积和电容为代价的。

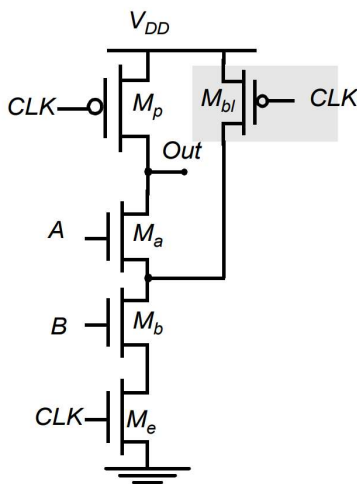


图 8.20 预充电内部节点来解决电荷分享问题

电容耦合

输出节点相对较高的阻抗使得电路对串扰的影响非常敏感。一条导线布在一个动态节点之上或邻近时，它可能会产生电容耦合而破坏这个浮空节点的状态。另一个同样重要的电容耦合形式是回栅（*backgate*），即输出至输入耦合。考虑图 8.21（a）中的电路，一个动态的两输入 NAND 门驱动一个静态 NAND 门。静态门输入的翻转可能会造成该门的输出（ Out_2 ）变为低电平。这一输出变化又会通过晶体管 M_4 的栅-源和栅-漏电容耦合到这个门的另一个输入（动态节点 Out_1 ）。图 8.21（b）是这一影响的模拟结果。它显示了耦合如何引起动态门的输出 Out_1 显著降低。这进一步使静态 NAND 门的输出不能全程下降至 0V，而有少量的静态功耗损失。如果这一电压降低足够大，电路可能会产生求值错误，即 NAND 不能降到低电平。在设计和布置动态电路的版图时需要特别注意，应尽可能减少电容的耦合。

时钟馈通

电容耦合的一种特殊情况是时钟馈通，它是由在预充电器件的时钟输入和动态输出节点之间的电容耦合引起的效应。耦合电容由预充电器件的栅-漏电容组成，包括了覆盖电容和沟道电容。当下拉网络不导通时，这一电容耦合会在时钟由低至高翻转时引起动态节点的输出上升到 V_{DD} 以上。其次，快速上升和下降的时钟边沿会耦合到信号节点上，这在图 8.21（b）的模拟结果中非常明显。

时钟馈通的危险在于它可能使预充电管正常情况下的反偏结二极管变为正向偏置。这会使电子注入到衬底中，它们可能为附近处于“1”（电平）状态的高阻抗节点所收集，最终导致出错。CMOS 闩锁则是这一注入的另一种可能结果。因此在任何情况下，高速动态电路都应当仔细进行模拟以确保时钟馈通的影响处在允许的范围之内。

以上所有这些考虑表明动态电路的设计很有技巧性并且需要非常仔细。因此只有当

要求很高的性能或具备高水平的自动设计工具时才考虑用它。

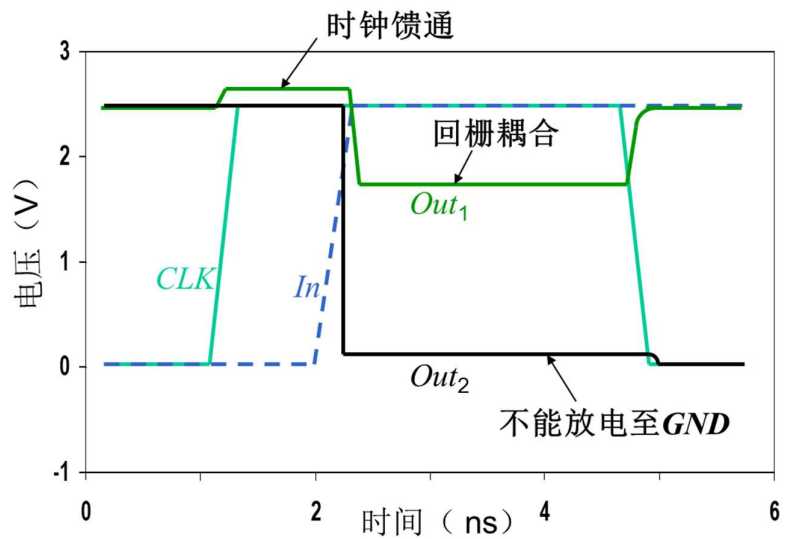
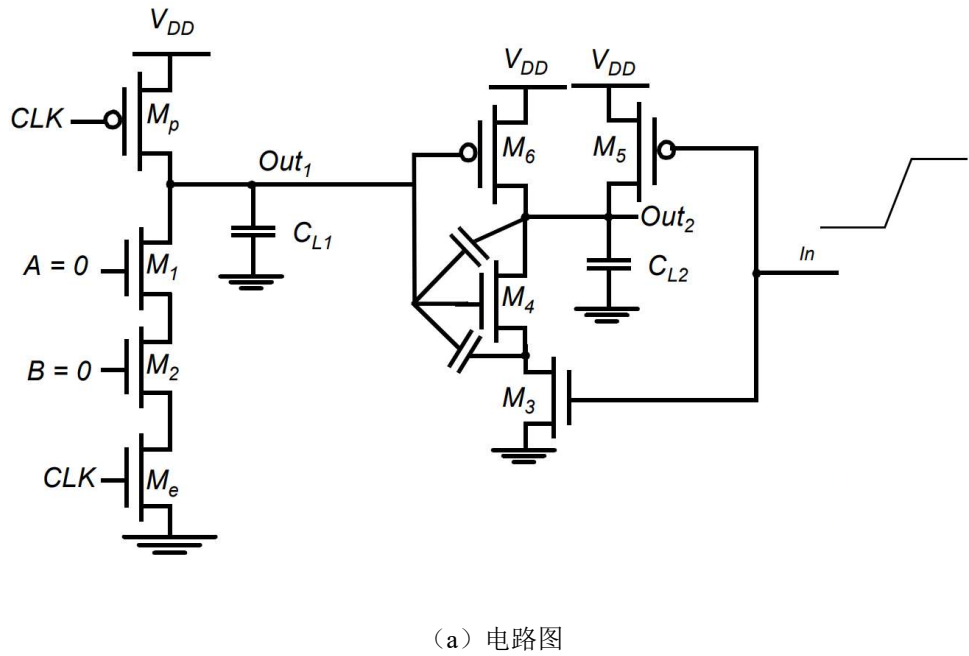


图 8.21 说明回栅耦合影响的例子

8.3.4 动态门级连

除了信号完整性问题而外，还有一个重要问题会使动态电路的设计复杂化：直接串联动态门形成多级逻辑结构的方法并不可行。这一问题可以用两个串联的 n 型动态反相器来说明，如图 8.22 (a) 所示。在预充电阶段（即 $CLK=0$ ），两个反相器的输出都被预

充电至 V_{DD} 。假设原始输入 In 为 $0 \rightarrow 1$ 的翻转[见图 8.22 (b)]，在时钟的上升沿，输出 Out_1 开始放电。第二个输出应当维持在预充电状态的 V_{DD} ，因为它的值预计是 1（在求值期间 Out_1 翻转到 0）。然而，输入使 Out_1 放电至 GND 需要有一定的传播延时，因此第二个输出也开始放电。只要 Out_1 仍然比第二个门的开关阈值（它大约等于 V_{Tn} ）高，在 Out_2 和 GND 之间就存在一条导电路径，“宝贵”的电荷就会从 Out_2 上流失。这一导电路径只有在 Out_1 降至 V_{Tn} 并关断 NMOS 下拉管时才被切断。这会使 Out_2 处于一个中间电平上。正确的电平将不会再恢复，因为动态门依靠电容存储，而不像静态门那样具有 dc 恢复功能。电荷损失导致噪声容限降低并可能引起功能出错。

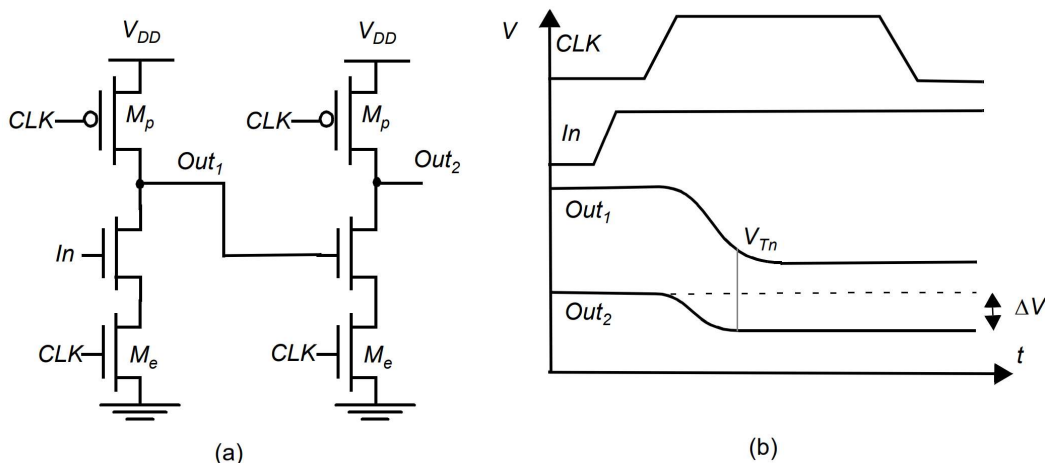


图 8.22 动态 n 型模块的级连

级连问题的出现是由于每一个门的输出（并且也是下一个门的输入）被预充电至 1，这在求值周期开始时可能造成无意的放电。在预充电期间置所有的输入为 0 可以解决这个问题。这样，在预充电之后下拉网络中所有的晶体管都被关断，因此在求值期间不会发生存储电容的错误放电。换言之，只要在求值期间输入只能进行单个的 $0 \rightarrow 1$ 翻转就能保证正确工作晶体管只有在需要的时候才导通，并且每周期最多一次。人们已经设想出许多符合这一规则的设计类型，下面讨论其中最重要的两个。

多米诺逻辑

基本概念 一个多米诺逻辑模块是由一个 n 型动态逻辑块后面接一个静态反相器构成的（见图 8.23）。在预充电期间， n 型动态门的输出被充电至 V_{DD} ，而反相器的输出则置为 0。在求值期间动态门有条件地放电，而反相器则有条件地完成 $0 \rightarrow 1$ 的翻转。如果假设一个多米诺门的所有输入是其他多米诺门的输出，那么就能保证在预充电阶段结束时所有的输入都置 0，因而在求值期间的惟一翻转就是 $0 \rightarrow 1$ 的翻转。因此它符合前面所述的规则。引入静态反相器还有另一个优点，即多米诺门的扇出由一个具有低阻抗输出的静态反相器驱动，因此提高了抗噪声能力。同时由于缓冲器隔离了内部和负载电容，因而减少了动态输出节点的电容。最后，反相器还可以用来驱动一个泄漏器件以抵抗漏电和电荷的重新分布，如图 8.23 的第二级所示。

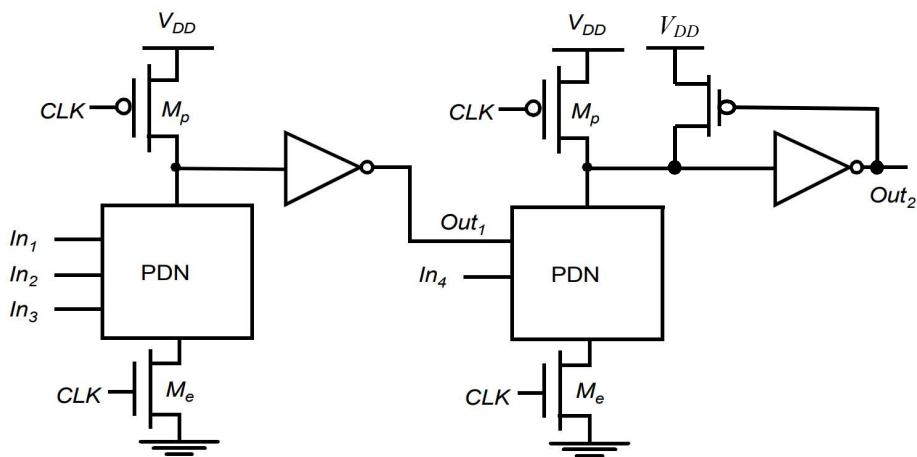


图 8.23 多米诺 CMOS 逻辑

现在考虑多米诺门串联链的工作情况。在预充电期间，所有的输入都置为 0。在求值期间，第一个多米诺块的输出或者停留在 0 或者从 0→1 翻转，从而影响第二个门。这一影响可以在整个多米诺链上传播，一个接着一个，有如一条崩塌的多米诺骨牌线，多米诺逻辑由此而得名。多米诺 CMOS 具有下列特点：

- 由于每一个动态门都有一个静态反相器，因此它只能实现非反相逻辑。虽然有一些办法可以解决这一问题，但这仍然是一个主要的限制因素，所以单纯的多米诺设计很少见。
- 可以达到非常高的速度：只存在上升沿的延时，而 t_{pHL} 等于零。可以调整反相器的尺寸使之与扇出匹配，这里扇出已比互补静态 CMOS 情形中小得多，因为每个扇出门只需要考虑一个栅电容。

由于在预充电期间多米诺门的输入在低电平，因此可以考虑取消求值晶体管，因为这可以减少时钟负载并提高下拉的驱动能力。然而取消求值器件会延长预充电周期——预充电过程也必须通过这一逻辑网络逐级传播。考虑图 8.24 所示的逻辑网络，这里已经取消了所有的求值器件。如果在求值期间原始输入 In_1 为 1，那么每一个动态门的输出求值为 0，而每一个静态反相器的输出为 1。在时钟下降沿预充电操作开始，进一步假设 In_1 为高至低的翻转。第二个门的输入最初为高电平， In_2 需要两个门的延时才能被驱动至低电平。而在这之前第二个门不能预充电它的输出，因为它的下拉网络会与预充电器件竞争。同样，第三个门必须等到第二个门预充电之后才能开始预充电。因此，预充电这个逻辑电路所需要的时间等于它的关键路径。另一个重要的负面影响是当上拉和下拉器件同时导通时的额外功耗。因此一个较好的做法是总是采用求值器件。

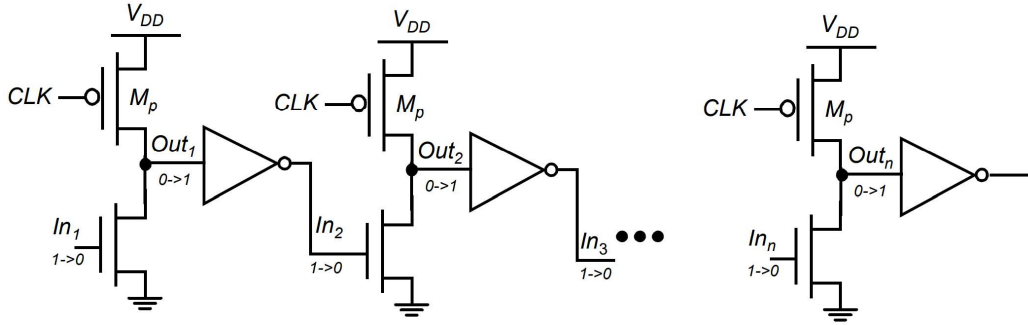


图 8.24 取消求值晶体管时预充电的传播效应

解决多米诺逻辑非反相的问题 多米诺逻辑的一个主要局限是只能实现非反相逻辑。这一要求限制了广泛采用纯多米诺逻辑。然而已有几种方法可以解决这个问题。图 8.25 为解决这个问题的一种方法——利用简单的布尔变换（如 De Morgan 定理）来重新组织该逻辑。可惜的是这类优化并不总是可行的，因此也许必须采用更为一般的方法。

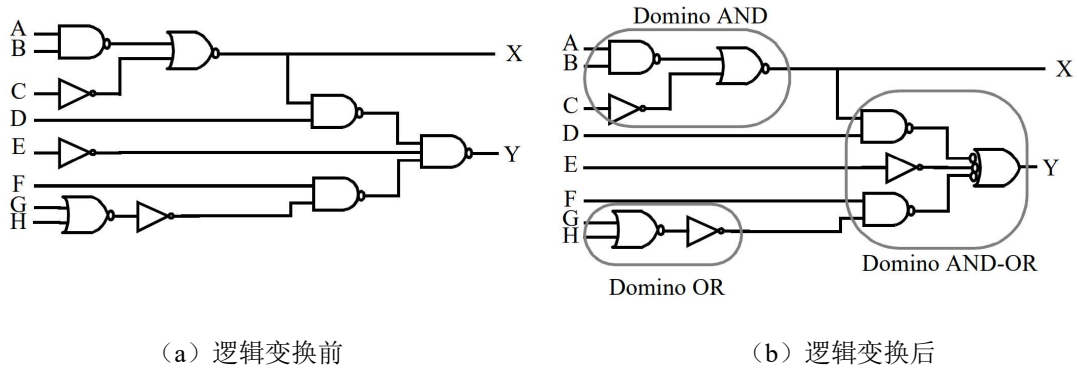


图 8.25 重构逻辑使之能采用非反相的多米诺逻辑实现

解决这个问题的一般方法（但代价较高）是采用差分逻辑。**双轨多米诺（Dual-rail domino）**在原理上类似于前面讨论的 DCVSL 结构，但它采用一个预充电负载而不是一个静态交叉耦合的 PMOS 负载。图 8.26 为一个 AND/NAND 差分逻辑门的电路图。注意，所有输入来自其他的差分多米诺门。它们在预充电阶段为低电平，而在求值期间完成有条件的 $0 \rightarrow 1$ 翻转。采用差分逻辑有可能实现任何随意的功能。但这是以增加功耗为代价的，因为不论输入为何值每个时钟周期必定有一次翻转，这是因为 0 或者 $\bar{0}$ 必定有一个要发生 $0 \rightarrow 1$ 的翻转。晶体管 M_{f1} 和 M_{f2} 的作用是在时钟较长时间处于高电平时仍保持该电路为静态（泄漏器）。注意这一电路不是有比电路，尽管存在 PMOS 上拉器件！由于它的高性能，所以这一差分方法非常流行，它已用在几个微处理器产品中。

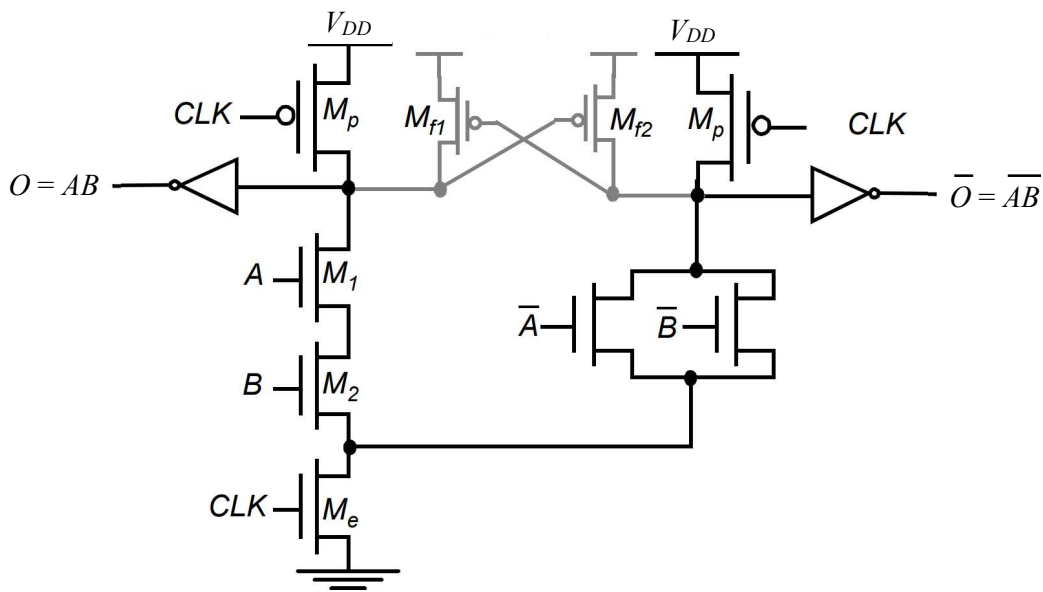


图 8.26 简单的双轨（差分）多米诺逻辑门

多米诺逻辑门的优化 有几种优化多米诺逻辑的方法。最明显的性能优化就是调整静态反相器中晶体管的尺寸。当多米诺电路中包含有求值器件时，所有的门都同时预充电，因此预充电只需两个门的延时——使动态门的输出充电至 V_{DD} 及驱动反相器的输出至低电平。求值期间的关键路径则是通过动态门的下拉路径及通过静态反相器的 PMOS 上拉晶体管。因此为了在求值期间加速电路，静态反相器的 β 比值应当较大，使它的开关阈值比较接近 V_{DD} 。这可以通过采用一个较小（最小尺寸）的 NMOS 器件和一个较大的 PMOS 器件来实现。最小尺寸的 NMOS 只影响预充电时间，这一时间一般很有限，因为所有的门同时预充电。采用大的 β 比值的惟一缺点是降低了噪声容限。因此设计者在确定器件尺寸的时候应当同时考虑降低噪声容限及性能的影响。

目前许多不同的多米诺逻辑已被提出。一种减少面积的优化方法是多输出多米诺逻辑（multiple-output domino logic）。它的基本原理显示在图 8.27 中。它利用了这样一个事实：某些输出是其他输出的子集，因此在一个门中可以产生许多逻辑功能。在本例中，由于 $O_3 = C+D$ 出现在所有三个输出中，所以它在下拉网络的底部实现。由于 O_2 等于 $B \cdot O_3$ ，所以它可以再次利用 O_3 。注意，内部节点必须预充电至 V_{DD} 以产生正确的结果。如果内部节点都要预充电到 V_{DD} 那么驱动预充电器件的器件数目并没有减少。然而求值晶体管的数目大大减少了，因为它们为多个输出所共享。

图 8.28 的组合多米诺（compounddomino）则代表对通用多米诺门的另一种优化，它也是使晶体管的数目尽可能减少。它不是每个动态门都驱动一个静态反相器，而是可以借助一个复合静态 CMOS 门把多个动态门的输出组合起来，如图 8.28 所示三个动态结构（即实现 $O_1 = \overline{ABC}$ ， $O_2 = \overline{DEF}$ 和 $O_3 = \overline{GH}$ ）的输出用一个实现 $O = \overline{(O_1 + O_2)} O_3$ 的复合 CMOS 静态门组合起来。以这一方式实现的总逻辑功能为 $O = ABCDEF + GH$ 。

组合多米诺是构成复杂动态逻辑门非常有用的工具。较大的（上下）堆叠的动态结构由扇出较小的并行结构及复合 CMOS 门所代替。例如，一个扇入很大的多米诺 AND 门可以实现为一组并行的扇入较小的动态 NAND 结构及一个静态的 NOR 门。在组合多米诺中一个重要的考虑是与回栅耦合相关的问题。必须仔细确保动态节点不受静态门输出与动态节点输出之间耦合的影响。

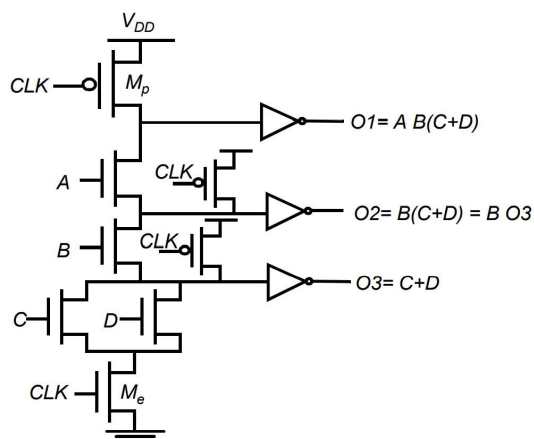


图 8.27 多输出多米诺逻辑

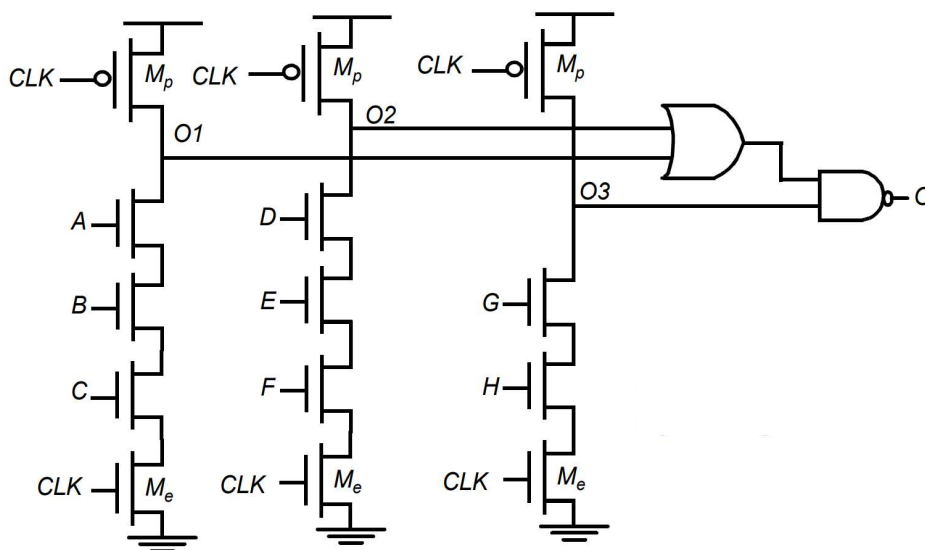


图 8.28 在动态门输出端使用复合静态门的组合多米诺逻辑

np-CMOS

串联动态逻辑的另一种方法是 *np-cmos*，它使用两种类型（*n* 型树和 *p* 型树）的动态逻辑，因而避免了在关键路径中由多米诺逻辑引入的额外静态反相器。在 *p* 型树逻辑门中，用 PMOS 器件构成一个上拉逻辑网络，包括一个 PMOS 求值管，如图 8.29 所示。NMOS 预充电管在预充电期间驱动输出至低电平。在求值期间取决于它的输入，输出将

有条件地进行 $0 \rightarrow 1$ 的翻转。

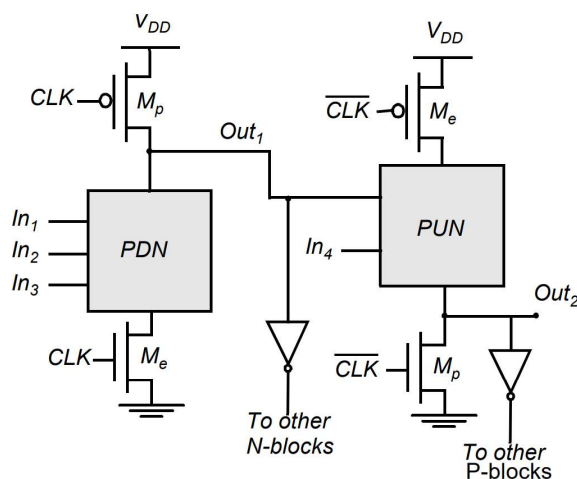


图 8.29 *np*-cmos 逻辑电路

np-cmos 逻辑利用了 *n* 型树和 *p* 型树逻辑门之间的对偶性来消除串级问题。如果 *n* 型树逻辑门由 *CLK* 控制, *p* 型树门用 $\overline{CLK}$ 控制, 则 *n* 型树门可以直接驱动 *p* 型树门, 反之亦然。与多米诺一样, *n* 型树门的输出在连至另一个 *n* 型树门时必须通过一个反相器。在预充电阶段 ($CLK=0$), *n* 型树门的输出 *Out1* 被充电至 V_{DD} , 而 *p* 型树门的输出 *Out2* 被预放电至 0V。由于 *n* 型树门与 PMOS 上拉器件相连, *p* 型树的 PUN 此时关断。在求值期间, *n* 型树门的输出只能进行 $1 \rightarrow 0$ 的翻转, 有条件地导通在 *p* 型树中的一些晶体管。这就保证了不会发生 *Out2* 的错误放电。同样 *n* 型树结构块连在 *p* 型树门后面不会有任何问题, 因为 *n* 型门的输入被预充电至 0V。*np*-CMOS 逻辑的缺点是由于在逻辑网络中 PMOS 管的电流驱动较弱, 所以 *p* 型树模块比 *n* 型树模块慢。要使它们的传播延时相等需要额外的面积。同样, 由于缺少缓冲器, 在门之间也存在与动态节点的连线。

第9章 组合逻辑电路设计

第一节 组合逻辑电路简介

数字逻辑电路分为组合逻辑电路（以下简称“组合电路”）和时序逻辑电路（以下简称“时序电路”）。组合逻辑电路由各种逻辑门构成, 电路在任何时刻的输出都由其当前的输入信号决定, 其逻辑功能可由一组布尔函数确定。时序逻辑电路由逻辑门和存储器构成, 其输出是当前输入和存储器状态的函数, 而存储器状态是历史输入的函数。因此, 时序电路的输出不仅依赖于其当前输入, 也依赖于其过去的输入, 其逻辑功能由输入和内部状态共同确定。

组合逻辑电路由相互连接的逻辑门构成, 逻辑门接收输入信号后产生输出信号。这个过程是将输入的二进制数据信息转换成需要的输出数据。图 9.1 所示是组合逻辑电路的框图, 其中 n 个二进制变量来自于外部信源, m 个输出变量送给外部接收者。每个输入和输出变量实质上对应于二进制信号的逻辑 1 和逻辑 0。在许多应用中, 信源和信宿都是寄存器, 如果组合电路中包含寄存器, 那么整个电路就要按时序电路对待。



图 9.1 组合逻辑电路框图

n 个输入变量有 2^n 种输入组合。对于每一种组合, 只有一种可能的输出值与其对应。把每种输入组合和与其对应的输出值列成真值表, 这样, 组合电路就能使用真值表来详细描述, 也可由 m 个布尔函数来描述, 一个函数对应一个输出变量, 每个输出是 n 个输入变量的函数。在第 2 章中, 我们学习了表达离散信息量的二进制数。二进制变量用来表示物理上的电压或其他类型信号。信号可以由数字逻辑门控制以实现特定功能。第 2 章介绍了布尔代数作为从代数上表达逻辑函数的手段, 并学习了怎样化简布尔函数以得到简单的门电路实现。本章的目的是运用前面学到的知识, 形成组合电路的系统分析和设计步骤。一些典型例子的解法将会给理解数字系统提供有用的重要基本函数样本。这里强调两项工作: (1) 分析给定逻辑电路的功能; (2) 综合实现给定功能的电路。

设计一个数字系统往往需要好几个组合电路。这些电路已经能集成为基本单元, 它们通常用来完成数字系统设计中的特定功能。在本章中, 我们介绍一些最重要的基本组合电路单元, 如加法器、减法器、比较器、译码器、编码器等。这些器件在集成电路中被归类为 MSI (中规模集成) 电路, 也作为标准单元用于复杂的 VLSI 电路, 如专用集

成电路（ASIC）。标准单元函数在 VLSI 内部的相互连接方法与其在多芯片的 MSI 电路设计中相同。

第二节 组合逻辑电路分析

组合电路的分析就是确定组合电路的实现功能。通过分析给定的逻辑图，得到一组布尔函数、真值表或电路功能的说明。如果待分析的逻辑图具有函数名或者功能实现的说明，那么分析问题就简化为对所述功能的验证。分析的执行可通过人工求出布尔函数或真值表，也可利用计算机仿真程序实现。

分析的第一步是确认给定电路是组合电路而非时序电路。组合电路只有逻辑门，没有反馈路径或存储元件。所谓反馈路径是指将第一个门的输出连接到第二个门的输入，同时第二个门的输出连接到第一个门的输入。存在反馈路径的数字电路称为时序电路，其分析有其他方法，这里不予考虑。

一旦逻辑图被确认为组合电路，就可以进一步得到其输出布尔函数或者真值表。如果该电路的功能还没有得到确定，那么就需要从布尔函数或真值表来解释该电路的功能。这个工作需要经验的积累，并对各种数字电路非常熟悉。

从逻辑图得到输出布尔函数的基本步骤如下：

1. 将所有输入变量函数的逻辑门输出用任意符号标出。确定每个门输出的布尔函数。
2. 标出输入变量函数的逻辑门，前面标注过的逻辑门采用其他任意符号。求出这些逻辑门的布尔函数。
3. 重复第 2 步的操作，直到求得电路的输出。
4. 把前面的函数重复代入，得到仅有输入变量的输出布尔函数。

以图 9.2 所示的组合电路为例来说明上述基本分析步骤。该电路有三个二进制输入变量 A 、 B 和 C ，两个二进制输出 F_1 和 F_2 。各个门的输出用中间符号标出：仅有输入变量函数的门输出有 T_1 和 T_2 ，输出 F_2 容易从输入推导出来。三个输出的布尔函数为：

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

接下来，把门的输出看作已定义符号的函数：

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

为了把 F_1 表示成 A 、 B 和 C 的函数，需要做以下一系列代换：

$$\begin{aligned}
 F_1 &= T_3 + T_2 = F_2' T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC \\
 &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\
 &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \\
 &= A'BC' + A'B'C + AB'C' + ABC
 \end{aligned}$$

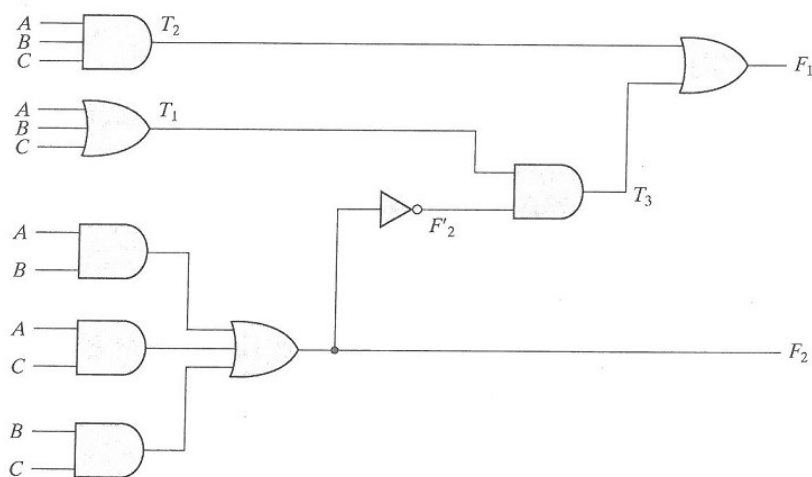


图 9.2 逻辑图分析实例

如果想进一步研究和确定这个电路实现的信息传输任务，可以根据布尔表达式画出电路图并且设法识别常见的运算。 F_1 和 F_2 的布尔函数实现电路相当于一个全加器电路，将在后面进一步讨论。

若输出布尔函数已知，依据真值表可以直接推导出电路。也可以省略布尔函数的推导，直接从逻辑电路图得到真值表，步骤如下：

1. 确定电路中输入变量的个数 n ，建立 2^n 种可能的输入组合，并把这些从 0 到 2^n-1 之间的二进制数列在一张表上。
2. 把选中门的输出用任意符号标示出来。
3. 得到那些仅有输入变量函数的门的输出真值表。
4. 继续求取先前定义过函数值的门的输出真值表，直到所有输出都确定为止。

仍用图 9.2 所示电路来举例说明这个过程。在表 9.1 中，对应于三个输入变量形成了八种可能的输出组合。 F_2 的真值表由 A 、 B 和 C 的值直接决定，有两个或三个输入等于 1 时， F_2 的值为 1。 F_2' 的真值表是对 F_2 的求反。 T_2 和 T_1 的真值表是输入变量的与、或函数。 T_3 的值由 T_1 和 F_2' 推出：当 T_3 和 F_2' 都等于 1 时， T_3 等于 1；否则 T_3 等于 0。最后， F_1 等于 1 的条件是 T_2 和 T_3 至少有一个等于 1。由真值表可得， A 、 B 、 C 、 F_1 和 F_2 组合分别与后面介绍的全加器真值表中的 x 、 y 、 z 、 S 和 C 相同。

表 9.1 图 9.2 逻辑图的真值表

A	B	C	F_2	F_2'	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

另外一种分析组合电路的方法是逻辑仿真。由于能够产生有意义输出的输入数可能非常巨大，仿真没有实际意义。但是，它在电路的功能与特性的验证方面有非常实用的价值。

第三节 组合逻辑电路设计步骤

组合电路设计根据设计目标要求，画出逻辑电路图或得到逻辑图的一组布尔函数。其步骤如下：

1. 根据电路要求，确定需要的输入数和输出数，并给每个变量分配一个符号。
2. 列出输入和输出之间关系的真值表。
3. 求输出函数的最简表达式。
4. 画出逻辑电路图，并验证设计的正确性。

真值表由输入栏和输出栏构成，输入栏包含对应于 n 个变量的 2^n 个二进制数，输出的二进制值则根据设计要求确定。真值表中定义的输出函数对组合电路做了准确描述。将设计要求正确翻译成真值表是很重要的一步，设计要求通常是不完全的，任何错误的翻译都可能导致不正确的真值表。

对真值表所列出的输出二进制函数要设法进行化简，例如代数化简法、卡诺图法或计算机化简程序等。因此，有多种不同的简化表达式可供选择。在特殊用途中，会有特殊的选择标准。在实际设计过程中，必须考虑一些约束条件，如门的个数、门的输入数、信号通过门的传播时延、连线数、每个门的驱动能力以及在集成电路设计中要考虑的各种其他标准。由于每个约束的重要程度取决于具体应用，因此很难对一个可行的电路方案进行概述。大多数情况下，首先是为满足基本目标而进行化简，例如把化简后的布尔函数表示成标准式，然后采用其他措施进一步化简，以满足其他性能标准。

码变换举例

同一个离散单元信息可以用许多不同的代码表示，这就导致了不同数字系统用的是

不同的代码。有时需要将系统的输出作为另一个系统的输入，如果这两个系统用不同的代码表示相同的信息，那么就需要在两个系统间加入变换电路。因此，码变换器是使两个系统协调工作的电路，即使这两个系统使用不同的二进制码。

将二进制码 A 变换为二进制码 B ，输入线路必须要支持代码 A 所用到的元素组合，而输出线路必须能够产生对应的代码 B 的组合。组合电路通过逻辑门来实现这个转换。

一个 n 位二进制码就是一个 n 位的比特组合，它有 2^n 种不同的 0、1 组合，每种组合都代表一个编码。一个四元素集可以用两个比特来编码，每个元素被赋予四种比特组合中的一个：00、01、10、11。八元素集需要三个比特，而十六元素集则需要四个比特。一个 n 位码所表示的二进制数从 0 到 2^n-1 。每个元素必须分配唯一的比特组合，两个元素不能有相同的编码值，否则码的分配就会产生混乱。如果要编码的元素个数不是 2 的幂次，则二进制编码会有一些码组无处分配，10 个十进制数集就是如此。区分 10 个元素的二进制码至少要有 4 位，而 4 位二进制码可以有 16 种不同的组合。因此，其中有 6 种编码没有被分配。将 4 位排列为 10 种不同的组合就能得到不同的二进制码。表 9.2 列出了两种二进制码，其中一种是十进制数最常用的码对应的二进制数分配，也就是二进制编码十进制数（二-十进制码），通常简称为 **BCD 码**。另一种是余 3 码，其每个码由对应的二进制数值加上 3 而来。

表 9.2 十进制数的两种不同的二进制码

十进制符	BCD 码	余 3 码
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

下面通过把二-十进制（BCD）码转化为余 3 码的例子来说明前述的设计步骤。

由于两种码都采用四位二进制表示一个十进制数，就必然存在四个输入变量和四个输出变量。四个输入变量记为 A 、 B 、 C 、 D ，四个输出变量记为 w 、 x 、 y 、 z 。输入和输出真值表见表 9.3。

输入码组和它们对应的输出可从表 9.2 直接得到。四个二进制变量有十六种组合，但真值表中只列出了十个，没被列出的六个输入变量的组合为“无关项”，这些值在 BCD

码中无意义，我们假设它们在实际电路中不会出现。因此，可以给输出变量随意分配 1 或 0，从而得到更简单的电路。

表 9.3 BCD 码变换举例的真值表

输入 BCD 码				输出余 3 码			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

对图 9.3 中的卡诺图画圈，可以得到输出布尔函数的最简式。每个卡诺图与电路的四个输出之一相对应，而电路是四个输入变量的函数。标 1 的方格对应输出为 1 的最小项，它们可以由真值表中的输出栏得到。例如，输出 *z* 有五个 1，因此，*z* 的卡诺图就有五个 1，每个对应一个使 *z* 等于 1 的最小项。10 到 15 这六个无关的最小项用 X 表示。每个卡诺图下面列了输出函数积之和式的最简表达式。

从化简后的布尔表达式可以直接得到每个输出的两级门电路逻辑图。可能还有一些其他的逻辑图用来实现这个电路。为了用普通的逻辑门得到两个或更多的输出，可以对图 9.3 中的表达式进行代数变换。这个操作如下所示，用以说明三级以上门电路构成多输出系统的灵活性：

$$\begin{aligned}
 z &= D' \\
 y &= CD + C'D' = CD + (C + D)' \\
 x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\
 &= B'(C + D) + B(C + D)' \\
 w &= A + BC + BD = A + B(C + D)
 \end{aligned}$$

这些表达式的实现电路如图 9.4 所示。注意，输出为 $C + D$ 的或门对三个输出函数都起作用。

不计输入的非门，积之和式的实现需要 7 个与门和 3 个或门。图 9.4 所示电路用了 4 个与门、4 个或门和 1 个非门。

如果输入只能为原变量，则第一种实现就需要对变量 *B*、*C* 和 *D* 进行取反，第二种实现则需要对变量 *B* 和 *D* 取反。

这样，三级逻辑电路就需要更多的逻辑门，所有逻辑门的输入不超过两个。

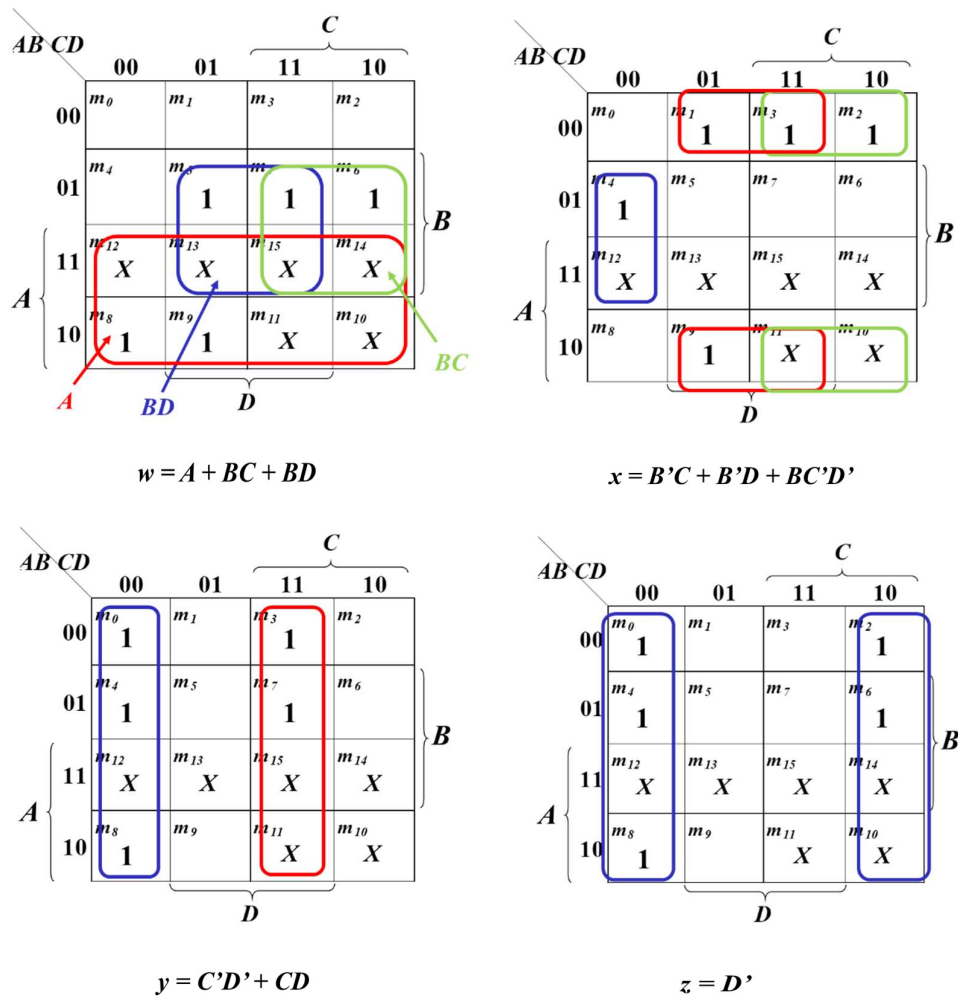


图 9.3 BCD 码到余 3 码转换电路的卡诺图

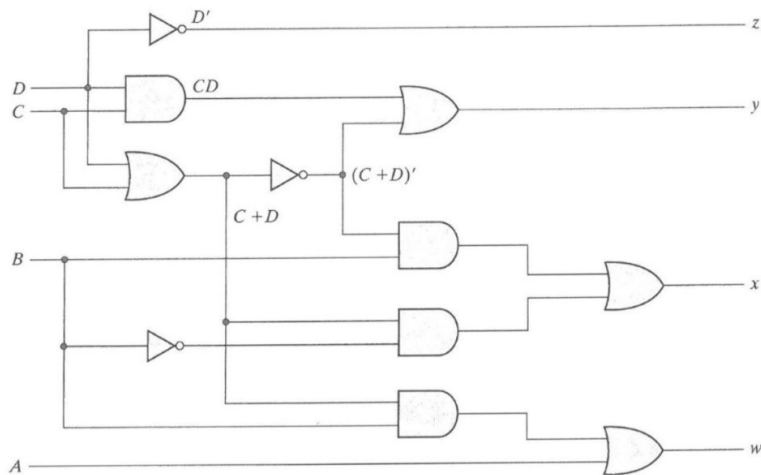


图 9.4 BCD 码到余 3 码转换器的逻辑

第四节 数值比较器

两个数的比较就是确定一个数是大于、小于还是等于另一个数，数值比较器是对两个数 A 和 B 进行比较，以确定它们相对大小的组合电路，比较结果是由三个二进制变量区分 $A > B$ 、 $A = B$ 和 $A < B$ 三种情况。

一方面，两个 n 位数比较电路的真值表有 2^{2n} 项，即使当 $n=3$ 时也十分烦琐。另一方面，比较器电路有一定的规律性，那么一个固有的、有明确规律的数学函数通常可以用算法来设计。这个算法就是详细说明解决问题的方法。我们从 4 位数值比较器的设计推导描述这种方法的算法。

对于两个 4 位二进制数： $A=A_3A_2A_1A_0$ 和 $B=B_3B_2B_1B_0$ ，如果所有对应的位都相等，则这两个数相等。每组的相等关系可以用异或函数表示：

$$x_i = A_i B_i + A_i' B_i' \quad i = 0, 1, 2, 3$$

$$(A = B) = x_3 x_2 x_1 x_0$$

只有当两个数的所有对应位都相等时，该二进制变量 $(A=B)$ 才等于 1。

为了确定 A 和 B 的大小，从最高位开始检查对应的每组数的相对大小。如果两个数相等，就比较较低一位的一组数，一直比较到数不相等为止。

$$(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$$

$$(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$$

符号 $(A > B)$ 和 $(A < B)$ 是二进制输出变量，分别在 $A > B$ 或 $A < B$ 时等于 1。 $(A = B)$ 、 $(A > B)$ 和 $(A < B)$ 三个输出变量逻辑门实现比较简单，包含了一定数量的重复元件，不相等的输出可以共用产生相等的输出的逻辑门。四位数值比较器如图 9.5 所示。

四个 x 输出由**异或非**电路产生，加到一个与门上以得到输出二进制变量 $(A = B)$ 。其他两个输出用变量 x 去产生前面列出的布尔函数组。这是一个多级实现并且有规则的方式。从这个例子可以清楚地了解获得多于四位的二进制数值比较器电路的方法。

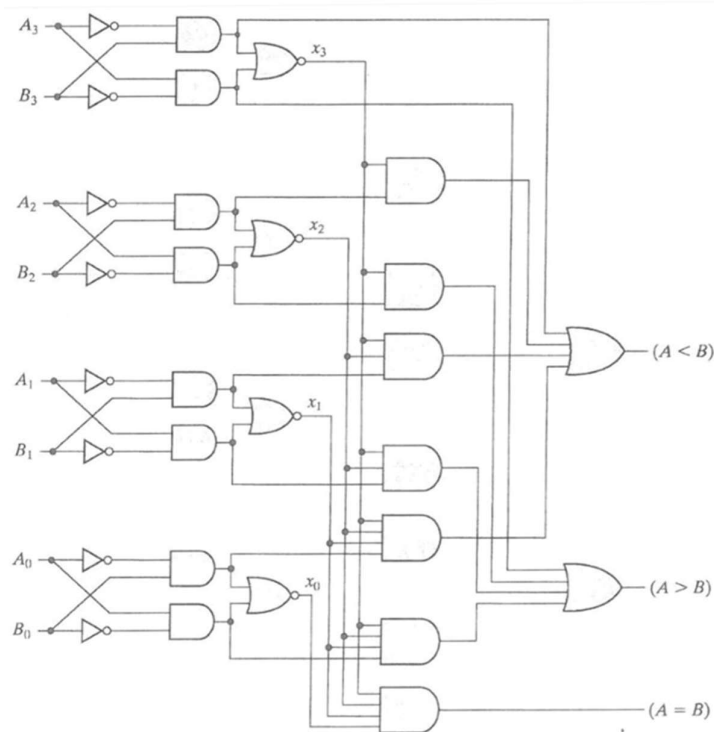


图 9.5 四位数值比较器逻辑电路图

第五节 译码器

数字系统中的离散信息量用二进制码来表示。一个 n 位二进制码可以表示 2^n 种不同的编码信息单元。译码器就是一个把来自 n 条输入线的二进制信息转化为 2^n 条输出线的组合电路。如果 n 位编码信息有没用的组合，译码器的输出可能少于 2^n 个。

上述译码器称为 n - m 线译码器，此处 $m < 2^n$ 。其目的是对 n 个输入变量产生 2^n (或小于) 个最小项，每种输入组合对应一个唯一的输出。译码器也可以用于其他关联的代码转换电路，例如 BCD-七段译码器。

以图 9.6 所示的 3-8 线译码器为例，3 个输入被译成 8 个输出，而每个输出代表 3 个输入变量的一个最小项，3 个非门提供反变量输入，8 个与门分别产生最小项中的一项。这种译码器的一个特殊应用就是二-十进制数的转换。输入变量代表一个二进制数，输出则代表了十进制系统中的 8 个数字。不管怎样，3-8 线译码器可以把任何 3 位码字译成八个输出，每个输出对应一个码字。

译码器可由与非门构成，以反相形式来产生译码器的最小项非常经济，也可以加上一个或多个使能端控制电路的工作。

一个由与非门构成的、带有一个使能输入的 2-4 线译码器如图 9.7 所示。

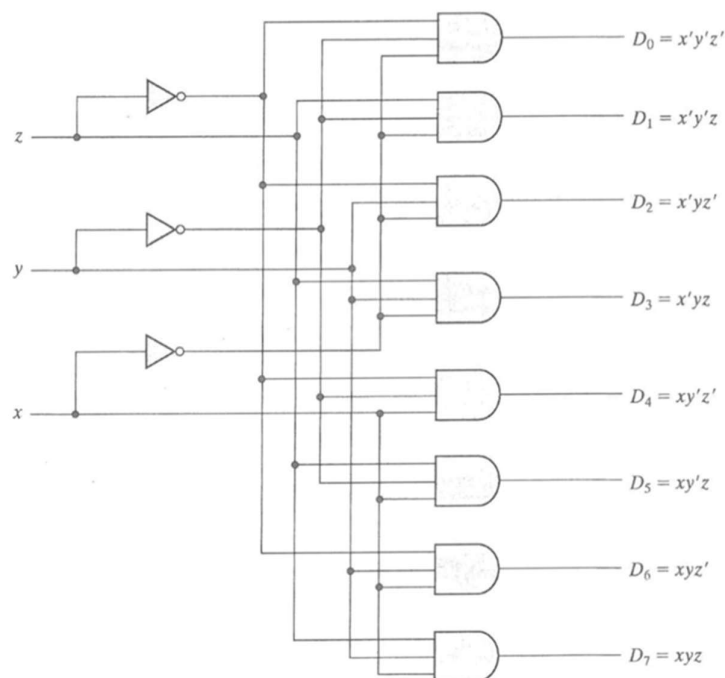


图 9.6 3-8 线译码器逻辑电路图

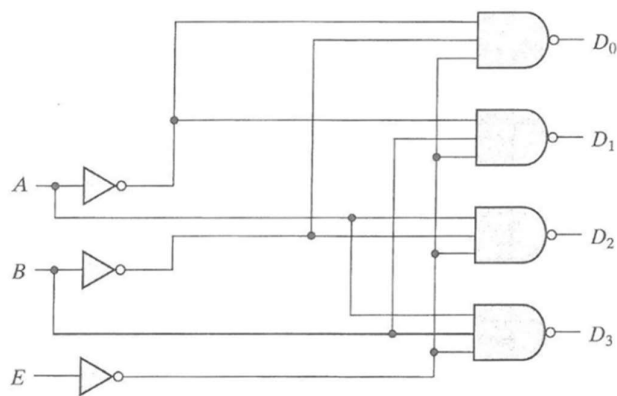


图 9.7 带使能输入的 2-4 线译码器

该电路的工作是以反相输出和低电平有效使能输入，当 E 等于 0 时译码器工作。可以看出，任何时候只有一个输出等于 0，其余所有的输出都为 1。值为 0 的输出代表了由输入 A 和 B 选择的最小项。

当 E 等于 1 时，不管其他两个输入值为何值，该电路被禁用。当此电路被禁用时，没有输出等于 0，因而没有最小项被选择。

总之，译码器可以采用反相输出也可以采用同相输出。使能输入端可用 1 或者 0 信号来激发。

有些译码器为了满足在给定逻辑条件下使电路开始工作，可以有两个或者更多的使

能输入。

带使能输入的译码器可用作数据选择器。数据选择器是一种从一条线路上接收信息，并可产生 2^n 条可能的输出线路的电路。一个特定输出的选择由 n 个选择线路的组合来控制。

当 E 作为数据输入线，而 A 、 B 作为选择输入时，图 9.7 所示的译码器可以当作 1-4 线数据选择器，唯一的输入变量 E 有到所有 4 个输出的路径，但是输入信息只由两条选择线 A 和 B 的二进制组合确定的其中一条路线输出，这一点可以由电路的真值表来验证。例如，如果选择线 $AB=10$ ，输出 D_2 就会和输入 E 的值一样，而所有其他的输出会保持在 1。因为译码器和数据选择器由相同的电路来实现，所以带使能端的译码器就称为译码器/数据选择器。

带使能端的译码器可以连在一起形成一个大型的译码器。例：两个带使能端的 3-8 线译码器连在一起构成了一个 4-16 线译码器。

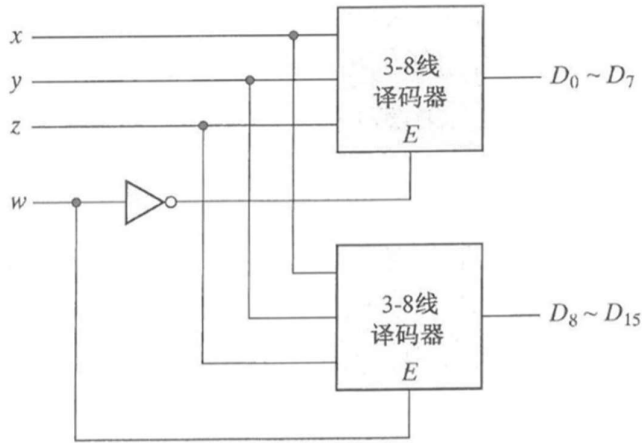


图 9.8 由两个 3-8 线译码器构成的 4-16 线译码器

当 $w=0$ 时，上面译码器工作，下面译码器不工作，下面译码器的所有输出都为 0，上面译码器产生从 0000 到 0111 的最小项。

当 $w=1$ 时，使能条件相反，下面译码器产生从 1000 到 1111 的最小项，而上面译码器的输出都是 0。

这个例子说明了使能输入在译码器以及其他组合逻辑单元中的作用。总的来说，使能输入对于连接两个或多个基本单元，以便将其扩展成为具有多个输入和输出的相同功能部件是非常有价值的。

用译码器实现组合逻辑

译码器提供 n 个输入变量的 2^n 个最小项，其输出由每一组输入唯一确定。任何布尔函数可以表示成最小项之和，就可以用译码器产生最小项，并且用外部或门来产生逻辑

辑和。用这种方法，任何有 n 输入、 m 个输出的组合电路就可以用一个 $n-2^n$ 译码器和 m 个或门来实现。

用译码器和或门实现组合电路的步骤要求：

先把电路的布尔函数用最小项之和的形式表示出来，然后选择译码器来产生输入变量的所有最小项，每个或门的输入从函数最小项译码器的输出中选择。

下面用一个全加器电路的实现来说明该步骤。

例 9.1 全加器电路的实现

采用最小项之和表示的组合电路的函数：

$$S(x, y, z) = \sum(1, 2, 4, 7),$$

$$C(x, y, z) = \sum(3, 5, 6, 7)$$

因为有 3 个输入和 8 个最小项，需要一个 3-8 线译码器，其实现如图 9.9 所示。

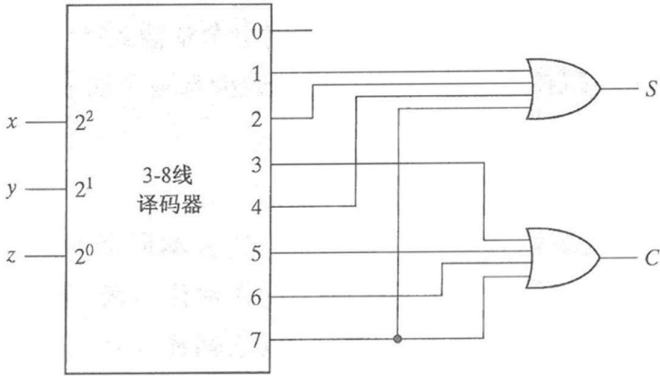


图 9.9 译码器实现组合逻辑的例子

译码器产生 x, y, z 的 8 个最小项，输出为 S 的或门产生最小项 1、2、4 和 7 的逻辑和，输出为 C 的或门产生最小项 3、5、6 和 7 的逻辑和。

有一长串最小项的函数需要一个多输入的或门。有 k 个最小项的函数可以表示为其 2^n-k 个最小项的反函数 F' 。

如果函数的最小项数目大于 $2^n/2$ ，那么 F' 可以用更少的最小项来表示。在这种情况下，用一个或非门来得到 F' 的最小项之和是很方便的。或非门的输出对这个逻辑和求反，产生正常的输出 F 。

如图 9.7 那样在译码器中使用与非门，则外部逻辑门必须是与非门而不是或门。这是因为两级与非门电路实现最小项函数的逻辑和，就相当于一个两级与或电路。

第六节 编码器

编码器是一种实现译码器反运算的数字电路，有 2^n 个输入线和 n 个输出线。例：8-3 线编码器，其真值表如表 9.4 所示，它有 8 个输入（每一组输入表示一个八进制数）和 3 个对应的二进制数输出。

假设在任何时候只有一个输入值等于 1。

编码器可以用或门来实现，其输入由真值表直接确定。当输入的八进制数是 1、3、5、7 时，输出 z 等于 1；对于八进制制数 2、3、6 或 7，输出 y 为 1；对于 4、5、6 或 7，输出 x 为 1。

表 9.4 8-3 线编码器真值表

输 入								输 出		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

表 9.4 所定义的编码器有一个局限，就是任何时候只有一个输入是有效的。如果两个输入同时有效，输入就会产生一个没有定义的组合。例如，如果 D_3 和 D_6 同时为 1，编码器输出将会是 111，三个输出都等于 1。这既不代表二进制数 3，也不代表二进制数 6。为了解决这个不确定因素，编码器电路必须建立输入优先机制，使得只有一个输出被编码。如果我们确定下标数高的输入有高的优先权，则当 D_3 和 D_6 同时为 1 时，输出将是 110， D_6 比 D_3 的优先级高。

8-3 线编码器中的另一个不确定性是，全 0 输出由全 0 输入产生；而这个输出与 D_0 等于 1 时的情况相同。这个矛盾可以用一个附加输出指示最少有一个输入等于 1 的方法来解决。

优先编码器

优先编码器是包含优先级函数的编码器电路。优先编码器中，如果有两个或多个输入同时有效，拥有最高优先级的输入将处于优先地位。

下表 9.5 是一个 4 输入优先级编码器的真值表。除了两个输出 x 和 y 以外，该电路还有第三个输出 V 。 V 是一个有效位指示符，当有一个以上的输入有效时，它就被设

定为 1。如果所有的输入无效， V 就等于 0。当 V 等于 0 时，另外的两个输出就被当成无关条件项而不用检查了。注意，输出栏中任何地方的 X 都代表无关条件，而输入栏中的 X 却对真值表的简化很有用。不用列出四个变量的所有 16 个最小项，真值表用一个 X 就代表了 1 或是 0。例如，X100 代表了两个最小项 0100 和 1100。

根据表 9.5，下标的数字越大，输入的优先级就越高。输入 D_3 具有最高的优先级，所以不管其他输入为何值，只要该输入为 1，输出 xy 就是 11（二进制数 3）。 D_2 具有第二优先权。如果 $D_3=0$ 而 $D_2=1$ ，不管其他两个低优先级的输入为何值，此时输出为 10。只有当更高优先级的输入为 0 时， D_1 对应的输出才会产生。下面以此类推。

表 9.5 4 输入优先级编码器的真值表

输 入				输 出		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

用于化简输出 x 和 y 的卡诺图如图 9.10 所示。两个输出函数的最小项是从表 9.5 推出的。尽管这个表只有 5 行，当一行中的每个 X 先后被 0 和 1 代替后，我们就得到了 16 个可能的输入组合。例如，表中第四行的 XX10 就代表了 4 个最小项 0010、0110、1010、1110。通过卡诺图化简，可以得到优先编码器的简化布尔表达式，输出 V 是所有输入变量的“或”。优先编码器的实现电路如图 9.11 所示，它对应于以下布尔函数：

$$\begin{aligned} x &= D_2 + D_3 \\ y &= D_1 D_2' + D_3 \\ V &= D_0 + D_1 + D_2 + D_3 \end{aligned}$$

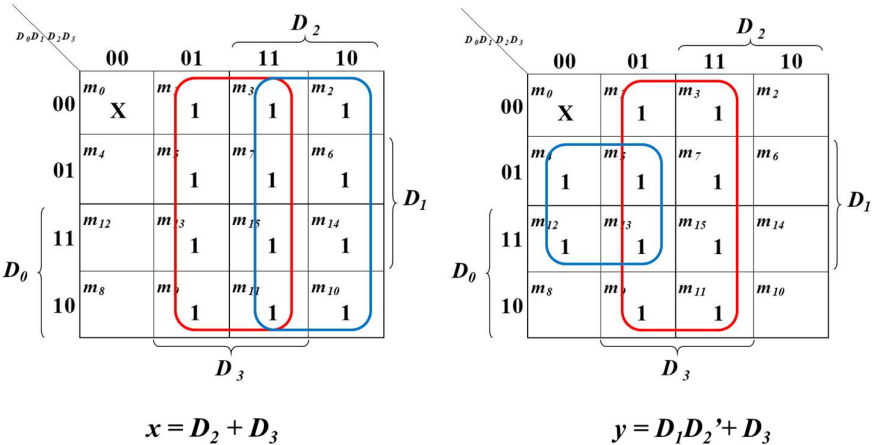


图 9.10 优先编码器卡诺图

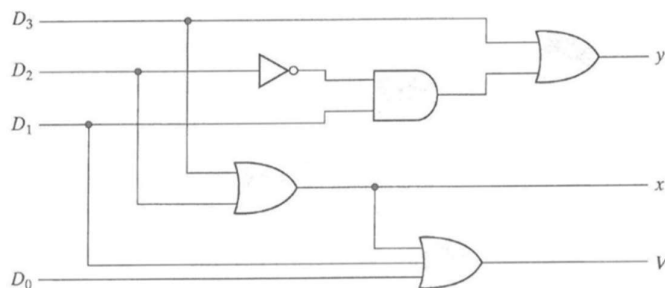


图 9.11 四输入优先编码器电路图

第七节 多路选择器

多路选择器（有时也称为数据选择器）是从多路输入线中选择其中的一路二进制信息到输出线上的一种组合电路。由一组选择线来控制，通常有 2^n 条输入线和 n 条选择线，选择线的组合决定哪条输入线被选中。

2 选 1 数据选择器是从两路二进制信源中选择一路连接到同一输出端，如图 9.12 所示。

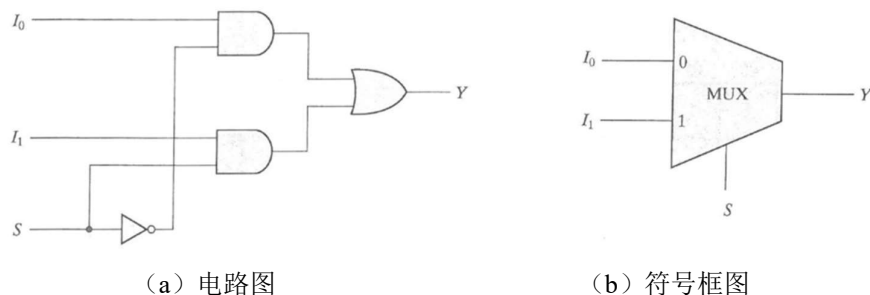


图 9.12 2 选 1 数据选择器

该电路有两条数据输入线、一条输出线，还有一条选择线 S 。当 $S = 0$ 时，上面的与门工作， I_0 连接到输出。而当 $S = 1$ 时，下面的与门工作， I_1 连接到输出。数据选择器就像个电子开关，用来选择两路信号中的一个信号，数据选择器的模块图有时用如图 9.12 (b) 所示的楔形符号表示，它直观地显示了某个被选中的多路数据源如何被传送到单个终端。数据选择器用符号 MUX 来表示。

图 9.13 (a) 所示的传输门电路常常作为 CMOS 工艺实现的多路选择器。这一电路用 4 个 MOSFET 实现路径选择逻辑（每个传输门有两个晶体管）。如果包括一对用于选择信号的缓冲反相器，那么总的晶体管数目增加到 8 个。该电路的主要问题是传输门具有寄生电阻和电容，会使响应减慢。图 9.13 (b) 为只用 NMOS 传输管的相同电路形式。包括选择信号驱动器在内，晶体管数目减少到 6 个。但在输出端增加了一个反相驱动器以补偿 NMOS 只能通过电压范围 $0V$ 至 $V_{\max}$ 的问题： $V_{\max} = V_{DD} - V_{Tn}$ 。这一反相器帮助把输出恢复至全摆幅的电压范围 $[0, V_{DD}]$ 。尽管这时晶体管数目与用传输门电路时相同，

但版图的布线将比较容易。

较大的多路选择器可以用基本门或级联 2 选 1 数据选择器来设计。4 选 1 数据选择器如图 9.14 所示。 I_0 到 I_3 的 4 个输入中每一个都接到一个与门的输入端。选择线 S_1 和 S_0 用来选择某一特定的与门。各与门的输出加到一个提供单线输出的或门。函数表中列出的是根据选择线的不同二进制组合传送至输出的数据输入。为了描述该电路的操作，考虑 $S_1S_0 = 10$ 的情况。从图 9.14 可知， I_2 所连的与门有两个输入等于 1，第三个输入为 I_2 ，其他的三个与门至少有一个输入等于 0，其输出就等于 0，此时或门的输出就等于 I_2 的值，提供了一条从选中的输入到输出的路径。

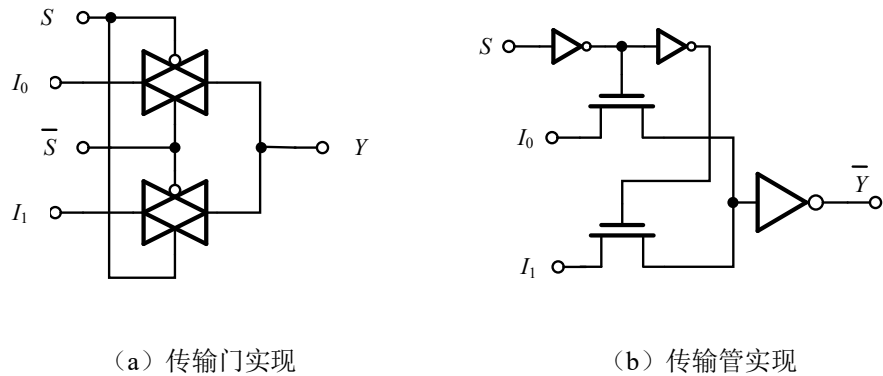


图 9.13 用传输门和传输管实现 2 选 1 多路选择器

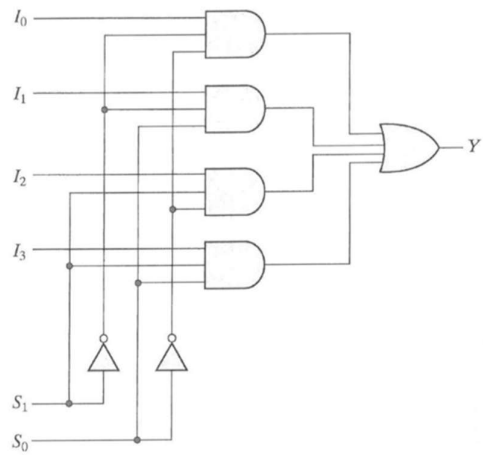


图 9.14 4 选 1 数据选择器电路图

数据选择器中的与门和非门实际上类似于译码器电路，它们对所选的输入线进行译码。

总之，一个 2^n 选 1 的数据选择器由一个 $n-2^n$ 译码器加上 2^n 个输入线构成，每个输入线接到一个与门上，与门的输出再接到一个或门上。数据选择器的规模由 2^n 个数据输入线和一个输出线决定。 n 个选择线就意味着有 2^n 个数据线。如译码器那样，数据选择器可以有一个使能输入来控制该部件的功能。当使能输入无效时，输出被禁止；而

当使能输入有效时，电路的功能就是一个正常的数据选择器。

数据选择器电路可以通过选择输入的组合来提供多位选择逻辑。例如，一个四位 2 选 1 数据选择器如图 9.15 所示。

该电路有四个数据选择器，每个可以选择两个输入线中的一个。输出 Y_0 可以选择 A_0 或 B_0 。类似地，输出 Y_1 的值可能是 A_1 或 B_1 ，依此类推。

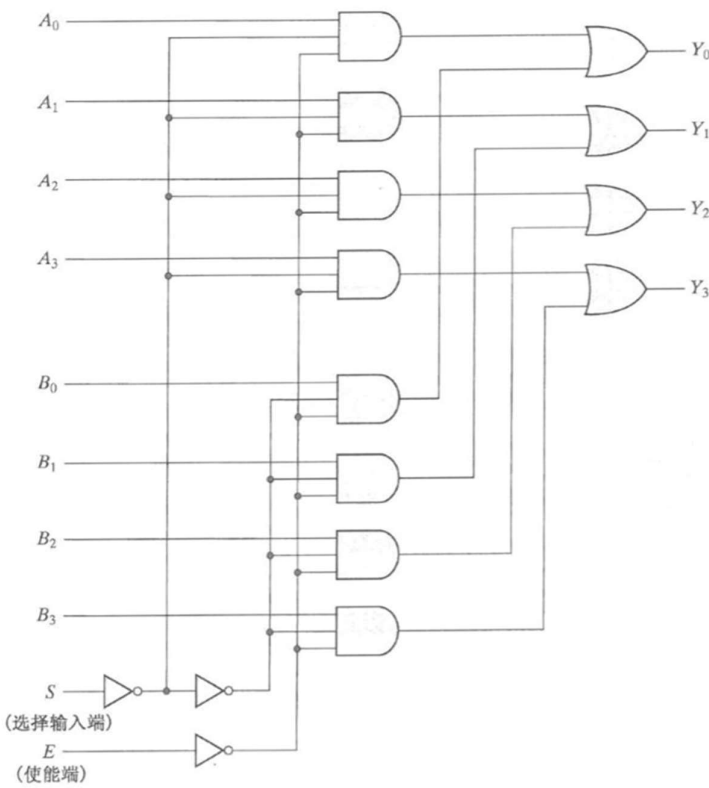


图 9.15 四位 2 选 1 数据选择器电路

输入选择线 S 选择 4 个数据选择器中的一组输入。正常工作时，使能输入 E 必须处于有效状态。尽管该电路包含了 4 个二选 1 数据选择器，但我们更喜欢把它视为一个从两组 4 位数据中选择一组数据的电路。当 $E=0$ 时，该单元工作，若 $S=0$ ，则 A 数据的 4 个输入有一条通道通向 4 个输出。另一方面，若 $S=1$ ，则 B 数据的 4 个输入被传送到输出。而当 $E=1$ 时，不管 S 为何值，输出都为 0。

用数据选择器实现布尔函数

前面介绍过译码器外加或门可以用来实现布尔函数。由数据选择器逻辑图可以看出，数据选择器实质上就是一个带或门的译码器。在数据选择器中，函数的最小项由与选择输入相连的电路产生，单个最小项可以由输入数据来选择。

这里提供一种方法，可以用一个有 n 个选择输入和 2^n 个数据输入的数据选择器来实现布尔函数，其中每个数据输入对应一个最小项。

另一种更有效的方法是，用一个有 $n-1$ 个选择输入的数据选择器来实现 n 个变量的布尔函数。函数的前 $n-1$ 个变量连到数据选择器的选择输入端，剩下的一个变量用作数据输入。

如果是单个变量 z ，那么数据选择器的每个数据输入将会是 z 、 z' 、1 或 0。例如下面的三变量布尔函数：

$$F(x, y, z) = \sum(1, 2, 6, 7)$$

该三变量函数可以用图 9.16 所示的 4 选 1 数据选择器来实现。

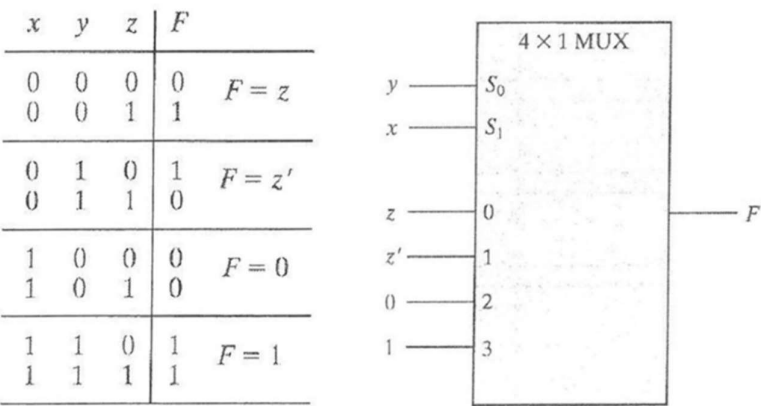


图 9.16 用数据选择器实现三变量布尔函数

两个变量 x 和 y 按顺序提供给选择线； x 连接到 S_1 输入端， y 接到 S_0 输入端。数据输入线的值由函数真值表决定。当 $xy=00$ 时，输出 F 等于 z ，这是因为当 $z=0$ 时， $F=0$ ，而当 $z=1$ 时， $F=1$ 。这就要求 z 变量被赋给数据输入 0。该数据选择器的操作是这样的：当 $xy=00$ 时，数据输入 0 有一条路径通到输出并且使 F 等于 z 。相同的方式下，可以根据当 $xy=01$ 、10 和 11 时， F 的值各不相同来确定所需的输入数据线 1、2 和 3。这个特殊的例子描述了数据输入中可以获得的所有四种可能。

用 $n-1$ 个选择输入和 2^{n-1} 个数据输入的数据选择器实现任意 n 个变量的布尔函数，一般步骤可参照前面的例子。

- 首先列出布尔函数的真值表，真值表的前 $n-1$ 个变量被用作数据选择器的选择输入；
- 对于输入变量的每个组合，把输出当作最后一个变量的函数，这个函数可以是 0、1、原变量、反变量；
- 将这些值以正确的顺序加到数据输入端 D 。

例如下面的布尔函数：

$$F(A, B, C, D) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$$

该函数用一个 3 选择输入的数据选择器来实现，如图 9.17 所示。

注意，第一个变量 A 必须接选择输入 S_2 ，这样 A 、 B 和 C 就分别对应选择输入 S_2 、 S_1 和 S_0 。

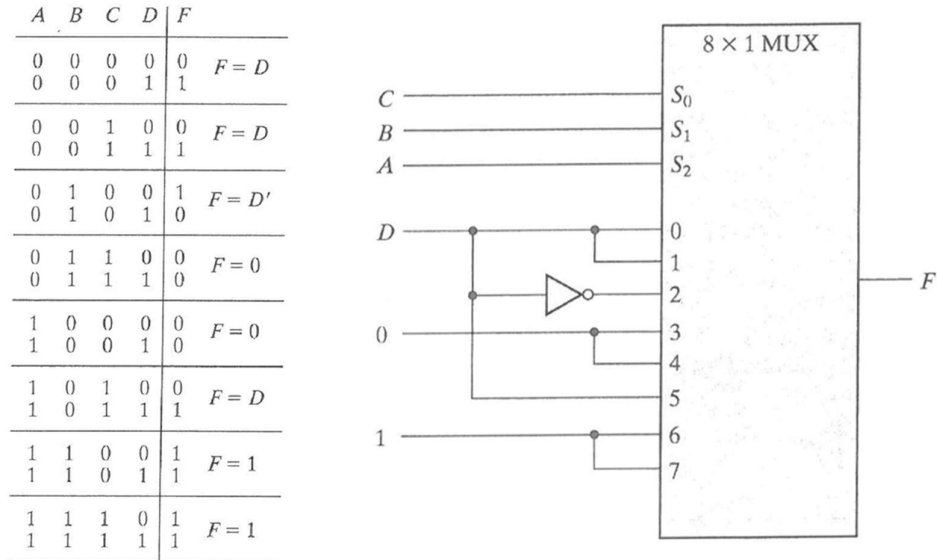


图 9.17 用数据选择器实现 4 输入布尔函数

数据输入线所接的输入由真值表列出的数字决定。相应的数据线由 ABC 的二进制组合决定。例如，当 $ABC=101$ 时，表中显示 $F=D$ 。这样输入变量 D 就提供给数据输入线 D_5 。

二进制常数 0 和 1 对应两个固定的信号值，当用于集成电路时，逻辑 0 对应于信号地，而逻辑 1 相当于电源信号（比如 3V），具体取决于电路的工艺技术。

三态门

数据选择器可以用三态门来构造。三态门是一种呈现三种状态的数字电路，其中的两种状态相当于传统的逻辑门的逻辑 1 和 0，第三种状态是高阻态。

高阻态呈现为开路，也就是输出表现为无连接，该电路此时没有逻辑意义。

三态门可以用来实现任何一个普通逻辑门，如与门和与非门。不过，最常用的还是用作缓冲门。

三态缓冲器的图形符号如图 9.18 所示，它通过在门的图标底部加上一条输入控制线来和普通的缓冲器相区别。

该缓冲器有一个数据输入 A 、一个输出 Y 和一个决定输出状态的控制输入 C 。当控

制输入 C 等于 1 时，输出使能，该逻辑门作为普通的缓冲器，输出 Y 就等于正常的输入 A 。当控制输入 C 为 0 时，输出无效，不管正常输入为何值，该逻辑门输出处于高阻态。

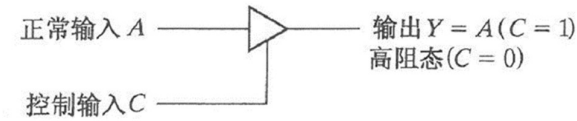


图 9.18 三态缓冲器图形符号

三态门的高阻态提供了一种其他门没有的特性。因为这个特性，可以将大量的三态门输出用线连在一起形成一个共用线。

用三态门组成的数据选择器结构如图 9.19 所示。图 9.19 (a) 部分显示了用两个三态门和一个反相器构成的 2 选 1 数据选择器的结构。

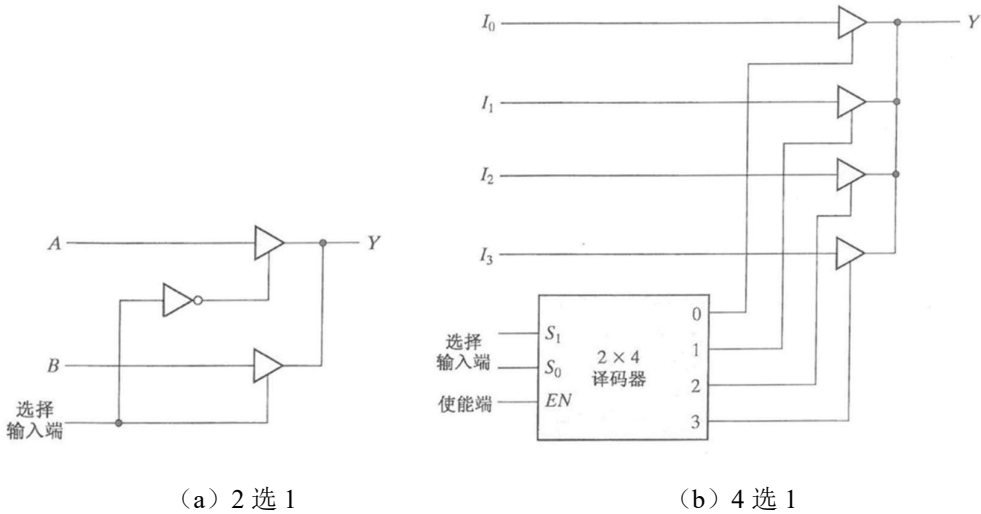


图 9.19 用三态门构成的数据选择器

两个输出被连接在一起以形成单条输出线（通常非三态输出逻辑门不能做这种连接）。当选择输入为 0 时，上面的缓冲器由输入控制使能，而下面的缓冲器无效，输出 Y 等于输入 A ；当选择输入为 1 时，下面的那个缓冲器使能，输出 Y 等于 B 。

4 选 1 数据选择器结构如图 9.19 (b) 所示。四个三态门的输出连在一起形成单一输出线。缓冲器的控制输入决定四个普通输入 I_0 到 I_3 其中之一将与输出线相连。任一时间，起作用的缓冲器数不会超过一个。已连接的缓冲器必须得到控制，以便只有一个三态缓冲器通向输出，而所有其他缓冲器均保持在高阻态。为确保在任何时候只有一个控制输入有效的方法是用译码器。当译码器的使能输入为 0 时，它的四个输出全是 0，因为所有的四个缓冲器都被禁止，所以输出总线呈现高阻态。当使能输入有效时，译码器的选择输入值将确定其中一个三态缓冲器有效。仔细研究将会发现，该电路以另外一

种方式构造了一个 4 选 1 数据选择器。

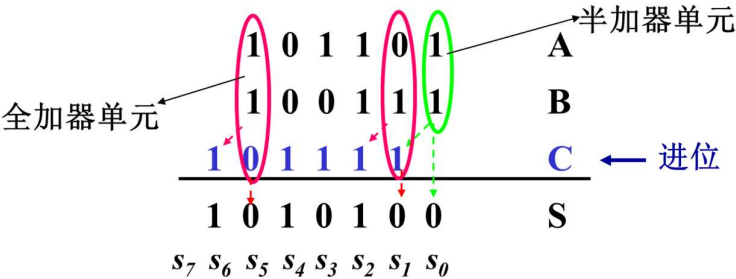
第八节 加法器设计

9.8.1 二进制加法器

数字计算机能实现多种不同的信息处理任务。在遇到的函数中有多种多样的数学运算，绝大多数基本的数学运算是两个二进制数的加法。这个简单加法有四种可能的基本算式： $0+0=0$ 、 $0+1=1$ 、 $1+0=1$ 和 $1+1=10$ 。前三种算式产生的和是一位数，但是当加数和被加数都为 1 时，二进制数的和就由两位数组成，运算结果的最高有效位被称为进位。若加数和被加数包含了多个有效数字，则两个数相加后的进位就被送至相邻的高位。

$$\begin{array}{r} \text{被加数:} \quad 101101 \\ \text{加数:} \quad + \quad 100111 \\ \hline \text{和:} \quad 1010100 \end{array}$$

二进制加减器是实现二进制数加法和减法运算的组合电路。其中实现两个二进制比特相加的组合电路称作半加器（Half-Adder, HA）；实现三位二进制数（两个有效数和一个进位输入）加法的组合电路是全加器（Full-Adder, FA）。这样命名是因为两个半加器组合在一起可以实现一个全加器的功能。



下面，首先进行半加器的设计，然后再将其升级到全加器。将 n 个全加器进行级联，可实现 n 位二进制加法器。

半加器（HA）

实现两位二进制数加法，需要两个二进制输入和两个二进制输出。两个输入记为 x 和 y ，两个输出记为 S （和）和 C （进位）。半加器的真值表列在表 9.6 中。只有当两个输入全为 1 时，输出 C 才是 1。输出 S 代表和的最低有效位。两个最简输出布尔函数可由真值表直接得到：

$$\begin{aligned} S &= x'y + xy' \\ C &= xy \end{aligned}$$

半加器的积之和式逻辑实现如图 9.20 (a) 所示。它也可以用一个异或门和一个与门来实现，如图 9.20 (b) 所示。这种实现方式显示了两个半加器可以构成一个全加器。

表 9.6 半加器真值表

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

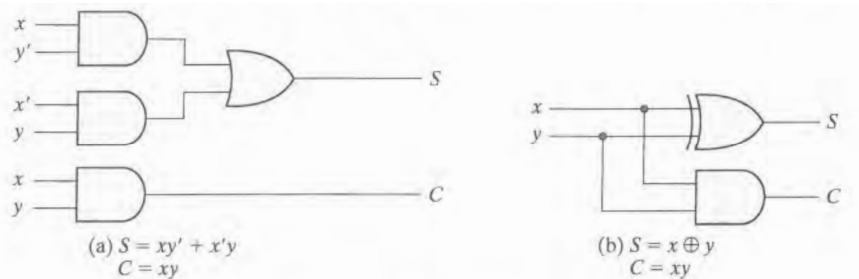


图 9.20 半加器的实现电路

全加器 (FA)

全加器实现 3 位二进制数算术和，它有 3 个输入和两个输出。两个输入变量 x 和 y 分别代表两个待加的有效位，第三个输入变量 z 代表来自低有效位上的进位；输出必须有两个，因为三个二进制数算术和的范围是 0 到 3，而二进制数 2 或 3 的表示则需要两位。两个输出变量中符号 S 代表和，符号 C 代表进位。 S 给出了和的最低有效位值， C 给出了输出进位值，该进位由当前位与前一位的进位求和产生。全加器的真值表列在表 9.7 中。输入变量栏里的八行给出了输入的三个变量所有可能的组合，输出变量由输入位的算术和决定。当输入全为 0 时，输出也是 0。当仅有一个输入等于 1 或者三个输入都等于 1 时，输出 S 等于 1。如果两个或者三个输入等于 1，输出进位 C 等于 1。

表 9.7 全加器的真值表

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

全加器两个输出布尔函数经化简后可得到：

$$\begin{aligned}
 S &= x'y'z + x'yz' + xy'z' + xyz \\
 C &= xy + yz + xz
 \end{aligned}
 \tag{9.1}$$

全加器逻辑图积之和的实现结果如图 9.21 所示。

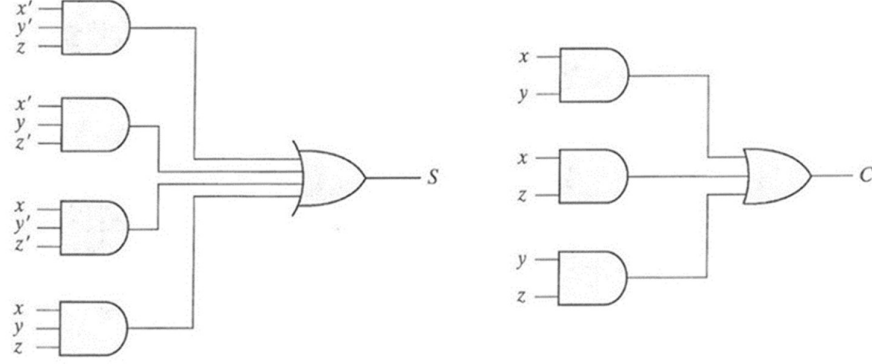


图 9.21 积之和式全加器的实现

全加器电路也可用两个半加器和一个或门实现，如图 9.22 所示。第二个半加器的输出 S 由 z 和第一个半加器的输出相异或而得：

$$\begin{aligned}
 S &= z \oplus (x \oplus y) = z'(x'y + xy') + z(x'y' + xy) \\
 &= x'y'z + x'yz' + xy'z' + xyz \\
 C &= z(x'y + xy') + xy = x'yz + xy'z + xy
 \end{aligned}
 \tag{9.2}$$

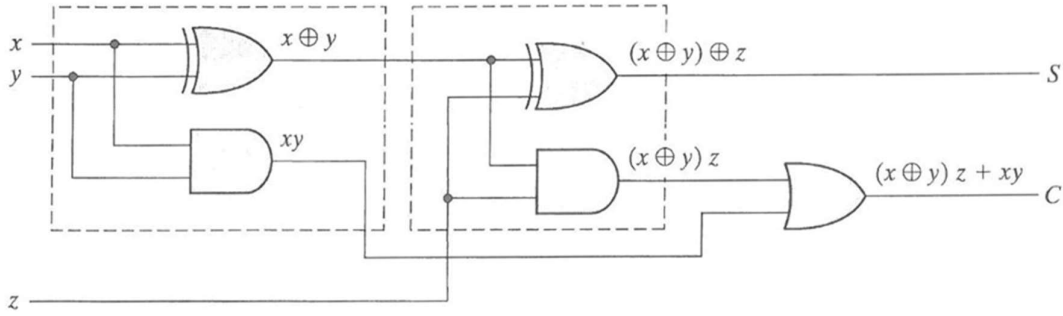


图 9.22 用两个半加器和一个或门实现的全加器

二进制加法器

二进制加法器是用来产生两个二进制数算术和的数字电路。它可以通过全加器的级联来实现，即每个全加器的进位输出和下一个全加器的一个输入相连，形成链状结构。 n 位二进制数加法需要 n 个全加器级联，或一个半加器和 $n-1$ 个全加器级联实现。在前者情况下，最低有效位上的输入进位需要置 0。图 9.23 所示为 4 个全加器级联而成的 4 位逐位进位加法器或行波进位加法器 (*ripple-carry adder*)，被加数 A 和加数 B 由带下标的字母从右到左指定，下标为 0 就代表最低位。全加器之间的进位连接成链状。加法器的输入进位为 C_0 ，它依次穿过四个全加器到达输出进位 C_4 。输出 S 产生需要的二进

制和。每个全加器的输出进位连接到高位全加器的输入进位上，这样 n 个全加器就能构成一个 n 位加法器。

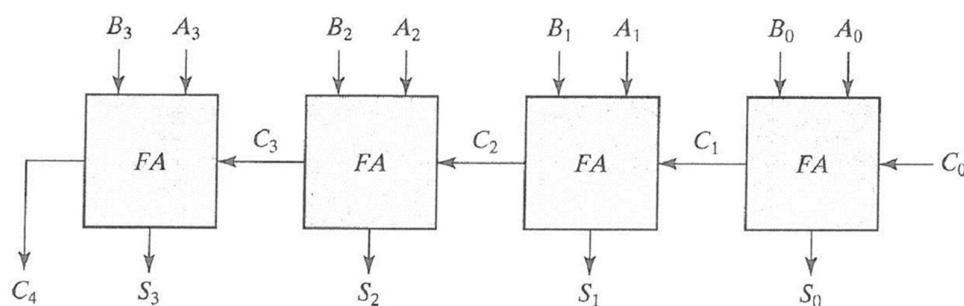


图 9.23 四位加法器

并行的两个二进制数加法是各位加数与被加数同时计算。对于任何组合电路，信号必须经过逻辑门的传输才能在输出端得到正确的运算结果。总的传输时间等于一个典型门的延时乘以电路中逻辑门的级数。加法器中最长的传播延时是进位通过各全加器所用的时间。既然输出结果的每一位都与输入进位有关，那么只有当输入进位已经传输到该级，全加器任何一级的 S_i 值才是稳定的。例如图 9.23 中的输出 S_3 ，输入信号一加到加法器上，输入 A_3 和 B_3 就有效。然而直到前一级进位 C_2 送到，输入进位 C_3 才能稳定在最终的值。以此类推， C_2 要等 C_1 ，一直等到 C_0 。这样，只有在进位传输穿过各级电路后，最终的输出 S_3 与进位 C_4 才能稳定在正确的值。

通过该电路的延时取决于传播必须通过的逻辑级的数目并且与所加的输入信号有关。对于某些输入信号完全不会发生行波进位效应，而对于另一些信号，进位必须从最低有效位 (*least significant bit, LSB*) 一直波动到最高有效位 (*most significant bit, MSB*)。这样一个结构（也称为关键路径，*critical path*）的传播延时定义为对所有可能的输入图形在最坏情形下的延时。

在逐位进位加法器中，最坏情形的延时发生在当最低有效位上产生的进位一直全程传播到最高有效位时。这一进位最终在最后一级上被吸收以产生和。因此，延时正比于输入字的位数 N 并近似为：

$$t_{adder} \approx (N-1)t_{carry} + t_{sum} \quad (9.3)$$

式中， t_{carry} 和 t_{sum} 分别等于全加器中从输入进位 (C_i) 至输出进位 (C_o) 及和 (S) 的传播延时。

进位传输经过门的级数可以从全加器电路中得到。为方便起见，我们把电路重绘于图 9.24 中。输入和输出变量用下标 i 代表加法器的 i 级。 P_i 和 G_i 信号在传输过程中要保持稳定，这两个信号对所有的全加器来说仅仅由输入的二进制加数与被加数来确定。从输入进位 C_i 到输出进位 C_{i+1} 的信号传输经过了一个与门和一个或门，构成了两级门电路。如果加法器里有四个全加器，则输出进位 C_4 必须穿过 C_0 和 C_4 之间的 $2 \times 4 = 8$ 级

门电路。对于 n 位加法器，从输入进位到输出进位需要穿过 $2n$ 级门电路。

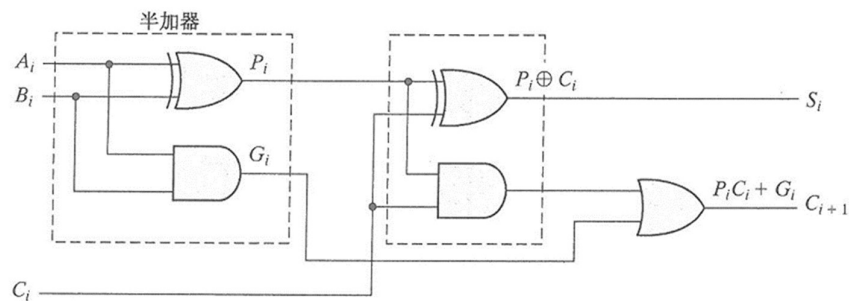


图 9.24 带有 P 和 G 的全加器

考虑图 9.24 所示的全加器电路，如果我们定义如下二进制变量

$$G_i = A_i B_i$$

$$D_i = \bar{A}_i \bar{B}_i$$

$$P_i = A_i \oplus B_i \quad (9.4)$$

输出的和与进位可表示为：

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i \quad (9.5)$$

G_i 称为进位产生，不管输入进位 C_i 为何值，当 A_i 和 B_i 都为 1 时，它产生一个进位。 P_i 称为进位传播，因为它与从 C_i 到 C_{i+1} 的进位传播有关。 D_i 称为进位取消，不管输入进位 C_i 为何值，当 A_i 和 B_i 都为 0 时，将取消一个进位。

例 9.2 逐位进位加法器的传播延时

推导 A_k 和 B_k ($k = 0, \dots, N-1$) 的值以得到逐位进位加法器最坏情形下的延时。最坏情形是在 LSB（最低有效位）上产生了一个进位，由于第一个全加器的输入进位 C_0 总是 0，这意味着 A_0 和 B_0 必须同时等于 1。其他所有各级必须处于传播模式。因此， A_i 或 B_i 必为高电平。最后，我们可以实际度量一下在 MSB（最高有效位）的和位上发生翻转的延时。假设 S_{N-1} 的初值为 0，我们必须安排一个 $0 \rightarrow 1$ 的翻转。这可以通过使 A_{N-1} 和 B_{N-1} 同时为 0（或同时为 1）来实现，于是在输入进位为 1 时就产生一个高电平的和位。

例如，以下 A 和 B 的值将导致一个 8 位加法最坏情形下的延时：

$$A: 00000001; B: 01111111$$

为了得到一个延时最长的翻转，可以使所有的输入在 A_0 由 $0 \rightarrow 1$ 翻转时保持为常数。

最左边的位代表这个二进制表达式中的最高有效位。注意，这只是许多最坏情形中的一个。这一情形使最终的和经历 $0 \rightarrow 1$ 的过渡延时。

进位传输时间对两个数相加的速度是限制因素。尽管加法器或所有组合电路在其输出端总会有一些值，但是如果不能给予足够的时间让信号传输通过输入和输出之间的门电路，其输出就不会正确。既然所有其他的数学运算都可以用连续加法来实现，因此加法过程中的耗时就很关键。减少进位时延的一个有效方法就是使用速度快、时延小的逻辑门电路。然而，电路的物理性能总是有限的。另一个解决办法是提高电路的复杂度以减少进位时延。目前，已有多种技术可以降低并行加法器的进位传播时延。

从公式 (9.3) 中可以得出两个重要结论：

- 逐位进位加法器的传播延时与 N 成线性关系。在设计当前和未来计算机所希望的宽数据通路 ($N = 16, \dots, 128$) 加法器时这一点变得日益重要。

- 在设计一个快速逐位进位加法器的全加器单元时，优化 t_{carry} 比优化 t_{sum} 重要得多，因为后者对 t_{adder} 总延时值的影响较小。

在深入讨论全加器单元的电路设计之前，值得提及的是全加器的另一个逻辑性质：

把一个全加器的所有输入反相，它的所有输出也反相。

这一性质也称为加法器的反相特性，可用下列两个公式来描述：

$$\begin{aligned}\bar{S}(A, B, C_i) &= S(\bar{A}, \bar{B}, \bar{C}_i) \\ \bar{C}_o(A, B, C_i) &= C_o(\bar{A}, \bar{B}, \bar{C}_i)\end{aligned}\quad (9.6)$$

这在优化逐位进位加法器的速度时非常有用。它说明图 9.25 左右两个电路（在逻辑关系上）完全等效。

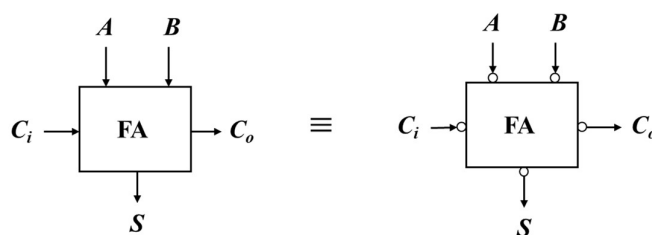


图 9.25 全加器的反相特性（图中的小圆圈代表反相）

9.8.2 全加器：电路设计考虑

静态加法器电路

实现全加器电路的一种方法是把逻辑方程 (9.1) 直接转变为互补 CMOS 电路。进行某些逻辑变换可以帮助减少晶体管的数目。例如，只要不减慢进位产生的速度（如前所述进位产生是最关键的部分）则在“和”与“进位产生”的子电路之间共享某些逻辑非常

有利。以下是这样一个重新组织方程组的例子：

$$C_o = AB + BC_i + AC_i \quad (9.7)$$

和

$$S = ABC_i + \bar{C}_o (A + B + C_i)$$

很容易验证它与原方程组等价。相应的采用互补静态 CMOS 的加法器设计见图 9.26，它需要 28 个晶体管。除了消耗较大的面积而外，这一电路的速度也较慢。因为：

- 在进位产生与和产生电路中堆叠着许多 PMOS 管。
- C_o 信号的本征负载电容很大，包括两个扩散电容、6 个栅电容，再加上连线电容。
- 在进位产生电路中信号传播通过两个反相级。正如前面提到的，最小化进位路径的延时是高速加法器电路设计者的首要目标。如果进位链输出上的负载（扇出）很小，那么有两个逻辑级就显得太多，因而引起额外的延时。
- 和的产生要求一个额外的逻辑级，但这并不那么重要。因为这一项在逐位进位加法器的传播延时公式（9.3）中只出现一次。

这一电路尽管慢，但却体现了某些聪明的设计技巧。注意，在进位产生电路第一个门的设计中 C_i 信号放在串联数较少的 PMOS 管支路上从而使它的逻辑努力降低为 2。同时，连接到 C_i 的 NMOS 和 PMOS 管尽可能地放在靠近这个门的输出端。这直接应用了 6.4.2 节中讨论过的电路优化技术——关键路径上的晶体管应当尽可能放在接近门的输出端。例如在加法器的第 k 级，信号 A_k 和 B_k 在 $C_{i,k} (= C_{o,k-1})$ 到达之前就已经存在并已稳定了很长时间。因此，在晶体管链中内部节点的电容事先已被预充电或放电。当 $C_{i,k}$ 到达时，只有节点 X 的电容必须充（放）电。如果把晶体管放在靠近 V_{DD} 和 GND 处，则不仅需要充（放）电节点 X 的电容，而且也要充（放）电内部节点的电容。

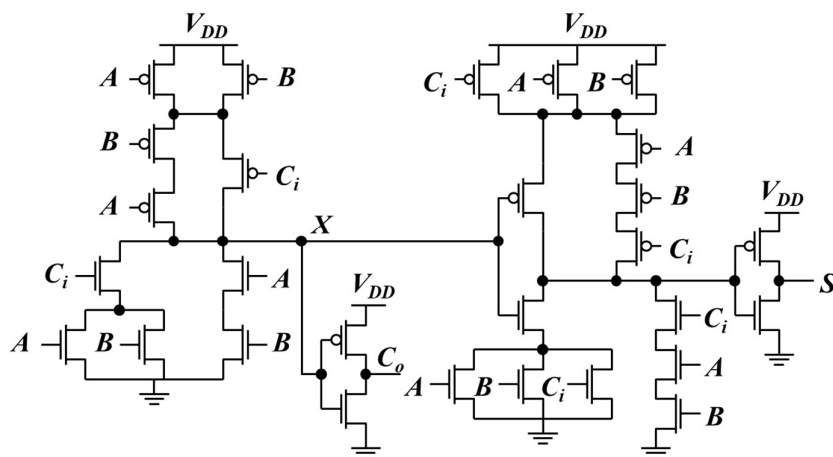


图 9.26 互补静态 CMOS 实现的全加器

现在，这一电路的速度可以利用前面讨论过的加法器的某些性质逐步加以改善。首先，利用加法器的反相特性，即把一个全加器单元的所有输入反相则它的所有输出也反相，这可以减少进位路径中反相级的数目。如图 9.27 所示，这一特性允许我们在一个进位链中消除反相器。

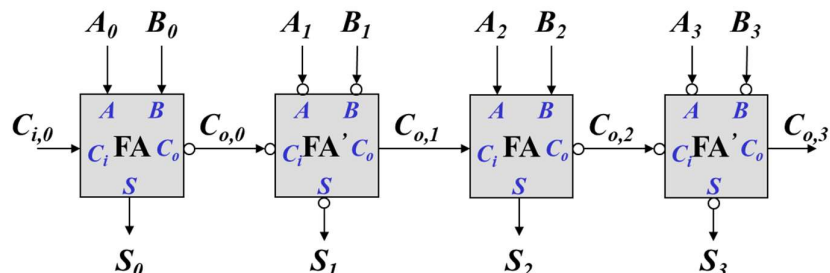
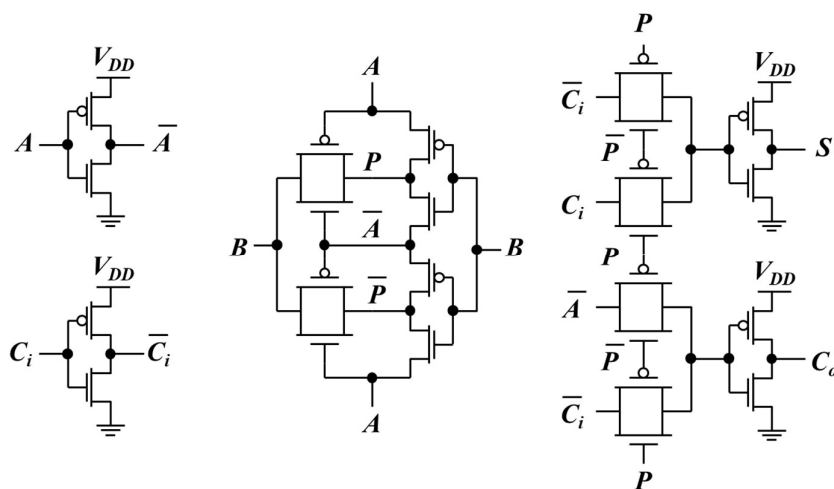


图 9.27 在进位路径中取消反相器（FA' 表示进位路径中没有反相器的全加器）

镜像加法器设计

图 9.28 是一个经过改进的加法器电路，也称为镜像加法器（*mirror adder*）。它的工作基础是公式 (9.5)，但它的进位产生电路值得分析。首先，如前面所建议的，取消了进位反相门。其次，门的 PDN 和 PUN 网络不再是对偶的，而是巧妙地实现了进位传播/产生/取消功能——当 D 或者 G 为高时， $\bar{C}_o$ 分别被置为 V_{DD} 或 GND 。满足进位传播条件时（即 $P=A+B$ 为 1）输入进位（以反相的形式）传播到 $\bar{C}_o$ 。这一结构使面积和延时都有相当程度的减少。以下一些事实值得考虑：

- 该全加器单元仅需要 24 个晶体管。
- NMOS 和 PMOS 链完全对称，由于求和与进位功能的自对偶性（*self-duality*）它同样能产生正确的功能。但这一结果使在进位产生电路中最多只有两个晶体管串联。
- 连接 C_i 的晶体管放在最接近门的输出端处。
- 只有在进位电路中的晶体管才需要优化尺寸以改善速度。在求和电路中的所有晶体管都可以采用最小尺寸。当设计该单元的版图时，最关键的问题是使节点 $\bar{C}_o$ 处的电容最小。共享扩散区可以减少堆叠（晶体管）的节点电容。
- 在图 9.26 的加法器单元中，可以独立确定反相器的尺寸来驱动下一级加法器的 C_i 输入。如果图 9.28 中的进位电路尺寸对称，则其每一个输入的逻辑努力为 2。这意味着优化尺寸以达到最小延时的最优扇出数应当为 $(4/2)=2$ 。然而，这一级的输出驱动两个内部的栅电容和 6 个所连下一级加法器单元的栅电容。保持每一级晶体管尺寸相同的一个巧妙办法是把进位级的尺寸增加至大约为求和级尺寸的 3 至 4 倍。这仍保持优化扇出为 2。所得到的晶体管的尺寸标在图 9.28



曼彻斯特 (Manchester) 进位链加法器

图 9.29 中的进位传播电路可以通过增加进位产生和进位消除信号来简化, 如图 9.30 (a) 所示。进位传播路径不充电, 如果进位传播信号 ($A_i \oplus B_i$) 为真, 则 C_i 被传送到输出 C_o 。如果传播条件不满足, 则输出或者由信号 D_i 下拉或者由 $\bar{C}_i$ 上拉。动态实现[见图 9.30 (b)]甚至可以使电路更加简单。由于动态电路中的晶体管是单方向工作的, 所以可以只用 NMOS 传输管来代替传输门。预充电输出使电路不再需要进位取消信号 (对于进位链传播的是进位信号的反信号时的情形)。

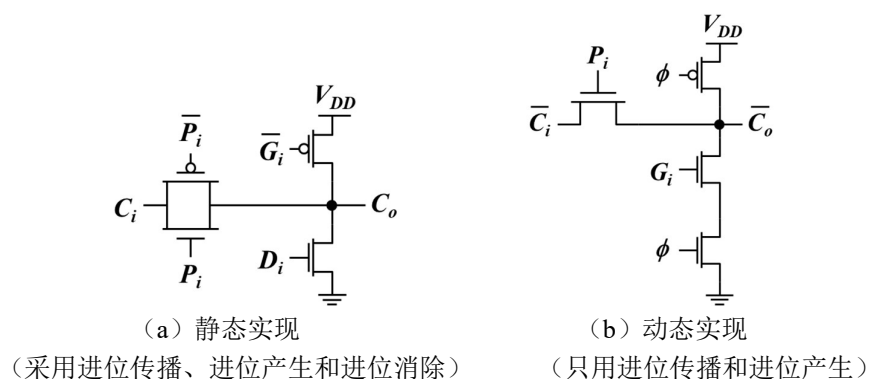


图 9.30 曼彻斯特进位门

曼彻斯特进位链加法器用串联的传输管来实现进位链。图 9.31 是以图 9.30 介绍的动态电路为基础的一个例子。在预充电阶段 ($\phi = 0$), 传输管进位链中的所有中间节点都被预充电至 V_{DD} 。在求值阶段, 当有输入进位且传播信号 P_k 为高电平时, 或者当第 k 级的进位产生信号 (G_k) 为高电平时, 节点 $\bar{C}_k$ 放电。

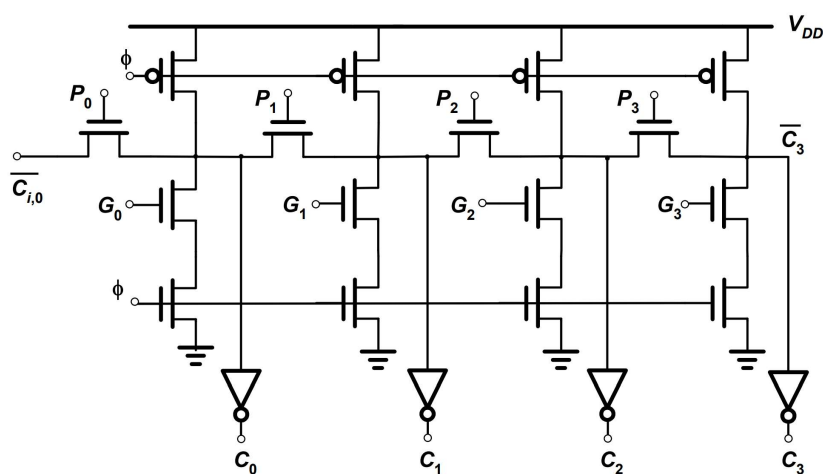


图 9.31 以动态逻辑实现的曼彻斯特进位链加法器 (四位部分)

将图 9.31 中加法器进位链在最坏情形下的延时模拟成图 9.32 中的线性 RC 网络。

如在第 5 章中所推导的，当所有的 $C_i=C$ 且 $R_j=R$ 时，这样一个网络的传播延时等于：

$$t_p = 0.69 \sum_{i=1}^N C_i \left(\sum_{j=1}^i R_j \right) = 0.69 \frac{N(N+1)}{2} RC \quad (9.8)$$

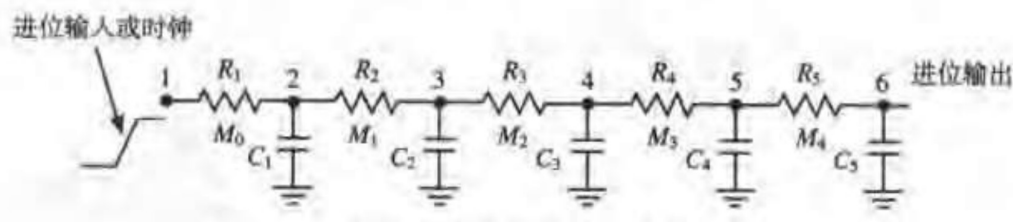


图 9.32 决定进位链传播延时的等效网络

例 9.3 确定曼彻斯特进位链的尺寸

进位链上每一节点的电容等于四个扩散电容、一个反相器输入电容以及与这一单元尺寸成正比的导线电容。反相器和 PMOS 预充电晶体管可以保持在单位尺寸，加上导线电容，固定电容可以估计为 15fF（就我们的工艺而言）。如果一个宽度为 W_0 的单位尺寸晶体管的电阻为 10KΩ，扩散电容为 2fF，则宽度为 W 的晶体管链的 RC 时间常数为：

$$RC = (6\text{fF} \cdot (W/W_0) + 15\text{fF}) \cdot 10\text{K}\Omega \cdot (W_0/W)$$

增加晶体管的宽度能缩小这一时间常数，但这也增加了前一级门的负载。因此，晶体管的尺寸受输入负载电容的限制。

很遗憾，进位链的分布 RC 特性使传播延时与位数 N 成平方关系。为了避免这一点，需要插入信号缓冲反相器。正如在已讨论过的，缓冲器的最佳级数取决于反相器的等效电阻以及传输管的电阻和电容。在我们的工艺中，以及其他大多数实际情形中，这一数目在 3 到 4 之间。增加反相器使总传播延时与 N 成线性关系，这与逐位进位加法器的情形一样。

9.8.3 二进制加法器：逻辑设计考虑

逐位进位加法器只适用于实现较小字长的加法。大多数台式计算机采用 32 位的字长，而服务器则要求 64 位，诸如大型机、超级计算机或多媒体处理器等速度很快的计算机要求的字长达 128 位。加法器延时与位数的线性关系使得采用逐位进位加法器很不实际。因此需要进行逻辑优化，使加法器的 $t_p < O(N)$ 。下面将简短地讨论一些优化方法，我们将集中在电路设计方面，因为这里介绍的大部分结构在传统的逻辑设计文献中是熟知的。

进位旁路加法器 (Carry-Bypass Adder)

考虑图 9.33 (a) 中的 4 位加法器模块。假设 A_k 和 B_k ($k = 0, \dots, 3$) 的值使所有的进位传播信号 P_k 为高电平。在这一状况下，一个进位输入 $C_{i,0} = 1$ 传播通过整个加法器链并使进位输出 $C_{o,3} = 1$ 。换言之：

$$\text{如果 } (P_0 P_1 P_2 P_3 = 1), \text{ 则 } C_{o,3} = C_{i,0}$$

否则发生进位消除或进位产生 (9.9)

这一信息可以用来加速加法器的操作，如图 9.33 (b) 所示。当 $BP = P_0 P_1 P_2 P_3 = 1$ 时，进位输入通过旁路晶体管 M_b 立即送至下一个模块，因而称之为进位旁路加法器 (carry-bypass adder) 或进位跳越加法器 (carry-skip adder)。如果 $P_0 P_1 P_2 P_3 \neq 1$ ，则通过正常路径得到进位输出。

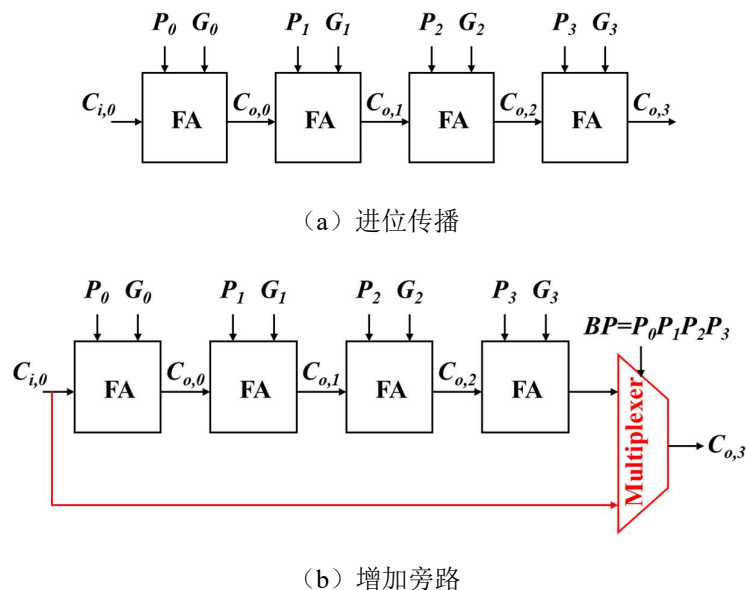


图 9.33 进位旁路结构——基本原理

例 9.4 曼彻斯特进位链加法器中的进位旁路

图 9.34 为用曼彻斯特进位链实现的加法器电路中可能的进位传播路径。图中显示了进位旁路如何加速加法运算：进位通过旁路传播，或者在进位链中的某处产生进位。在这两种情形中，延时都小于通常的逐位进位结构。因增加旁路而增加的面积很小，一般在 10~20% 的范围内。但是，增加旁路破坏了规则的位片结构。

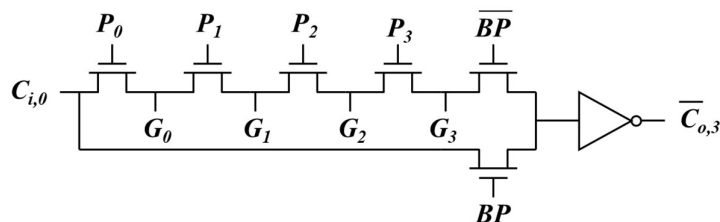


图 9.34 用曼彻斯特进位链实现的旁路加法器

下面我们来计算一个 N 位加法器的延时。首先假设整个加法器被划分成 (N/M) 个等长的旁路级，每一级含有 M 位。由图 9.35 可以推导出这个加法器总传播时间的近似表达式 (9.10)。即，

$$t_{adder} = t_{setup} + Mt_{carry} + (N/M - 1)t_{bypass} + (M - 1)t_{carry} + t_{sum} \quad (9.10)$$

其中各部分参数定义如下：

- t_{setup} ：形成进位产生信号和进位传播信号所需要的固定时间。
- t_{carry} ：通过一位的传播延时。最坏情形下通过具有 M 位的一个级的进位传播延时大约为 t_{carry} 的 M 倍。
- t_{bypass} ：通过一级旁路多路开关的传播延时。
- t_{sum} ：产生最后一级的“和”所需要的时间。

关键路径在图 9.35 的方框图中以灰色表示。由公式 (9.10) 可知 t_{adder} 与位数 N 仍然为线性关系，因为在最坏情况下进位产生于第一位的位置，逐位通过第一个模块，跃过 $(N/M - 2)$ 个旁路级，并且被吸收在最后一位的位置上而不产生输出进位。每一旁路模块的最优位数是由工艺参数来决定的，如旁路选择多路开关的额外延时、在进位链中的缓冲要求以及通过逐位进位和通过旁路的延时的比值。

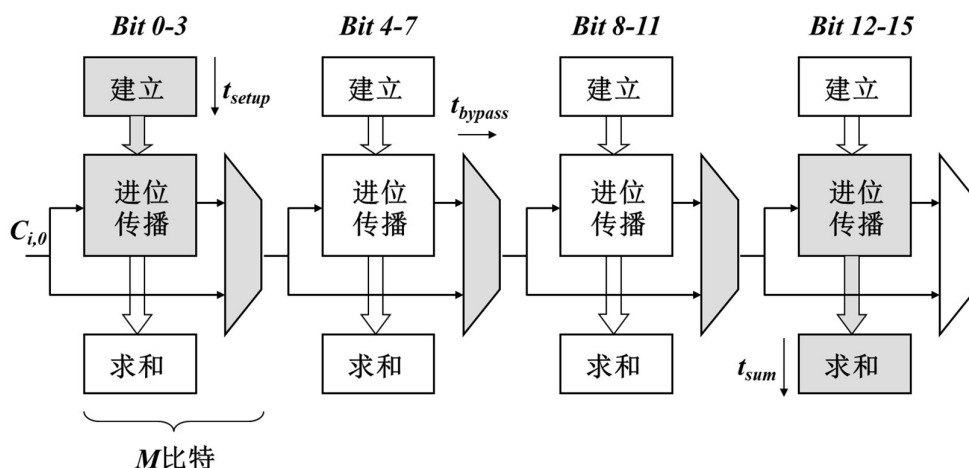


图 9.35 $(N=16)$ 进位旁路加法器的组成（灰色部分为最坏情形的延时路径）

尽管进位旁路加法器的延时与位数 N 仍然是线性关系，但旁路加法器延时增加的斜率比在逐位进位加法器中要平缓，如图 9.36 所示。这一差别对于较大的加法器是很明显的。注意，逐位进位加法器实际上在 N 值较小时比较快，这时额外的旁路多路开关的开销使旁路结构并不具有什么意义。交叉点（即何时采用进位旁路）取决于工艺考虑，通常 N 在 4 到 8 位之间。

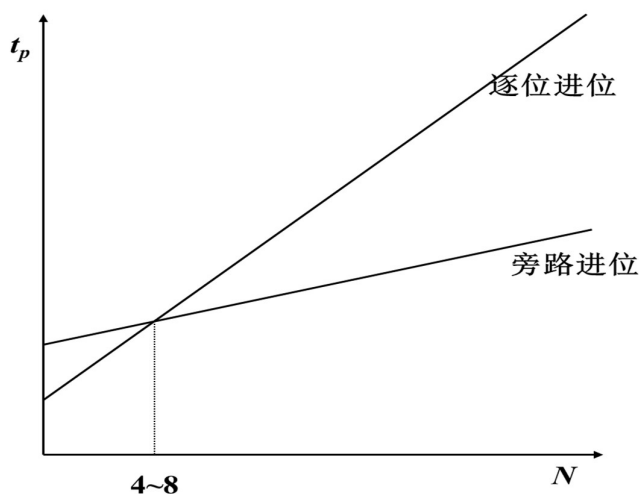


图 9.36 逐位进位加法器与进位旁路加法器传播延时的比较

线性进位选择加法器 (Linear Carry-Select Adder)

在逐位进位加法器中，每一个全加器单元必须等待输入进位到达之后才能产生一个输出进位。避免这一线性依赖关系的一种方法是预先考虑进位输入两种可能的值，并提前计算出针对这两种可能性的结果。一旦输入进位的确切值已知，正确的结果就可以通过一个简单的多路开关级很容易地选出。这一设想的实现非常恰当地称为进位选择加法器 (*carry-select adder*)，如图 9.37 所示。考虑加法器相加第 k 位至第 $k+3$ 位的模块，它不再等待第 $k-1$ 位的输出进位到达，而是对 0 和 1 两种可能性都进行分析。从电路的角度来看，这意味着实现了两个进位路径。当 $C_{o,k-1}$ 最终确定下来后，就可以通过多路开关选择 0 或者 1 路径的结果，这可以用一个最小的延迟时间来完成。从图 9.37 中可以清楚地看到，进位选择加法器所增加的硬件开销限于一个额外的进位路径和一个多路开关，大约等于逐位进位结构的 30%。

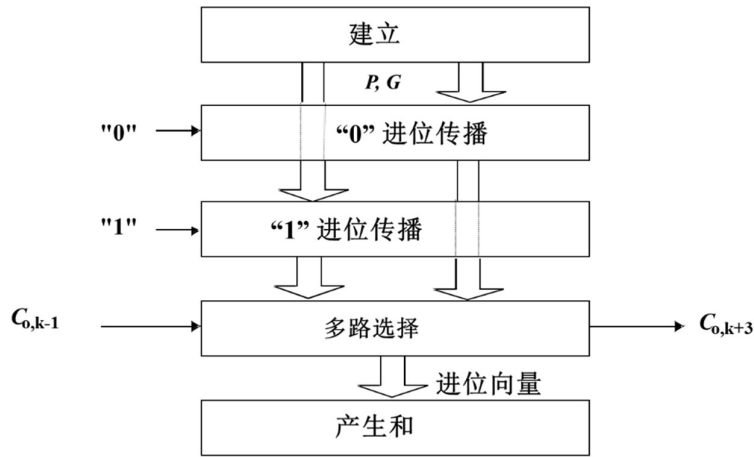


图 9.37 4 位进位选择模块——拓扑结构

一个完整的进位选择加法器现在就可以如进位旁路方法那样通过链接许多相等长度的加法器级来构成（见图 9.38），图中关键路径以灰色显示。通过观察这一电路，可以推导出这一模块最坏情形下传播延时的一阶模型，即可写为：

$$t_{adder} = t_{setup} + Mt_{carry} + \frac{N}{M}t_{mux} + t_{sum} \quad (9.11)$$

式中， t_{setup} 、 t_{sum} 和 t_{mux} 是固定延时， N 和 M 分别代表总位数和每一级的位数， t_{carry} 是进位通过单个全加器单元的延时。通过单个模块的进位延时正比于该级的长度，即等于 Mt_{carry} 。

加法器的传播延时再次线性地正比于 N [公式 (9.11)]。形成这一线性关系的原因是在最坏情形下选择 0 或 1 的运算结果的模块选择信号仍然必须逐一通过所有的级。

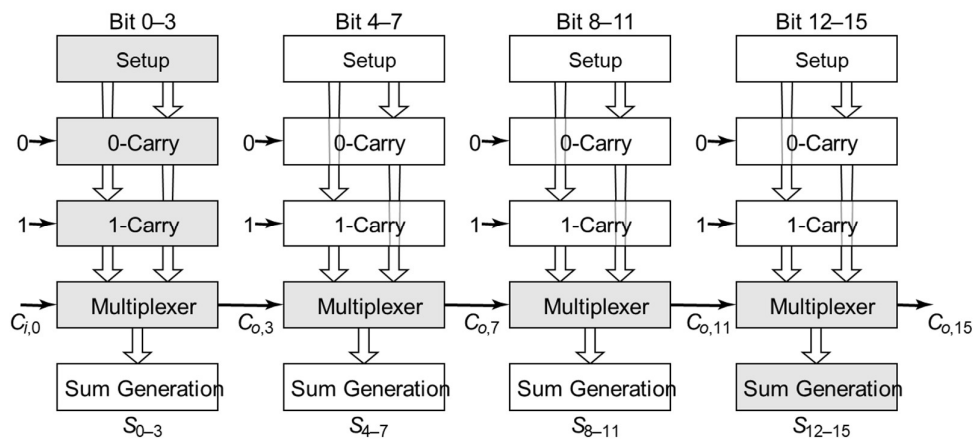


图 9.38 16 位线性进位选择加法器（关键路径以灰色显示）

平方根进位选择加法器（Square-Root Carry-Select Adder）

为了优化设计，最重要的是首先确定关键时序路径的位置。考虑一个 16 位线性进

位选择加法器的情形。为了简化讨论，假设全加器和多路开关单元具有相同的传播延时，并都等于一个归一化的值 1。在最坏情形下相对于输入时间的信号到达电路不同节点的时间标注在图 9.39 (a) 上。这一分析显示加法器的关键路径逐级通过后续各级的多路开关电路。

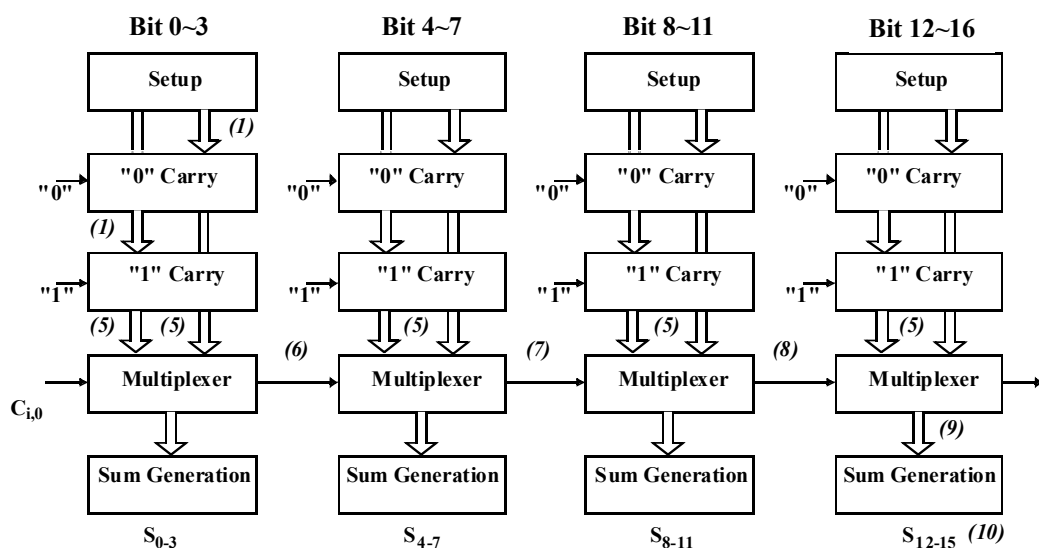
这里出现了一个极好的机会。考虑最后一级加法器的多路开关门。这个多路开关的输入是这模块的两个进位链以及来自前一级模块的多路开关信号。可以观察到这些信号的到达时间有明显的差别，进位链的结果（多路开关的两路数据输入）在多路开关控制信号到达之前很久就已稳定下来。因此使这两条路径的延时相等是有意义的。这可以通过在该加法器中逐渐增加后续各级的位数来实现，因而后续各级需要有更多的时间来产生进位信号。例如，第一级可以相加两位，第二级是 3 位，第三级是 4 位，等等，如图 9.39 (b) 所示。图中标出的到达时间表明这一加法器的拓扑结构虽然增加了额外一级，但还是比线性结构快。事实上，一个 20 位加法器的传播延时也和它一样。注意，在各个多路开关节点上两条路径到达时间的差别已消除。

实际上，这一加法器逐级加长的简单方法使加法器结构具有亚线性延时特性。这可以通过以下分析来说明：假设一个 N 位加法器含有 P 级，第一级相加 M 位。后续各级依次增加一位，于是下述关系成立：

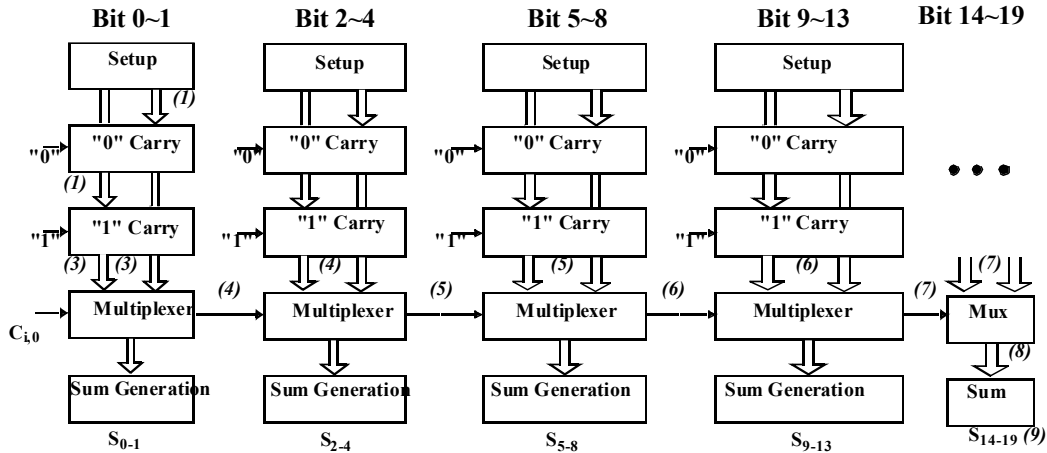
$$N = M + (M+1) + (M+2) + \dots + (M+P-1) = P^2/2 + P(M-1/2) \quad (9.12)$$

如果 $M \ll N$ （例如 $M=2$, $N=64$ ），则第一项起主要作用，公式 (9.12) 可简化为：

$$N \approx P^2/2 \quad \text{或} \quad P = \sqrt{2N} \quad (9.13)$$



(a) 线性结构



(b) 平方根结构

图 9.39 进位选择加法器中最坏情形下信号到达的时间（信号到达时间表示在括号内）

利用公式 (9.13) 重写公式 (9.11)，可以把 t_{adder} 表示成 N 的函数：

$$t_{adder} = t_{setup} + Mt_{carry} + \sqrt{2N}t_{mux} + t_{sum} \quad (9.14)$$

所以，对于大的加法器 ($N \gg M$) 延时与 $\sqrt{N}$ 成正比，即 $t_{adder} = O(\sqrt{N})$ 。这一平方根关系具有重要意义，这可以从图 9.40 看出，图中画出了线性进位选择加法器和平方根进位选择加法器的延时与位数 N 的关系曲线。可以看到，当 N 值很大时， t_{adder} 几乎变为常数。

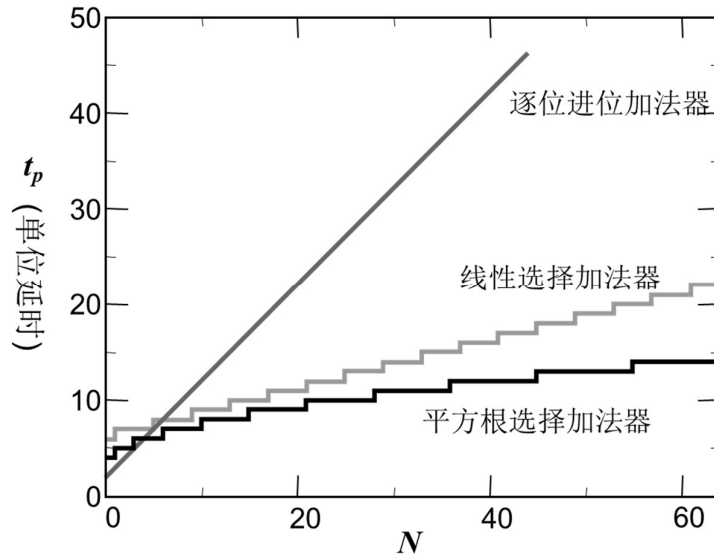


图 9.40 平方根进位选择加法器与逐位进位加法器以及线性选择加法器传播延时的比较
(采用单位延时模型模拟单元延时)

超前进位加法器 (Carry-lookahead Adder)

单一超前进位加法器 在设计更快速的加法器时，避免逐级进位效应至关重要，而这一效应仍然以某一种形式存在于进位旁路和进位选择加法器中。超前进位原理提供了一种有可能解决这一问题的方法。如前所述，在 N 位加法器中每一位的位置上都存在下列关系：

$$C_{o,k} = f(A_k, B_k, C_{o,k-1}) = G_k + P_k C_{o,k-1} \quad (9.15)$$

通过展开 $C_{o,k-1}$ 可以消除 $C_{o,k}$ 对 $C_{o,k-1}$ 的依赖关系：

$$C_{o,k} = G_k + P_k C_{o,k-1} = G_k + P_k (G_{k-1} + P_{k-1} C_{o,k-2}) \quad (9.16)$$

$C_{o,k}$ 的完全展开式为：

$$C_{o,k} = G_k + P_k (G_{k-1} + P_{k-1} (\dots + P_1 (G_0 + P_0 C_{i,0}))) \quad (9.17)$$

其中 $C_{i,0}$ 通常等于零。

这一展开关系可以用来实现一个 N 位的加法器。每一位的进位输出与“和”位输出都与前面的位无关，因而有效地消除了逐位进位效应，因此加法时间应当与位数无关。图 9.41 是一个超前进位加法器整体组成的方块图。

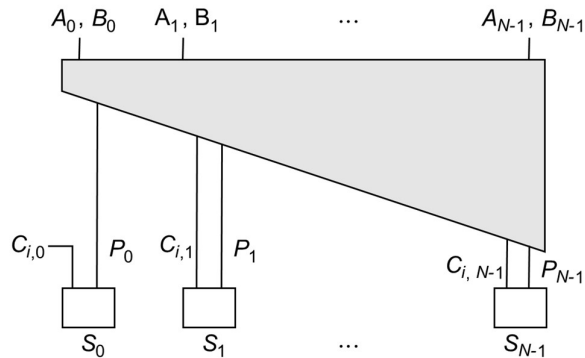


图 9.41 超前进位加法器的原理图

这一高层次的模型包括了某些隐含的依赖关系。当我们研究这一加法器的细节电路图时就会很清楚地发现，与位数无关的加法时间只是一种希望的想法，真正的延时至少随位数呈线性地增加。这一点表现在图 9.42 中，它是 $N=4$ 时公式 (9.17) 的一种可能的电路实现。注意，该电路利用超前进位公式的自对偶性和递归性建立了一镜像结构，与图 9.28 的一位全加器的形式相类似。这一电路的大扇入使它在 N 值较大时极慢。如果用较简单的门来实现，则要求多个逻辑层次。这两种情况都会使传播延时增加。此外，某些信号上的扇出常常过度增加，从而使加法器变得甚至更慢。例如， G_0 和 P_0 信号出现在后续每一位的表达式中。因此在这些线上的电容是很可观的。最后，这一电路实现的面积也随 N 而逐渐增加。因此，这种超前结构只在 N 值较小 (≤ 4) 时有效。

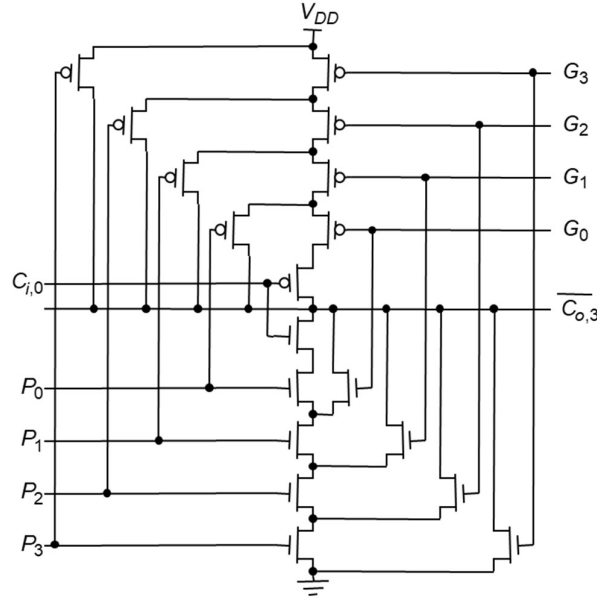


图 9.42 4 位超前进位加法器镜像实现的电路图

对数超前进位加法器——原理 对于一个 N 位的超前进位组，它的晶体管实现具有 $N+1$ 个并行分支并且最多有 $N+1$ 个晶体管堆叠在一起。由于门的分支和晶体管的堆叠较多使性能较差，所以超前进位计算在实际中至多只能限于 2 或 4 位。为了建立非常快速的加法器，需要把进位传播和进位产生组织成递归的树形结构。一个比较有效的实现方法是把进位传播层次化地分解成 N 位的子组合：

$$\begin{aligned}
 C_{o,0} &= G_0 + P_0 C_{i,0} \\
 C_{o,1} &= G_1 + P_1 C_{o,0} = G_1 + P_1 G_0 + P_1 P_0 C_{i,0} = (G_1 + P_1 G_0) + (P_1 P_0) C_{i,0} \\
 &= G_{1:0} + P_{1:0} C_{i,0} \\
 C_{o,2} &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{i,0} = G_2 + P_2 C_{o,1} \\
 C_{o,3} &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{i,0} \\
 &= (G_3 + P_3 G_2) + (P_3 P_2) C_{o,1} = G_{3:2} + P_{3:2} C_{o,1}
 \end{aligned} \tag{9.18}$$

在公式 (9.18) 中，进位传播过程被分解成两位的子组合。 $G_{i,j}$ 和 $P_{i,j}$ 分别表示一组位（从第 i 位至第 j 位）的进位产生和进位传播函数，因而我们称之为块进位产生和块进位传播信号。如果该组产生一个进位，则 $G_{i,j}$ 等于 1，而与输入进位无关。如果一个输入进位传播通过整个这一组，则 $P_{i,j}$ 即为 1。这一条件等同于前面讨论过的进位旁路。例如，当进位产生于第 3 位或当进位产生于第 2 位并传播通过第 3 位时，则 $G_{3:2}$ 等于 1（即 $G_{3:2} = G_3 + P_3 G_2$ ）。当输入进位传播通过这两位时， $P_{3:2}$ 为 1（即 $P_{3:2} = P_3 P_2$ ）。

注意，新的进位表达式的形式与原来的一样，只是进位产生和进位传播信号由块进位产生和块进位传播信号所代替。符号 $G_{i,j}$ 和 $P_{i,j}$ 一般化了原来的进位公式，因为 $G_i = G_{i:i}$ 和 $P_i = P_{i:i}$ 。另一个可能的一般化是把产生和传播函数看做是一对 $(G_{i,j}, P_{i,j})$ ，而不

是单独的函数。现在可以引入一个新的布尔运算，称为点操作 ($\bullet$)，这一操作针对这些成对的函数并且允许对一个块的位进行组合和运算：

$$(G, P) \bullet (G', P') = (G + PG', PP') \quad (9.19)$$

利用这一操作现在我们可以分解 $(G_{3:2}, P_{3:2}) = (G_3, P_3) \bullet (G_2, P_2)$ 。点操作服从结合律，但不符合交换律。

例 9.5 用点操作表示逐位进位加法器

利用点操作，一个 4 位的逐位进位加法器可以重写为：

$$(C_{0,3}, 0) = [(G_3, P_3) \bullet (G_2, P_2) \bullet (G_1, P_1) \bullet (G_0, P_0)] \bullet (C_{i,0}, 0)$$

利用结合律可以重写这一函数，并把 $C_{0,3}$ 表示为两组进位的函数：

$$\begin{aligned} (G_{3:0}, P_{3:0}) &= [(G_3, P_3) \bullet (G_2, P_2)] \bullet [(G_1, P_1) \bullet (G_0, P_0)] \\ &= (G_{3:2}, P_{3:2}) \bullet (G_{1:0}, P_{1:0}) \end{aligned}$$

利用点操作的结合律，可以构成一个树结构来有效地计算所有 2^i-1 个位置（即 1, 3, 7, 15 等）上的进位 ($i=1, \dots, \log_2(N)$)。最重要的优点是在位置 2^i-1 上的进位只需要 $\log_2(N)$ 步就可以计算出来。换言之，一个 N 位加法器的输出进位可以在 $\log_2(N)$ 的时间内计算出来。这比起前面所描述的加法器来有很大的改进。例如，对于一个 64 位的加法器，线性加法器的传播延时正比于 64，对于一个平方根选择加法器，它减小到正比于 8，而对于一个对数加法器，这一比例常数减为 6。这显示在图 9.43 中，图中画出了一个 16 位对数加法器的原理图。位置 15 处的进位通过联合块(0:7)和(8:15)的结果计算出。这两个结果中的每一个又是由各层次组成的。例如，(0:7)是由(0:3)和(4:7)组成的，而(0:3)是由(0:1)和(2:3)组成的，等等。

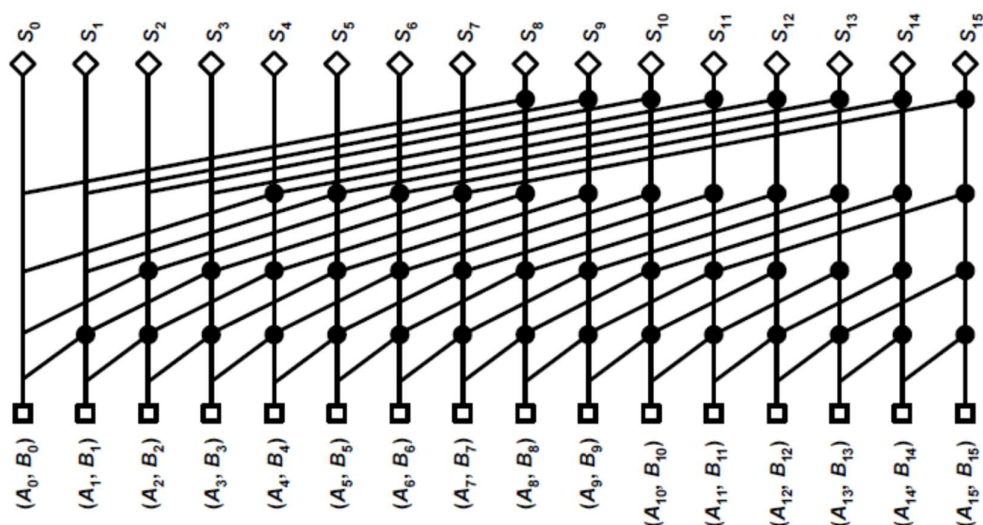


图 9.43 Kogge-Stone 16 位超前进位对数加法器的原理图

(图中□表示建立 P 和 G 信号, ●表示点操作, ◇表示产生“和”)

只计算在位置 2^i-1 上的进位显然是不够的, 还需要推导出在中间位置上的进位信号。实现这一点的一个方法是在每一位上都重复这一树结构, 如图 9.43 对于 $N=16$ 的情形所示。例如, 位置 6 处的进位通过组合块(6:3)和(2:0)的结果来计算。这整个结构常常称为 *Kogge-stone* 树, 它是基数为 2 的树结构中的一种。基数为 2 是指这一结构是二进制树: 它在每一结构层次上一次只组合两个进位字。整个加法器需要 49 个复杂逻辑门, 每个都实现点操作。此外, 还需要 16 个逻辑模块在第一层上产生进位传播和进位产生信号 (P_i 和 G_i) 以及 16 个门产生和。

设计举例——用动态逻辑实现一个超前进位加法器

如果最终目的是实现很高的性能, 那么联合超前进位 (CLA) 技术和动态逻辑似乎是一种理想的方法。因此经历一个动态 CLA 设计的全过程是很有益的。

第一个模块产生进位传播和进位产生信号, 如图 9.44 所示。所附加的单独用来驱动保持器的反相器稍难理解一些, 这一技术在驱动较大扇出的门中是很有用的。在翻转开始后通过使保持器的驱动器与该电路的扇出间失去联系可以使保持器很快脱离工作。反之, 驱动后续逻辑门的反相器则被优化来驱动两个 (对于 G 输出) 或三个 (对于 P 输出) NMOS 下拉网络的扇出。

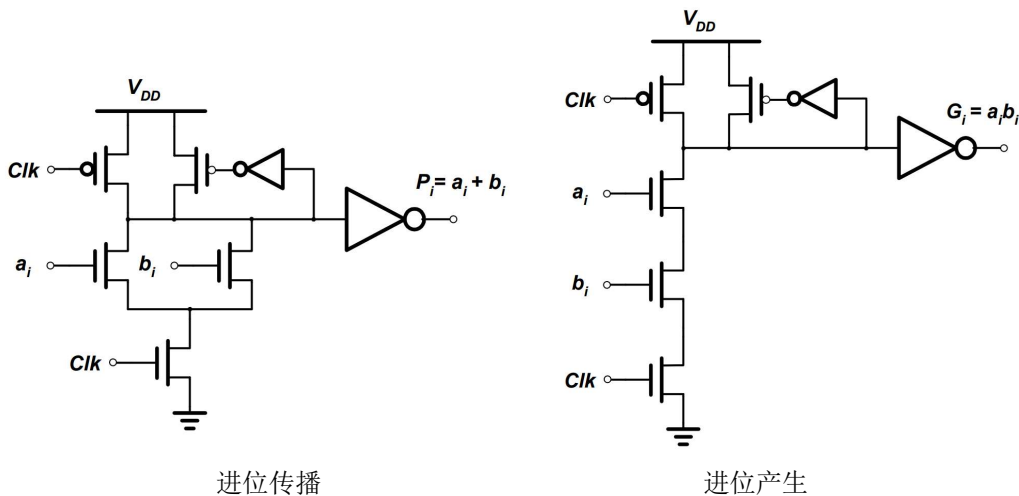


图 9.44 进位传播和进位产生信号的动态实现

图 9.43 中每个黑圆点代表计算块级的进位传播和进位产生信号的两个门, 如图 9.45 所示。由于这两个门的位置不在一条流水线的开始处, 所以如在第 8 章已讨论过的, 求值晶体管 (也称为最底下开关) 为可选件。这一方法通常用在动态数据通路中。在预充电阶段, 多米诺门的所有输出都保证处于低电平, 从而关断了在后续多米诺级中的任何放电路径。在除第一级以外的任何一级都取消最底下开关可以减小该门的逻辑努力, 而加速求值过程。例如图 9.45 中产生进位传播信号的门的逻辑努力为 $2/3$ 而不是 1 (假设反相器是对称的)。然而这一方法有一个缺点: 在预充电阶段, 没有最底下开关的所有门中都存在一个短路电流, 直到它们的输入被预充电为止。为了避免这一短路电流, 每

一级的时钟 Clk_k 比它的前一级延迟一点时间。这一方法称为时钟延迟多米诺 (*clock-delayed domino*)。注意, 对于每一级位片单元时钟都延迟相同的时间, 从而简化了电路实现。

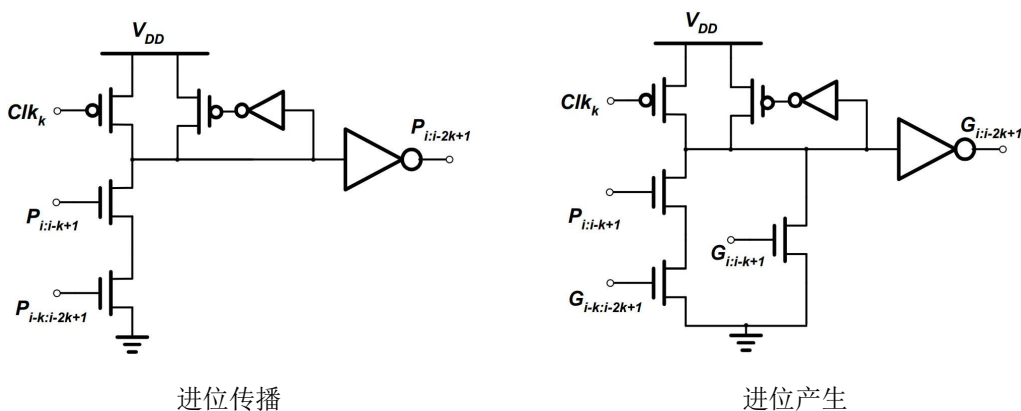


图 9.45 动态逻辑中实现的点操作: 在第 k 级第 i 位处

把几个逻辑级按位片式放在一起, 即在 P - G 产生之后有 6 个点操作, 就可以构成一个 64 位的加法器。为完成动态加法器设计还缺少的一个辑级是最终和的产生。和的产生需要有 XOR 功能, 而它不能很容易地用多米诺逻辑来实现。虽然可以采用静态 XOR 门, 但这些门会产生非单向的过渡, 因此不能用来驱动其他多米诺门。这本身也许不是一个问题, 因为和的产生一般都是加法运算的最后一级。然而在和产生之后的锁存器却不能是透明的, 因为这可能会违反后续多米诺级的翻转规则。

用多米诺实现求和的一种方法是通过求和选择 (*sum selection*) 电路, 在此电路中求和的两种条件计算为 $S_i^0 = \overline{a_i \oplus b_i}$ 及 $S_i^1 = a_i \oplus b_i$ 。于是图 9.46 的动态门就可以根据输入进位选择其中的一种可能性。

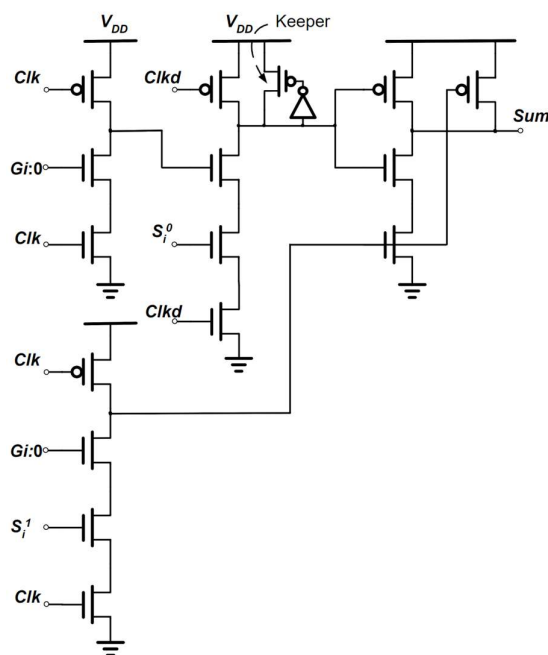


图 9.46 动态逻辑实现和选择电路

多路开关门的实现要求三个逻辑层，这是因为在多米诺逻辑中没有任何互补的进位。与通常一样，在所有动态节点上都应当放有保持器，但图中所示的那个保持器绝对是最重要的。前面两个多米诺级（在左上部）违反了多米诺逻辑的设计规则：两个动态门串联在一起而没有插入反相器。如果这两个门用同一个时钟来求值，那么在第二个门的输出处就有可能发生毛刺（*glitch*）。使第二个门的时钟（*Clkd*）延迟有助于解决这个问题，尽管延迟时钟要求有一个严格的时序界限，即第二个门的所有输入都必须在时钟上升之前完成它们的翻转。

应当强调的是当设计者选择采用这些设计技术时，电路的稳定性不能牺牲。所有的时钟都会出现随机的偏差，但延迟时钟必须具有足够的余量以吸收最坏情形下的时钟偏差。如果 *Clkd* 提前到达，则加法器将会误操作。然而使保持器有合适的尺寸将有助于保证该设计稳定可靠，它在时钟提前到达的情况下可以抑制在第二级输出端的毛刺。保持器的尺寸必须足够大以减少这一虚假翻转，但同时又不能太大以免影响性能。

另一种选择是用差分多米诺逻辑来实现加法器，由于此时两种信号极性都存在，所以避免了反相问题。然而整个设计的功耗还是非常大的。

另一种对数超前进位加法器 快速加法器的设计者在综合考虑面积、功耗或性能时常常回过头来采用其他类型的树结构。我们简短讨论一下另外两种比较常用的结构，即 Brent-Kung 加法器及四进制加法器。

图 9.43 的 Kogge-Stone 树结构具有某些有意义的特性。首先它的互连结构比较规则，这使其非常容易实现。而且整个树结构各处的扇出，特别是在关键路径上的扇出，

基本上是一个常数。因此简化了为达到最优性能确定晶体管尺寸的任务，但与此同时，为产生中间进位需要复制进位树结构，这造成了在面积和功耗方面较大的代价。设计者常常选择较不复杂的树结构，以较长的延时来换取较少的面积和功耗。一个较为简单的树结构只计算 2 的幂次位上的进位，如图 9.47 对于 $N=16$ 的情形所示。

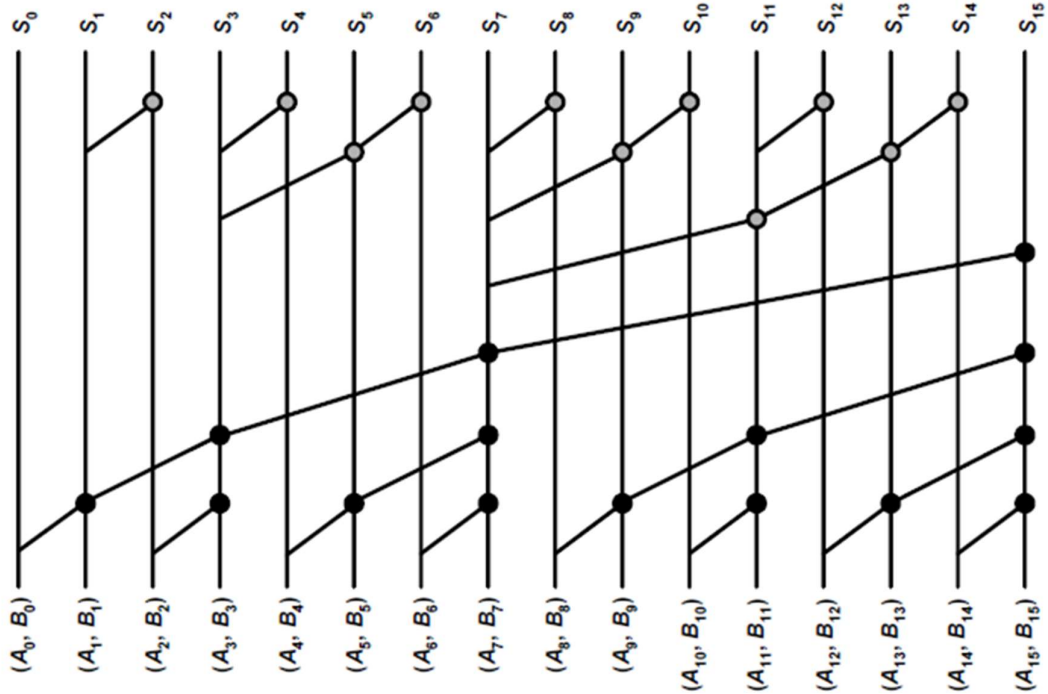


图 9.47 16 位 Brent-Kung 树结构（反向二进制树用灰色表示）

正向二进制树（forward binary tree）只实现在 2^N-1 位置上的进位信号：

$$\begin{aligned}
 (C_{0,0}, 0) &= (G_0, P_0) \cdot (C_{i,0}, 0) \\
 (C_{0,1}, 0) &= [(G_1, P_1) \cdot (G_0, P_0)] \cdot (C_{i,0}, 0) = (G_{1:0}, P_{1:0}) \cdot (C_{i,0}, 0) \\
 (C_{0,3}, 0) &= [(G_{3:2}, P_{3:2}) \cdot (G_{1:0}, P_{1:0})] \cdot (C_{i,0}, 0) = (G_{3:0}, P_{3:0}) \cdot (C_{i,0}, 0) \\
 (C_{0,7}, 0) &= [(G_{7:4}, P_{7:4}) \cdot (G_{3:0}, P_{3:0})] \cdot (C_{i,0}, 0) = (G_{7:0}, P_{7:0}) \cdot (C_{i,0}, 0) \\
 &\dots
 \end{aligned} \tag{9.20}$$

正向二进制树结构不足以产生全部进位位，因此需要一个反向二进制树（*inverse binary tree*）来实现其他进位位（在图 9.47 中用灰线表示）。这一结构组合中间的结果以产生其余的进位位。读者可以去验证这一结构确实产生所有进位位的正确表达式。由此得到的结构常称为 Brent-Kung 加法器，它采用了 27 个点操作的门，这几乎是完全的二进制树所需要的 49 个点操作门数的一半，并且它所需要的导线也较少。然而布线结构不太规则，并且各个门的扇出也不相同，这使优化性能比较困难。尤其是对于中间节点 $(C_{0,7})$ 的扇出，在本例中它等于一个求和以及 4 个点操作，是一个应主要考虑的问题，这一事实使 Brent-Kung 加法器很不适于非常大（大于 32 位）的加法器。

减少树结构深度的一种选择是在每一结构层次上每次组合四个信号。由此得到的树

结构就是四进制树结构，因为它采用的基本构造模块是四线的，如图 9.48 所示。一个 16 位的加法只需要两级进位逻辑。但应当知道这里每一个门是比较复杂的，因此使逻辑级较少并不总能实现较快的操作。

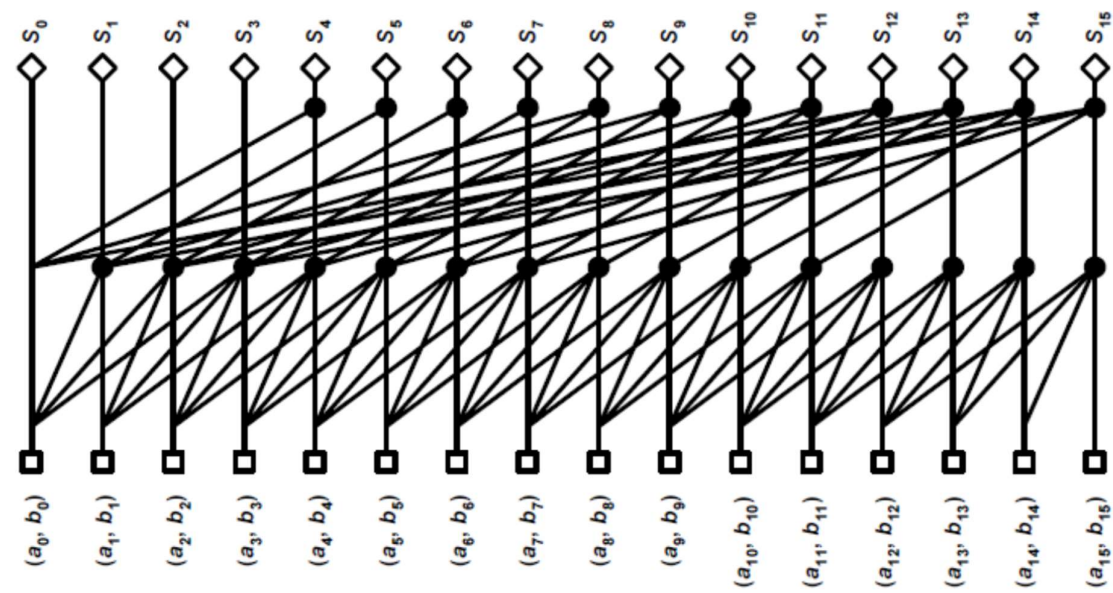


图 9.48 16 位操作数的四进制 Kogge-Stone 树结构

平均而言，一个超前进位加法器要比一个逐位进位加法器大数倍，但对于较大的操作数，它有非常明显的速度优势。对数特性使它在 N 值较大时优于旁路加法器和选择加法器。其交叉点的确切值很大程度上取决于工艺和电路设计。

对于加法器的讨论还没有结束。鉴于它对计算结构性能的重要影响，快速加法器电路的设计已是许多论文的主题。甚至还有可能设计出传播延时与位数无关的加法器结构。这些加法器的例子是进位保留 (carry-save) 结构和冗余二进制算法结构 (redundant binary arithmetic structure)。这些加法器要求有数字的编码和解码步骤，它们的延时是 N 的函数。因此，它们只有在嵌入到较大的结构 (如乘法器或高速信号处理器) 中时才有意义。

第九节 十进制加法器

以十进制数直接进行算术运算的计算机或计算器，一般以二进制数形式来表示十进制数。作为这样一台计算机的加法器，必须要有算术电路来接收编码的十进制数，并以相同的编码表示计算结果。对于二进制数加法，只要考虑一对有效位以及前面的进位就足够了。十进制加法器最少需要 9 个输入和 5 个输出，因为每个十进制数需要四位二进制来编码，并且电路必须要有一个输入和一个输出进位。十进制加法器有多种电路，它取决于所代表的十进制数的编码。这里我们主要讨论 BCD 码的十进制加法器。

BCD 加法器

考虑两个十进制数 BCD 码的算术加法，包括来自前一级的进位输入。既然每个输入数不超过 9，输出和就不会大于 $9 + 9 + 1 = 19$ ，其中 1 代表一个输入进位。假设将两个 BCD 码加到一个 4 位二进制加法器上，该加法器就会产生一个二进制和，且计算结果范围为 0 至 19。这些二进制数被列在表 9.8 中，并用符号 K 、 Z_8 、 Z_4 、 Z_2 和 Z_1 来标识。

表 9.8 BCD 加法器的推导

二进制和					BCD 和					十进制数
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

表中 K 代表进位， Z 的下标 8、4、2、1 代表权值，分配给 BCD 码的四个位。二进制和的那一栏列出了 4 位加法器的输出二进制值。两个十进制数的和输出必须表达成 BCD 码，用列在 BCD 和那一列的形式表示。下面的问题就是要找到一个规则，将二进制和转换成正确的 BCD 和。

从表 9.8 可以看出：当二进制和等于或者小于 1001 时，其对应的 BCD 数是相等的，因而不需要进行转化。当二进制和大于 1001 时，得到的是一个无效的 BCD 表示。解决办法是将二进制数 6（0110）与二进制和数相加，得到正确的 BCD 表示，并产生所需的输出进位。

用于校正的逻辑电路可从表格中得出。当二进制和有一个输出进位 $K=1$ 时就需要校正。其他的从 1010 到 1111 的六种组合也需要进行校正，它们在 Z_8 的位置上都为 1。为了将它们与二进制数 1000 和 1001 加以区分，我们进一步限定 Z_4 或 Z_2 上也必须是 1。校正的条件以及输出进位可以用布尔表达式表示如下：

$$C = K + Z_8Z_4 + Z_8Z_2$$

当 $C=1$ 时，需要将 0110 与二进制和相加并给下一级提供一个输出进位。

两个 BCD 数相加产生 BCD 和的加法器如图 9.49 所示。两个十进制数与输入进位首先加到上面的 4 位加法器上，以产生二进制和。当输出进位等于 0 时，二进制和保持不变。而当其等于 1 时，将二进制数 0110 与下面 4 位加法器产生的和相加。下面加法器产生的输出进位可以被忽略，因为它提供的信息可以在输出进位端得到。一个并行的 n 位十进制加法器需要 n 级 BCD 加法器，其中低一级输出进位必须接到高一级的输入进位上。

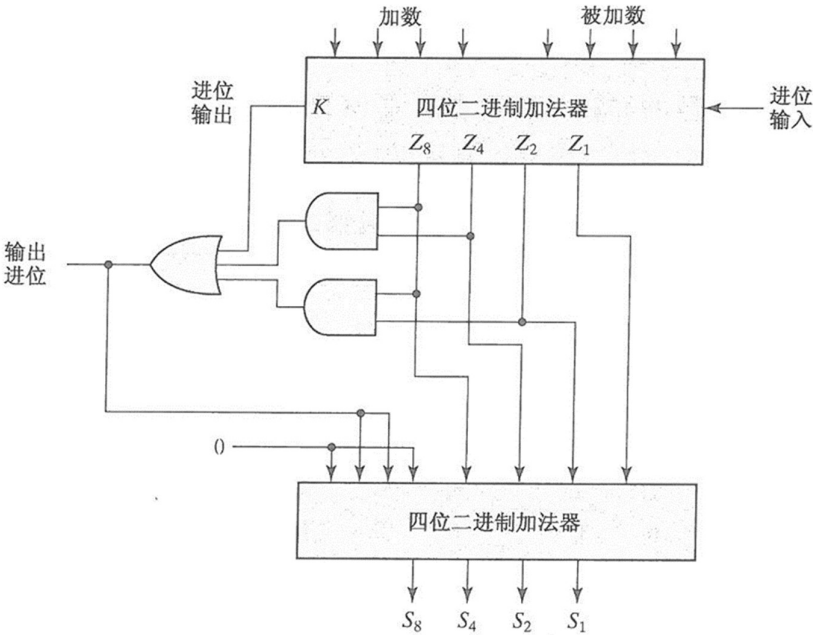


图 9.49 BCD 加法器框图

第十节 二进制减法器

9.10.1 补码

补码在数字计算机中用于简化减法运算和逻辑操作，从而使电路更加简单、价格更加便宜。每个 r 进制系统都有两种类型的补码：补码（*radix complement*）和反码（*diminished radix complement*）。第一种可看作 r 的补码，第二种为 $(r-1)$ 的补码。这

里讨论二进制数的 2 的补码和 1 的补码。

反码

给定一个有 n 个数字的 r 进制数 N ，其 $(r-1)$ 的补码定义为 $(r-1) - N$ ，即反码。对于二进制数， $r=2$ 和 $r-1=1$ ，所以 N 的反码是 $(2^n-1) - N$ 。 2^n 是一个由 1 后面加上 n 个 0 组成的二进制数， 2^n-1 是一个用 n 个 1 表示的二进制数。例如，如果 $n=4$ ，有 $2^4=(10000)_2$ 和 $2^4-1=(1111)_2$ 。因此，二进制数的反码是用 1 减去其每个数字来得到的。然而，当用 1 去减二进制数时，有 $1-0=1$ 或 $1-1=0$ ，从而将一位从 0 变为 1 或者从 1 变成 0。因此，二进制数的反码在形式上就是将 1 改为 0 或将 0 改为 1。如：

1011000 的反码是 0100111

0101101 的反码是 1010010

补码

一个有 n 数字的 r 进制数 N ，其 r 的补码定义为 $r^n - N$ （分 $N \neq 0$ 和 $N=0$ 两种情况）。与前面的反码相比， r 的补码实质就是反码加 1，即 $r^n - N = [(r^n-1) - N] + 1$ 。因此，二进制数 101100 的补码是 $010011+1=010100$ ，就是将它的反码值加 1 得到的。

2^n 是用一个 1 后面加上 n 个 0 表示的数。因此，上述二进制数的补码也可以这样来构成，不改变两个低位的 0 和第一个 1，再将其其他更高位的有效数字用相反的数字代替，即用 0 代 1，用 1 代替 0。因此

二进制数 1101100 的补码是 0010100

和

二进制数 0110111 的补码是 1001001

第一个数的补码是通过不改变低两位的 0 以及第一个 1，而将其其他四个高位有效数字取反，即用 0 代替 1，用 1 代替 0。第二个数的补码是不改变最低有效位的 1，而将所有其他位取反。

在前面的定义中，都是假设那些数没有小数点。如果原始数 N 包含小数点，应该将小数点临时去掉以计算 r 补码或 $(r-1)$ 反码，然后再在相同的位置上将小数点恢复。同样值得强调的是，补码的补码又重新等于它的原码， N 的补码是 $r^n - N$ ，其补码的补码是 $r^n - (r^n - N) = N$ ，刚好等于原码。

补码的减法

利用借位概念的直接减法在小学就学过了。这种方法是当被减数比减数小时就从高位借 1。人们用纸和笔进行减法运算时，这种方法确实很好，而要用硬件实现减法运算时，这种方法的效率就不如使用补码的方法了。

两个有 n 个数字的 r 进制无符号数做减法 $M - N$ 可以按以下步骤来进行：

1. 将被减数 M 加上减数 N 的补码，即 $M + (r^n - N) = M - N + r^n$ 。
2. 如果 $M \geq N$ ，结果的和将产生可以被丢弃的进位，剩下的就是 $M - N$ 的结果。
3. 如果 $M < N$ ，那么和就不会产生进位，等于 $r^n - (N - M)$ ，也就是 $(N - M)$ 的补码。

常见的计算结果是取和的补码并在其前面加上一个负号。

例 9.6 给定两个二进制数 $X=1010100$ 和 $Y=1000011$ ，用补码来实现：(a) $X - Y$ ，(b) $Y - X$

$ \begin{array}{r} X = 1010100 \\ Y \text{ 的补码} = + 0111101 \\ \hline \text{和} = 10010001 \\ \text{抛弃末端进位 } 2^7 = - 10000000 \\ \hline \text{结果: } X - Y = 0010001 \end{array} $ (a)	$ \begin{array}{r} Y = 1000011 \\ X \text{ 的补码} = + 0101100 \\ \hline \text{和} = 1101111 \end{array} $ (b)
---	--

因此， $Y - X = - (1101111 \text{ 的补码}) = - 0010001$ 。

无符号数的减法也可以利用 $(r-1)$ 的反码进行。要知道 $(r-1)$ 的反码只比补码小 1，因此，将被减数和减数的反码相加的结果产生一个和数。当有最后进位时，该和数比正确结果少 1，丢弃该最后进位，并将和数加 1。

9.10.2 带符号的二进制数

正整数（包括零）可以表示成无符号数。然而，为了表示负整数，需要表示符号。在普通算术里，一般用“-”号表示负数，用“+”号表示正数。由于硬件限制，计算机必须用二进制数表示任何事物。通常，二进制数最左边的那个位表示其符号，按惯例 0 表示正号，1 表示负号。

要注意，不论是带符号的二进制数还是无符号的二进制数，在计算机中都是用一组比特来表示的，这一点很重要。究竟是无符号数还是带符号数由用户来确定。若二进制数是带符号的，则其最左边的位表示符号，其余位表示数值；而若二进制数被假定是无符号数，则最左边的位是该数的最高有效位。例如，01001 可以被认为是 9（无符号数）或是 +9（带符号数），最左边的那位为 0。11001 作为无符号数时等于 25，而作为有符号数时是 -9，因为此时其最左边位置上的 1 表示负号，而其余的四位表示二进制数 9。如果预先知道数的类型，一般就不会混淆。

上一个例子中，那些带符号数的表示方式称为“符号-数值”表示法。在该表示法中，数由数值和符号（+或-）或表示符号的位（0 或 1）两部分组成，这就是普通算术中带符号数的表示。当算术运算在计算机中实现时，使用“符号-补码”系统表示负数是比较方便的。在这个系统中，负数用它的补码表示。“符号-数值”系统通过改变数的符号对其取反，而“符号-补码”系统则是通过求补码将其取反。因为正数最左边的位总是以 0 开始，所以负数的补码总以 1 开始。“符号-补码”系统可以采用反码（1 的补码）或补码（2

的补码)，但最常用的是补码。

例如，我们用八位表示二进制数 9。+9 表示为 00001001，即最左边是符号位 0，跟在后面的是等于 9 的二进制序列。注意，所有八位都必须有一个值。因此，在符号位和第一个 1 之间要插入一些 0。尽管只有一种方法来表示+9，而用八位表示-9 却有三种不同的表示方法：

符号-数值表示法： 10001001
符号-反码表示法： 11110110
符号-补码表示法： 11110111

在“符号-数值”表示法中，-9 是通过把+9 最左边位的 0 改为 1 得到的。在“符号-反码”表示法中，-9 是通过把+9 的所有位取反得到的，包括符号位。“符号-补码”表示的 -9 是通过取+9 的二进制补码得到的，也包括符号位。

表 9.9 列出了所有可能的四位带符号二进制数的三种表示法，与此相对应的十进制数也列在其中作为参考。注意，三种表示法中所有正数是完全相同的，且最左边位（最高有效位）为 0。“符号-补码”表示法中 0 只有一种表示，且为正数，而其他两种表示法中既有正 0 也有负 0，这在普通算术中是不会遇到的。所有负数的最高有效位都是 1，据此我们将其与正数区分开来。四位可以表示 16 个二进制数，在“符号-数值”和“符号-反码”表示中，有八个正数和八个负数，包括两个零。在“符号-补码”表示中，有八个正数（其中包括一个零）和八个负数。

表 9.9 带符号的二进制数

十 进 制 数	符号 - 补码	符号 - 反码	符号 - 数值
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

“符号-数值”表示一般用于普通算术运算中，但要用于计算机运算却很麻烦，要分开处理符号和数值。因此，计算机通常采用的是“符号-补码”表示。反码存在着一些难点，在算术运算中很少使用。因为将 0 和 1 互换相当于逻辑求反操作，这一点在逻辑运算中很有用。下面讨论带符号的二进制数时只用“符号-补码”来表示负数，相同的步骤可应用于“符号-反码”表示法，只要如同处理无符号数一样包含最后进位就行了。

算术加法

两个数的加法在“符号-数值”表示中遵循普通算术规则。如果两个数的符号相同，我们可以将两个数值相加，而相加之和的符号保持不变。如果两个数的符号不同，就用绝对值大的数减去绝对值小的数，并将绝对值大的那个数的符号赋给运算结果。例如， $(+25) + (-37) = -(37 - 25) = -12$ ，就是用绝对值大的数 37 减去绝对值小的数 25，运算结果使用数 37 的符号。上述步骤需要比较符号和绝对值，然后再进行加或减。该步骤同样也适用于“符号-数值”法所表示的二进制数。不同的是，对于用“符号-补码”表示的二进制数，在进行两个数相加时，并不需要比较符号或相减运算，而只有加法运算。

两个以“符号-补码”形式表示的二进制符号数的加法，就是把这两个数相加，包括它们的符号位。符号位上所产生的进位要丢弃。

数的加法举例如下：

$+ 6$	00000110	$- 6$	11111010
$+13$	00001101	$+13$	00001101
$+19$	00010011	$+ 7$	00000111
$+ 6$	00000110	$- 6$	11111010
-13	11110011	-13	11110011
$- 7$	11111001	-19	11101101

注意，负数必须首先转换为补码形式。如果相加后得到的和是负数，则给出的结果也是补码形式。例如， -7 表示为 11111001 。

上述四个例子的每一个所进行的运算都是包括符号位的加法。符号位上所产生的进位都要被丢弃，最后得到的负数结果自动就是 2 的补码形式。

为了得到正确结果，我们必须保证要有足够的位数表示和数。如果两个 n 位的数相加，得到的结果却是 $n+1$ 位的和数，称为产生了溢出。当人们用纸笔进行加法运算时，溢出并不是问题，因为我们没有对纸的宽度做出限制，我们只要再添一个 0 到正数或再添一个 1 到负数作为它们的最高有效位，从而将其扩展成为 $n+1$ 位，然后再进行相加。溢出在计算机中之所以成为问题，是因为计算机保存数的位数（字长）是有限的，结果哪怕超过限定位数 1 位都不行。

用补码形式表示的负数与“符号-数值”法所表示的负数是不同的。为了确定以“符号-补码”形式表示的负数的值，有必要将其转换为正数，这样的形式我们更为熟悉。例如，

带符号的二进制数 11111001 是个负数，因为其最左边位上是 1。该二进制负数的补码是 00000111，等于二进制数+7，因此原负数等于-7。

算术减法

当以补码形式表示负数时，两个二进制带符号数的减法是非常简单的，具体步骤如下：

取减数的补码（包括符号位），将其与被减数（包括符号位）相加，丢弃符号位上所产生的进位。

这种方法的产生是因为在算术运算中，如果改变减数的符号，就可以用加法代替减法运算。下面的关系式可以说明：

$$(\pm A) - (+B) = (\pm A) + (-B)$$

$$(\pm A) - (-B) = (\pm A) + (+B)$$

把一个正数变为负数是很容易的，只要取它的 2 的补码就行了。反过来也很容易，因为负数补码的补码可产生等值的正数。我们来看减法 $(-6) - (-13) = +7$ ，用八位二进制数可将其写成 $(11111010 - 11110011)$ 。通过取减数 (-13) 的 2 的补码可以得到 $(+13)$ ，减法变成了加法。在二进制数的算术运算中，就是 $11111010 + 00001101 = 100000111$ ，丢弃最后进位，我们得到正确的答案是 00000111 (+7)。

对于用“符号-补码”法表示的二进制数，如果按照无符号数的基本运算法则进行相同的加法和减法运算，那是没有意义的。因此，计算机只需要普通的硬件电路来处理这两种类型的算术运算。鉴于此，计算机系统中几乎所有的算术单元都采用了“符号-补码”系统。用户或程序员必须要了解的是，加法或减法运算结果的差异将取决于这些数究竟是带符号数还是无符号数。

9.10.3 二进制减法器

如前所述，无符号二进制数的减法可通过补码方式很方便地实现。减法 $A-B$ 可通过将 B 的补码与 A 相加来实现。二进制数补码的实现可以先求其反码，然后在反码的最低有效位上加 1。反码可通过非门实现，1 可以通过输入进位与和相加。

减法 $A-B$ 的实现电路包括一个加法器，并且在每个数据输入 B 与其对应的全加器输入之间都有一个非门。当进行减法运算时，输入进位 C_0 必须要等于 1。这样，该运算就变成了 A 加上 B 的反码，再加 1。对于无符号数，如果 $A \geq B$ 就给出 $A-B$ ；若 $A < B$ ，则得到 $B-A$ 的补码。对于带符号数，假设没有溢出，结果就是 $A-B$ 。

加法和减法运算可包含在一个普通的二进制加法器电路里。可以通过将每个全加器加上一个异或门来实现，得到的四位加减器电路如图 9.50 所示。

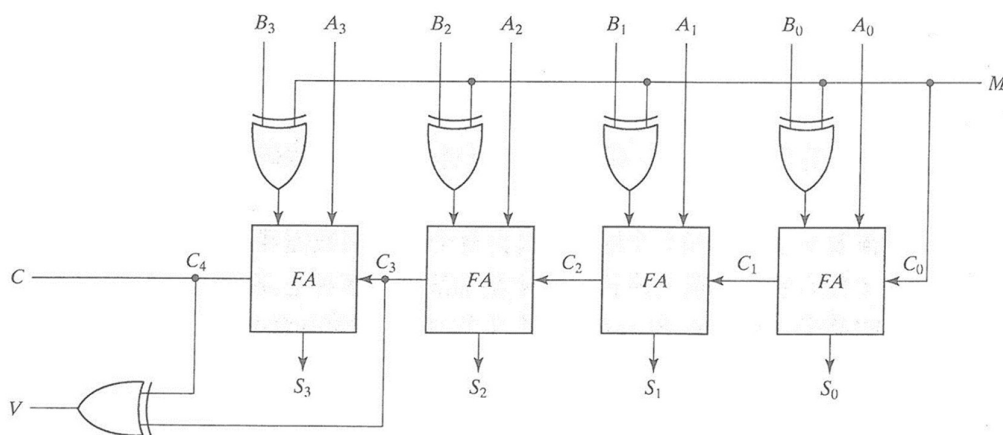


图 9.50 四位加减法器（带溢出检测）

模式输入 M 控制运算类型，当 $M=0$ 时，此电路就是加法器；而当 $M=1$ 时，该电路就成了减法器。每个异或门都收到输入 M 和 B_i ($i=0, \dots, 3$)。当 $M=0$ 时，有 $B \oplus 0 = B$ 。全加器接收到 B 的值，输入进位为 0，电路实现的是 $A+B$ 。当 $M=1$ 时，有 $B \oplus 1 = B'$ ，且 $C_0=1$ ，输入 B 被取反，并且通过输入进位加上 1。该电路实现了 A 加上 B 的补码的运算（输出为 V 的异或门是为了检测溢出）。

溢出

如果两个 n 位数相加的和是 $n+1$ 位，则称为溢出。不管对二进制数还是十进制数，也不管是否有符号，这一点是肯定的。当用纸笔进行加法操作时，溢出不是问题，因为求解时纸的页面大小不受限制。而在数字计算机中，由于计算机字长的限制，无法容纳 $n+1$ 位的数，溢出就成为了问题。正由于这个原因，许多计算机都会检测溢出的发生。一旦发生溢出，就会有相应提示，使用户能够察觉。

要检测出两个二进制数相加后是否溢出，取决于这些数是带符号数还是无符号数。当两个无符号数相加时，溢出要在最高有效位的最后进位检测。而在带符号数的情况下，最左边位总是代表符号，且负数用补码表示。当两个带符号数相加时，符号位被当作数的一部分，且最后进位不表示为溢出。

既然一个正数和一个负数相加产生的结果比原先两个数中大的数还要小。因此，若加法运算中有一个数是正数而另一个数为负数，就不会产生溢出。若两个数都为正数或者都为负数，则相加的结果就可能产生溢出。下面我们通过一个例子来看看溢出是如何发生的。假设两个带符号数 +70 和 +80 存储在两个 8 位寄存器里。每个寄存器能容纳数的范围从二进制的 +127 到 -128。因为这两个数相加的和是 +150，超过了 8 位寄存器容量。如果两个数都为正或都为负，就可能发生这种情况。两个二进制加法及最后的两个进位描述如下：



要注意，本该为正数的 8 位数却有一个负的符号（第 8 位），而本该为负数的 8 位数却有了一个正的符号。然而，如果符号位上的进位作为结果的有效位，则这样获得的 9 位结果才是正确的。因为结果超出了 8 位数，我们就说产生了溢出。

溢出的产生可以由进入到符号位上的进位和符号位上的输出进位来检测。若这两个进位不相等，则产生了溢出。上面例子中的两个进位清楚地说明了这一点。将这两个进位加到一个异或门，若该门的输出等于 1，表明发生了溢出。要使这种方法工作正确，补码必须要通过反码加 1 来计算。这就要注意最大负数求反的情况。

输出为 C 和 V 的二进制加减器如图 9.49 所示。如果把两个二进制数当作无符号数，则 C 位就会检测到一个相加后的进位或相减后的借位。若把这些数当成带符号数，则 V 位就检测溢出。若相加或相减后 $V = 0$ ，就表明没有溢出，且 n 位的结果正确。如果相加或相减后 $V = 1$ ，则运算结果有 $n + 1$ 位，但是该数只有右边的 n 位有效，因此产生了溢出。第 $n + 1$ 位实际上是符号，但被移出了位置。

第 10 章 时序逻辑电路设计

前面几章所介绍的电路都是组合电路，这类电路的特点是：在稳定状态下，任一时刻的输出完全取决于该时刻的输入。尽管任何数字系统都可能含有组合逻辑电路，但实际中遇到的系统往往都需要保存状态信息，这就需要另一类称为时序逻辑的电路。在这样的电路中，输出不仅取决于当前输入值，也取决于以前的输入值。也就是说，一个时序电路可以存储、保留该系统过去的一些历史，并在以后需要的时候检索相关信息，即它具有记忆功能。

第一节 时序电路基本概念

时序电路的框图如图 10.1 所示，它由组合电路和存储单元（有时称为存储元件或时序元件）构成。存储单元作为反馈、用于存储二进制信息。在任一给定时刻，存储单元存储的二进制信息被定义为该时刻时序电路的状态。时序电路从外部输入接收信息，这些输入连同存储单元的当前状态（现态）一起决定了输出信号。同时，它们也是存储单元状态改变的条件。从图中可以看出，时序电路的输出既与当前输入有关，又与存储单元的现态有关。存储单元的下一个状态（次态）是当前外部输入和现态的函数。因此，时序电路是由输入、输出和内部状态的时序决定的。相反，组合逻辑电路的输出仅由当时的输入决定。

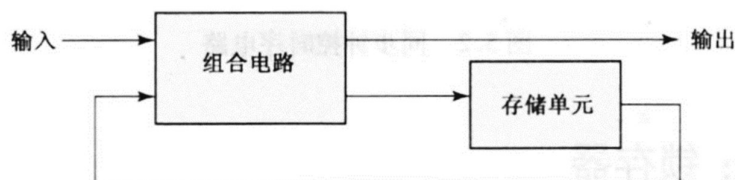


图 10.1 时序电路

从信号的时序关系上看，时序电路分同步时序电路和异步时序电路两大类。同步时序电路的输出可以由离散时刻的信号确定，异步时序电路的输出取决于输入信号和输入变化的顺序。在门电路构成的异步系统中，存储单元由逻辑门构成，它们为传输延迟提供了必要的存储功能。异步时序电路可认为是带反馈的组合电路，由于逻辑门的反馈，异步时序电路经常表现出不稳定，这给设计者带来了许多难题。因此，一般的时序系统较少采用异步时序电路，而是以同步时序电路为主。

同步时序电路只在离散时刻才对存储单元产生作用。同步是通过时钟发生器这样的定时器获得的，时钟发生器能够产生周期性的脉冲序列（时钟信号），通常用标识符 *clock* 和 *clk* 表示。同步时序系统中，所有存储单元均在一个统一的时钟信号（称为全局时钟）

控制之下，存储单元中存储的信息只有在时钟脉冲到来时才会受到影响。实际上，时钟脉冲和其他信号(包括外部输入信号等)一起确定存储单元的存储内容和输出是否改变。例如，一个带二进制加法和存储功能的电路，当时钟脉冲信号出现时，计算数字之和并将其存储下来。时钟作用于存储单元的时序电路有时称为钟控时序电路 (*clocked sequential circuits*)。在实际中经常遇到同步时序电路，它们没有不稳定的问题，时序 (*timing*) 可以很容易分解为单独的时段，然后对每一个时段分别考虑。

同步时序电路中的时序元件通常采用触发器 (*flip-flop*，有时称为寄存器) 或锁存器 (*latch*)。触发器是能够存储二进制信息的存储单元，在稳定状态下，触发器的输出为 0 或 1。下图 10.2 (a) 所示是一种使用触发器作为存储元件的同步时序电路框图，电路的输出既可以来自组合电路，也可以来自触发器。当时钟脉冲发生时，新的值被存储(也就是触发器被更新)。在时钟脉冲发生之前，产生下一个触发器值的组合逻辑必须要稳定下来。因此，组合逻辑电路的工作速度是非常重要的。由图 10.2 (b) 中的时序波形图可知，触发器从组合电路接收输入信号，从时钟获得固定间隔发生的脉冲。传播延迟应在电路工作允许的最小间隔脉冲之间。触发器的状态只有在时钟脉冲变化时(例如，当时钟脉冲从 0 变为 1)才会改变。当时钟信号无效时，即使组合电路的输出发生变化，触发器的输出也不会改变。这样，反馈回路相当于开路。因此，触发器从一个状态到另一个状态的转变仅仅发生在时钟脉冲变化的时间上。

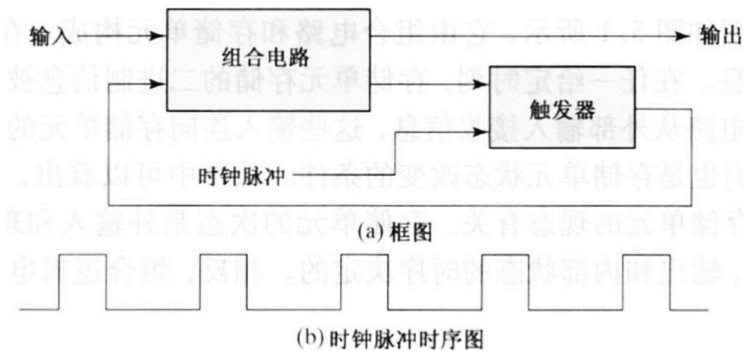
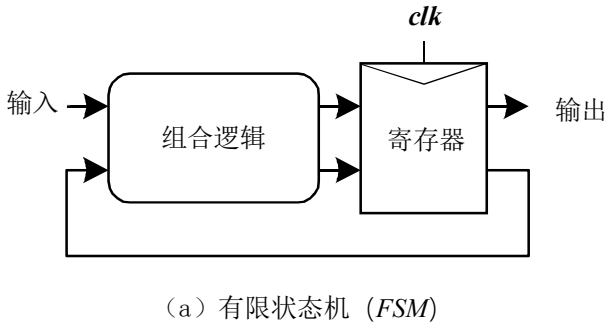
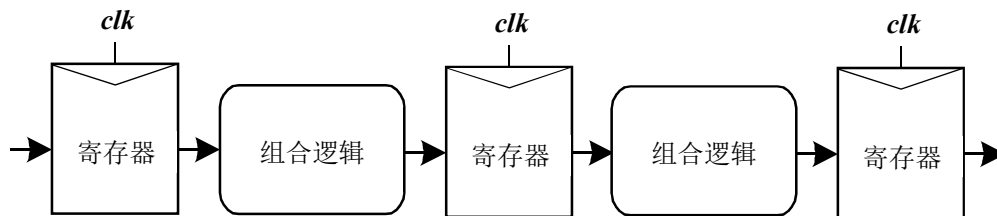


图 10.2 同步时序电路

有限状态机和流水线是两种最典型的时序电路例子，如下图 10.3 所示。





(b) 流水线 (Pipeline)

图 10.3 时序电路举例

10.1.1 存储单元的分类

前台存储器与后台存储器

存储器在高层次上可以分为前台存储器和后台存储器。嵌入在逻辑中的存储器称为前台存储器，经常组织成单个的寄存器或寄存器组 (*register bank*)。大量的集中存储内核称为后台存储器 (将在以后的课程中进行专门介绍)，它通过有效使用阵列结构、以性能及稳定性为代价换取小尺寸以达到较高的面积密度。本章中将集中介绍前台存储器。

静态存储器与动态存储器

存储器可以是静态的或动态的。只要接通电源，静态存储器就会一直保存存储的状态。它是基于正反馈或再生性原理构成的，其电路拓扑结构有意识地把一个组合电路的输出和输入连在一起。当存储的内容在较长时间内不被更新时静态存储器最为适用，例如上电时装入的系统设置数据 (静态数据)。大多数采用有条件时钟控制 (即门控时钟) 的处理器也属这种情形，在这些处理器中，不运行模块的时钟被关闭掉，因此不能确定时钟对寄存器的控制将有多频繁，需要由静态存储器来保持静态信息。基于正反馈的存储器属于多谐振荡器电路类型，双稳态单元则是多谐振荡器电路中最普遍的代表，其他单元入单稳态和不稳态电路也常使用。

动态存储器的数据只能保存很短一段时间，例如几毫秒。它们的工作原理是在与 MOS 器件相关的寄生电容上暂时存储电荷，达到保存信息的目的。与前面讨论的动态逻辑一样，这些电容必须周期性地刷新以弥补泄漏的电荷。动态存储器往往结构比较简单，因而具有明显较小的面积、较高的性能和较低的功耗。它们最常用在要求较高性能水平和采用周期时钟控制的数据通路电路中。甚至当电路由条件时钟控制时，只要一个模块在进入闲置模式时它的状态可以不用保留，就仍然可以采用动态电路。

锁存器与寄存器

锁存器是构成边沿触发器 (有时也称为寄存器) 的主要部件。它是一个电平敏感电路，即在时钟信号为高电平时把输入端 (D) 信号传送到输出端 (Q)，此时锁存器处于透明 (*transparent*) 模式。当时钟为低电平时，在时钟信号由高电平变为低电平时 (下降沿) 被采样的输入数据在输出端处整个阶段都保持稳定，此时锁存器处于维持 (*hold*)

模式。按这种方式工作的锁存器称为**正锁存器**。与正锁存器相对应，一个**负锁存器**在时钟信号为低电平时把输入 D 传送到输出 Q ，在时钟为高电平时输出端保持稳定。正锁存器和负锁存器也分别称为高电平透明(*transparent high*)和低电平透明(*transparent low*)锁存器。下图 10.4 为正锁存器和负锁存器的电路符号及信号波形(时序)。

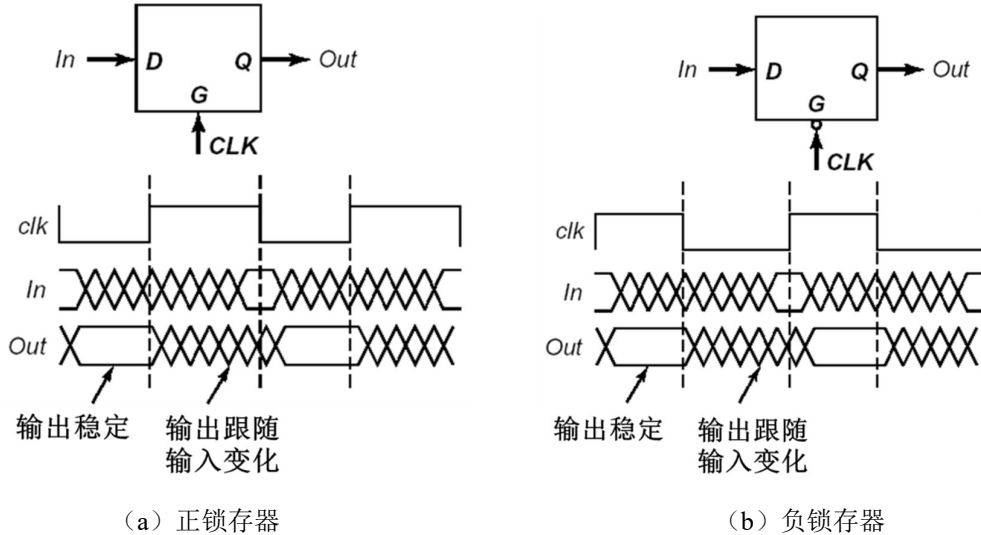


图 10.4 正锁存器于负锁存器时序

不同于电平敏感锁存器，边沿触发的寄存器只在时钟翻转时才采样输入： $0 \rightarrow 1$ 翻转时采样称为正沿（上升沿）触发寄存器，而 $1 \rightarrow 0$ 翻转时采样称为负沿（下降沿）触发寄存器。它们通常是用图 10.4 的锁存器基本单元构成的。一个经常出现的寄存器形式是主-从（*master-slave*）结构，它把一个正锁存器和一个负锁存器串联起来。寄存器也可以用单脉冲（*one-shot*）时钟信号发生器构成“毛刺”寄存器（“*glitch*” register），或用其他特殊结构构成。

10.1.2 时序电路的时间参数

有三个重要的时序参数与寄存器有关，它们都表示在下图 10.5 中。建立时间（ t_{su} ）是在时钟翻转（对正沿触发寄存器为 $0 \rightarrow 1$ 的翻转）之前数据输入（ D ）必须有效的的时间。维持时间（ t_{hold} ）是在时钟边沿之后数据输入必须仍然有效的的时间。假设建立时间和维持时间都满足要求，那么输入端 D 处的数据最晚在传播延时 t_{c-q} （相对于时钟边沿）之后被复制到输出端 Q 。

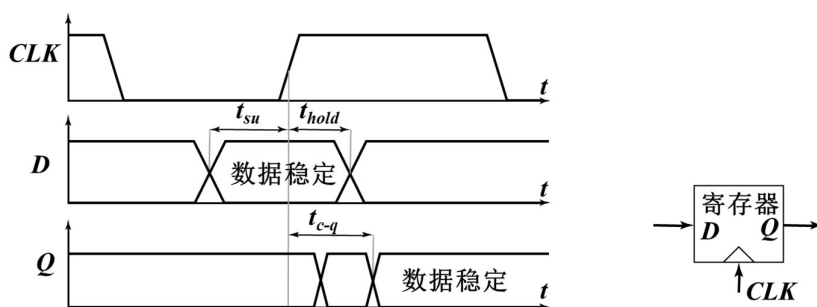


图 10.5 同步寄存器的建立时间、维持时间和传播延时的定义

一旦了解了寄存器和组合逻辑模块的时序信息，就可以推导出系统级的时序约束条件。在同步时序电路中，对时钟激励做出响应的开关事件是同时发生的。运行的结果必须等到下一个时钟翻转时才能进入到下一级。也就是说，只有在当前所有的计算都已经完成且系统开始闲置时下一轮的运算才能开始。因此时序电路工作的时钟周期 T 必须能容纳电路中任何一级的最长延时。假设一个逻辑最坏情形下的延时等于 t_{logic} ，而它的最小延时——也称为污染延时（*contamination delay*）——为 t_{cd} 。时序电路正确工作所要求的最小时钟周期 T 为：

$$T \geq t_{c-q} + t_{logic} + t_{su}$$

时序电路正确工作的另一个约束时对寄存器维持时间的要求，即：

$$t_{cd, register} + t_{cd, logic} \geq t_{hold}$$

这里， $t_{cd, register}$ 是寄存器的最小传播延时（或称污染延时）。这一约束保证了时序元件的输入数据在时钟边沿之后能够维持足够长的时间，而不会因新进入的数据流而过早改变。

正如从上式看到的，使得与寄存器有关的时序参数值最小非常重要，因为它们直接影响时序电路能被时钟控制的速度。事实上，现代高性能系统的特点是逻辑深度很低，因此寄存器的传播延时和建立时间在时钟周期中占的比例较大。例如，DEC Alpha EV6 微处理器的最大逻辑深度是 12 个逻辑门，它的寄存器时间开销（建立时间和传播延时）大约占据了时钟周期的 15%。一般来说，维持时间的约束不难满足，虽然当寄存器之间存在很少或没有逻辑电路时这可能会成为一个问题。

第二节 存储单元电路：静态锁存器和寄存器

10.2.1 双稳态原理

静态存储器用正反馈来建立双稳电路，这一电路具有两个稳定状态，分别代表 0 和 1。这一基本思想表示在下图 10.6 (a) 中，图中显示了两个反相器相串联的电路以及这样一种电路典型的电压传输特性，图中还画出了第一个反相器的 VTC（即 V_{o1} 对于 V_{i1} 的关系）以及第二个反相器的 VTC（ V_{o2} 对于 V_{o1} 的关系），后一曲线被翻转以强调 $V_{i2} =$

V_{o1} 。现假设第二个反相器的输出 V_{o2} 连到第一个反相器的输入 V_{i1} ，如图 10.6 (a) 中的虚线所示。由此得到的电路只有三个可能的工作点 (A , B 和 C)，如合在一起的 VTC 所示。很容易验证以下重要的推测：

当翻转区中反相器的增益大于 1 时，只有 A 和 B 是稳定的工作点，而 C 是一个亚稳态工作点。

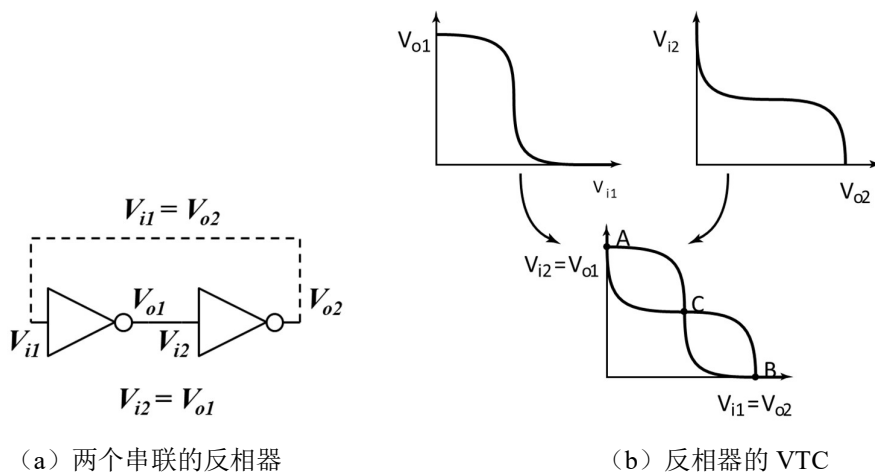


图 10.6 两个串联的反相器和它们的 VTC

假设交叉耦合的一对反相器偏置在 C 点，那么对这一偏置点的一个小偏移（也许由噪声引起）会沿这一电路环路被放大和再生。这是由于沿此环路的增益大于 1 造成的。这一影响显示在图 10.7 (a) 中假设一个小偏移 δ 加在 V_{i1} 上（偏置在 C 点）。这一偏移被第一个反相器的增益所放大，放大的偏差又加到第二个反相器并再次被放大。于是偏置点从 C 点移开，直到达到 A 或 B 中的一个工作点为止。因此 C 是一个不稳定的工作点。每一个偏移（甚至是最小的偏移）都会使工作点远离它原来的偏置。交叉耦合的一对反相器偏置在 C 点并保持在那里的机率是很小的。具有这一特性的工作点称为亚稳态 (*metastable*)。

反之， A 和 B 是稳定的工作点，如图 10.7 (b) 所示。在这些点上，环路增益比 1 要小很多。即使从这两个工作点有相当大的偏移也会被减小直至消失。

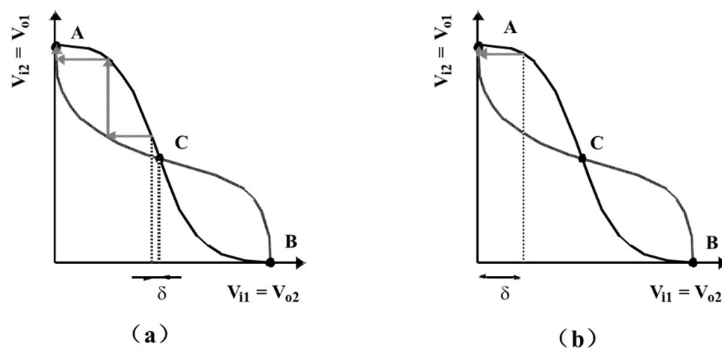


图 10.7 亚稳态与稳态工作点

因此交叉耦合的两个反相器形成了双稳电路——也就是一个电路具有两个稳定状态，每一个对应一个逻辑状态。这一电路可用做存储器，存放 1 或 0（相应于位置 A 和 B ）。

为了改变存放的值，我们必须能把电路从状态 A 变为 B ，反之亦然。由于稳定性的先决条件是环路增益 G 小于 1，所以可以通过使 G 增加到一个大于 1 的值而使 A （或 B ）暂时不稳定来达到改变状态的目的。一般的做法是在 V_{i1} 或 V_{i2} 处加一个触发脉冲。例如，假设该系统处在位置 A （ $V_{i1}=0$ ， $V_{i2}=1$ ）。迫使 V_{i1} 变为 1 会引起两个反相器同时导通一小段时间，并使环路增益大于 1。正反馈再生了触发脉冲的效应，于是电路移向另一状态（在这里为状态 B ）。触发脉冲的宽度只需稍大于沿该电路环路的传播延时，也就是反相器平均传播延时的 2 倍。

总之，双稳电路具有两个稳态。在不存在任何触发的情形下，电路保持在单个状态（假设电源一直加在该电路上），因而记忆了一个值。

双稳电路另一个常用的名字是触发器（flip-flop）。触发器只有在能使电路从一种状态变为另一种状态时才有用。这一般可以采用如下两种方法来实现：

- **切断反馈环路。**一旦反馈环路打开，一个新的值就能很容易地写入 Out （即 Q ）。这样的锁存器称为多路开关型锁存器，因为注意到一个同步锁存器的逻辑表达式与多路开关的公式相同：

$$Q = \overline{Clk} \cdot Q + Clk \cdot D$$

这一方法在当前的锁存器中非常普遍。

- **触发强度超过反馈环。**在触发器的输入端加上一个触发信号，因其强度超过存储值而迫使一个新的值进入该单元。要做到这一点需要仔细地确定反馈环和输入电路中晶体管的尺寸。一个弱的触发电路可能无法胜过一个强的反馈环路。这一方法在早期的数字设计中通常比较流行，但现在已逐渐失去了人们的青睐。然而它是实现静态后台存储器（将在以后的课程中详细讨论）的主要方法。

10.2.2 静态锁存器

SR 锁存器

如前所述，使双稳元件改变状态的一种方法是在其输入端加上一个触发信号并使其强于反馈环路。最简单的具体实现方法是人们熟知的 SR （*set-reset*）锁存器，其电路是由两个相互耦合的“或非”门（也可以是“与非”门）构成的。它有两个输入端，标注 S 的是置位端（*set*），标注 R 的是复位端（*reset*）。由两个 NOR 门相互耦合构成的 SR 锁存器电路如下图 10.8 所示。这个电路与交叉耦合的反相器对类似，只是用 NOR 门代替了反相器。NOR 门的第二个输入连到了触发器输入（ S 和 R ），从而有可能强迫 Q 和 Q' 输出进入到一个指定的状态。锁存器有两个状态 0 和 1。当输出 $Q = 1$ 和 $Q' = 0$ 时，锁

存器处于置位状态；当输出 $Q = 0$ 和 $Q' = 1$ 时，锁存器处于复位状态。输出信号 Q 和 Q' 通常是互补的。然而，当两个输入端同时为 1 时，两个输出都将被强迫为 0，锁存器处于不稳定状态。因此，在实际应用中，锁存器输入端绝不允许同时为 1。在一般情况下，锁存器的两个输入端为 0，这样，状态维持不变。此时，如果给输入端 S 施加 1，将导致锁存器进入置位状态。若使锁存器处于复位状态，应给输入端 R 施加 1。为避免出现不确定状态， S 端在 R 端变化前必须先回到 0。由图 10.8 (b) 所示的功能表可知，两种输入条件下电路进入置位状态。第一种情况是： $S=1, R=0$ 。此时，通过输入端 S 有效来使电路进入置位状态。第二种情况是： $S=0, R=0$ 。把有效信号从 S 端撤开，电路维持置位状态不变。在两个输入端回到 0 之后，才能给复位端 R 加 1，使电路进入复位状态，把 1 从 R 端撤走，电路维持复位状态不变。因此，当输入端 S 和 R 都为 0 时，锁存器或处于置位状态，或处于复位状态，关键是要看在此之前，是哪个输入端信号有效。

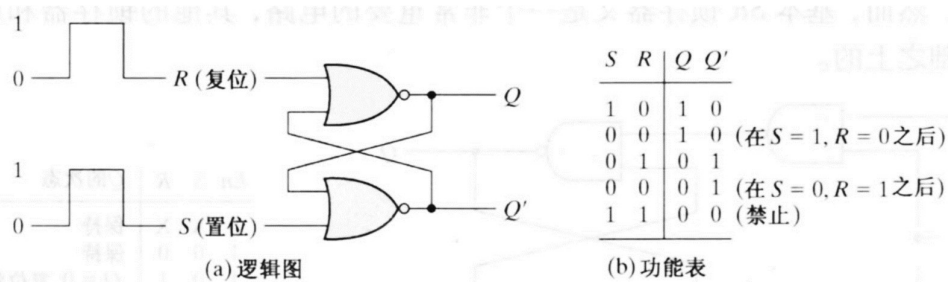


图 10.8 由或非门构成的 SR 锁存器

如果 1 同时加到了输入端 S 和 R ，两个输出都强迫为 0。但当两个输入端同时回到 0 之后，次态到底是 0 还是 1 则无法确定，因此将产生不确定状态。另外，输出同时为 0 也与输出必须互补相矛盾。在通常情况下，应避免出现这种现象，采取的方法是两个输入端不能同时有效。

由与非门构成的 SR 锁存器电路如下图 10.9 所示。一般情况下，该电路两个输入都为 1，状态维持不变。只有需要改变锁存器的状态时，才改变输入信号。对输入端 S 加 0 将导致输出为 1，使锁存器进入置位状态。当 S 恢复到 1 后，电路仍旧维持置位状态不变。若两个输入端同时为 1，然后输入端 R 加 0，将导致电路进入复位状态，即使两个输入端回到 1，状态还将保持不变。对于与非门构成的锁存器，两个输入端同时为 0 时，不确定的状态发生，这种输入组合应避免。

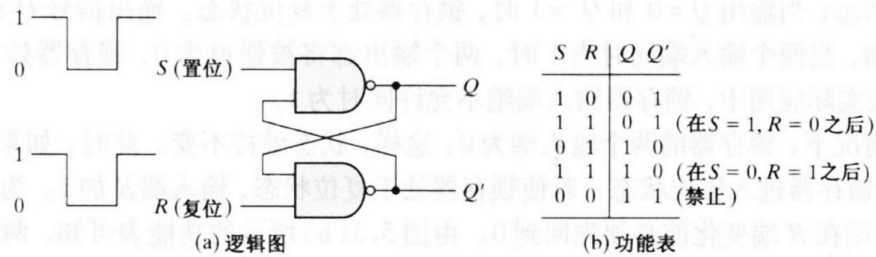


图 10.9 由与非门构成的 SR 锁存器

将与非门构成的锁存器和或非门构成的锁存器做个对比，可以发现，前者输入低电平有效，而后者高电平有效，两者恰恰相反。

前面我们讨论的 *SR* 锁存器完全是非同步的，它与当前主流集成电路所偏爱的同步设计方法不太匹配。对基本 *SR* 锁存器结构进行修改，利用一个额外的控制输入端来决定锁存器状态改变的时机，从而实现同步控制。带有控制输入端的 *SR* 锁存器如下图 10.10 所示，它由一个基本 *SR* 锁存器和两个与非门构成。控制输入端 (*Clk*) 是另外两个输入信号的使能信号。只要控制输入信号 *Clk* 为 0，与非门的输出就维持 1 不变。当 *Clk* 变为 1 时，输入 *S* 和 *R* 对 *SR* 锁存器有作用。当 *S*=1, *R*=0 和 *Clk*=1 时，电路进入置位状态。为使锁存器进入复位状态，输入信号必须满足：*S*=0, *R*=1 和 *Clk*=1。当 *Clk* 恢复到 0 时，电路将维持当前的状态不变。控制输入信号为 0 时，无论 *S* 和 *R* 取什么值，输出状态都不会改变。另外，当 *Clk*=1, *S* 和 *R* 都等于 0 时，电路的状态也不发生改变。这种锁存器称为时钟控制 *SR* 锁存器（简称钟控 *SR* 锁存器）。

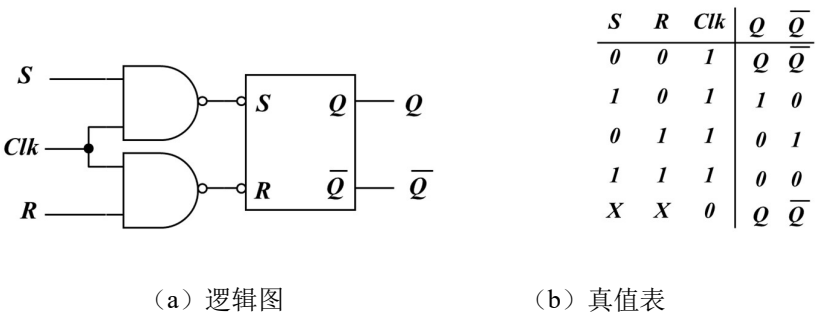
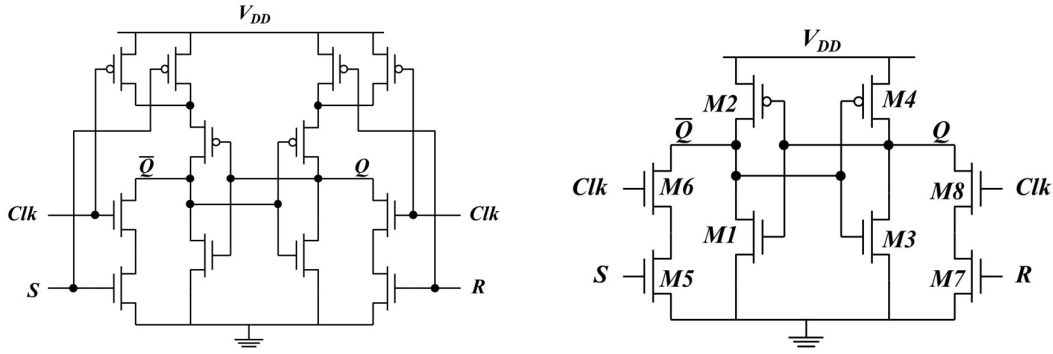


图 10.10 带时钟控制输入的 *SR* 锁存器

当三个输入都为 1 时，会发生一种不确定的情况。此时，基本 *SR* 锁存器的两个输入端都为 0，使电路进入不稳定状态。当控制信号回到 0 后，由于无法确定 *S* 和 *R* 哪一个将先回到 0，所以也就无法确定次态是什么。这种不确定的情况使电路非常难以维护，因此在实际中很少使用。然而，基本 *SR* 锁存器又是一个非常重要的电路，其他的锁存器和触发器都是建立在其基础之上的。

下图 10.11 所示是钟控 *SR* 锁存器的 CMOS 电路形式，其中图 (a) 是直接以互补 CMOS 电路实现，图 (b) 是简化的时钟同步 *SR* 锁存器。它包括一对交叉耦合的反相器，加上 4 个额外的晶体管来驱动锁存器从一种状态转变到另一种状态，并实现同步。在稳定状态，一个反相器处于高电平状态，而另一个则在低电平。*V_{DD}* 和 *GND* 之间不存在静态路径，然而晶体管的尺寸对保证锁存器在需要时能从一种状态转变到另一种状态至关重要。



(a) 互补 CMOS 实现的钟控 SR 锁存器

(b) 简化的钟控 SR 锁存器

图 10.11 CMOS 钟控 SR 锁存器

下面来分析上图 10.11 (b) 中的时钟控制 SR 锁存器的晶体管尺寸要求。

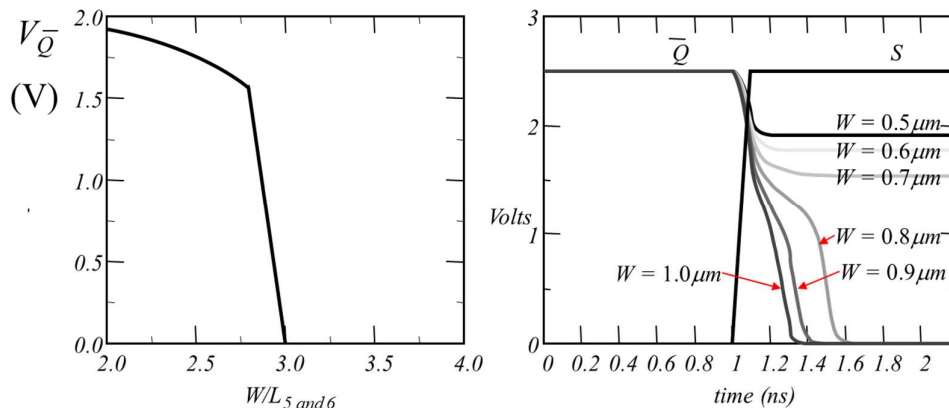
考虑 Q 位于高电平并加入一个脉冲 R 的情况。为了使锁存器切换，必须要成功地使 Q 降低到反相器 $M1$ - $M2$ 的开关值。一旦做到这一点，正反馈会使锁存器变为相反状态。为满足这一要求，需要增加晶体管 $M5$, $M6$, $M7$ 和 $M8$ 的尺寸。晶体管 $M4$, $M7$ 和 $M8$ 的组合形成了一个有比反相器。假设把这对交叉耦合的反相器设计成反相器的阈值 V_M 处在 $V_{DD}/2$ 。对于 $0.25\mu\text{m}$ CMOS 工艺选择如下管尺寸: $(W/L)_{M1} = (W/L)_{M3} = (0.5\mu\text{m}/0.25\mu\text{m})$ 和 $(W/L)_{M2} = (W/L)_{M4} = (1.5\mu\text{m}/0.25\mu\text{m})$ 。现假设 $Q=0$ ，确定保证该器件能切换的 $M5$, $M6$, $M7$ 和 $M8$ 的最小尺寸。

为了使锁存器能从 $Q=0$ 切换到 $Q=1$ 状态，这一有比的伪 NMOS 反相器($M5$ - $M6$)- $M2$ 的低电平应当低于反相器 $M3$ - $M4$ 的开关阈值($V_M = V_{DD}/2$)。可以假设只要 $V_{Q'} > V_M$ ， V_Q 就等于 0，于是晶体管 $M2$ 的栅接地。晶体管尺寸的边界条件可以通过使 $V_{Q'} = V_{DD}/2$ 时反相器中的电流相等来推导，如下面的方程所示（这里忽略了沟道长度调制效应）。

$$k'_n \left(\frac{W}{L} \right)_{5-6} \left((V_{DD} - V_{Tn}) V_{DSATn} - \frac{V_{DSATn}^2}{2} \right) = -k'_p \left(\frac{W}{L} \right)_2 \left((-V_{DD} - V_{Tp}) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right)$$

电流取决于饱和电流，因为 $V_S = V_{DD} = 2.5\text{V}$ ，而 $V_M = 1.25\text{V}$ 。假设 $M5$ 和 $M6$ 的尺寸相同， $(W/L)_{5-6}$ 是串联器件的等效比。在这一条件下，下拉网络可以用一个晶体管 M_{5-6} 来模拟，其长度是单个器件长度的两倍。

使用 $0.25\mu\text{m}$ 工艺的参数，由上面方程得到等效比的限制条件为 $(W/L)_{M5-6} \geq 2.26$ 。这意味着 $M5$ 或 $M6$ 的单个器件尺寸比必须大于 4.5。下图 10.12 (a) 显示 $V_{Q'}$ 与 $M5$ 和 $M6$ 单个器件尺寸关系的 DC 曲线。我们注意到当单个器件的尺寸比大于 3 时就足以使的电压达到反相器的开关值。手工分析和模拟之间的差别来自二阶效应，如沟长调制效应和 DIBL。图 10.12 (b) 画出了不同器件尺寸的瞬态响应，并证实了单个器件 W/L 比大于 3 是克服反馈回路并使锁存器状态切换的必要条件。

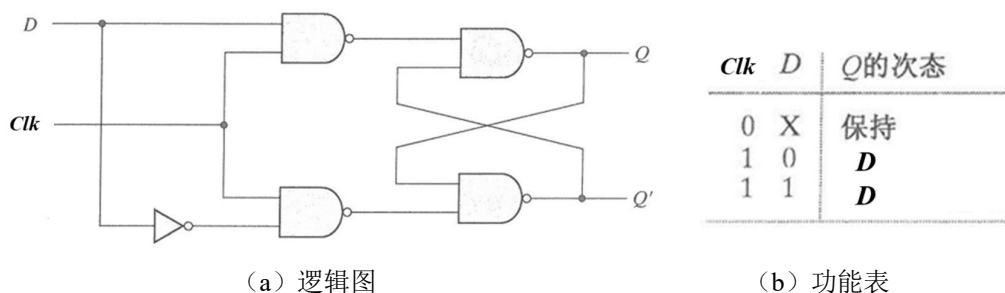


(a) DC 输出电压和下拉器件尺寸 $M_{5,6}$ 的关系 (b) 瞬态响应表明 M_5 和 M_6 各自的 W/L 比应大于 3 以切换 SR 锁存器器
($W/L_2=1.5\mu\text{m}/0.25\mu\text{m}$)

图 10.12 SR 锁存器电路的器件尺寸选择

D 锁存器

对于 SR 锁存器中的不确定状态，有一种消除的方法是确保输入信号 S 和 R 不同时为 1， D 锁存器就可以满足这种要求，如下图 10.13 所示。这种锁存器仅有两个输入端： D （数据）和 Clk （使能，或时钟）。输入端 D 直接与输入端 S 相连，它的反相输出连接到输入端 R 。只要使能信号为 0，相互耦合连接的 SR 锁存器的两个输入就都为 1，这样无论 D 取什么值，电路都不会改变状态。当 $Clk=1$ 时，输入端 D 被采样。若 $D=1$ ， Q 输出为 1，电路进入置位状态；若 $D=0$ ，输出 Q 为 0，电路进入复位状态。 D 锁存器具有存储信息的功能，因此适合作为临时存储单元使用。只要使能信号有效， D 锁存器数据输入端出现的信号将被传输至输出端 Q ，且输出数据会随着输入数据的改变而改变。这就提供了一条从输入端 D 到输出端 Q 的通道，因此这种电路也称为透明锁存器。当使能信号无效时，出现在数据输入端的信息将一直保持在输出端，直到使能信号再次有效为止。



(a) 逻辑图

(b) 功能表

图 10.13 D 锁存器

建立一个 D 锁存器最稳妥和最常用的技术是采用传输门多路开关。下图 10.14 是一个以多路开关为基础的正、负静态 D 锁存器的实现。对于一个负锁存器，当时钟为低电平时选择多路开关的输入端 0，并将输入 D 传送到输出。当时钟信号为高电平时，选择与锁存器输出相连的多路开关的输入端 1，只要时钟维持在高电平，反馈就能保证有一个稳定的输出。同样，在正锁存器中，当时钟信号为高电平时，选择 D 输入，而在时钟信

号为低电平时输出保持原状（通过反馈）。

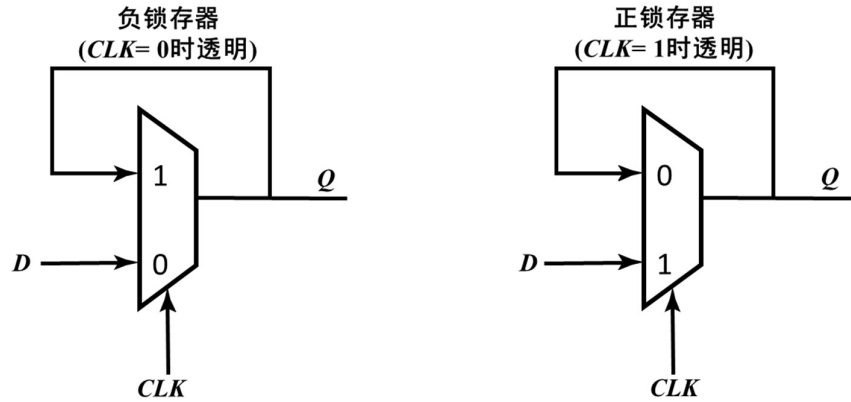


图 10.14 多路开关型正锁存器和负锁存器

下图 10.15 是基于多路开关的正锁存器晶体管级的实现。当 CLK 为高时下面的传输门导通，因而锁存器是透明的，即输入 D 被复制到输出 Q 上。在这一阶段，反馈环断开，因为上面的传输门是断开的。因此晶体管的尺寸对于实现正确功能并不重要。但从功率角度看，时钟驱动的晶体管数目是一个重要的衡量指标，因为时钟的活动性系数为 1。从这一角度来看，本例的锁存器效率不高：它对于 CLK 信号有 4 个晶体管的负载。

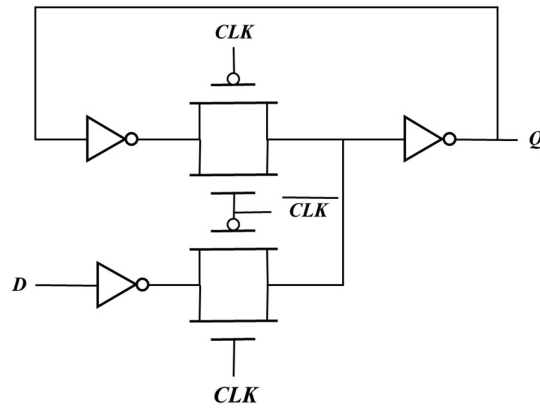


图 10.15 用传输门构成正锁存器的晶体管级实现

如果仅用 NMOS 传输管实现多路开关，则可以将锁存器的时钟负载晶体管减至两个，如下图 10.16 所示。当 CLK 为高电平时，锁存器采样输入 D ，而低电平的时钟信号则使反馈环路导通，从而使锁存器处于维持状态。这一方法虽然简单，但仅用 NMOS 传输管会使传送到第一个反相器输入的高电平下降为 $V_{DD}-V_{Tn}$ 。这对噪声容限和开关性能都会有影响，特别是在 V_{DD} 值较低而 V_{Tn} 值较高时更为如此。它也造成在第一个反相器中的静态功耗，因为该反相器的最大输入电压为 $V_{DD}-V_{Tn}$ ，所以反相器的 PMOS 器件不能完全关断。

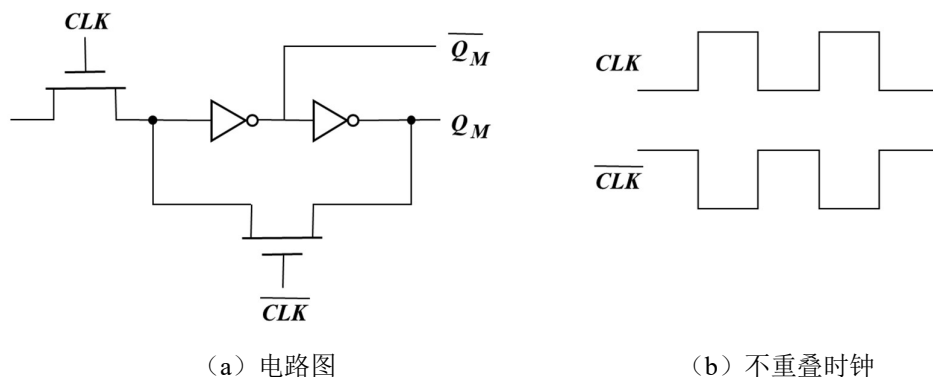


图 10.16 仅用 NMOS 传输管构成多路开关的多路开关型锁存器

10.2.3 主从边沿触发寄存器

用两个 D 锁存器和一个非门构成的主从边沿触发寄存器（有时也称为 D 触发器）如下图 10.17 所示。第一个锁存器称为主锁存器，第二个锁存器称为从锁存器。电路仅在控制时钟 Clk 的下降沿时，根据输入端 D 的值，决定输出信号 Q 的值。当时钟为 0 时，非门的输出为 1，从锁存器使能端有效，输出 Q 与主锁存器输出 Y 相同。时钟 $Clk=0$ 使得主锁存器不工作，变为 1 时，从外部 D 输入来的数据被传送给了主锁存器，只要时钟维持 1 不变，从锁存器的控制输入 En 为 0，功能被禁止，输入端的变化只影响到主锁存器的输出 Y ，但不影响从锁存器的输出。当时钟脉冲回到 0 时，主锁存器不工作，与输入端 D 断开，同时，从锁存器使能端有效， Y 的值传送给触发器输出 Q 。因此，触发器的输出只能在时钟从 1 到 0 的变化瞬间（下降沿）发生改变。

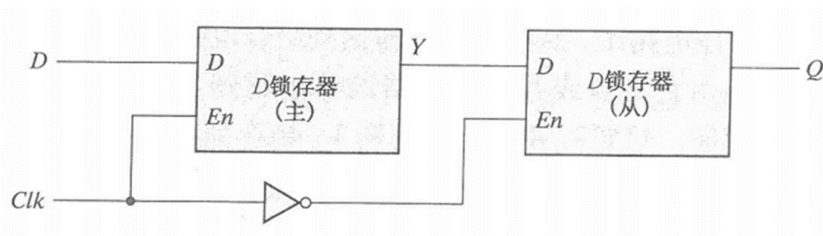


图 10.17 主从边沿触发寄存器（ D 触发器）

从上面主从触发器的行为特性可以看出，触发器输出只在时钟的下降沿发生改变。当然，我们也可以设计出电路，使得触发器的输出在时钟的上升沿变化。这种上升沿触发的触发器需要在 Clk 端和非门与主锁存器 En 端连接节点之间加一个非门即可。从下降沿触发的主从触发器可知，时钟的下降沿影响主锁存器，上升沿影响从锁存器和输出端。

采用多路开关型锁存器构成的边沿触发主从寄存器如下图 10.18 所示。在时钟的低电平阶段，主级是透明的，输入 D 被传送到主级的输出端 Q_M 。在此期间，从级处于维持状态，通过反馈保持它原来的值。在时钟的上升沿期间，主级停止对输入采样，而从

级开始采样。在时钟的高电平阶段，从级对主级的输出端(Q_M)采样，而主级处于维持状态。由于 Q_M 在时钟的高电平阶段不变，因此输出每周期只翻转一次。由于 Q 的值就是时钟上升沿之前的 D 值，因此该寄存器具有上升沿（正沿）触发效应。下降沿（负沿）触发寄存器可以用同样的原理构成，只需简单地改变一下正负锁存器的位置就可以了（即把正锁存器放在前面）。

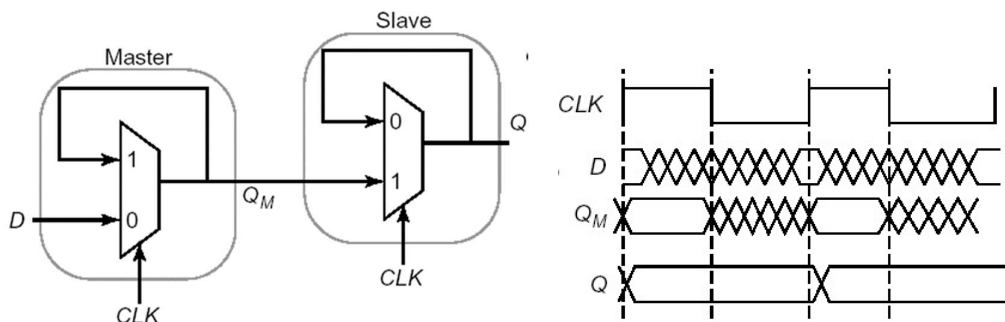


图 10.18 基于主从结构的上升沿触发寄存器

下图 10.19 是一个完整的主从正沿触发寄存器的晶体管级实现。其中，多路开关采用前面讨论过的传输门来实现。当时钟处于低电平时（ $CLK=1$ ），传输门 T_1 导通 T_2 关断，输入 D 被采样到节点 Q_M 上。在此期间，传输门 T_3 和 T_4 分别关断和导通。交叉耦合的反相器（ I_5, I_6 ）保持从锁存器的状态。当时钟上升到高电平时，主级停止采样输入并进入维持状态 T_1 关断 T_2 导通，交叉耦合的反相器 I_2 和 I_3 保持 Q_M 状态。同时， T_3 导通 T_4 关断， Q_M 被复制到输出 Q 上。

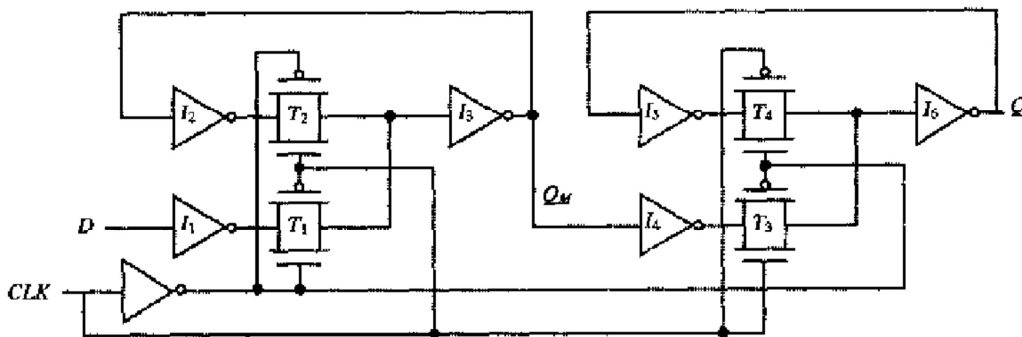


图 10.19 利用多路开关构成的主从型正沿触发寄存器

多路开关型主从寄存器的时序特性分析

寄存器有三个重要的特征时序参数：建立时间、维持时间和传播延时。了解影响这些时序参数的因素并建立手工估计它们的直观方法是很重要的。假设每一个反相器的传播延时为 t_{pd_inv} ，传输门的传播延时为 t_{pd_tx} 。同时假设污染延时（contamination delay）为 0，而且由 CLK 产生 $\overline{CLK}$ 的反相器的延时也为 0。

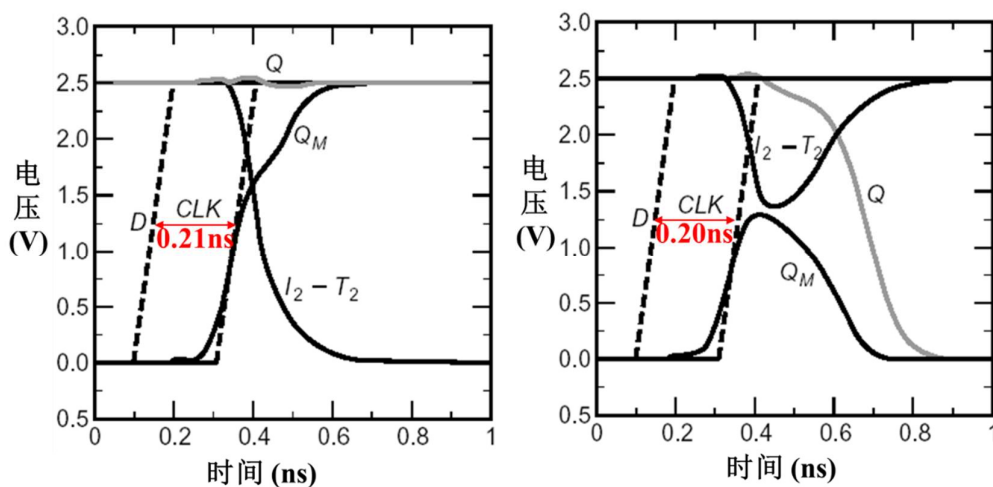
建立时间是输入数据 D 在时钟上升沿之前必须有效的的时间。这就相当于问：在时钟上升沿之前输入 D 必须稳定多长时间才能使 Q_M 采样的值是可靠的？对于传输门多路开关型寄存器，输入 D 在时钟上升沿之前必须传播通过 I_1 , T_1 , I_3 和 I_2 。这就保证了在传输门 T_2 两端的节点电压值相等。否则，交叉耦合的一对反相器 I_2 和 I_3 就可能会停留在一个不正确的值上。因此建立时间等于 $3 \times t_{pd_inv} + t_{pd_tx}$ 。

传播延时是 Q_M 值传播到输出 Q 所需要的时间。注意，由于我们在建立时间中已包括了 I_2 的延时，意味着 I_4 的输出在时钟上升沿之前已有效。因此延时 t_{c-q} 就只是通过 T_3 和 I_6 的延时 ($t_{c-q} = t_{pd_tx} + t_{pd_inv}$)。

维持时间表示在时钟上升沿之后输入必须保持稳定的时间。在这里的情形下，当时钟为高电平时，传输门 T_1 关断。由于 D 输入和 CLK 在到达 T_1 之前都要通过反相器，所以在时钟变为高电平之后输入上的任何变化都不会影响输出。因此维持时间为 0。

利用 SPICE 进行时序分析

为了用 SPICE 得到寄存器的建立时间，我们将输入 D 的翻转（从左边）逐渐靠近时钟边沿直到电路失效。图 10.20 显示了假设输入与时钟边沿偏差 210ps 和 200ps 时建立时间的模拟结果。对于 210ps 的情况，输入 D 的采样值是正确的（在这一情况下，输出 Q 维持在 V_{DD} 的值）。对于偏差 200ps 的情况，传送到输出的值是错误的，因为输出变化到了 0。节点 Q_M 开始上升，而 I_2 的输出（传输门 T_2 的输入）开始下降。然而时钟在传输门 T_2 两端的节点稳定到同一个值之前就已有效，因此造成不正确的值写入主锁存器。所以这一寄存器的建立时间是 210ps。



(a) D 早于时钟沿 210ns 变化

(b) D 早于时钟沿 200ps 变化

图 10.20 建立时间模拟

维持时间也可以用类似的方法来模拟。 D 的输边沿再次（从右边）靠近时钟信号上升沿直至电路工作失效。对于这一设计维持时间为 0（即输入可以在时钟边沿处改变）。最后，对于传播延时，输入变化要在时钟上升沿的至少一个建立时间之前完成，延时是

从 CLK 边沿的 50% 点处计算到 Q 输出的 50% 点处。从这一模拟中（见图 10.21）得到， $t_{c-q(LH)}$ 为 160ps，而 $t_{c-q(HL)}$ 为 180ps。

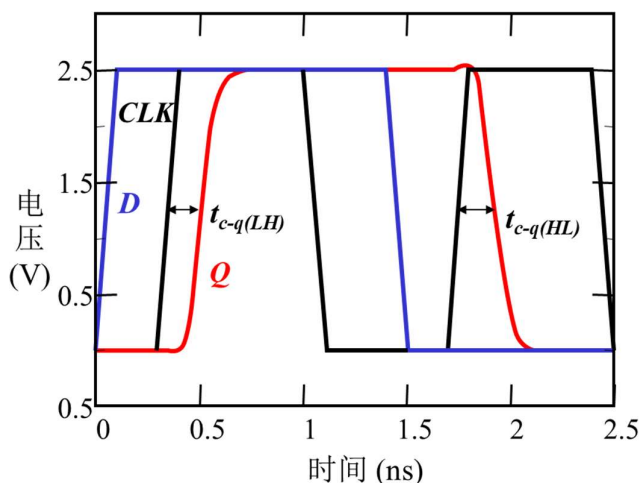


图 10.21 传输门寄存器的传播延时模拟

传输门寄存器的缺点是时钟信号的电容负载很大。每个寄存器的时钟负载是很重要的，因为它直接影响时钟网络的功耗。如果忽略使时钟信号反相所需要的开销（因为反相器的开销可以分摊在多个寄存器上），那么每个寄存器具有 8 个晶体管的时钟负载。以稳定性为代价降低时钟负载的一个方法是使电路成为有比电路。下图 10.22 显示可以直接用交叉耦合反相器来省去反馈传输门。

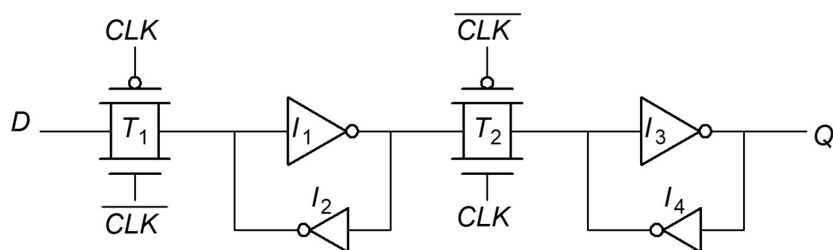


图 10.22 减少了时钟负载的静态主从寄存器

减少时钟负载的代价是增加了设计过程的复杂性。传输门 T_1 以及它的源驱动器必须比反馈环路反相器 I_2 更强才能切换交叉耦合反相器的状态。传输门所要求的尺寸可以用类似于分析电平恢复器件尺寸的方法来推导。反相器 I_1 的输入必须超过它的开关阈值以便能够产生翻转。如果在传输门中要使用最小尺寸的器件，一定要注意把反相器 I_2 的晶体管设计得更弱（较小的宽长比）。小的宽长比可以通过使晶体管的沟长大于最小尺寸来实现。在传输门中希望使用最小尺寸或近于最小尺寸的器件，以降低锁存器和时钟分配网络的功耗。

这种方法存在的另一个问题是反向传导—第二级可能反过来影响第一级锁存器的状态。当主从寄存器中的从级导通时（见下图 10.23）， T_2 和 I_4 有可能共同影响存储在

I_1 - I_2 锁存器中的数据。好在只要 I_4 是一个弱器件，这就不会成为什么大问题。

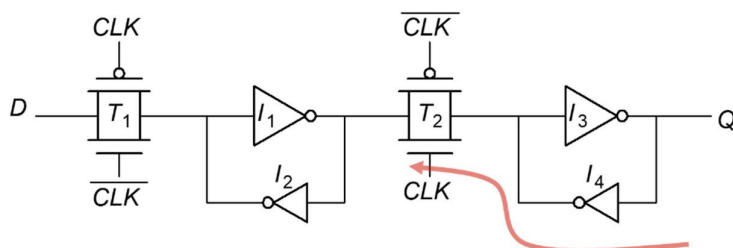


图 10.23 传输门中可能发生的反向传导

非理想时钟信号

至此我们一直假设 CLK 与 $\overline{CLK}$ 完全相反，或换言之，产生反相时钟信号的反相器的延时为 0。即便这有可能实现，但仍然不是一个很好的假设。因为布置两个时钟信号的导线会有差别，或者负载电容可以因存储在所连接的锁存器中的数据不同而变化。这一影响称为时钟偏差（*clock skew*）。这是一个很重要的问题，它会造成两个时钟信号的重叠，如图 10.24（b）所示。时钟重叠可以引起两种类型的错误，我们用图 10.24（a）中仅用 NMOS 管的主从寄存器的情况来说明这一点。

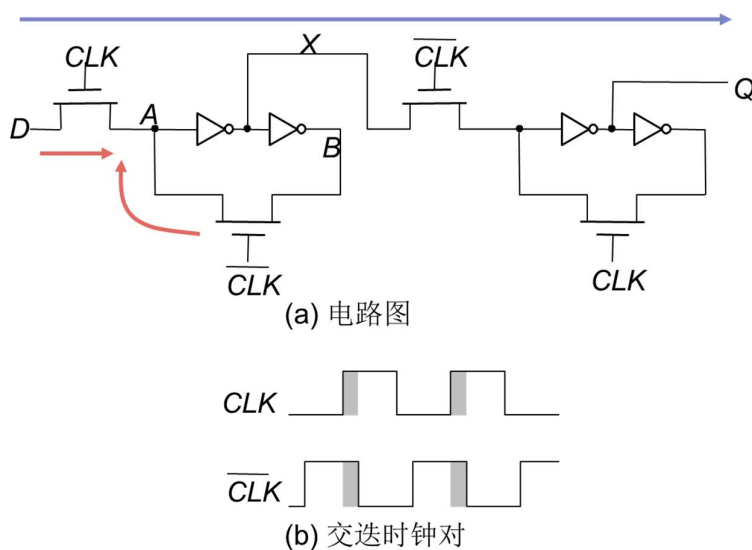


图 10.24 仅用 NMOS 传输管的主从寄存器

1. 当时钟变为高电平时，从级应当停止对主级输出的采样并进入维持状态，然而由于 CLK 和 $\overline{CLK}$ 在一个很短的时间（重叠期）内都为高电平，两个采样传输管都导通，因此在输入 D 和输出 Q 之间有直接通路（如图中蓝色箭头所示）。结果造成输出端的数据可以在时钟的上升沿改变，这是一个负沿触发寄存器所不希望的。这被称为竞争（*race*）情况，此时输出 Q 的值取决于输入 D 是在 CLK 的下降沿之前还是之后到达节点 X 。如果节点 X 在亚稳态时被采样，那么输出将会切换到由系统噪声决定的一个值（这是一个不确定值）。

2. 多路开关型寄存器的基本优点是在采样期间反馈环断开，因此器件的尺寸对功能的影响不大。然而如果在 CLK 和 $\overline{CLK}$ 之间存在时钟重叠，那么节点 A 同时被 D 和 B 驱动，造成不确定的状态（如图中红色箭头所示）。

为了避免这些问题，可以采用两相不重叠时钟 PHI_1 和 PHI_2 （见下图 10.25），并保持两相时钟之间的不重叠时间 $t_{non_overlap}$ 足多长，因而甚至在存在时钟布线延时的情况下也不会有任何重叠发生。在不重叠时间内，触发器处在高阻抗状态—反馈环开路，环路增益为零，并且输入被断开。如果这一情况保持的时间太长，漏电将破坏这一状态，因此称其为伪静态：根据时钟的不同状态，寄存器采取了静态或动态的存储方式。

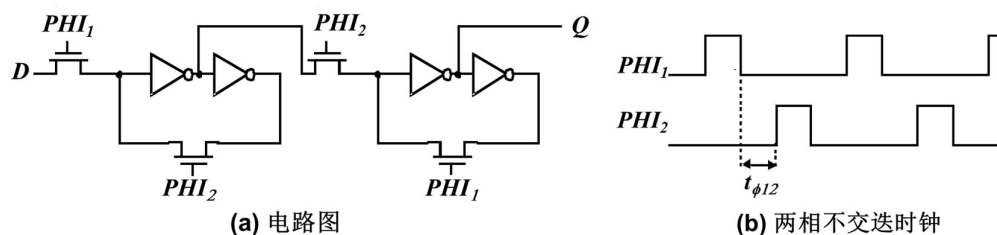


图 10.25 伪静态两相位 D 寄存器

产生不重叠时钟

下图 10.26 (a) 是产生两相不重叠时钟的时钟产生电路的一种实现方法。假设每个门具有一个单位的门延时，可以推导出输入时钟与两个输出时钟的时序关系，如图 10.26 (b) 所示。如果需要增加不重叠时间，可以在反馈路径上增加延时，如图 10.26 (c) 所示。

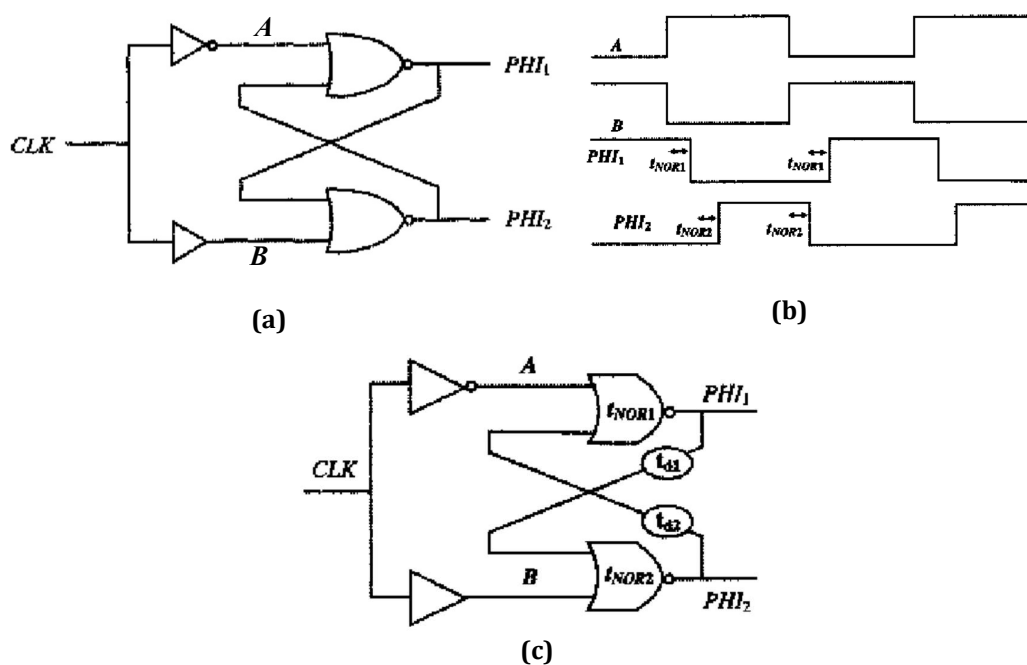


图 10.26 产生两相不重叠时钟

构成边沿触发器的另外一种有效方法是使用三个 *SR* 锁存器，如下图 10.27 所示。两个锁存器共用外部数据输入端 *D* 和时钟输入端 *Clk*，第三个锁存器产生触发器的输出。在 *Clk*=0 时，输出锁存器的 *S* 和 *R* 维持 1 不变，这使得输出状态保持不变，可能 *D*=0 或者 *D*=1。如果 *Clk* 变为 1 时，*D*=0，*R* 为 0，触发器进入复位状态，*Q*=0。如果 *Clk*=1 时，输入端 *D* 发生变化，*R* 端保持 0 不变，此时触发器被锁定，不会对输入变化做出响应。当时钟恢复到 0 时，*R* 变为 1，锁存器的输出不会改变。类似地，当 *Clk* 从 0 到 1 时，*D*=1，*S* 变为 0，这将使电路进入置位状态，*Q*=1。当 *Clk*=1 时，*D* 发生变化不会影响输出。

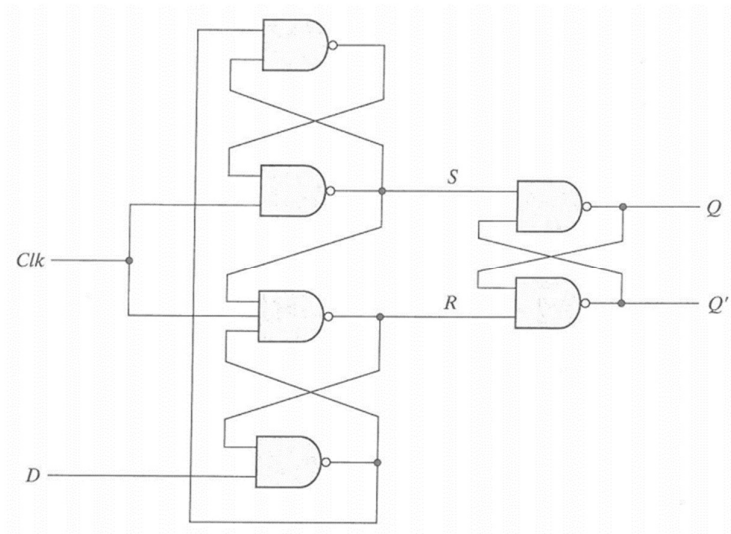


图 10.27 三个 *SR* 锁存器构成的上升沿 *D* 触发器

总之，对于上升沿触发器来说，只有当输入时钟发生上升沿变化时，*D* 的值才能传送给 *Q*。从 1 到 0 的下降沿变化不会影响触发器的输出，*Clk* 信号为稳定的 1 或 0 都不会影响输出，这种类型的触发器只对从 0 到 1 的上升沿响应。

边沿触发器图形符号如下图 10.28 所示。除了在字母 *Clk* 旁边有一个箭头符号外，其他与锁存器相似。这里的箭头符号表示动态输入，这种动态符号说明触发器只对时钟的边沿响应。在方框外与动态符号一起的圆圈表示下降沿触发，没有圆圈表示上升沿触发。

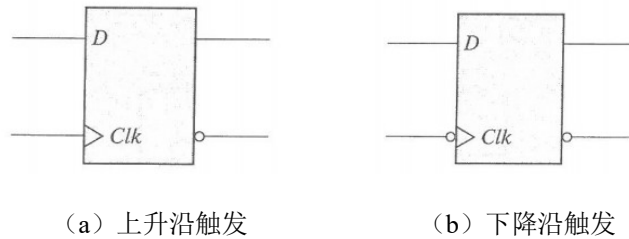


图 10.28 边沿 *D* 触发器的图形符号

10.2.4 其他触发器

最经济和最有效的触发器电路是边沿 D 触发器，它需要的门电路个数最少，其他类型的触发器可以由 D 触发器和附加的门电路构成。有两种触发器在数字系统中使用广泛，分别是 JK 和 T 触发器。

JK 触发器具有三种功能：置位（输出为 1）、复位（输出为 0）和输出翻转。下图 10.29（a）所示是由 D 触发器和门电路一起组成的 JK 触发器电路图。输入端 J 的作用是将触发器置位为 1，输入端 K 的作用是将触发器复位为 0。当这两个输入端同时有效时，输出翻转。

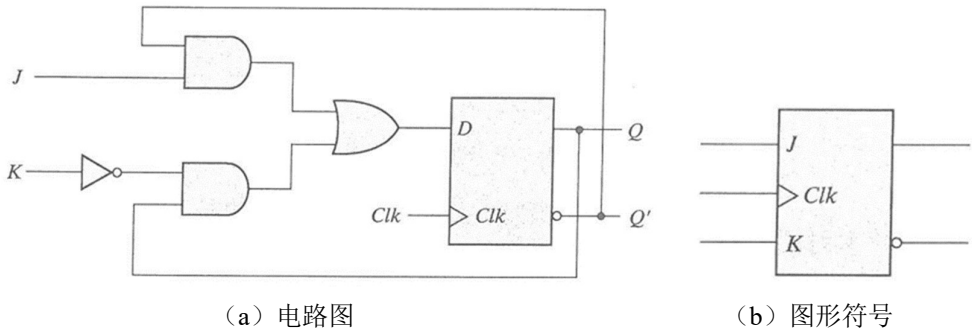


图 10.29 JK 触发器

JK 触发器的三种功能可以写成下面的表达式： $D = JQ' + K'Q$ 。当 $J=1$ 和 $K=0$ 时， $D = Q' + Q = 1$ ，因此下一个时钟沿到来时，输出将置位为 1。当 $J=0$ 和 $K=1$ 时， $D=0$ ，下一个时钟沿到来时，输出将复位为 0。当 $J=K=1$ 时， $D=Q'$ ，下一个时钟沿到来时，输出翻转。当 $J=K=0$ 时， $D=Q$ ，时钟沿到来状态保持不变。 JK 触发器的图形符号示于图 10.29（b）。可见，除了输入端命名为 J 和 K 外，它与 D 触发器的图形符号非常相似。

如果将 JK 触发器的 J 和 K 连接在一起就得到了 T （翻转）触发器，如图 10.30（a）所示。当 $T=0$ （ $J=K=0$ ）时，时钟边沿到来对状态不产生影响。当 $T=1$ （ $J=K=1$ ）时，时钟边沿到来使得输出反相。翻转触发器在设计二进制计数器时非常有用。 T 触发器也可以由 D 触发器外加一个异或门得到，如图 10.30（b）所示。 D 触发器输入端 D 的表达式为： $D = T \oplus Q = TQ' + T'Q$ 。当 $T=0$ 时， $D=Q$ ，输出状态维持不变；当 $T=1$ 时， $D=Q'$ ，输出反相。 T 触发器的图形符号中在输入端有个字母 T 。

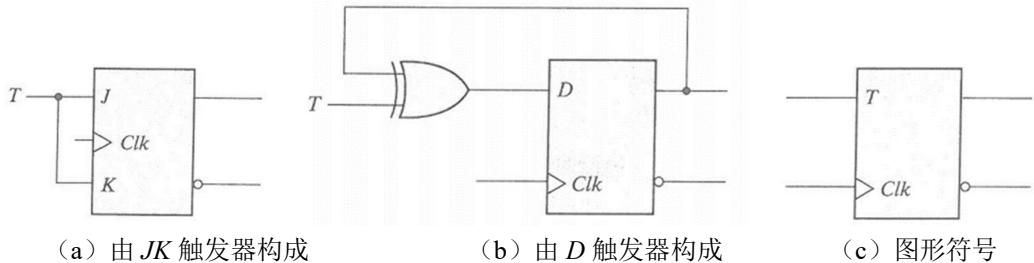


图 10.30 T 触发器

特征表

特征表是以表格形式描述触发器的逻辑功能。前面介绍的三种触发器的特征表见表 10.1, 表中把次态定义成输入和现态的函数。 $Q(t)$ 表示在某个时钟边沿到来之前的现态, $Q(t+1)$ 是时钟之后的次态。注意: 时钟边沿输入没有包含在特征表中, 隐含在时刻 t 和 $t+1$ 之间发生。

表 10.1 三种触发器的特征表

JK 触发器			
J	K	Q(t+1)	
0	0	Q(t)	保持
0	1	0	复位
1	0	1	置位
1	1	Q'(t)	翻转

D 触发器		
D	Q(t+1)	
0	0	复位
1	1	置位

T 触发器		
T	Q(t+1)	
0	Q(t)	保持
1	Q'(t)	翻转

由 JK 触发器的特征表可知: 当 J 和 K 都等于 0 时, 次态与现态相同, 可以表示成 $Q(t+1)=Q(t)$, 可见时钟对状态改变没有作用。当 J=1 和 K=0 时, 触发器复位, $Q(t+1)=0$ 。当 J=1 和 K=0 时, 触发器置位, $Q(t+1)=1$ 。当 J 和 K 都为 1 时, 次态和现态互为反相, 表示成 $Q(t+1)=Q'(t)$ 。

D 触发器的次态只与输入 D 有关, 与现态无关, 表示为 $Q(t+1)=D$, 这说明次态与 D 相同。注意: D 触发器没有一种保持不变的情况。如果要触发器状态保持不变, 可以通过两种方法实现, 一种是禁止时钟信号有效; 另一种是不理会时钟信号, 把输出反馈回输入端 D。

T 触发器的特征表只有两种情况: 当 T=0 时, 时钟边沿不影响状态; 当 T=1 时, 时钟边沿会使触发器的状态翻转。

特征方程

触发器的逻辑功能除了可以用特征表描述之外, 还可以用特征方程来描述。

D 触发器的特征方程表示为:

$$Q(t+1) = D$$

它表示输出的次态与现时刻输入 D 的值相同。

JK 触发器的特征方程可以由特征表或图 10.29 所示的电路得到, 结果为:

$$Q(t+1) = JQ' + K'Q$$

其中， Q 是触发器在时钟边沿到来之前的输出。

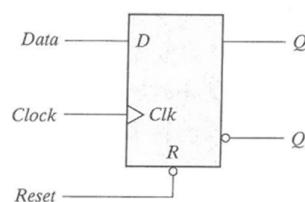
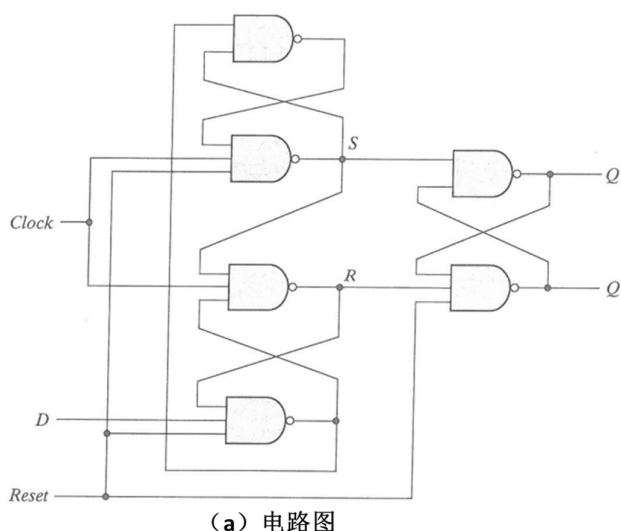
T 触发器的特征方程由图 10.30 得到，结果为：

$$Q(t+1) = T \oplus Q = TQ' + T'Q$$

直接输入

有些触发器具有与时钟无关的异步输入端，用于强迫触发器进入某个特殊状态。将触发器置位为 1 的输入称为置位端（或直接置位端）。将触发器清除为 0 的输入端称为复位端（或直接复位端）。数字系统一开机时，触发器的状态是未知的，此时，直接输入端非常有用，可以在有效时钟到来前强迫系统中所有触发器进入一个已知的初始状态。

带异步复位端的边沿 D 触发器如下图 10.31 所示，图中除了有连到三个与非门输入的复位端外，其他与图 10.27 相同。当复位信号为 0 时，将强迫 Q' 为 1，同时这个 1 反过来又将 Q 清零，因此，触发器被复位。从复位端连出的另外两个输入端保证第三个锁存器的 S 维持 1 变，此时无论 D 和 Clk 是何值，复位输入端都将为 0。带直接置位端的触发器使用字母 S 表示异步置位输入。



Reset	Clock	D	Q	Q'
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

(c) 功能表

图 10.31 带异步复位端的 D 触发器

第三节 存储单元电路：动态锁存器和寄存器

静态时序电路的存储依赖于如下概念：一对交叉耦合的反相器形成了一个双稳元件并且因此可以用来记忆二进制值。这个方法有一个很有用的特性，即只要电源电压加在该电路上，它所保存的值就一直有效，因此称为静态电路。然而，静态门的主要缺点是它比较复杂。当寄存器在频率较高的时钟控制的计算结构（如流水线数据通路）中使用

时，可以显著降低这一存储元件应当能在相当长一段时间内维持状态的要求。

由此产生了一类将电荷暂时储存在寄生电容上的电路。其原理与动态逻辑完全一样——存储在电容上的电荷可以用来代表一个逻辑信号。没有电荷表示 0，而存在电荷则代表存储了 1。可惜的是，任何电容都不是理想的，总会有一些电荷泄漏。因此一个存储的值只能保存有限的一段时间，一般在毫秒的范围内。如果要保持信号的完整性，需要周期性地刷新该值，因此称为动态存储。如果从一个电容中读出被存放信号的值而又不破坏电荷，则要求有高输入阻抗的器件。

10.3.1 动态传输门边沿触发寄存器

下图 10.32 是一个基于主从概念的完全动态的正沿触发寄存器。当 $CLK=0$ 时输入数据在存储节点 A 被采样，该节点具有一个等效电容 C_1 ，它由反相器 I_1 的栅电容、传输门 T_1 的结电容和 T_1 的栅重叠电容组成。在此期间，从级处于维持模式，其节点 B 处于高阻抗（浮空）状态。在时钟上升沿时，传输门 T_2 导通，于是恰在上升沿之前在节点 A 上所采样的值就传送到输出 Q （注意，在时钟高电平阶段，由于第一个传输门关断，节点 A 是稳定的）。现在节点 B 存储了节点 A 的相反状态。这一边沿触发器的实现是非常高效的，因为它只需要 8 个晶体管。采样开关可以用纯 NMOS 传输管来实现，这样就更加简单，只需要 6 个晶体管。晶体管数目的减少对于高性能低功耗系统极具吸引力。

这一电路的建立时间就是传输门的延时，它相应于节点 A 采样 D 输入所需要的时间。维持时间近似为 0，因为传输门在时钟边沿处关断，因此不受输入进一步变化的影响。传播延时（ t_{c-q} ）等于两个反相器的延时加上传输门 T_2 的延时。

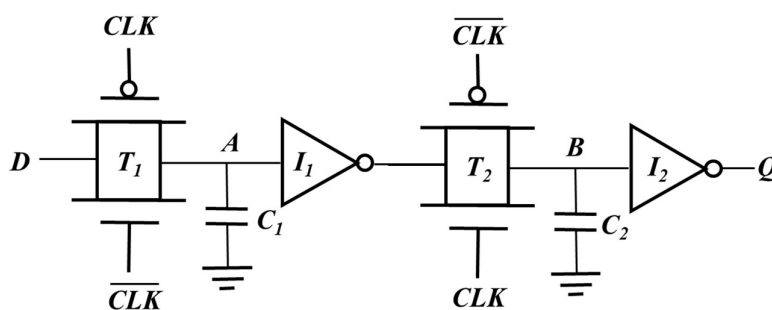


图 10.32 动态边沿触发寄存器

对于这样一个动态寄存器，需要好好考虑的是存储节点（即状态）必须周期性地刷新，以防止因电荷泄漏、二极管泄漏或亚阈值电流引起的电荷丢失。在数据通路电路中刷新的频率不是一个问题，因为寄存器受到时钟周期性的控制，所以存储节点在不断更新。

时钟重叠是这个寄存器的重要问题。考虑下图 10.33 的时钟波形。在 0-0 重叠期间，传输门 T_1 的 PMOS 和 T_2 的 PMOS 同时导通，形成数据从寄存器的 D 输入流到 Q 输出

的直接通路。换言之，此时出现了竞争情况。如果重叠期间很长，输出 Q 可能在下降沿时就改变，显然，对于正沿触发寄存器这是一个不希望有的效应。对于 1-1 重叠区也是一样，这时存在一条通过 T_1 的 NMOS 和 T_2 的 NMOS 的输入至输出路径。后一种情况可以通过强加一个维持时间约束来解决，即数据必须在高电平重叠期间稳定。而前一种情况（0-0 重叠）可以这样来解决，即保证在 D 输入和节点 B 之间有足够延时，以使主级采样的新数据不会传送到从级。一般来说，内置单个反相器的延时就足够了。对重叠期间的限制条件为：

$$t_{overlap0-0} < t_{T1} + t_{I1} + t_{T2}$$

同样，对 1-1 重叠的限制条件为：

$$t_{overlap1-1} < t_{hold}$$

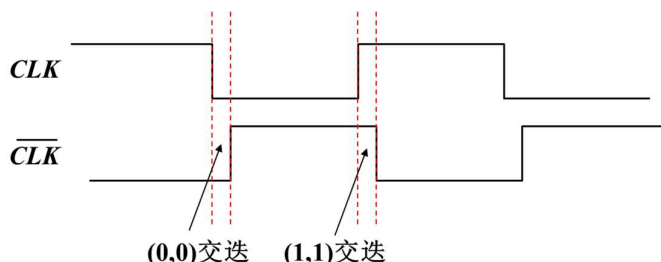


图 10.33 重叠时钟的影响

警告：本节所介绍的动态电路在复杂性、性能和功耗方面都是非常有吸引力的。遗憾的是，在稳定性方面的考虑限制了它们的应用。在如图 10.32 这样的完全动态的电路中，一个被电容耦合到内部存储节点上的信号节点会注入相当大的噪声而破坏状态。这一点在 ASIC 设计流程中特别重要，因为它对信号节点与内部动态节点之间的耦合缺乏控制。漏电电流引起的另一个问题是：大多数现代处理器要求时钟能够减慢或者完全停止，以便在低活动性期间节省功率。最后，内部动态节点并不跟踪电源电压的变化。例如，在图 10.32 的电路中，当 CLK 为高电平对节点 A 维持它的状态，但它不会跟踪由所看到的电源电压的变化。其结果是降低了噪声容限。

大多数这些问题都可以通过增加一个弱的反馈反相器使电路成为伪静态来解决（如图 10.34）。虽然这要增加少量的延时，但却明显改善了抗噪声的能力。除非寄存器是用在高度控制的环境中（例如定制设计的高性能数据通路），它们都应当设计成伪静态的或静态的。这对本节讨论的所有锁存器和寄存器都适用。

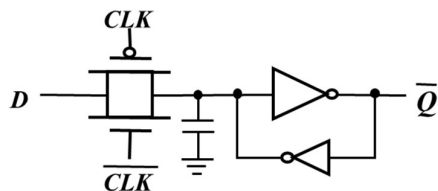


图 10.34 使动态锁存器成为伪静态

10.3.2 C²MOS —— 一种对时钟重叠不敏感的方法

C²MOS 寄存器

下图 10.35 是一个非常巧妙的基于主从概念并对时钟重叠不敏感的正沿触发寄存器。这一电路称为 C²MOS (Clocked CMOS) 寄存器，其工作过程分为两个阶段：

1. $CLK=0$ ($\overline{CLK}=1$)：第一个三态驱动器导通，此时主级像一个反相器在内部节点 X 上采样 D 的反相数据，因此主级处于求值模式。同时从级处在高阻抗模式，即维持模式。晶体管 M_7 和 M_8 均关断，切断了输出与输入的联系。输出 Q 维持其原来存储在输出电容 C_{L2} 上的值。
2. $CLK=1$ 时恰好相反：主级部分处在维持模式 (M_3 - M_4 关断)，而第二部分求值 (M_7 - M_8 导通)。存放在 C_{L1} 上的值经过从级传送到输出节点，此时从级的作用像一个反相器。

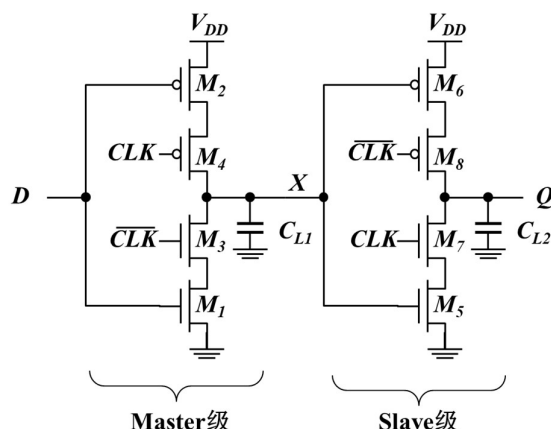


图 10.35 C²MOS 主从正沿触发寄存器

整个电路作为一个正沿触发的主从寄存器，非常类似于前而介绍过的传输门型寄存器。然而，它们有一个重要的差别：

只要时钟边沿的上升和下降时间足够小，具有 CLK 和 $\overline{CLK}$ 时钟控制的这一 C²MOS 寄存器对时钟的重叠是不敏感的。

为了证明这一结论，同时考察 (0-0) 和 (1-1) 重叠的情况 (见图 10.33)。在 (0-0) 重叠情况下，电路简化成图 10.36 (a) 所示的电路，在这段时间，电路中的两个 PMOS 器件都导通。为了能正确工作，不应当有任何在重叠窗口期间采样的新数据传送到输出 Q ，因为输出数据在正沿触发寄存器的负边沿上不应当有变化。确实，在重叠期间新数据通过串联的 PMOS 器件 M_2 - M_4 采样到节点 X 上，因此节点 X 可以从 0 翻转到 1。但这一数据不能传送到输出，因为 NMOS 器件 M_7 是关断的。在这一重叠期结束时 $\overline{CLK}=1$ ，于是 M_7 和 M_8 均关断，使从级处于维持状态。因此在从级输出端 Q 看不到任何在时钟下降边沿采样的新数据，因为从级状态直到时钟下一个上升沿之前一直是关断的。由于

电路由两个串联的反相器构成，信号的传播需要在一个上拉之后跟着一个下拉，或反过来亦可。而这在图中所示的情形下显然是不可能的。

两个 NMOS 器件 M_3 和 M_7 同时导通的(1-1)重叠情况更值得讨论，如图 10.36(b)。问题仍然是在重叠期间采样（在时钟刚开始上升之后）的新数据是否会传送到输出 Q 。一个正沿触发的寄存器只允许传送在上升沿之前存在于输入端的数据。如果 D 输入在重叠期间改变，节点 X 可能发生由 1 至 0 的翻转，但不能进一步传播。然而只要重叠期一结束，PMOS M_8 就导通，于是 0 传送到输出，而这并不是所希望的。这一问题可以通过对输入数据 D 规定一个维持时间的约束条件来解决，换言之，即数据 D 应当在重叠期间保持稳定。

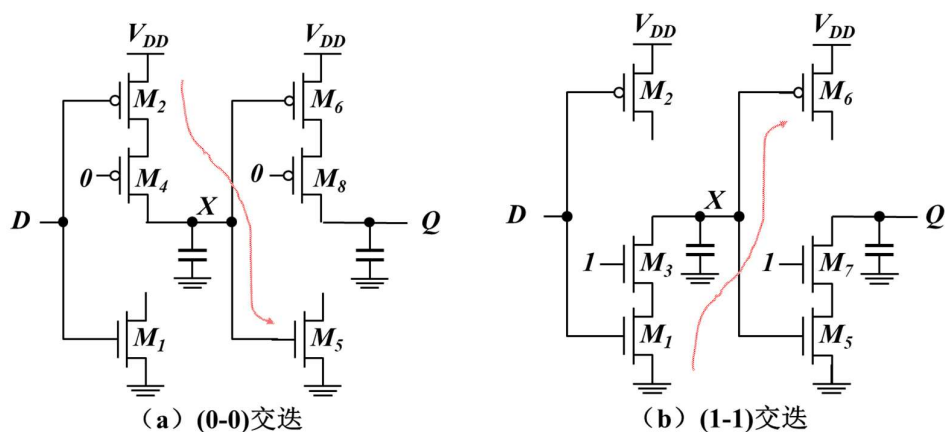


图 10.36 重叠期间的 C²MOS D 型触发器 (DFF)。

(如图中箭头所示，在 D 和 Q 之间不可能有信号通道存在)

总之，可以说 C²MOS 锁存器对于时钟重叠是不敏感的，因为虽然这些重叠会使锁存器的上拉网络或下拉网络导通，但不会同时使它们有效。然而，如果时钟的上升和下降时间足够慢，那么就会存在一个 NMOS 和 PMOS 管同时导通的时间间隙。它在输入和输出之间形成了一条通路，可以破坏电路的状态。模拟显示只要时钟上升时间（或下降时间）约小于寄存器传播延时的 5 倍，该电路就能正确工作。这一条件并不十分苛刻，它在实际的设计中很容易达到。上升和下降时间的影响显示在下图 10.37 中，图中画出了当时钟斜率分别为 0.1ns 和 3ns 时一个 C²MOS DFF 瞬态响应的模拟结果。时钟较慢时，存在发生竞争状况的可能性。

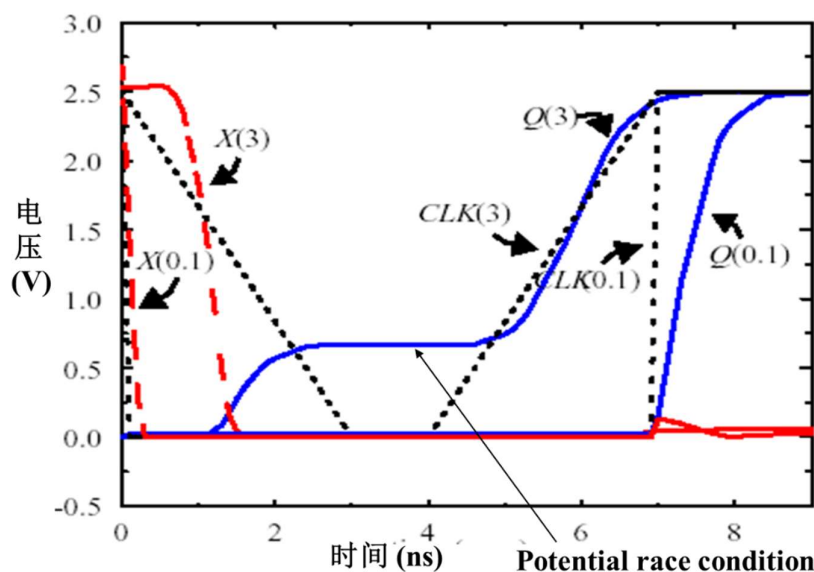


图 10.37 时钟上升/下降时间为 0.1ns 和 3ns 时 C²MOS DFF 的瞬态响应（假设 D=1）

第四节 钟控时序电路分析

如果逻辑电路中所有的触发器都共用时钟端，这样的电路被称为钟控时序电路（或同步时序电路）。钟控时序电路的功能取决于输入、输出和触发器的状态。输出和次态都与输入和现态有关。时序电路的分析就是要寻求输入、输出以及内部状态之间的时序关系表或波形图，也可以用布尔表达式来描述时序电路的功能，这些表达式需要直接或间接体现信号之间的时序关系。

时序逻辑电路图可以包括、也可以不包括组合电路，触发器可以是任何类型。本节将引入代数表示法，用代数式把次态描述成现态和输入的函数。另一种代数法是确定时序电路的逻辑图。对给定电路的功能分析将在某种工作条件之下进行，下面举例说明这些方法。

状态方程

同步时序电路的功能可采用状态方程来描述。状态方程也称转移方程，就是把次态表示成现态和输入的函数。如图 10.38 所示的时序电路，它有两个 D 触发器 A 和 B，有一个输入 x 和一个输出 y。由于触发器的输入 D 决定次态的值，因此可以写出电路的状态方程为：

$$A(t+1) = A(t)x(t) + B(t)x(t)$$

$$B(t+1) = A'(t)x(t)$$

状态方程是确定触发器状态转移条件的代数表达式。方程左边的 (t+1) 表示触发器时钟边沿到来后的次态，方程右边是使次态为 1 的现态和输入的布尔表达式。由于布

尔表达式中所有的变量都是现态的函数，这里为了方便起见，我们省略每个变量之后的标号(t)，这样状态方程可以表示为更加紧凑的形式：

$$\begin{aligned} A(t+1) &= Ax + Bx \\ B(t+1) &= A'x \end{aligned}$$

状态方程的布尔表达式可直接由组成时序电路的组合电路导出。

输出的现态可以在代数上表示成：

$$y(t) = [A(t) + B(t)] x'(t)$$

把现态中的符号(t)去掉，则可以将输出布尔方程写成：

$$y = (A + B) x'$$

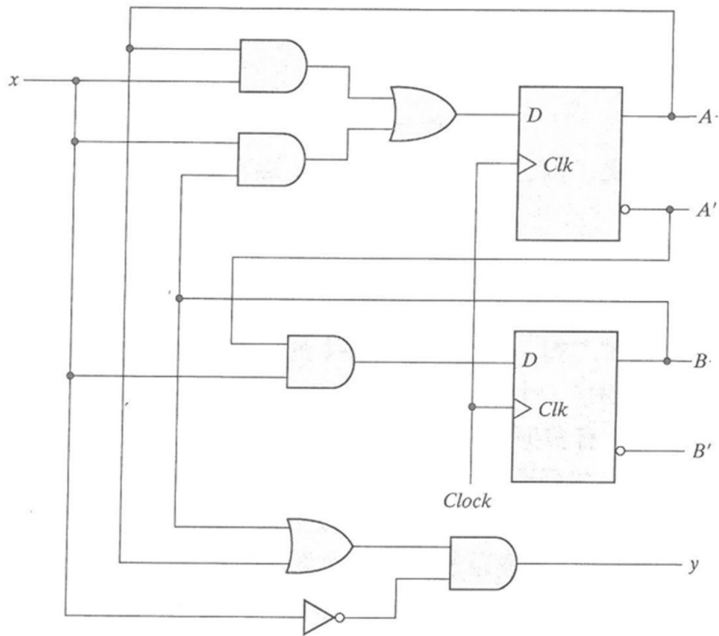


图 10.38 时序电路举例

状态表

输入、输出和触发器状态的时序可以用状态表（也叫转移表）描述出来。图 10.38 所示电路的状态表见下表 10.2。

该表由 4 个部分组成，分别为现态、输入、次态和输出。现态部分列出了在给定时刻 t 条件下，触发器 A 和 B 的状态。输入部分列出了每种可能的现态取值所对应的 x 值。次态部分列出一个时钟周期后，在 $t+1$ 时刻触发器的状态。输出部分列出了与每个现态和输入相对应的 t 时刻的输出 y 值。

状态表需要首先列出现态和输入所有可能的二进制组合。本例中，从 000 到 111 共

八个二进制组合。次态值从逻辑图或者状态方程得到。触发器 A 的次态必须满足的状态方程为：

$$A(t+1) = Ax + Bx$$

状态表中次态部分 A 的一列有 3 个 1，与之对应的现态和输入满足的条件是：触发器 A 的现态和输入 x 都等于 1，或者触发器 B 的现态和输入 x 都等于 1。类似地，触发器 B 的次态也可以从下面的状态方程中得到：

$$B(t+1) = A'x$$

当 A 的现态为 0，输入 x 为 1 时，上式左边等于 1。

表 10.2 图 10.38 中电路的状态表

现 态		输 入	次 态		输 出
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

输出对应的列可以从输出方程中获得：

$$y = Ax' + Bx'$$

由 D 触发器构成的时序电路状态表可按上述过程求得。通常，如果时序电路有 m 个触发器和 n 个输入端，状态表就要有 2^{m+n} 行，从 0 到 $2^{m+n} - 1$ 的二进制数值被罗列在现态和输入部分。次态部分有 m 列，每一列对应一个触发器。次态的二进制数值直接由状态方程得到。输出部分的列数与输出变量相同，有多少个输出变量，就有多少个输出列。其二进制数值可以如同写真值表一样从电路或者布尔函数中得到。

有时候也可将状态表稍微进行一下改动，这样可以带来一些方便。改动之后，状态表只有三个部分：现态、次态和输出，输入的可能取值在次态和输出的下面罗列出来。对表 10.2 重新描述的结果如下表 10.3。对于每一个现态，次态和输出都有两种可能，分别对应输入的两种取值。到底哪种方法好用，还要看具体的应用场合。

状态图

状态表中的有用信息也可用状态图的图形方式表示出来。在这种图形中，状态表示成圆圈，使用有向连线从一个圆圈连到另一个圆圈表示状态之间的转移。图 10.38 所

示的时序电路用状态图描述如下图 10.39 所示。

表 10.3 第二种状态表的表述方法

现 态		次 态				输 出	
		x=0		x=1		x=0	x=1
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

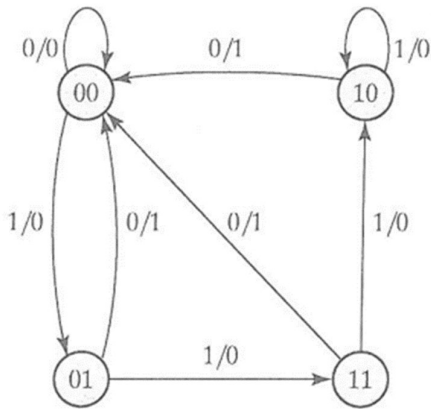


图 10.39 图 10.38 时序电路的状态图

状态图功能与状态表相同，可以直接从表 10.2 或表 10.3 得到。每一个圆圈内的二进制数值表示触发器状态。有向连线采用二进制数值标注，需要分隔时用斜线(/)分隔，先标出现态时的输入值，斜线下方标出给定输入的现态下得到的输出。注意，沿着有向连线标注出的输出只发生在现态和特定输入条件下，与转移到的次态没有关系。例如，从 00 到 01 的有向连线被标注了 1/0，其含义是：当时序电路的现态是 00，输入是 1 的时候，输出为 0，在下一个时钟到来后，电路进入次态 01。如果输入为 0，那么输出就变为 1；但如果输入维持 1 不变，输出也维持 0 不变。上述内容可使用状态图中以状态 01 圆圈为起始的两个有向连线来加以验证。用有向连线从一个圆圈连到它本身，这种连接方式说明状态保持不变。

除了描述方式不一样之外，状态表和状态图之间没有任何区别。状态表比较容易从逻辑图和状态方程中得到，接下来状态图从状态表获得。状态图能直观地显示状态之间的转移关系，比较适合于人们理解电路工作过程。例如，图 10.39 中的状态图就非常明显地表明，状态从 00 开始，只要输入维持 1 不变，输出就为 0。在输入一串 1 之后再 来一个 0，就会使输出变 1，并且电路回到初始状态 00。

触发器输入方程

时序电路的逻辑电路图是由触发器和门电路构成的。门电路之间的相互连接构成了组合电路。组合电路可以用布尔表达式进行代数法描述。已知触发器类型，并且还知道组合电路的布尔表达式，这样就可以画出时序电路的逻辑图。组合电路部分产生外部输出，可以用一组布尔函数表示，称之为输出方程。产生触发器输入的部分电路也可以用布尔函数描述，称之为触发器输入方程（也叫激励方程）。采用触发器输入字母来表示输入方程变量，下标表示触发器输出的名称。例如，下面的方程确定的是一个或门输入方程，输入 x 和 y 通过一个或门连到触发器的 D 输入端，触发器的输出用字母 Q 表示。

$$D_Q = x + y$$

图 10.38 所示的时序电路是由两个 D 触发器、一个输入 x 和一个输出 y 组成。电路的逻辑图用代数法表示成两个触发器输入方程和一个输出方程。

$$D_A = Ax + By$$

$$D_B = A'x$$

$$y = (A + B)x'$$

三个方程确定了时序电路逻辑图的结构与组成。字母 D_A 表示标号为 A 的 D 触发器， D_B 表示第二个标号为 B 的 D 触发器，与这两个变量以及输出 y 相关的布尔表达式隐含了时序电路中的组合部分。

触发器输入方程是用于确定时序电路图的比较便捷的方式，它们将触发器的类型隐含在使用的字母上，可以完全确定出驱动触发器的组合电路。注意，因为特征方程中次态等于输入 D 的数值 $Q(t+1)=D_Q$ ，对于 D 触发器的输入方程与对应的状态方程来说，它们在表达式上是一致的。

由 D 触发器构成的时序电路分析

下面我们通过一个简单的例子来归纳由 D 触发器构成的钟控时序电路分析步骤。该电路的输入方程为：

$$D_A = A \oplus x \oplus y$$

字母 D_A 隐含它是标号 A 的 D 触发器的输出， x 和 y 是电路的输入。由于没有给出输出方程，所以隐含着输出直接由触发器的输出提供。从输入方程可以求得逻辑图，如下图 10.40 (a) 所示。

状态表有一列用于触发器的现态，两列代表两个输入，还有一列是 A 的次态。在 A 、 x 和 y 下面罗列出从 000 到 111 的二进制数值，如图 10.40 (b) 所示。次态可由状态方程求得：

$$A(t+1) = A \oplus x \oplus y$$

电路只有一个触发器和两个状态。状态图包含两个圆圈，一个圆圈对应一个状态，如图 10.40 (c) 所示。现态和输出可以是 0 或者 1，如圆圈中数值所示。有向连线上的斜线此时没有，是因为组合电路没有输出。对应一个状态转移的两个输入组合用逗号分隔，这样可以简化描述结果。

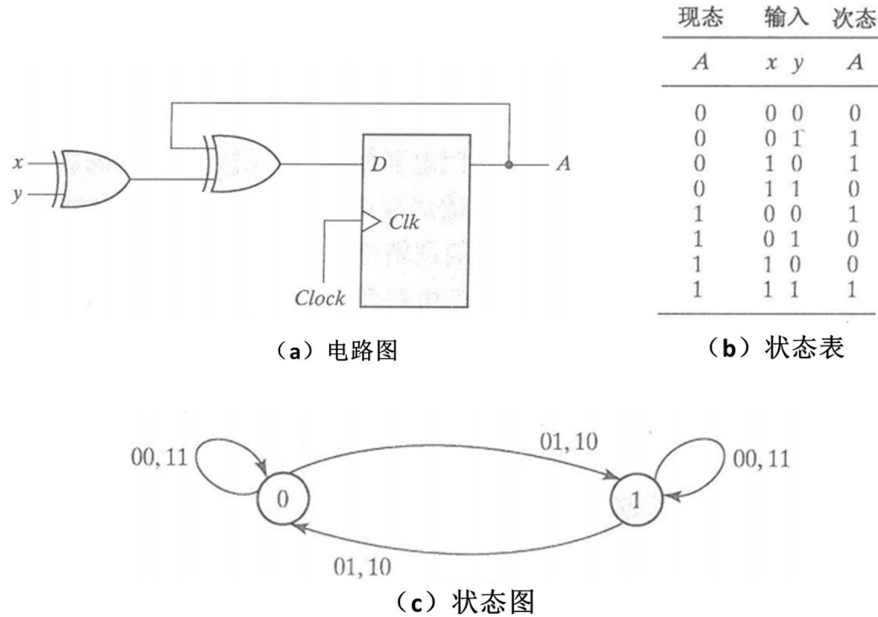


图 10.40 由 D 触发器构成的时序电路

由 JK 触发器构成的时序电路分析

状态表包括四个部分：现态、输入、次态和输出。前面两个部分罗列出所有二进制的组合，次态值由状态方程得到。对 D 触发器来说，状态方程与输入方程一致。当使用的触发器不是 D 触发器，而是 JK 或者 T 触发器时，就有必要参照对应的特征表或者特征方程来得到次态值。下面首先用特征表求次态值，然后用特征方程求次态值。

如果一个时序电路是由 JK 或者 T 触发器构成的，那么求其次态的步骤为：

1. 把触发器输入方程表示成现态和输入变量的函数；
2. 列出每个输入方程的二进制数值；
3. 利用对应触发器的特征表确定状态表中的次态值。

例如，两个 JK 触发器 A 、 B 和一个输入 x 构成的时序电路如下图 10.41 所示。电路没有输出，因此状态表中不需要输出列。（在这种情况下，可以认为触发器的状态就是电路的输出。）

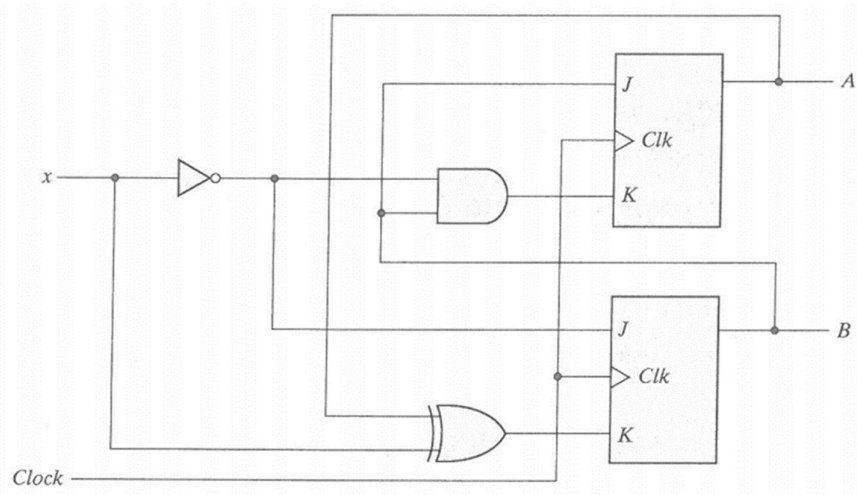


图 10.41 由 JK 触发器构成的时序电路

触发器输入方程为：

$$\begin{aligned} J_A &= B & K_A &= Bx' \\ J_B &= x' & K_B &= A \oplus x = A'x + Ax' \end{aligned}$$

电路的状态表如表 10.4 所示。现态和输入部分列出了八种二进制组合，在表的右侧的触发器输入下面列出的二进制数值不是状态表的一部分，但它们在步骤 2 中对求得次态很有帮助，这些二进制数值可以直接从四个输入方程得到，与根据布尔表达式填真值表的方法相同。每个触发器次态要由相应的 J 和 K 以及表 10.1 中的 JK 触发器特征表共同得到。

表 10.4 由 JK 触发器构成的时序电路状态表

现 态		输 入	次 态		触发器输入			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

有四种情况需要考虑：当 $J=1$ 和 $K=0$ 时，次态是 1；当 $J=0$ 和 $K=1$ 时，次态是 0；当 $J=K=0$ 时，状态不发生改变，次态等于现态；当 $J=K=1$ 时，次态是现态的反相。例如，表中 AB 是 10 且输入 x 为 0 时， J_A 和 K_A 都等于 0，而 A 的现态为 1，因此 A 的次态保持 1 不变。在表的同一行， J_B 和 K_B 都等于 1，由于 B 的现态是 0，所以 B 的次态翻转为 1。

从特征方程导出状态方程，然后再求出次态方程的步骤如下：

1. 把触发器的输入方程表示成现态和输入变量的函数；
2. 将输入方程代入到触发器的特征方程中，得到状态方程；
3. 使用相应的状态方程求出状态表的次态。

图 10.41 中两个 JK 触发器输入方程在前面已经给出。分别用 A 和 B 替代 JK 触发器特征方程表达式中的 Q ，得到：

$$A(t+1) = J_A A' + K_A' A$$

$$B(t+1) = J_B B' + K_B' B$$

用输入方程将 J_A 和 K_A 替代掉，可以得到 A 的状态方程为：

$$A(t+1) = J_A A' + K_A' A = BA' + (Bx')'A = A'B + AB' + Ax$$

由状态方程得到状态表中 A 的次态一系列数值。类似地，触发器 B 的状态方程也可以通过将特征方程中的 J_B 和 K_B 替代掉而得到。触发器 B 的状态方程为：

$$B(t+1) = J_B B' + K_B' B = x'B' + (A \oplus x)'B = B'x' + ABx + A'Bx'$$

状态方程提供了状态表中 B 的次态一系列数值。（注意，表 10.4 中右侧触发器输入的几列在使用状态方程时可以不用。）

时序电路的状态图如下图 10.42 所示。我们注意到，由于电路没有输出，从圆圈出来的有向连线只标注了输入 x 对应的二进制值。

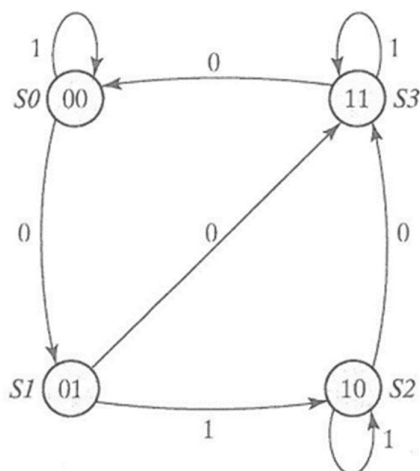


图 10.42 图 10.41 的状态转换图

由 T 触发器构成的时序电路分析

由 T 触发器构成的时序电路分析步骤与 JK 触发器构成的时序电路的分析步骤相同，状态表的次态值可以从特征表或特征方程得到。 T 触发器的特征方程为：

$$Q(t+1) = T \oplus Q = T'Q + TQ'$$

考虑如下图 10.43 所示的时序电路。它有两个触发器 A 、 B ，一个输入 x 和一个输出 y 。

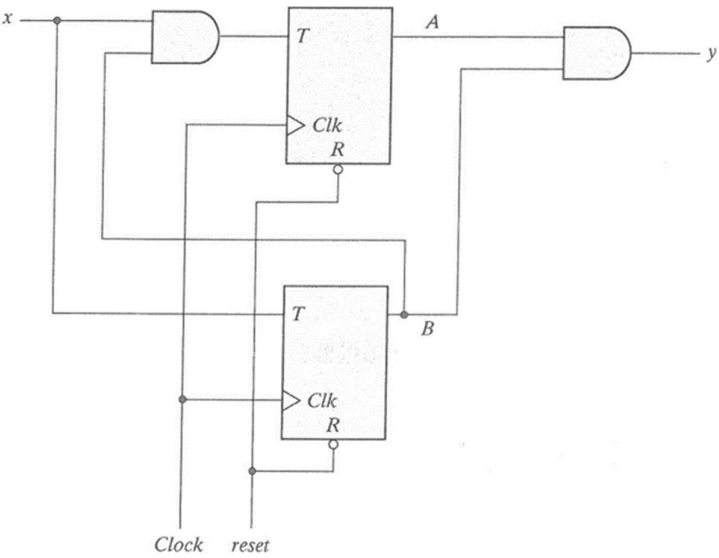


图 10.43 由 T 触发器构成的时序电路

用两个输入方程和一个输出方程描述为：

$$\begin{aligned} T_A &= Bx \\ T_B &= x \\ y &= AB \end{aligned}$$

电路的状态表见下表 10.5。 y 的值从输出方程得到，次态的值从状态方程得到，状态方程是将特征方程中的 T_A 和 T_B 替代后得到的。具体的状态方程为：

$$\begin{aligned} A(t+1) &= (Bx)'A + (Bx)A' = AB' + Ax' + A'Bx \\ B(t+1) &= x \oplus B \end{aligned}$$

表 10.5 由 T 触发器构成的时序电路状态表

现 态		输 入	次 态		输 出
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

状态表中 A 和 B 的次态值从上面这两个方程求得。

电路的状态图如下图 10.44 所示。只要输入 z 为 1，电路如同二进制计数器一样工作，状态序列是 00、01、10、11，然后回到 00；当 x 为 0 时，电路状态保持不变；当现态为 11 时，输出 y 为 1。输出只与现态有关，而与输入无关。每个圆圈里被斜线隔开的两个数值分别代表现态和输出。

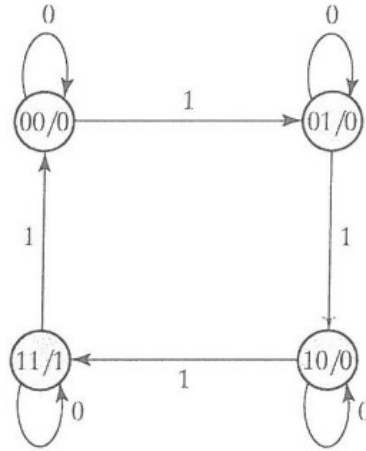


图 10.44 图 10.43 时序电路的状态图

有限状态机的两种模型—米利和摩尔模型

一般的时序电路都有输入、输出和内部状态。通常，时序电路可以划分为两大类：米利型和摩尔型，如下图 10.45 所示，它们的区别在于产生输出的方式上。在米利型电路中，输出是现态和输入的函数。在摩尔型电路中，输出只是现态的函数。对这两种类型电路，有的教材和技术资料也把时序电路称作有限状态机，简称为 FSM。时序电路的米利型称作米利 FSM 或者米利机，摩尔型称作摩尔 FSM 或者摩尔机。

米利型电路的例子如图 10.38 所示。输出 y 既是输入、又是现态 A 和 B 的函数。图 10.39 中给出了对应的状态图，状态间的有向连线上的标注用斜线将输入和输出分开。摩尔型电路的例子如图 10.41 所示，图中，输出只是现态的函数。图 10.42 是电路对应的状态图，输入被标注在有向连线上，触发器的状态被标注在圆圈内部。另一个摩尔型电路的例子如图 10.43 所示。电路的输出只与触发器的状态有关，也就是说只是现态的函数。状态图中的输入被标注在有向连线上，但是输出却与现态一起被标注在圆圈内部。

在摩尔型时序电路中，输出与时钟同步，这是因为与输出有关的触发器输出是与时钟同步的。对于米利型时序电路，在时钟周期中，如果输入发生变化，输出也可能发生变化。除此之外，输出可能还会有瞬间的错误出现。其原因是输入变化时间和触发器输出变化时间不一致。为了使米利型时序电路也符合同步要求，时序电路的输入必须与时钟同步，并且输出仅仅在时钟边沿被采样。

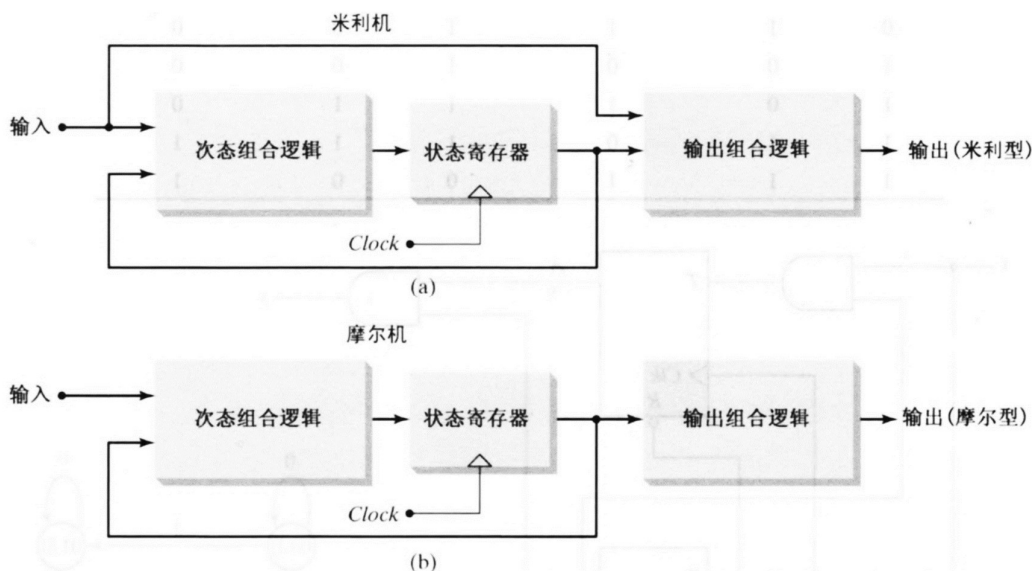


图 10.45 米利型和摩尔型有限状态机框图

第五节 时序电路的设计过程

设计的目的是确定实现预定功能的硬件电路。小的电路可以使用人工设计来完成，但是工业上，往往使用综合工具来设计大规模集成电路。综合工具用来实现 JK 和 T 触发器使用的基本构件是 D 触发器，辅助使用门电路。实际上，设计者通常并不关心使用的触发器类型，只要综合工具实现的时序电路功能满足要求即可。这里，我们介绍使用 D 、 JK 和 T 触发器来设计的人工方法。

同步时序电路的设计首先是给出一系列指标，最后得到逻辑图或者能产生逻辑图的布尔方程。组合电路可以由真值表确定，时序电路用状态表表示其功能。时序电路设计的第一步是画出状态表，也可以是等效的表示方法，如状态图等。

同步时序电路由触发器和门组成。电路的设计包括选择触发器，以及确定组合电路结构等。组合电路和触发器有机结合在一起，产生能实现预定功能的时序电路。触发器数目由电路状态确定，组合电路可以通过求状态表中的触发器输入和输出方程来得到。实际上，一旦触发器的类型和数目确定后，设计过程就从时序电路问题转化成了组合电路问题。因此，就可以使用组合电路的设计方法。

时序电路的设计过程总结为如下几个步骤：

1. 理解所要求的功能描述和指标，画出电路的状态图；
2. 简化状态的数目（如果有必要）；
3. 状态编码（为状态分配二进制数值）；
4. 画出二进制编码状态表；

5. 选择触发器的类型；
6. 求出最简的触发器输入方程和输出方程；
7. 画出逻辑图。

电路功能的说明常常需要读者熟悉数字逻辑术语。由于字面说明不是非常完整和精确，所以设计者有必要凭借自身的经验正确理解电路的功能。从指标可以得到状态图，接下来才有可能使用已知的设计方法来完成设计。步骤中的第 2 步和第 3 步是状态化简和分配，都有常规的方法来完成。设计过程中的第 4 步到第 7 步可以用精确的算法实现，因此实际中都是自动进行的。使用自动工具软件完成设计的过程称为综合（synthesis）。设计者如果要使用逻辑综合工具软件辅助设计，遵循的步骤可以简化。首先是将状态图转换成硬件描述语言程序，然后利用综合软件对程序进行综合，这样就可以得到电路图。

第 1 步最具挑战性，也往往是设计的关键。下面通过一个简单的例子来说明如何从字面说明得到状态图。

假设我们希望设计这样一个电路，其功能是检测输入的一串二进制序列，当连续输入三个或三个以上的 1 时，输出为 1。否则，输出为 0。

根据上面的描述，得出电路的状态图如图 10.46 所示。

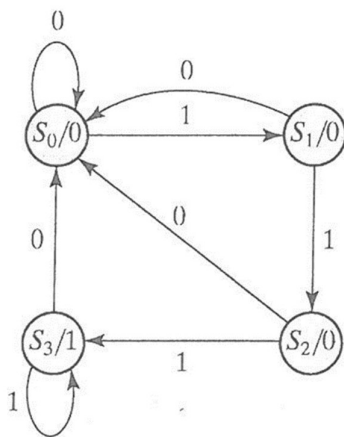


图 10.46 序列检测器的状态图

起始状态是 S_0 ，如果输入 0，电路状态不变；如果输入是 1，状态转到 S_1 ，说明已经检测到了一个 1，再来一个 1，状态变为 S_2 ，说明已经收到了两个连续的 1，但如果接下来输入是 0，状态回到 S_0 ，如果接下来输入 1，连续的第三个 1 将电路状态转到 S_3 。此后，如果检测到了更多的 1，电路将维持在 S_3 不变，只要输入为 0，状态就回到 S_0 。因此，当收到三个或三个以上的 1 时，电路将在 S_3 不变。该电路是摩尔型的时序电路，当电路处于状态 S_3 时输出为 1，否则输出为 0。

使用 D 触发器进行设计

一旦得到状态图后，需要对状态进行编码（例：顺序二进制），列出编码后的状态

表。表 10.6 是编码后的状态表，它是对图 10.46 中的状态图直接顺序编码得到的。

表 10.6 序列检测器的状态表

现 态		输 入	次 态		输 出
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

需要使用两个 D 触发器来表示四个状态，分别为 A 和 B ，还有一个输入端 x 和一个输出端 y 。 D 触发器的特征方程是 $Q(t+1) = D$ ，隐含着状态表中的次态由触发器的输入 D 确定。触发器的输入方程直接从次态 A 和 B 求得，用最小项表示为：

$$A(t+1) = D_A(A, B, x) = \Sigma(3, 5, 7)$$

$$B(t+1) = D_B(A, B, x) = \Sigma(1, 5, 7)$$

$$y(A, B, x) = \Sigma(6, 7)$$

A 和 B 是触发器 A 和 B 的现态值， x 是输入， D_A 和 D_B 是输入方程。输出 y 的最小项从状态表的输出一列得到。

用卡诺图对输入布尔方程进行简化，得到：

$$D_A = Ax + Bx$$

$$D_B = Ax + B'x$$

$$y = AB$$

使用 D 触发器设计的优点是描述触发器输入的布尔方程可以直接从状态表求得。时序电路的逻辑图如下图 10.47 所示。

使用 JK 触发器进行设计

采用 JK 触发器设计时序电路的过程与采用 D 触发器基本相同。唯一的区别在于， JK 触发器的输入方程不是直接从状态表中获得，而要先从**激励表**中得到从现态转移到次态的条件，然后才能求得。下面先介绍一下激励表。

使用 D 触发器之外的其他触发器（如 JK 触发器和 T 触发器）设计时序电路时，输入方程不能从状态表直接得到，为了求触发器的输入方程，需要从状态表导出输入方程

The logic diagram shows a sequential circuit with two D flip-flops and four AND gates. The inputs are x and y . The outputs are A , B , and B' . A common clock signal, labeled *Clock*, is connected to the *Clk* input of both flip-flops. The circuit logic is as follows:

- The first AND gate has inputs x and A . Its output is connected to the *D* input of the first flip-flop.
- The second AND gate has inputs x and B . Its output is connected to the *D* input of the second flip-flop.
- The third AND gate has inputs x and y . Its output is connected to the *D* input of the first flip-flop.
- The fourth AND gate has inputs x and y . Its output is connected to the *D* input of the second flip-flop.
- The output of the first flip-flop is A .
- The output of the second flip-flop is B .
- The output B' is the complement of B .

表 10.1 是触发器的特征表。当已知输入和现态时, 可求出次态。特征表对分析时序电路和确定触发器的功能非常有效。在设计时, 在已知状态发生转移的情况下, 将导致这种转移的触发器输入列在表中, 这样的表称为激励表。

表 10.7 JK 触发器和 T 触发器激励表

$Q(t)$	$Q(t+1)$	J	K	$Q(t)$	$Q(t+1)$	T
0	0	0	X	0	0	0
0	1	1	X	0	1	1
1	0	X	1	1	0	1
1	1	X	0	1	1	0

(b) T 触发器

135

导致状态从 0 变为 1。

表 10.7 (b) 是 T 触发器的激励表。从特征表可知，当输入 $T=1$ 时，触发器的状态翻转； $T=0$ 时，触发器的状态保持不变。因此，若触发器状态保持不变，条件是 $T=0$ 。若触发器发生翻转，条件是 T 必须等于 1。

下面说明如何利用激励表，用 JK 触发器设计时序电路。为了说明这个过程，我们以设计表 10.8 的时序电路为例。除了有现态列之外，输入和次态如一般的状态表，表中也给出了触发器输入的条件，从中可以得到输入方程。这些触发器的输入是由从触发器的状态表和激励表一起获得的。例如，在表 10.8 的第一行，触发器 A 发生从现态 0 到次态 0 的转移。从表 10.7 的 JK 触发器激励表可知，状态从现态 0 变为次态 0 的条件是输入 J 为 0，输入 K 任意。因此，0 和 X 写在了 J_A 和 K_A 的下方第一行。由于第一行中的触发器 B 从现态 0 变为次态 0，因此，0 和 X 也写在了 J_B 和 K_B 的下方第一行。表中的第二行是触发器 B 从现态 0 变为次态 1，从激励表可知，0 变为 1 的条件是 J 为 1 和 K 任意。因此，1 和 X 写在 J_A 和 K_A 的下方第二行。表中的每一行都遵循这样的步骤，对照触发器的状态转移情况，从激励表获得输入条件，填入相应位置。

表 10.8 状态表和 JK 触发器的输入

电路状态表					触发器的输入条件			
现 态		输 入	次 态		触发器输入			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	1	0	1	X	X	1
0	1	1	0	1	0	X	X	0
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1

表 10.8 把输入方程表示成现态 A 和 B 以及输入 x 的函数。输入方程可以用卡诺图进行函数化简。在化简过程中，次态值并不使用，但能从中体会使用 JK 触发器设计时序电路的优点。因为有如此多的任意态，任意态有助于产生简单的表达式，因此，输入方程对应的组合电路可能非常简单。

两个 JK 触发器对应的 4 个输入方程如下：

$$\begin{aligned}
 J_A &= Bx' & K_A &= Bx \\
 J_B &= x & K_B &= (A \oplus x)'
 \end{aligned}$$

设计得到的时序电路逻辑图如下图 10.48 所示。

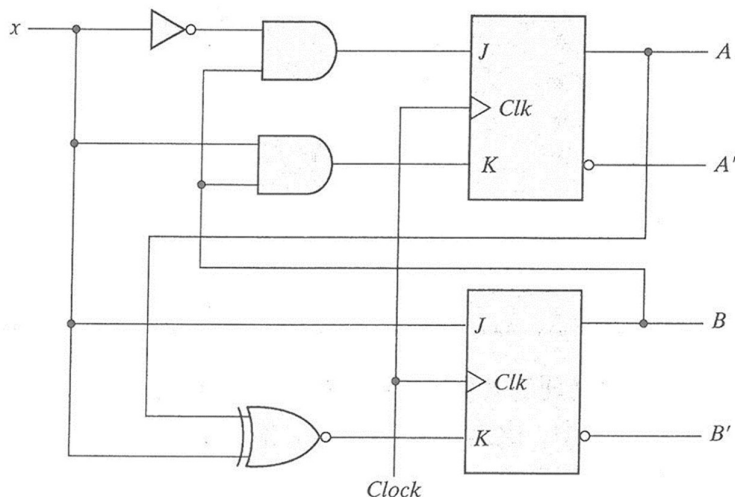


图 10.48 用 JK 触发器构成的时序电路逻辑图

使用 T 触发器进行设计

采用 T 触发器的设计过程可以从二进制计数器构成的角度来解释。一个 n 位的二进制计数器由 n 个触发器组成，计数值从 0 到 2^n-1 。三位二进制计数器的状态图如下图 10.49 所示。

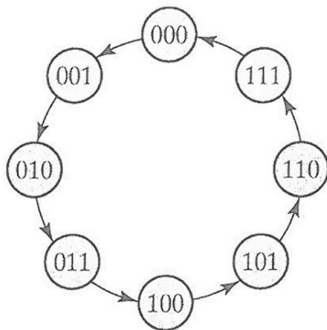


图 10.49 三位二进制计数器的状态图

从图中每个圆圈内的二进制状态可见，触发器输出重复二进制计数序列，从 111 再返回到 000。两个圆圈之间的有向连线没有被标注输入和输出值，这点与其他的状态图不一样。记住：同步时序电路的状态转移发生在时钟的边沿，如果没有有效时钟信号，触发器仍旧保持状态不变。因此，没有在状态图或者状态表中作为输入信号直接标注出时钟来。这里，输入只有时钟信号，输出由触发器的状态提供。计数器次态完全取决于现态，只要时钟边沿到来，状态就会发生转移。

表 10.9 是三位二进制计数器的状态表。三个触发器用符号 A_2 、 A_1 和 A_0 表示。

考虑到它们的翻转特性，二进制计数器电路使用 T 触发器来构成。触发器的激励信号 T 可以通过观察 T 触发器激励表得出。例如，对于触发器输入行 001，现态是 001，

次态是 010。比较这两个数值，可以看出， A_2 从 0 到 0，也就是说当时钟边沿到来时，触发器 A_2 状态不变，因此 T_{A2} 填 0； A_1 从 0 到 1，触发器在下一个时钟边沿发生翻转，因此 T_{A1} 填 1，类似地， A_0 从 1 到 0，说明也发生翻转， T_{A0} 填 1。由于最后一行现态 111 的次态是第一行计数值 000，所有的 1 都变为 0，说明三个触发器都发生翻转，因此对应的触发器输入都为 1。

表 10.9 三位二进制计数器的状态表及 T 触发器输入条件

现 态			次 态			触发器输入		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A2}	T_{A1}	T_{A0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	1	1
1	1	1	0	0	0	1	1	1

触发器的输入方程可用卡诺图进行简化， T_{A0} 的 8 个最小项都为 1，计数器的最低有效位在每个时钟到来时都发生翻转。一个函数表达式如果包括了全部的最小项，那么该函数恒等于 1。化简后的输入方程为：

$$\begin{aligned}
 T_{A2} &= A_1 A_0 \\
 T_{A1} &= A_0 \\
 T_{A0} &= 1
 \end{aligned}$$

把这些函数与触发器合在一起，就可以得到计数器的逻辑图，如下图 10.50 所示。

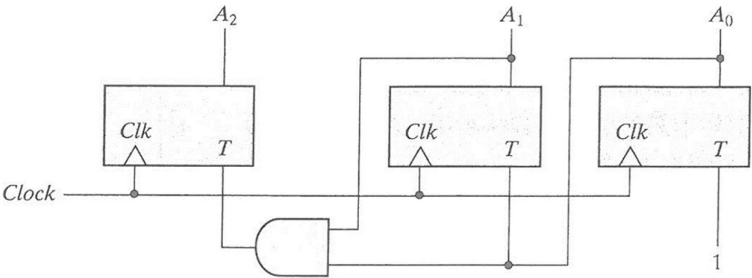


图 10.50 用 T 触发器设计的三位二进制计数器电路逻辑图

第六节 寄存器和计数器设计

10.6.1 寄存器设计

钟控时序电路由一组触发器和组合电路组成。触发器是必需的，因为没有它们，电路就变成了单纯的组合电路。凡是包含触发器的电路，即使没有组合电路部分，也认为是时序电路。常用的时序电路按功能分类有两种，分别是寄存器和计数器。

寄存器由一组触发器构成，而每个触发器能够存储一位信息。 n 位的寄存器包含 n 个触发器，能够存储 n 位的二进制信息。除了触发器，寄存器还可能含有组合电路，主要用于执行数据处理任务。从广义上说，寄存器包括一组触发器和能够影响状态转换的门电路。触发器保存二进制信息，门电路决定信息以何种方式传送给寄存器。

计数器从本质上说也是寄存器，不过它是在预先设定好的状态序列中转移。计数器中门电路的连接方式是为了产生这种状态序列。尽管计数器是寄存器的一种特殊形式，然而通常还是以不同的名称来加以区分。

电路中实际使用的寄存器类型有很多种。最简单的寄存器只包含触发器，没有任何组合门电路。如下图 10.51 所示就是这样的一种寄存器，它由 4 个 D 触发器构成，共用时钟输入端。在每个时钟脉冲的上升沿，所有触发器都被触发，4 位二进制数被传送给 4 位寄存器，时钟边沿之前的 (I_3, I_2, I_1, I_0) 值决定了时钟边沿之后的 (A_3, A_2, A_1, A_0) 值。4 个输出可以在任何时刻被采样，以获得存储在寄存器中的二进制信息。复位输入 ($Clear_b$) 连到了 4 个触发器的复位输入端，都是低电平有效。当该输入为 0 时，所有触发器被异步复位。该输入端能够在时钟工作之前将寄存器复位到 0，这一点在实际中非常有用。正常工作时，输入端 $Clear_b$ 必须保持在高电平。注意，在触发器操作中，一般使用 $Clear$, $Clear_b$ 或 $Reset$, $Reset_b$ 等将寄存器复位到全 0 状态。

带并行预置端的寄存器

带并行预置端的寄存器是数字系统中经常使用的基本模块之一。同步数字系统有一个主时钟发生器，它能连续产生时钟脉冲，时钟作用于系统中所有的触发器和寄存器。主时钟相当于泵，可对系统中所有部件产生经常性的冲刷。外部控制信号决定每一个时钟脉冲到来时的寄存器功能。将信息传送给寄存器的操作称为寄存器预置。在共用的时钟脉冲到来时，如果寄存器的所有输入数据都被同时预置进了寄存器，则这种预置是并行预置。如图 10.51 所示的寄存器，当时钟边沿加到输入端 $Clock$ 上时，所有四个输入数据并行预置进寄存器中。对于这种电路结构，如果寄存器的内容不需要更新，就要禁止时钟信号有效，最简单的方法是采用带使能端的逻辑门对时钟信号进行控制。然而，在时钟通路中插入逻辑门，意味着与时钟脉冲发生逻辑运算，会导致主时钟和触发器时钟之间产生不对称延时。为了使系统完全同步，必须保证系统中的所有时钟同时到达触发器的触发端，对时钟进行逻辑运算引入了延时，会使系统失去同步。因此，建议当需要控制寄存器工作时，尽量通过输入端 D 来控制，而不是控制触发器的时钟输入端 C 。

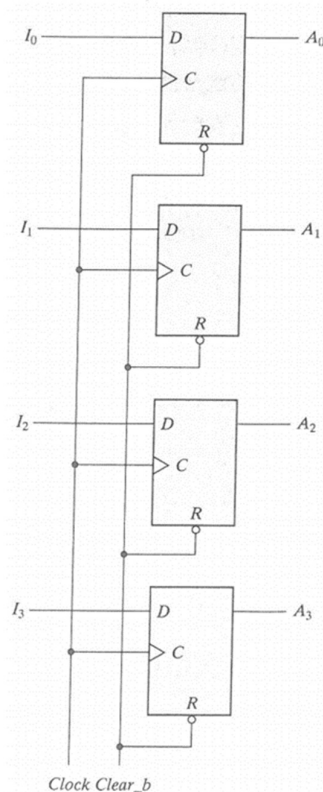


图 10.51 带异步复位端的 4 位寄存器

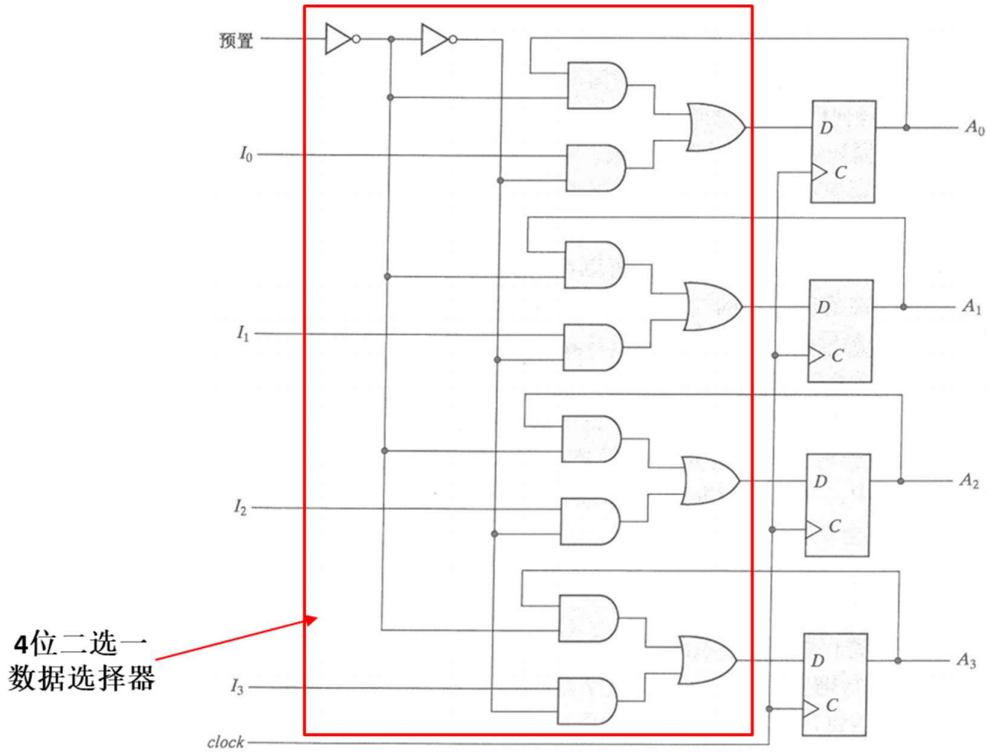
如下图 10.52 所示是一个带并行预置端的四位寄存器，其输出直接送给门电路，然后再反馈回触发器的输入端 D 。增加的门实现了一个两路 MUX 复用，它们可以通过数据总线或寄存器的输出驱动寄存器的输入。寄存器的预置端决定了时钟脉冲到来时的电路功能。当预置端为 1 时，四个输入端的数据在时钟下一个上升沿到来时进入到寄存器中；当预置端为 0 时，触发器的输出直接连到了各自的输入端 D 。从输出反馈到输入是必要的，因为 D 触发器没有“不变”的条件。当时钟有效边沿到来时，触发器的次态等于输入端 D 的值，次态等于现态，使触发器状态保持不变（即每一个时钟脉冲到来输出循环到输入）。时钟脉冲一直加在 C 输入端，预置端决定寄存器功能是预置还是保持不变，4 个触发器的预置操作发生在同一个时钟有效边沿。

10.6.2 移位寄存器

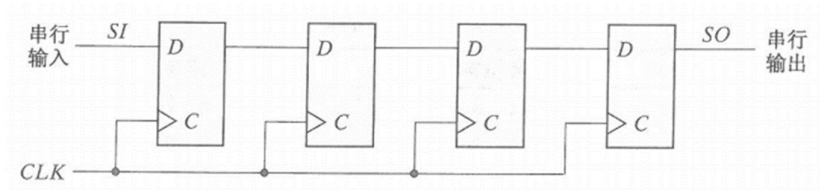
能够对信息进行单向或双向移位操作的寄存器称为移位寄存器。移位寄存器的逻辑结构包括一串级联起来的触发器，每个触发器输出连接到下一个触发器的输入端，所有触发器共用时钟脉冲。当时钟有效边沿到来时，信息从前一个触发器移位到下一个触发器中。

最简单的移位寄存器只使用触发器，如下图 10.53 所示。将触发器的输出连到右边触发器的输入端 D ，移位是单向的（由左向右），每一个时钟脉冲到来时，寄存器内容会一位一位地向右移，最右边的触发器产生串行输出。有时候也有必要控制这种移位，

使得只在某些脉冲到来时发生移位，对另一些脉冲不发生移位， 这就需要阻止这些脉冲到达寄存器的时钟输入端。高速电路中优先选择的是抑制时钟动作，而不是门控时钟信号。



后面我们将了解到，移位操作可以通过触发器的输入端 D 来加以控制，而不使用时钟输入端。然而，如果利用图 10.53 所示的移位寄存器执行移位操作，控制移位的方法则只能是将时钟输入与控制移位的输入信号相与后接到触发器的时钟端上。注意，这里是简化示意图，图中复位信号未标出，但实际中一定要使用复位信号。



下面通过串行传输和串行加法两个例子说明串行移位寄存器的应用。

串行传输

如果数字系统的数据路径每次只传送一位信息，则称它工作在串行方式下，信息从源寄存器移位到目的寄存器中。如果是并行传输，信息的传送是同时进行的。信息从寄

存器 *A* 到寄存器 *B* 的串行发送可以采用移位寄存器来完成，如图 10.54 (a) 所示。寄存器 *A* 的串行输出 (*SO*) 连到了寄存器 *B* 的串行输入 (*SI*)。为防止源寄存器中存储的信息发生丢失，将寄存器 *A* 的串行输出连接回串行输入，构成反馈。除非传送给第三个移位寄存器，否则，寄存器 *B* 的数据从它的串行输出端移出且被丢掉。移位控制输入端决定移位的时机和移位的次数，可以使用与门实现这种功能。只有当移位控制端有效时，与门才允许时钟通过到达触发器的时钟端，否则时钟被阻止。(这种做法可能会有问题，因为它可能会影响电路的时钟路径，正如前面所讨论的。)

假设图 10.54 中的移位寄存器 *A* 和 *B* 都是四位，各个输入端信号见图 10.54 (b) 所示的时序图。移位控制端信号与时钟同步，恰好在时钟下降沿之后电平发生改变，当移位控制端有效时，四个时钟脉冲信号 T_1 、 T_2 、 T_3 和 T_4 ；顺利通过与门到达寄存器 *A* 和 *B*，每个时钟上升沿到来时，两个寄存器都会移位一次，第四个时钟过后，移位控制信号变为 0，移位寄存器不工作。假设在移位前 *A* 的数据是 1011，*B* 是 0010。从 *A* 到 *B* 的串行传输分四步，如表 10.10 所示。与第一个脉冲 T_1 相对应，*A* 的最右边的位将移位给 *B* 的最左边的位，同时通过反馈传送回 *A* 的最左边位。*A* 和 *B* 的所有位都向右移动一个位置，将原先 *B* 的最右边位丢掉，该位从 0 变为 1。后面的三个脉冲执行同样的操作，每次把 *A* 和 *B* 的位置右移一次，移位 4 次之后，移位控制信号变为 0，寄存器 *A* 和 *B* 的值都是 1011。这样，*A* 的数据传送给 *B*，而 *A* 的内容仍旧保持不变。

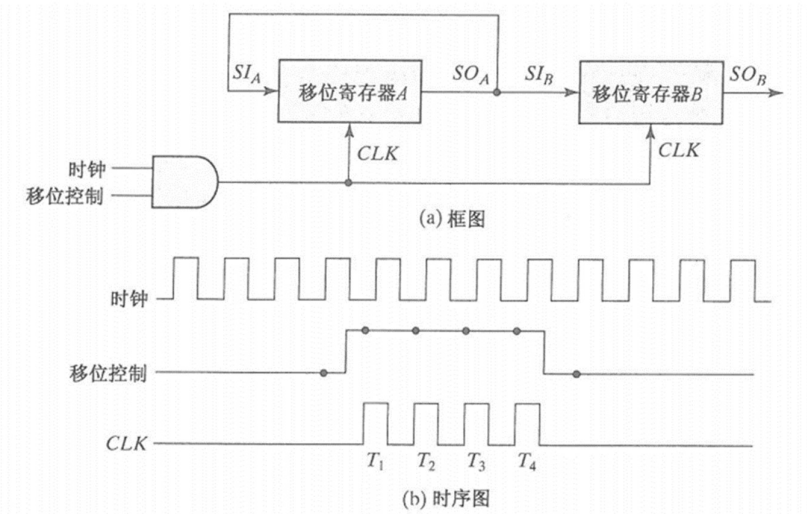


图 10.54 从寄存器 *A* 到寄存器 *B* 的串行传输

表 10.10 串行传输过程举例

定时脉冲	移位寄存器 A	移位寄存器 B
初值	1 0 1 1	0 0 1 0
T_1 后	1 1 0 1	1 0 0 1
T_2 后	1 1 1 0	1 1 0 0
T_3 后	0 1 1 1	0 1 1 0
T_4 后	1 0 1 1	1 0 1 1

串行加法

数字计算机中的运算通常采用快速并行模式。串行运算速度慢，优点是需要的器件少，在 VLSI 中，需要占用的芯片面积小。下面通过分析串行加法器的工作过程来说明串行工作方式。

串行相加的两个二进制数据存储在两个移位寄存器中，全加器用来对每一组的三个位相加，如下图 10.55 所示。全加器的进位输出连接到 D 触发器的输入端 D ， D 触发器的输出 Q 连接到全加器的进位输入端 z ，全加器的和位输出 S 连接到寄存器 A 的串行输入端 SI ，这样可以将每次相加的和传送给寄存器 A 。经过 A 的移位后，与寄存器 B 中的加数相加。寄存器 A 既保存被加数，又保存和位，寄存器 B 的串行输入端接收外部新的数据。

串行加法器的工作过程是：起初，被加数保存在寄存器 A 中，加数保存在寄存器 B 中，进位触发器清零。 A 和 B 的输出 SO 分别送给全加器的 x 和 y ，触发器的输出 Q 提供进位输入给 z ，移位控制端使两个寄存器和进位触发器都工作。这样，当下一个时钟脉冲到来时，两个寄存器都往右移位一次，全加器输出的和位 S 进入到寄存器 A 的最左边，这个过程一直进行下去，直到移位控制信号无效为止。因此，只使用一个全加器就可以完成对到来的数据及前一次相加产生的进位进行相加运算，每次相加后产生的和位传送给寄存器 A 。

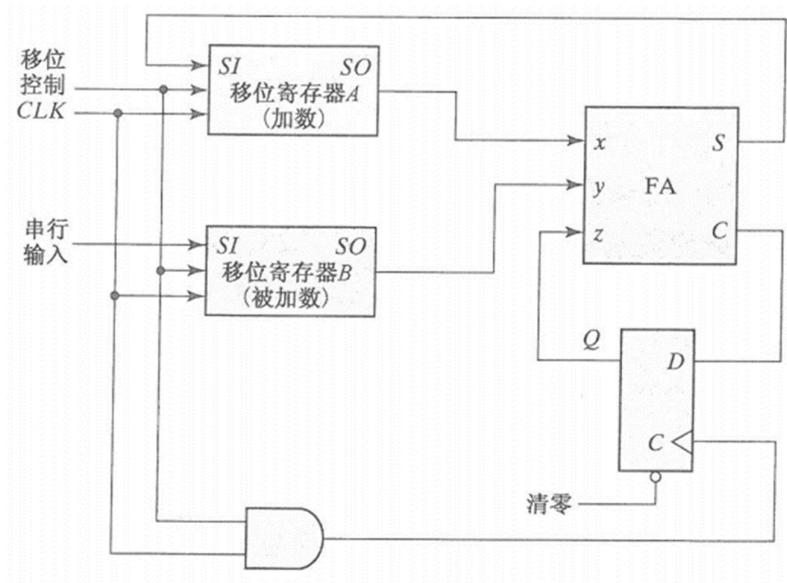


图 10.55 串行加法器

注意，寄存器 A 和进位触发器首先要被清零。 A 中数值与来自 B 的第一个数值相加， B 通过移位，将第二个数值串行输出，与寄存器 A 的数值相加。同时，第三个数值串行移位进寄存器 B 。按照这种方法可以完成两位、三位甚至更多位的相加，累加结果存储在寄存器 A 中。

为说明串行运算也可以采用时序电路设计方法来实现，下面使用状态表重新设计串行加法器。首先，假设用两个移位寄存器存储串行相加的二进制数，寄存器的串行输出用 x 和 y 表示，设计的时序电路首先不包括移位寄存器，设计完后再把寄存器插入进去，这样可以得到完整的电路。时序电路有两个输入端 x 和 y ；一个输出 S 表示和位，还有一个输出表示触发器的进位 Q 。电路的状态如表 10.11 所示， Q 的现态也是进位的现态， Q 与输入 x 和 y 一起产生和位 S ， Q 的次态代表了进位输出。注意，除了进位输入是 Q 的现态、进位输出是 Q 的次态之外，状态表的输入部分与全加器的真值表输入部分完全相同。

表 10.11 串行加法器的状态表

现 态	输 入		次 态	输 出	触发器输入	
Q	x	y	Q	S	J_Q	K_Q
0	0	0	0	0	0	X
0	0	1	0	1	0	X
0	1	0	0	1	0	X
0	1	1	1	0	1	X
1	0	0	0	1	X	1
1	0	1	1	0	X	0
1	1	0	1	0	X	0
1	1	1	1	1	X	0

如果用 D 触发器产生 Q ，电路简化结果如图 10.55 所示。假设用 JK 触发器产生 Q 需要确定输入 J 和 K 的数值，此时需要参考激励表（见表 10.7），表 10.11 的最后两列是 JK 触发器的激励表。触发器的输入方程和输出方程用卡诺图化简可以得到：

$$J_Q = xy$$

$$K_Q = x'y' = (x + y)'$$

$$S = x \oplus y \oplus Q$$

用 JK 触发器设计的串行加法器的电路图如下图 10.56 所示。电路中有四个门和一个 JK 触发器，图中还包括两个移位寄存器。这是一个完整的串行加法器电路，输出 S 不仅与 x 和 y 有关，还与 Q 的现态有关， Q 的次态与自身的现态及从移位寄存器串行输出的 x 和 y 有关。

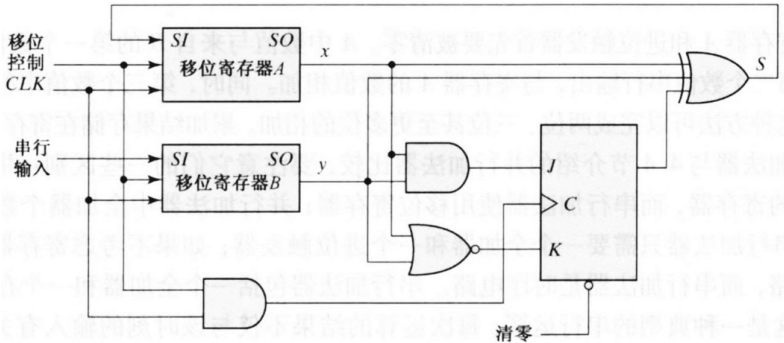


图 10.56 串行加法器第二种形式

通用移位寄存器

如果移位寄存器中的触发器输出是可存取的,则移位串行输入的信息也可以从触发器的输出并行取出。如果移位寄存器具有并行预置功能,则并行预置的数据可以通过移位寄存器串行移出。

有些移位寄存器带有并行传输的输入和输出端口,并且同时具有左移和右移的功能。最具代表性的移位寄存器具有以下功能:

1. 清零控制端将寄存器复位到 0。
2. 时钟输入端与操作同步。
3. 右移控制输入端使寄存器具有右移功能, 还有与右移有关的串行输入端和输出端。
4. 左移控制端使寄存器具有左移功能, 还有与左移有关的串行输入端和输出端。
5. 并行预置端使寄存器具有并行传输功能, 还有与并行传输有关的 n 个输入端。
6. n 个并行输出端。
7. 在没有时钟作用时, 寄存器中的信息保持不变。

也有一些移位寄存器只具有上述部分功能,但至少应具有移位功能。寄存器只能往一个方向移位就不是双向移位寄存器,只有能在两个方向上移位的寄存器才是双向移位寄存器。如果寄存器既具有双向移位,又具有并行预置功能,则称之为通用移位寄存器。

四位通用移位寄存器电路如下图 10.57 所示。

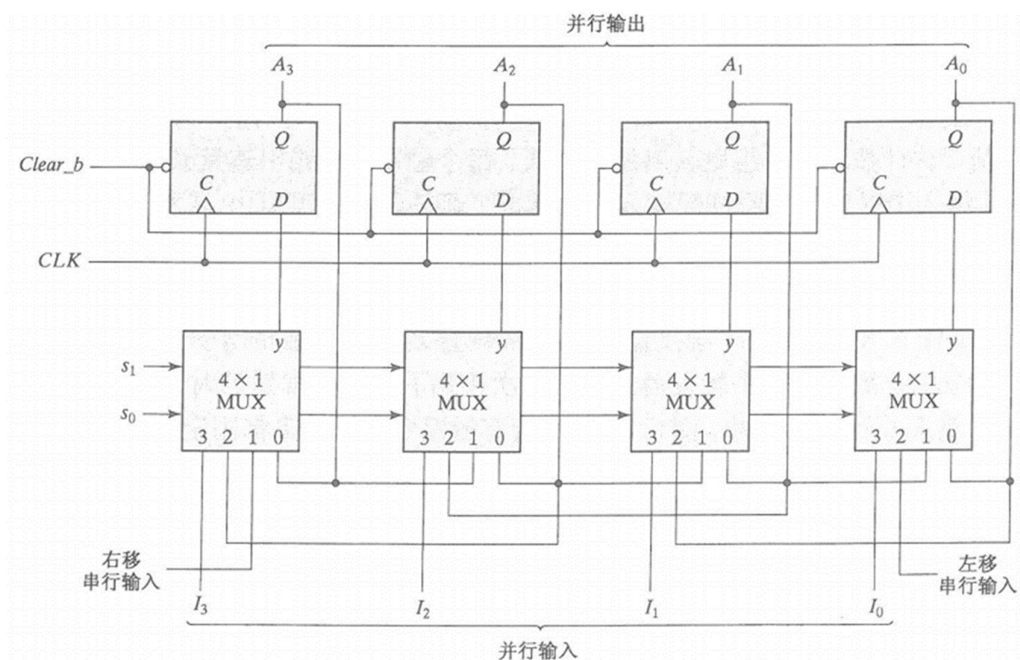


图 10.57 四位通用移位寄存器

该电路具有上述 7 条的全部功能。电路结构包括四个 D 触发器和四个 4 选 1 数据选择器。四个数据选择器共用两个选择输入端 S_0 和 S_1 ，若 $S_1S_0=00$ 时，每个数据选择器的 0 通道输入被选中；若 $S_1S_0=01$ 时，1 通道输入被选中；另外两种情况与此类似。选择不同的输入信号可以控制寄存器的工作模式，如下表 10.12 所示。当 $S_1S_0=00$ 时，寄存器的现态连到 D 触发器的输入端 D ，此时，每个触发器的输出反馈回各自的输入端，因此状态保持不变；当 $S_1S_0=01$ 时，数据选择器将 1 通道输入送给触发器的输入端 D ，此时的功能是右移，右移串行输入送给触发器 A_3 的数据输入端；当 $S_1S_0=10$ 时，此时的功能是左移，左移串行输入送给触发器 A_0 的数据输入端；最后，当 $S_1S_0=11$ 时，并行输入端的数值在时钟边沿到来时同时传输给寄存器。注意：对于右移操作，数据输入是 MSB_in ，对于左移操作，数据输入是 LSB_in 。 $Clear_b$ 是一个低电平信号，用来将触发器清零。

表 10.12 图 10.57 的寄存器功能表

控制模式		寄存器功能 Q
s_1	s_0	
0	0	保持
0	1	右移
1	0	左移
1	1	并行预置

移位寄存器常用于数字系统的接口：串并-并串转换。例如，假设在两点之间需要传输 n 位的数值，如果距离较远，使用 n 条线并行传输的代价高昂，此时可使用一条线进行串行传输，每次发送一位比较经济。发送端以并行方式从系统获得 n 位的数据，然后送给移位寄存器，在一条公共线上串行传输。接收端将收到的串行传输数据移位进寄存器，当所有 n 位都收到后，接收端的系统可从寄存器的输出并行读出。因此，发送端将数据进行并串转换，而接收端反过来进行串并转换。

10.6.3 行波计数器

寄存器在输入脉冲的作用下，按照一定的状态序列转移，称为计数器。输入脉冲可能是时钟脉冲，也可能是由外部信号源产生的有固定或随机时间间隔的脉冲序列。状态序列可以是按照二进制数值的顺序，也可以是其他状态序列。如果计数值按照自然二进制数值顺序变化，则称之为二进制计数器。 n 位的二进制计数器由 n 个触发器构成，计数值从 0 到 2^n-1 。常用的计数器有两种：行波计数器和同步计数器。在行波计数器中，每个触发器的输出被用作触发其他触发器。换句话说，部分或者全部触发器的时钟输入 C 并没有共用一个时钟脉冲，其触发是由其他触发器的输出提供的。在同步计数器中，所有触发器的时钟输入 C 都连接到相同的时钟端，同步计数器将在稍后介绍，这里以二进制和 BCD 码的行波计数器为例说明它们的工作过程。

二进制行波计数器

二进制行波计数器由一组翻转触发器构成，每个触发器的输出连接到下一个触发器的时钟输入端，第一个触发器接收外部输入的计数脉冲。翻转触发器可以由 JK 触发器转换而来（将 JK 输入连到一起），也可以用 T 触发器转换（ T 输入端接逻辑 1），还可以由 D 触发器转换（将输出反相连接到输入端 D ）。每来一个时钟脉冲，触发器发生翻转。两个四位二进制行波计数器的逻辑图如下图 10.58 所示，图 10.58（a）是由 T 触发器转换成翻转触发器而构成的计数器，而图 10.58（b）则是由 D 触发器构成的计数器。

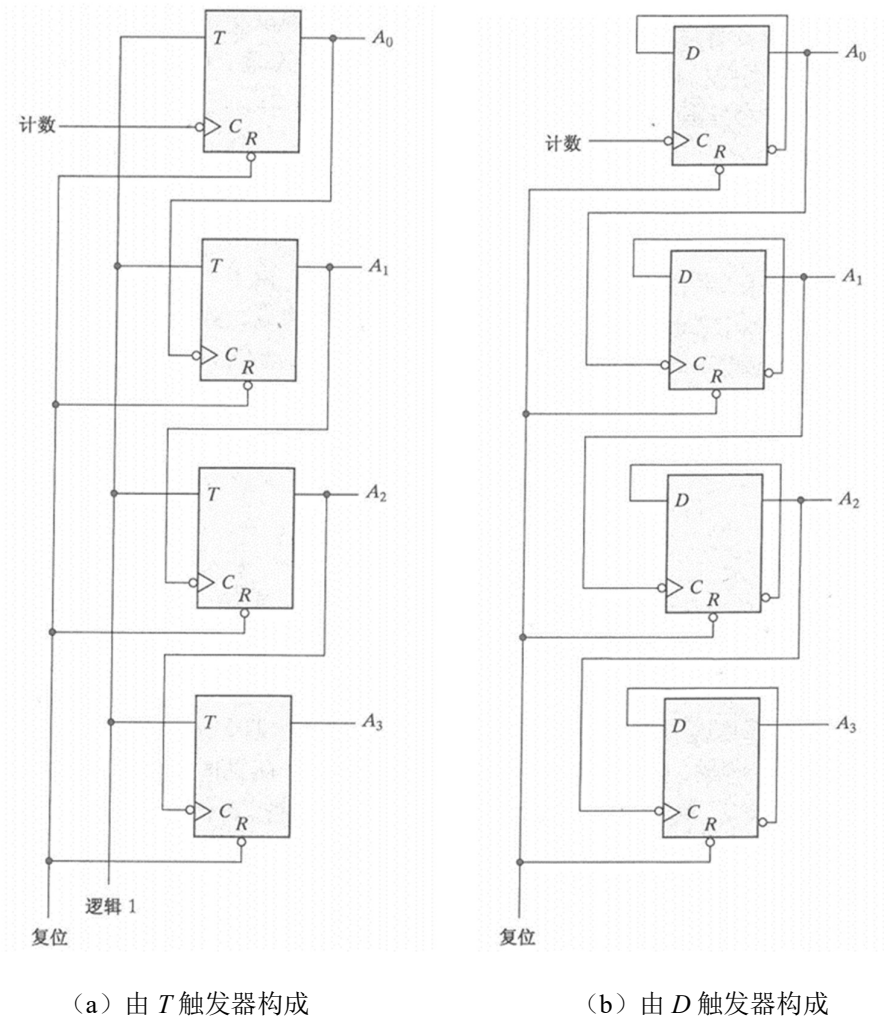


图 10.58 四位行波二进制计数器

每个触发器的输出依次接到下一个触发器的时钟端，第一个触发器时钟是外部输入计数器脉冲 C ，图 10.58（a）中所有触发器的输入 T 都固定接 1。时钟符号 C 前面的圆圈表示触发器时钟是下降沿有效，当前一个触发器的输出从 1 变为 0 时，会产生下降沿。

下表 10.13 中列出了前 9 个二进制计数。计数值起始值是 0，然后每来一个计数脉

冲加 1，在计数值为 15 后，返回到 0，以后重复这个计数过程。每来一个输入计数脉冲，最低有效位 A_0 翻转一次，每当 A_0 从 1 变到 0 时，信号 A_1 会翻转，每当 A_1 从 1 变到 0 时， A_2 也会翻转，当 A_2 从 1 变为 0 时， A_3 再翻转。

表 10.13 二进制计数序列

A_3	A_2	A_1	A_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0

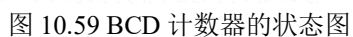
行波计数器的其他触发器工作类似。作为例子，我们来分析计数值从 0011 到 0100 变化时电路的工作情况。 A_0 在计数脉冲到来时发生翻转，由于 A_0 从 1 到 0，它产生的下降沿使 A_1 翻转，结果 A_1 从 1 变为 0， A_1 从 1 变到 0 又触发 A_2 翻转，使 A_2 从 0 变为 1 产生上升沿，不能触发 A_3 。因此，计数值从 0011 到 0100 变化，其实质是触发器状态依次变化得到的，计数值从 0011 到 0010，然后到 0000，最后回到 0100 稳定下来。触发器连续不断地依次变化，信号在计数器中以行波的方式从一级传送到下一级。

反向计数的二进制计数器称为二进制减法计数器。在减法计数器中，每来一个输入计数脉冲，二进制计数值减 1。四位减法计数器的计数值从 15 开始，然后依次经过 14、13、12、...、0，再回到 15。从二进制减法计数器的计数序列中可见，每来一个计数脉冲，最低有效位翻转一次。如果前一位发生从 0 到 1 的变化，后面的位就会发生翻转。因此，二进制减法计数器的逻辑图与图 9.58 基本相同，区别在于前者电路中的触发器是上升沿有效（时钟输入端 C 的圆圈去掉）。如果使用下降沿触发的触发器，每个触发器的时钟输入 C 必须连接到前一个触发器的反相输出。当输出从 0 变到 1 时，反相输出从 1 变到 0，使下一级触发器翻转。

BCD 行波计数器

十进制计数器有 10 个状态，在计数值为 9 之后回到 0。由于一位十进制数至少需用四位二进制数表示，所以十进制计数器也至少需要四个触发器构成，触发器的输出表示一位十进制数。使用 BCD 编码后的状态图如下图 10.59 所示，它与二进制计数器的区别在于 1001（十进制数 9 的编码）之后的状态是 0000（十进制数 0 的编码）。

使用 JK 触发器构成的 BCD 行波计数器如下图 10.60 所示。四个输出用字母 Q 表示，下标代表 BCD 码对应位的权重， Q_1 的输出既连到了 Q_2 ，又连到了 Q_8 的时钟输入端 C 。 Q_2 的输出连到了 Q_4 的时钟输入 C 。输入端 J 和 K 有的直接接高电平信号，有的接其他触发器的输出。



为求出 BCD 行波计数器发生状态转移时的输入,有必要首先列出这些状态序列。

如图 10.59 所示，每来一个时钟脉冲， Q_1 的状态变化一次；只要 $Q_8=0$ ，每次 Q_1 从 1 变为 0， Q_2 翻转一次；当 Q_8 为 1 时， Q_2 维持在 0 不变；每次 Q_2 从 1 变为 0， Q_4 翻转一次；只要 Q_2 或者 Q_4 为 0， Q_8 维持在 0 不变；只有当 Q_2 和 Q_4 都为 1，遇 Q_1 从 1 变为 0 时， Q_8 才翻转， Q_8 在 Q_1 的有效边沿被清零。

图 10.60 是一个模十 BCD 计数器，计数值从 0 到 9。为使计数值从 0 到 99，需要使用两个模十计数器，而从 0 到 999 的计数则需要使用三个模十计数器，这些计数器都是将模十 BCD 计数器级联起来实现的。三个模十 BCD 计数器的电路如下图 10.61 所示，第二个和第三个模十计数器的输入来自前一个计数器的输出 Q_8 。当某个模十计数器的 Q_8 从 1 变为 0 时，它将触发下一个计数器，而它自身的计数值从 9 变为 0。

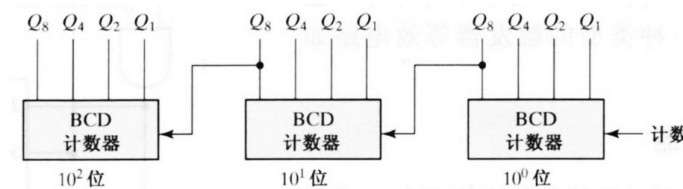


图 10.61 三个模十 BCD 计数器级联实现 3 位十进制计数 (0-999)

10.6.4 同步计数器

同步计数器与行波计数器的不同之处在于，前者的所有触发器共用一个时钟输入信号，共同的时钟将触发所有的触发器；后者触发器时钟不共用，每次只有一个触发器被触发，触发器是否翻转取决于时钟边沿到来时的输入端 T 或者 J 和 K 的取值。如果 $T=0$ 或者 $J=K=0$ ，触发器状态保持不变；如果 $T=1$ 或者 $J=K=1$ ，触发器发生翻转。同步时序电路的设计过程在本章前面已经介绍，图 10.50 是三位二进制计数器的设计结果。本节将通过一些典型的同步计数器来说明其工作原理。

二进制计数器

同步二进制计数器的设计非常简单，不需要按照时序电路设计过程进行设计。在同步二进制计数器中，处于最低有效位的触发器每遇一个时钟脉冲翻转一次，其他位的触发器在其所有低位都等于 1 时，遇时钟脉冲到来时也发生翻转。例如，如果一个四位计数器的现态是 $A_3A_2A_1A_0=0011$ ，下一个计数值将是 0100， A_0 总发生翻转， A_0 的现态为 1， A_1 也发生翻转， A_1A_0 的现态为 11， A_2 将发生翻转，然而，由于 $A_2A_1A_0$ 的现态是 011，不满足全为 1 的条件，所以 A_3 不发生翻转。

同步二进制计数器结构比较有规则，可以由翻转触发器和门来构成。下图 10.62 为四位同步二进制计数器电路，从图中可以看出这种规则：所有触发器的时钟输入共用，计数使能输入端连接到计数器的使能端，如果使能输入为 0，则所有 J 和 K 将都等于 0，计数器的状态保持不变；如果计数器使能输入为 1，第一级触发器的输入 J 和 K 将为 1。再看其他位触发器的情况，如果比它位置低的触发器输出为 1，并且计数器使能输入为

1，则该位置触发器的 J 和 K 也都为 1。电路中的与门用来产生每级触发器的输入 J 和 K 。触发器级数可以扩展，每一级都需要一个附加的触发器和一个与门。

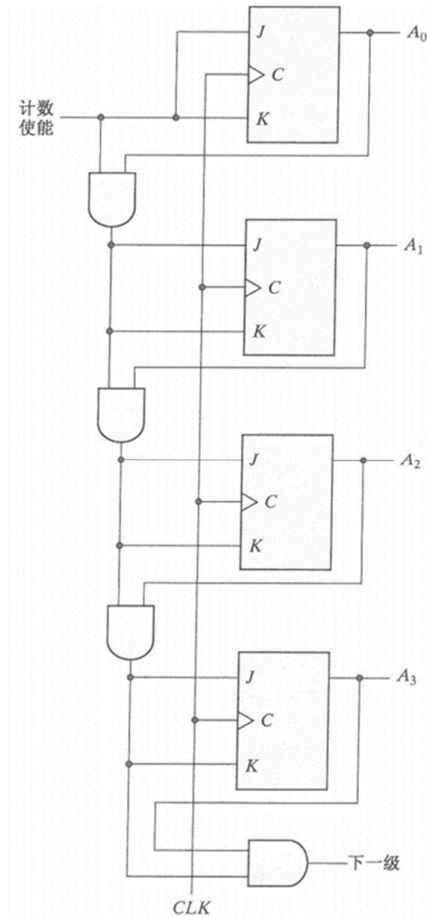


图 10.62 四位同步二进制计数器

注意，所有触发器都是在时钟上升沿触发，这点与行波计数器不同。同步计数器中的触发器可以是上升沿触发，也可以是下降沿触发，所使用的翻转触发器类型可以由 JK 、 T 或带异或门的 D 触发器转换而来。

可逆二进制计数器

同步减法二进制计数器的状态序列与二进制数的顺序相反，从 1111 到 0000，然后回到 1111，这样重复减 1 计数下去。采用常规方法设计减法计数器时，先来观察减法计数状态序列的规律。最低有效位每遇一个时钟脉冲翻转一次，其他位在其所有低位都为 0 时，遇时钟到来时也会翻转。例如，现态 0100 之后的次态是 0011，最低有效位总是翻转，由于第一位为 0，所以第二位也发生翻转，由于第一位和第二位都为 0，所以第三位将发生翻转，由于所有低位并不全为 0，所以第四位不翻转。

减法二进制计数器结构与图 10.62 所示电路大致相同，区别在于：与门的输入信号

不是来自前一个触发器的同相输出，而是触发器的反相输出。

把加法和减法功能集中在一个电路中得到可逆计数器。由 T 触发器构成的可逆二进制计数器如下图 10.63 所示，它有一个控制加法计数的输入端和一个控制减法计数的输入端。当加法计数输入为 1 时，由于输入端 T 来自前一个触发器的输出，电路加法计数；当减法计数输入端为 1，且加法计数输入端为 0 时，由于前一个触发器的反相输出接到输入端 T ，所以电路减法计数。当加法计数输入端和减法计数输入端都为 0 时，计数器状态保持不变。当加法计数输入端和减法计数输入端都为 1 时，电路加法计数，即加法计数输入端比减法计数输入端优先权高。

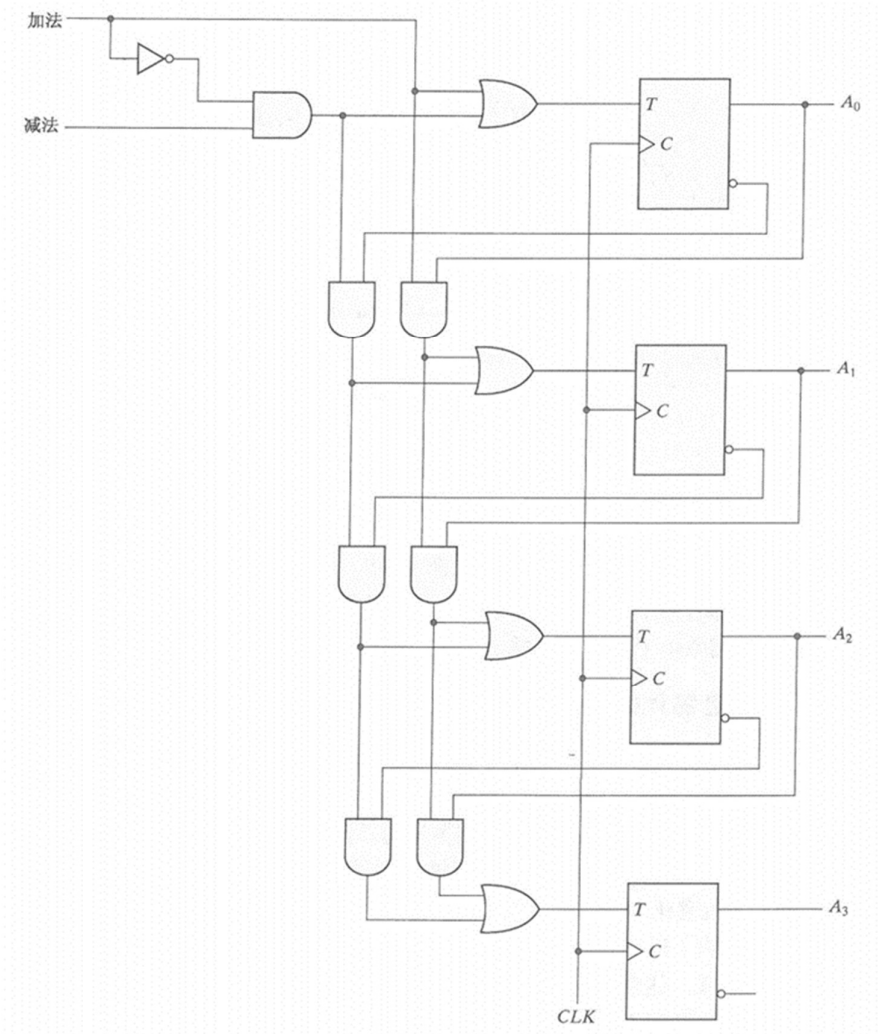


图 10.63 四位可逆二进制计数器

BCD 计数器

BCD 计数器的计数序列是二进制编码十进制，从 0000 到 1001，然后再回到 0000。由于 9 之后回到 0，BCD 计数器不如二进制计数器的结构那样规则。为导出 BCD 同步计数器电路，有必要按照时序电路的设计步骤进行设计。

BCD 计数器的状态列于表 10.14 中。 T 触发器输入函数可以从现态和次态的关系中得到，输出 y 也列于表中，当现态是 1001 时，输出为 1。因此，对于同一个脉冲，当现态从 1001 变为 0000 时，其输出 y 可以用来控制下一级计数器，使之计数。

表 10.14 BCD 计数器的状态表

现 态				次 态				输 出	触发器输入			
Q_8	Q_4	Q_2	Q_1	Q_8	Q_4	Q_2	Q_1	y	TQ_8	TQ_4	TQ_2	TQ_1
0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	0	1
0	0	1	1	0	1	0	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	0	1
0	1	0	1	0	1	1	0	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	0	1
1	0	0	1	0	0	0	0	1	1	0	0	1

触发器输入方程通过卡诺图化简，未使用的状态对应最小项 10 到 15，可认为是任意态，简化结果为：

$$\begin{aligned}
 T_{Q1} &= 1 \\
 T_{Q2} &= Q_8'Q_1 \\
 T_{Q4} &= Q_2Q_1 \\
 T_{Q8} &= Q_8Q_1 + Q_4Q_2Q_1 \\
 y &= Q_8Q_1
 \end{aligned}$$

该电路包括 4 个 T 触发器、5 个与门和 1 个或门。同步 BCD 计数器可以级联成任意长度的十进制计数器，级联结果示于图 10.61，输出 y 接到下一级计数器的计数使能输入端。

带并行预置的二进制计数器

数字系统中使用的二进制计数器大多数都需具有并行预置功能，这样才能使计数器在开始工作前，为计数器预置初始值。下图 10.64 是一个四位寄存器的逻辑电路图，它有并行预置功能，同时也可用作计数器。预置控制输入端为 1 时，计数器不工作，此时可以将四位输入数据传送给对应的四个触发器。如果预置控制输入和计数控制输入都为 0，时钟脉冲到来也不会使寄存器状态发生变化。

当计数使能端为 1 时，如果所有触发器输出为 1，进位输出也为 1，这也是使下一级触发器发生翻转的条件。进位输出用于计数器扩展，进位输出直接来自所有触发器的输出端，因此计数器速度得到提高，产生的进位延时下降，状态 1111 变为 0000，只时延一个门，而在图 10.62 所示电路中使用的与门串联会有 4 个门的延时。类似地，触发

器直接接收前一级触发器的输出，而不是从与门链中得到输入。

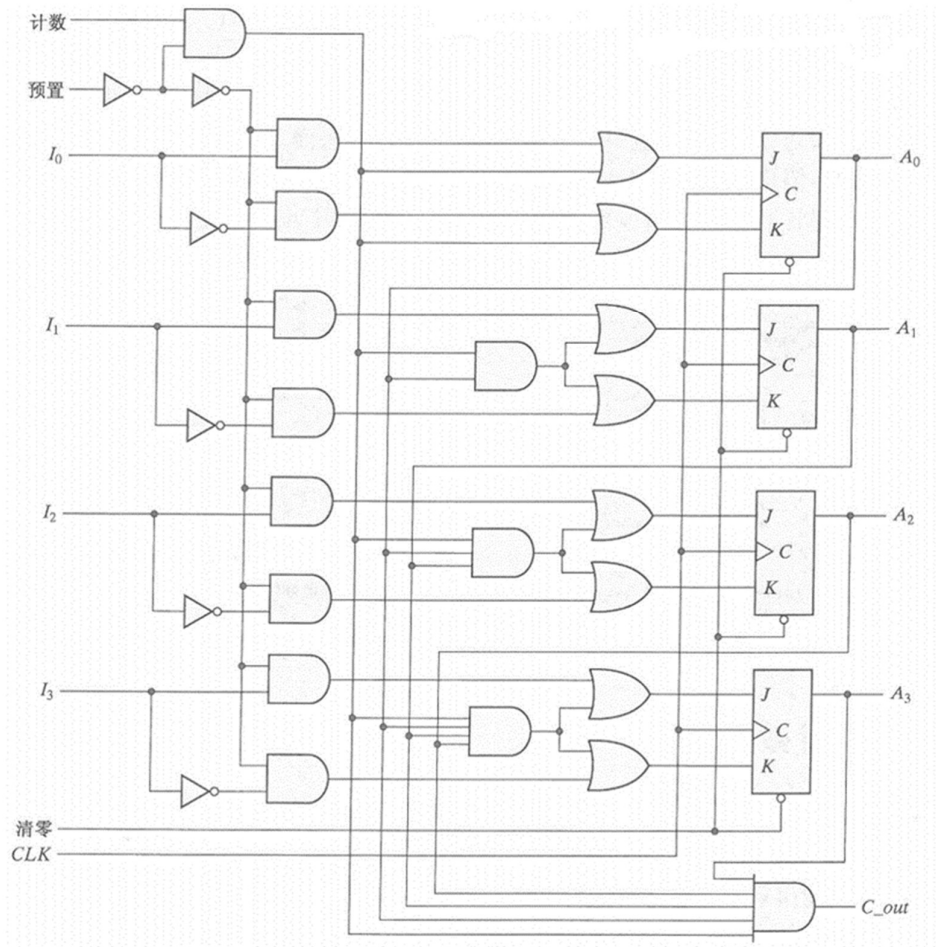


图 10.64 带并行预置的四位二进制计数器

计数器的功能列于下表 10.15。4 个控制输入信号为复位、时钟、预置和计数，它们共同决定了次态。复位信号是异步的，只要为 0，计数器将清零。无论此时时钟脉冲和其他信号如何，表中的符号 X 描述的是这种与其他输入无关的任意态。当计数器用于其他功能时，复位端必须为 1。当预置和计数使能输入端都为 0 时，即使时钟脉冲出现，输出也保持不变。预置输入为 1 时，输入 $I_0 \sim I_3$ 遇时钟上升沿被预置进寄存器，数据预置时与计数使能输入无关。当预置输入有效时，计数输入被阻隔。只有当预置输入为 0 时，计数输入信号才能控制计数器的功能。

表 10.15 图 10.64 中计数器的功能表

清	零	CLK	预	置	计	数	功	能
0		X	X	X				复位
1		↑	1	X				预置
1		↑	0	1				计数
1		↑	0	0				保持

带并行预置功能的计数器可用于产生任意计数序列。 下图 10.65 给出了两种由带并行预置计数器构成的 BCD 计数器方法。对于每种方法，计数使能控制为 1，确保时钟能作用于计数器。另外，计数器的预置控制端能使计数器不计数，而复位操作是异步的，与其他输入无关。

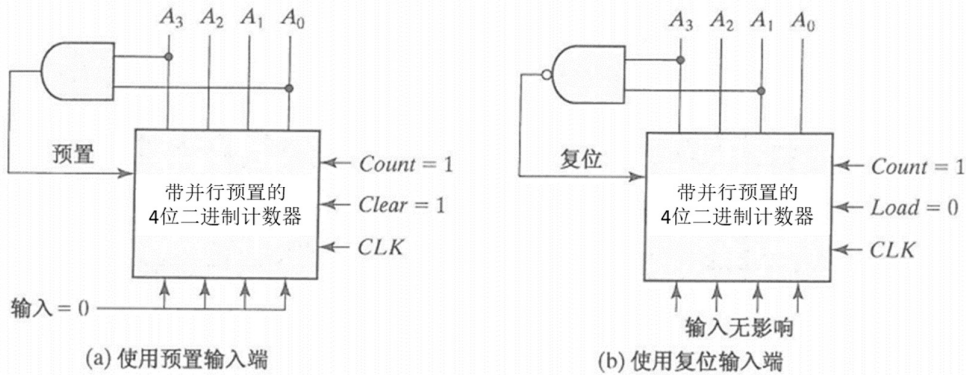


图 10.65 由带并行预置的计数器构成 BCD 计数器的两种方法

图 10.65 (a) 中与门的作用是检测状态 1001 是否出现。计数器起初被复位到 0，然后复位输入和计数使能输入都被置为 1，计数器开始计数。只要与门输出为 0，每来一个时钟上升沿，计数器就会加 1。当计数值达到 1001 时， A_0 和 A_3 都为 1，与门的输出也为 1，这时，预置输入有效。因此，在下一个时钟有效边沿，计数器预置。由于四个输入端都连到了逻辑 0，计数器在 1001 后的状态是 0000。因此，电路的计数值从 0000 到 1001，然后回到 0000，符合 BCD 计数器的功能要求。

在图 10.65 (b) 中，与非门用于检测计数值 1010。只要这个计数值产生，寄存器立即复位。由于寄存器变为 0，计数值 1010 没有机会存在很长时间。计数从 1001 到 1010，然后立即回到 0000，输出信号上出现瞬时的毛刺。这种瞬时的毛刺没有用处，因此不推荐使用这种配置方式。如果计数器具有同步复位输入端，可以在计数 1001 出现的下一个时钟，计数器复位到 0。

10.6.5 其他计数器

计数器可以用来产生需要的状态序列。被 N 整除计数器（也称为模 N 计数器）有 N 个状态。工作时，这些状态会周而复始地往复。状态序列可以是二进制计数值，也可以是其他任何序列。计数器用来产生控制数字系统的工作时序，也可用作移位寄存器。这里给出几个非二进制计数器的例子。

有无效状态的计数器

由 n 个触发器构成的电路有 2^n 个状态，这些状态并不都被使用，未使用的状态在状态表中不列出来。为简化输入方程，未使用的状态可认为是任意态，或者分配特定的次态。一旦电路被设计出来，外部干扰可导致电路进入那些不被使用的状态。因此，有

必要保证电路最终进入一个有效的状态，这样才可以正常工作，否则，如果时序电路在无效状态中循环，就没有办法让它进入到预定的状态转移序列。未使用状态的次态可以从设计好的电路分析加以确定。

举例来说，分析表 10.16 确定的计数器，计数器在六个状态序列间循环，触发器 B 和 C 重复二进制计数 00、01、10，触发器 A 每隔三个计数值就在 0 和 1 之间变换一次。计数器的计数序列不是自然的二进制数，有两个状态 011 和 111 没有出现在计数序列中，对应这两个状态转换的 JK 触发器的输入已列于表中，输入 K_B 和 K_C 的取值只能为 1 和 X，因此这些输入总为 1，其他触发器的输入方程通过将最小项 3 和 7 当作任意态来简化，简化结果为：

$$\begin{aligned} J_A &= B & K_A &= B \\ J_B &= C & K_B &= 1 \\ J_C &= B' & K_C &= 1 \end{aligned}$$

表 10.16 计数器的状态表

现 态			次 态			触发器输入					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	1	0	0	1	X	X	1	0	X
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	0	0	0	X	1	X	1	0	X

计数器的逻辑电路如图 10.66 (a) 所示。由于有两个未使用的状态，下面分析它们对电路造成的影响。如果电路因为信号发生错误进入了状态 011，那么电路将在下一个时钟脉冲作用下进入状态 100，这个结果可以由电路图观察到。我们注意到，当 $B=1$ 时，下一个时钟边沿将使 A 反相，对 C 清零；当 $C=1$ 时，遇时钟边沿将 B 反相。采用同样的方式，还可以求出现态 111 的次态是 000。

电路的全状态转换图如图 10.66 (b) 所示。如果电路因为外部干扰进入一个未被使用的状态，而下一个计数脉冲的到来将使它进入一个有效的状态，电路接下来正常计数，这样的计数器是能够自启动的。对于具有自启动特性的计数器，如果电路进入一个未用状态，在经过一个或者几个时钟脉冲之后最终都能到达正常的计数序列。

环形计数器

在数字系统中，控制工作时序的定时信号可以用移位寄存器，或者用计数器加译码器产生。环形计数器是可循环移位的寄存器，在每个特定时刻只有一个触发器被置位，而其他触发器都被复位。唯一的一个 1 从一个触发器移到另一个触发器，这样产生定时信号。下图 10.67 (a) 是一个四位移位寄存器用于环形计数器的例子。寄存器的初始值

是 1000，每来一个时钟，唯一的 1 右移一次，然后从 T_3 移回到 T_0 ，这样循环下去。四个时钟周期中，只有一个触发器输出为 1，产生的 4 个定时信号如图 10.67 (b) 所示。在时钟脉冲的下降沿之后，触发器输出为 1，直到下个时钟脉冲到来前都保持不变。

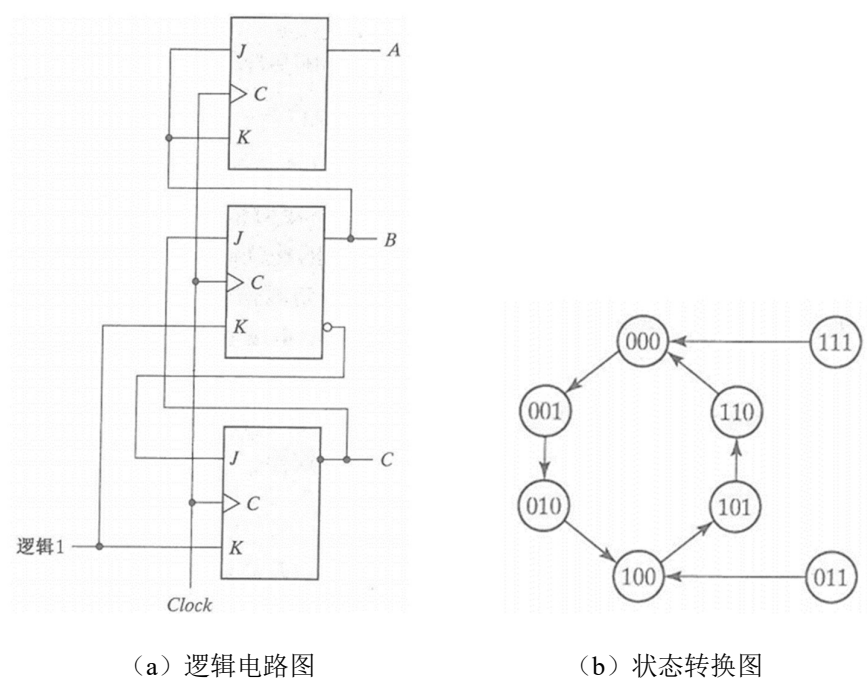


图 10.66 存在未使用状态的计数器

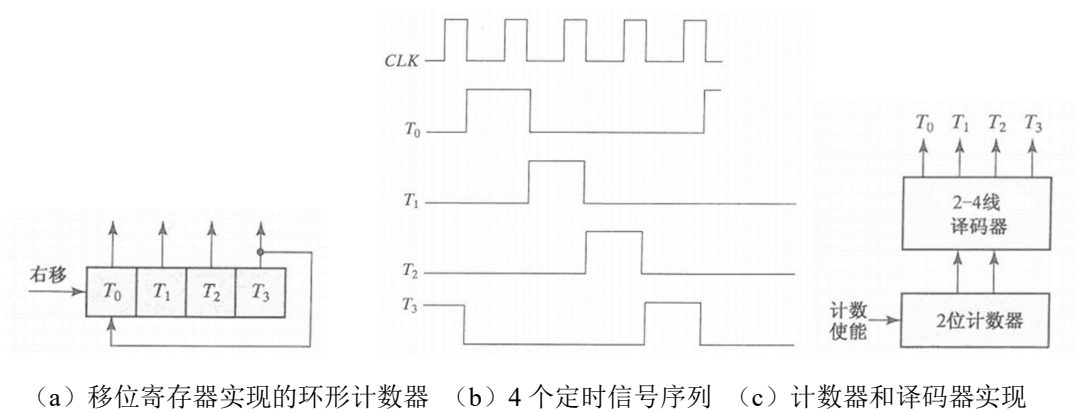


图 10.67 定时信号产生

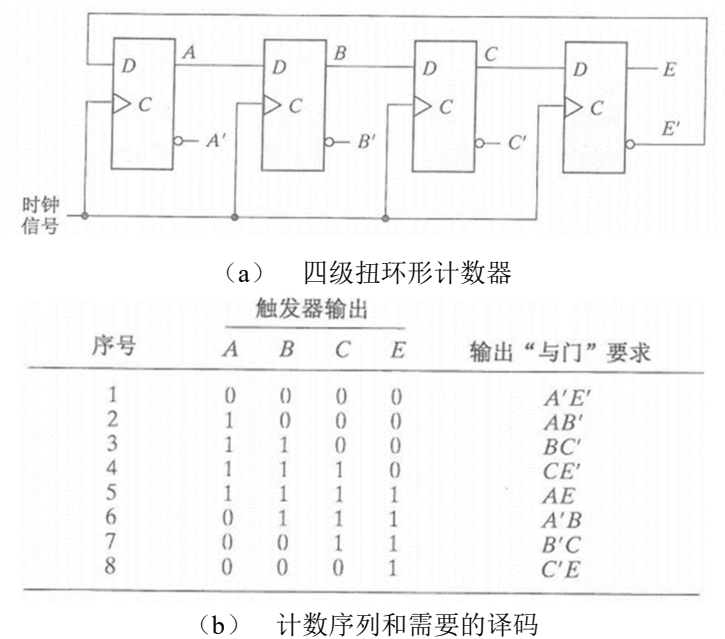
上述定时信号也可以用 2 位计数器产生，计数器在 4 个不同状态之间循环。图 10.67 (c) 中的译码器对计数器的 4 个状态进行译码，产生所需要的定时信号序列。

为了产生 2^n 个定时信号，需要由 2^n 个触发器构成的移位寄存器，或者是一个 n 位二进制计数器外加一个 $n-2^n$ 线的译码器。例如，要产生 16 个定时信号，可以用 16 位移位寄存器构成的环形计数器，或者用一个 4 位二进制计数器和一个 4-16 线译码器。第

一种情况需要 16 个触发器，第二种情况需要 4 个触发器和 16 个用于译码的 4 输入与门。还可以将移位寄存器和译码器组合在一起产生定时信号，使用这种方法时，触发器数目少于环形计数器，译码器只需要 2 输入的门，该电路称为约翰逊（Johnson）计数器。

约翰逊计数器

一个 n 位的环形计数器中，唯一的一个 1 在触发器之间移位，可以产生 n 种不同的状态。如果移位寄存器按照扭环形计数器方式连接，则状态的数目加倍。扭环形计数器也是一种循环移位寄存器，不过它是将最后一级触发器的反相输出连到第一级触发器的输入端。下图 10.68（a）所示就是这样的移位寄存器。每输入一个时钟脉冲，移位寄存器向右移位一次，同时触发器 E 的反相输出信号传送给触发器 A 。该扭环形计数器的状态序列从全零开始，一共有 8 个，如图 10.68（b）所示。



(b) 计数序列和需要的译码

图 10.68 约翰逊计数器的结构

通常，一个 n 位的扭环形计数器的状态序列有 $2n$ 个状态，从全 0 开始，每次移位左边增加一个 1，直到寄存器全为 1 为止，在后续的移位操作中，0 从左边插入，直到寄存器又为全 0 为止。

约翰逊计数器是在 n 位扭环形计数器基础上增加 $2n$ 个用于译码的门，这样可以产生 $2n$ 个定时信号。用于译码的门在图 10.68 中没有给出，但可以由表的最后一列确定。8 个与门列于表中，把它们接入其中就会得到完整的约翰逊计数器的电路图。由于每个门只对一个特定的状态序列起作用，因此门的输出为 8 个定时信号。

按照上述方式，对 n 位扭环形计数器的译码可以产生 $2n$ 个定时信号，全 0 状态被译码成最两边触发器反相输出的与，全 1 状态被译码成最两边触发器的同相输出的与，

其他状态译码由序列中相连的 1、0 或 0、1 决定。例如，序列 7 有一个连续的 0、1，对应触发器为 B 和 C ，译码时取 B 的反相和 C 的同相相与，这样结果为 $B'C$ 。

图 10.68 (a) 所示电路有一个缺陷，如果它进入了一个无效状态，将会一直从这个无效状态转移到另一个无效状态，永远无法进入有效状态中。可以通过修改电路以避免这种情况发生。一种办法是不要将触发器 B 的输出连接到触发器 C 的输入端 D ，而是对触发器 C 的输入端添加这样的函数：

$$D_C = (A + C) B$$

这里 D_C 是对应触发器 C 输入端 D 的输入方程。

约翰逊计数器可以用于产生任何个数的定时序列，需要的触发器数目只是定时信号的一半，用于译码的逻辑门电路的个数与定时信号相等，而且只使用 2 输入的逻辑门。

第七节 流水线概念和同步时序原理

10.7.1 流水线

流水线是一种非常普遍的设计技术，经常用来加速数字处理器的数据通路。这个概念可以用下图 10.69 (a) 的例子来解释，该电路的功能是计算 $\log(|a+b|)$ ，这里 a 和 b 都代表数字流（即计算必须针对很大的一组输入值来进行）。为保证正确求值，所需要的最小时钟周期 T 为：

$$T_{\min} = t_{c-q} + t_{pd,logic} + t_{su}$$

这里， t_{c-q} 和 t_{su} 分别为寄存器的传播延时和建立时间。假设寄存器为边沿触发的 D 寄存器。 $t_{pd,logic}$ 代表通过组合电路最坏情形下的延时路径，它由加法器、求绝对值和求对数的电路组成。在通常的系统中（即并不追求最新技术），组合电路的延时一般大大超过与寄存器相关的延时，因此是决定电路性能的主要因素。假设每一个逻辑模块的传播延时相等，那么每个逻辑模块起作用的时间只是时钟周期的 1/3（如果忽略寄存器的延时）。例如，加法器单元只在第一个 1/3 周期期间起作用而在其余的 2/3 周期期间一直闲置（没有进行有用的计算）。

流水线是一项提高资源利用率的技术，它增加了电路的数据处理量。假设我们在逻辑块之间插入寄存器，如图 10.69 (b) 所示。这使得一组输入数据的计算分布在几个时钟周期中，如下表 10.17 所示。数据组 (a_1, b_1) 的计算结果只是在三个时钟周期之后才出现在输出端。此时电路已在执行下一数据 (a_2, b_2) 和 (a_3, b_3) 的部分计算。这一计算过程以一种装配线的形式进行，因此得名流水线。

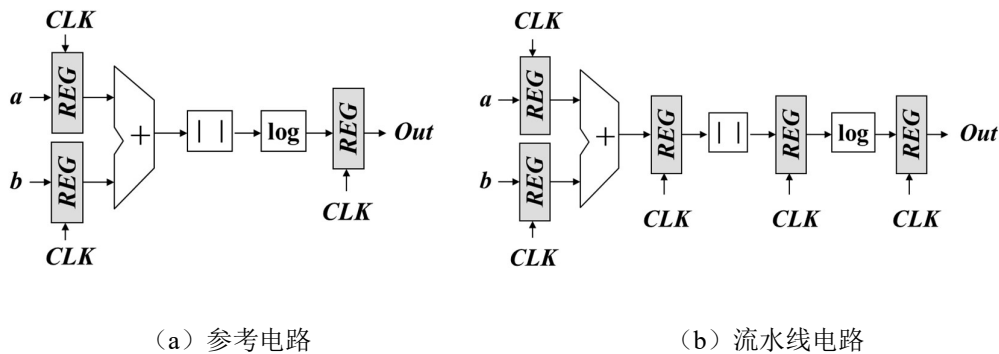


图 10.69 计算 $\log(|a+b|)$ 的数据通路

表 10.17 流水线计算的例子

时钟周期	加法器	求绝对值	求对数
1	$a_1 + b_1$		
2	$a_2 + b_2$	$ a_1 + b_1 $	
3	$a_3 + b_3$	$ a_2 + b_2 $	$\log(a_1 + b_1)$
4	$a_4 + b_4$	$ a_3 + b_3 $	$\log(a_2 + b_2)$
5	$a_5 + b_5$	$ a_4 + b_4 $	$\log(a_3 + b_3)$

流水线工作的优点可以从考察这一改进电路的最小时钟周期中看得非常清楚。组合电路块分成三个部分，每一部分比原来的总功能具有较小的传播延时，这就有效地减少了最小允许的时钟周期值：

$$T_{\min, \text{pipe}} = t_{c-q} + \max(t_{pd, \text{add}}, t_{pd, \text{abs}}, t_{pd, \text{log}}) + t_{su}$$

假设所有的逻辑块具有近似相等的传播延时，并且寄存器的延时相对于逻辑延时来说很小。在这些假设条件下，流水线电路的性能超过原来电路的三倍（即 $T_{\min, \text{pipe}} = T_{\min} / 3$ ）。这一性能提高的代价相对较小，只是增加两个寄存器和一个等待时间（latency）。这就解释了为什么流水线在实现高性能的数据通路时应用十分普遍。

（注：在这里等待时间定义为数据从输入传送至输出所需要的时钟周期数。在本例中，流水线使等待时间从 1 增加到 3。等待时间的增加一般是可接受的，但如不仔细处理也会引起全局性能的下降。）

10.7.2 同步时序原理

实际上目前设计的绝大部分数字系统都采用周期性的同步信号或时钟。时钟的产生和分布对系统的性能和功耗会产生显著的影响。假设一个正边沿触发系统，其中时钟的上升沿标志着一个时钟周期的开始和结束。在理想情况下，假设从中心分布点到每个寄

寄存器的时钟路径完全均衡，那么在系统不同点处的时钟相位（即相对于参照时间的时钟边沿的位置）也应当完全相同。下图 10.70 表示一个同步流水线数据通路的基本结构，在理想情形中，寄存器 R_1 和寄存器 R_2 的时钟具有相同的周期，并且在完全相同的时刻翻转。

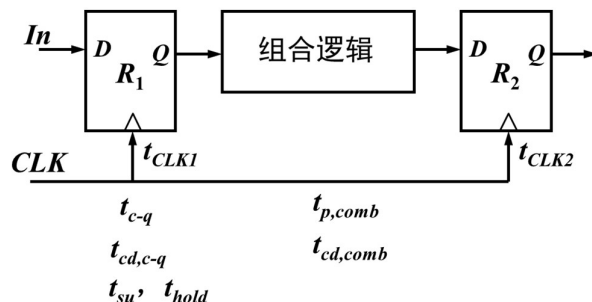


图 10.70 流水线数据通路电路及时序参数

假设已知下列时序电路的时序参数:

- 寄存器的“污染”或最小延时($t_{c-q,cd}$)和最大传播延时(t_{c-q})
- 寄存器的建立时间(t_{su})和维持时间(t_{hold})
- 组合逻辑的污染延时($t_{logic,cd}$)和最大延时(t_{logic})
- 时钟 $CLK1$ 和 $CLK2$ 的上升沿相对于全局参照时钟的位置(分别为 t_{CLK1} 和 t_{CLK2})

在理想情况下， $t_{CLK1} = t_{CLK2}$ ，因此这一时序电路要求的最小时钟周期仅取决于最坏情况的传播延时。时钟周期必须足够长，以便在时钟的下一个上升沿之前数据能够传播通过寄存器 R_1 和组合逻辑并在目标寄存器 R_2 处建立起来（满足建立时间要求），如前面所述，这一约束由以下表达式给出：

$$T > t_{c-q} + t_{logic} + t_{su}$$

与此同时，目标寄存器 R_2 的维持时间必须小于通过逻辑网络的最小传播延时（或者说，通过寄存器 R_1 和组合逻辑网络的最小传播延时必须大于寄存器 R_2 的维持时间要求）：

$$t_{hold} < t_{c-q,cd} + t_{logic,cd}$$

可惜的是，上述分析多少有点简单化，因为时钟永远也不会是理想的。实际上不同的时钟事件既不是理想周期性的也不是完全同步的。由于工艺和环境的变化，时钟信号会在空间和时间上同时发生偏差，这会导致性能下降或电路出错。

10.7.3 时钟不确定性*

时钟偏差（Clock Skew）

集成电路中一个时钟翻转的到达时间在空间上的差别通常称为时钟偏差。在一个 IC 上两点 i 和 j 之间的时钟偏差为 $\delta(i,j) = t_i - t_j$ ，这里 t_i 和 t_j 是该时钟上升沿相对于参照

时钟的位置。考虑图 10.70 中在寄存器 R_1 和 R_2 之间传送数据。

根据布线方向和时钟源的位置，时钟偏差可以有正有负，下图 10.71 显示的是正偏差情况下的时序图。如图所示，在第二个寄存器处时钟上升沿延迟了一个正的 δ 。

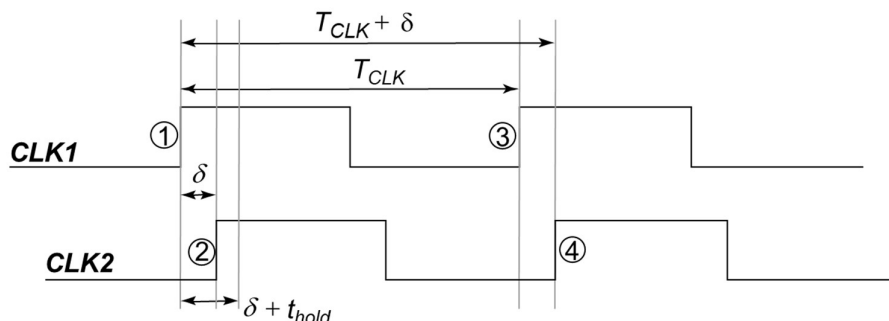


图 10.71 研究时钟偏差对性能和功能影响的时序图 ($\delta > 0$ 的情形)

时钟偏差是由于时钟路径的静态不匹配以及时钟在负载上的差异造成的。根据定义，各个周期的偏差是相同的。这就是说，如果在一个周期 $CLK2$ 落后于 $CLK1$ 一个 δ ，那么在下一个周期它也将同样落后 δ 。需要注意的是，时钟偏差并不造成时钟周期的变化，造成的只是相位的偏移。

时钟偏差现象无论对时序系统的性能还是功能都有很大的影响。首先，考虑时钟偏差对性能的影响。从图 10.71 中可以看到，由 R_1 在边沿①处采样的一个新输入 In 将传播通过组合逻辑并被 R_2 在边沿④处采样。如果时钟偏差为正，那么信号由 R_1 传播到 R_2 的可用时间就增加了一个时钟偏差值。组合逻辑的输出必须在 $CLK2$ 上升沿（点④）的一个建立时间之前有效。于是对这一最小时钟周期的约束就可以推导如下：

$$T + \delta \geq t_{c-q} + t_{logic} + t_{su} \quad \text{或} \quad T \geq t_{c-q} + t_{logic} + t_{su} - \delta$$

这一公式表示，时钟偏差实际上具有改善电路性能的可能。也就是说，电路可靠工作所要求的最小时钟周期随时钟偏差的增加而减小！这的确没错，但可惜的是，增加偏差会使电路对竞争情况更加敏感，而这有可能危及整个时序系统的正确工作。

这一点可以用以下的例子来说明：再次假设输入 In 在 $CLK1$ 的上升沿（即边沿①）处被采样进入 R_1 。 R_1 输出端的新数据传播通过组合逻辑并且应当在 $CLK2$ 的边沿④之前有效。然而，如果组合逻辑块的最小延时很小，那么 R_2 的输入就有可能在时钟边沿②之前就改变，导致求值出错。为了避免竞争，必须保证通过寄存器和组合逻辑的最小传播延时足够长，以使 R_2 的输入在边沿②之后的一段维持时间内保持有效（不变）。这一约束可以用公式表示为：

$$\delta + t_{hold} < t_{c-q,cd} + t_{logic,cd}$$

或

$$\delta < t_{c-q,cd} + t_{logic,cd} - t_{hold}$$

下图 10.72 显示的是 $\delta < 0$ 情况下的时序图。这时， $CLK2$ 的上升沿发生在 $CLK1$ 的上升沿之前。 R_1 在 $CLK1$ 的上升沿处采样一个新的输入，该新数据传播通过组合逻辑，并且在 $CLK2$ 的上升沿处（相应于边沿④）被 R_2 采样。正如图 10.72 及上面最小时钟周期约束公式显示的那样，负的时钟偏差对时序系统的性能有负面的影响。然而，假设 $t_{hold} + \delta < 0$ ，则负偏差意味着系统永远不会失误，因为边沿②发生在边沿①之前！

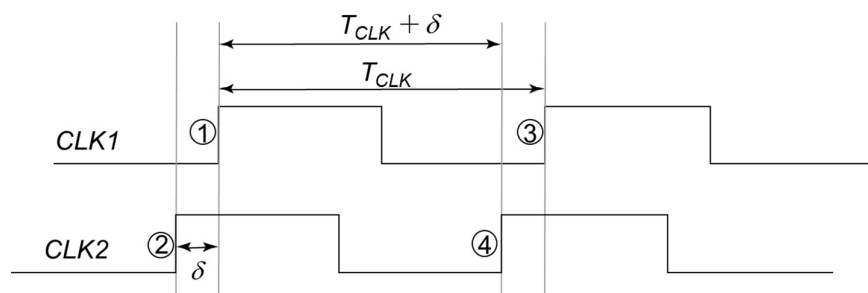


图 10.72 $\delta < 0$ 情况下的时序图（ $CLK2$ 的上升沿早于 $CLK1$ 上升沿到达）

正时钟偏差和负时钟偏差的例子见下图 10.73。

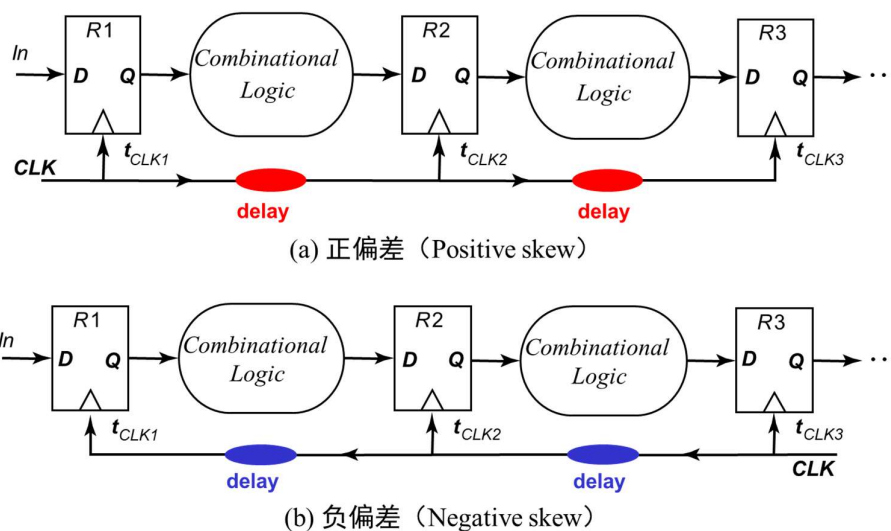


图 10.73 正时钟偏差和负时钟偏差的情形

- $\delta > 0$ — 这相当于时钟布线的方向与数据通过流水线的方向一致[见图 10.73 (a)]。在这种情形下，时钟偏差应严格控制并满足上面最小传播延时和维持时间的约束公式。如果不能满足这一约束，那么无论什么样的时钟周期电路都会出错。就是说，即使降低一个边沿触发电路的时钟频率并不能帮助解决时钟偏差问题！所以在设计时必须满足对维持时间的约束。反之，如前面最小时钟周期约束公式所示，正偏差能够增加电路的数据通过量，即时钟周期可以缩短 δ 。但这一改进的范围是有限的，因为较大的 δ 值很快就会导致违反维持时间约束公式的情况。

- $\delta < 0$ — 当时钟布线与数据方向相反时[见图 10.73 (b)], 时钟偏差为负值并显著提高了抗竞争的能力: 如果维持时间是 0 或负值, 竞争就可以被消除, 因为以上公式可以无条件成立! 由于偏差降低了可用于进行实际计算的时间, 所以时钟周期必须增加一个 $|\delta|$ 。总之, 在与数据相反的方向上布时钟线可以避免出错, 但会降低电路的性能。

遗憾的是, 由于在一般的逻辑电路中数据可以在两个方向上流动(如具有反馈的电路), 所以这种消除竞争的方法并不总能奏效。下图 10.74 显示的是根据数据传送的方向时钟偏差值可正可负。在这些情况下, 设计者必须考虑最坏情况下的时钟偏差。一般来说, 使时钟布线只发生负偏差是不现实的。因此, 设计一个偏差小的时钟网络才是最重要的。

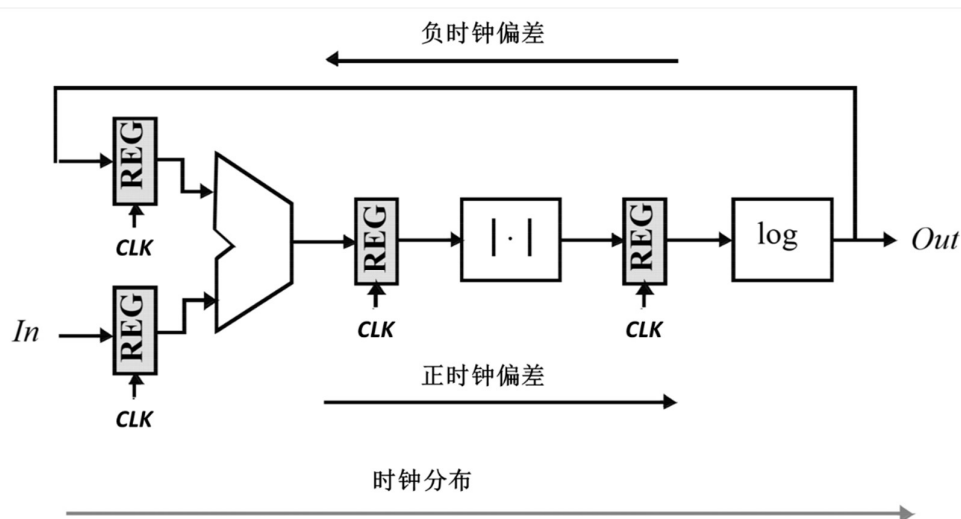


图 10.74 具有反馈的数据通路结构

例 10.1 估计传播延时和污染延时

考虑下图 10.75 中的逻辑电路。对给定的最坏情况下的逻辑门延时 t_{gate} , 确定该组合逻辑电路的污染延时和传播延时。也可以假设逻辑门的最大延时和最小延时相等。

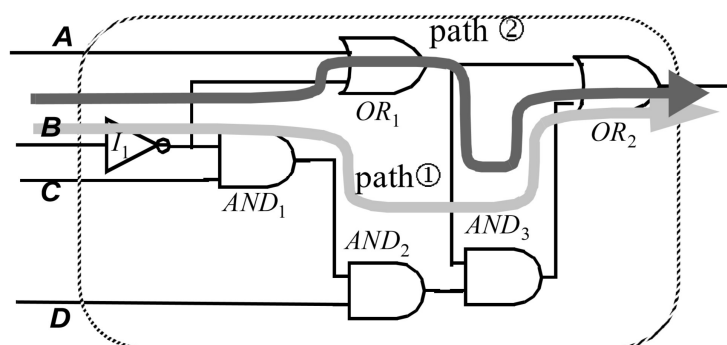


图 10.75 用于计算性能的逻辑电路

污染延时很容易求出，它是数据通过 OR_1 和 OR_2 的延时，等于 $2t_{gate}$ 。然而，计算最坏情况下的传播延时却并不那么简单。初看起来最坏情况似乎相应于路径① (path①)，因而延时为 $5 t_{gate}$ 。但当分析数据相关性时会发现路径①根本不会有信号经过，因而路径①称为虚假路径 (*false path*)。如果 $A=1$ ，那么关键路径经过 OR_1 和 OR_2 。如果 $A=0$ 及 $B=0$ ，则关键路径经过 I_1 ， OR_1 和 OR_2 (相应于延时为 $3 t_{gate}$)。对于 $A=0$ 而 $B=1$ 的情况，最长的路径经过 I_1 ， OR_1 ， AND_3 和 OR_2 。换言之，在这一简单 (但有创意) 的电路中，输出甚至与输入 C 和 D 无关 (即这里有冗余)。因此，实际的传播延时为 $4 t_{gate}$ 。如果给定传播延时和污染延时，电路所允许的最小偏差和最大偏差就很容易地计算出来。

注意：由于存在虚假路径，组合逻辑最坏情况的传播延时不能简单地通过相加各个逻辑门的传播延时来计算。关键路径很大程度上取决于电路的拓扑结构以及数据的相关性。

时钟抖动 (Clock Jitter)

时钟抖动是指在芯片的某一个给定点上时钟周期发生暂时的变化，即时钟周期在每个不同的周期上可以缩短或加长。时钟抖动是严格衡量时钟暂时不确定性的一项指标，并且经常针对某给定点进行说明。抖动可以用许多方法来衡量和表征，它是一个平均值为零的随机变量。绝对抖动 (t_{jitter}) 是指在某一给定位置处的一个时钟边沿相对于理想的周期性参照时钟边沿在最坏情形下的变化 (绝对值)。周期至周期抖动 (T_{jitter}) 一般是指单个时钟周期相对于理想参照时钟的时变 (time-varying) 偏离。对于一个给定的空间位置 i ， $T_{jitter}(n) = t_{clk,n+1} - t_{clk,n} - T_{CLK}$ ，这里， $t_{clk,n+1}$ 和 $t_{clk,n}$ 分别代表第 $n+1$ 个和第 n 个时钟边沿到达节点 i 的时间，而 T_{CLK} 则是名义时钟周期。在最坏情况下，周期至周期抖动的值等于绝对抖动的 2 倍 ($2 t_{jitter}$)。

抖动直接影响时序系统的性能。图 10.76 显示了一个名义时钟周期以及周期的变化。

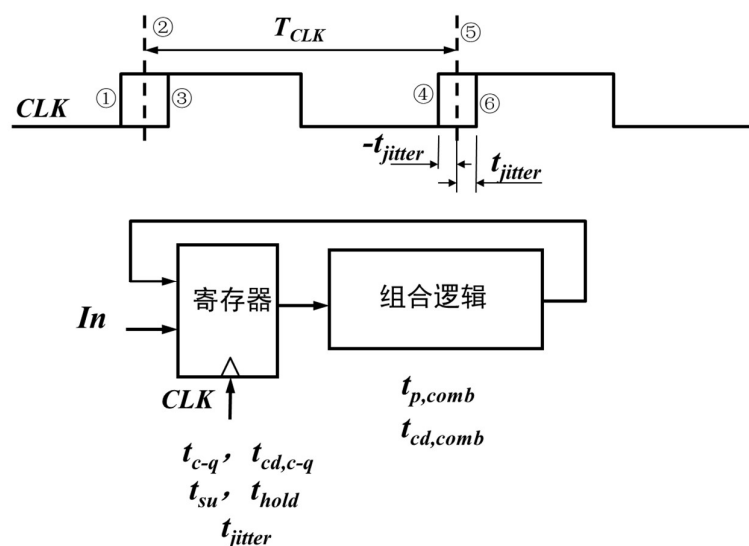


图 10.76 研究时钟抖动对性能影响的电路

在理想情况下，时钟周期起始于边沿②而结束于边沿⑤，其名义时钟周期为 T_{CLK} 。然而，最坏情况发生在当前时钟周期的上升沿因抖动而延后（边沿③），而下一个时钟周期的上升沿又因抖动而提前（边沿④）。结果，在最坏情况下可用来完成操作的总时间减少了 $2t_{jitter}$ ，这可由下式得出：

$$T_{CLK} - 2t_{jitter} \geq t_{c-q} + t_{logic} + t_{su} \quad \text{或} \quad T \geq t_{c-q} + t_{logic} + t_{su} - \delta + 2t_{jitter}$$

上式说明，抖动直接降低了一个时序电路的性能。如果性能是一个考虑因素，那么重要的是严格保证抖动在限定的范围之内。

偏差和抖动的共同影响

下面讨论通常的边沿触发时钟的偏差和抖动的共同影响。考虑下图 10.77 的时序电路。

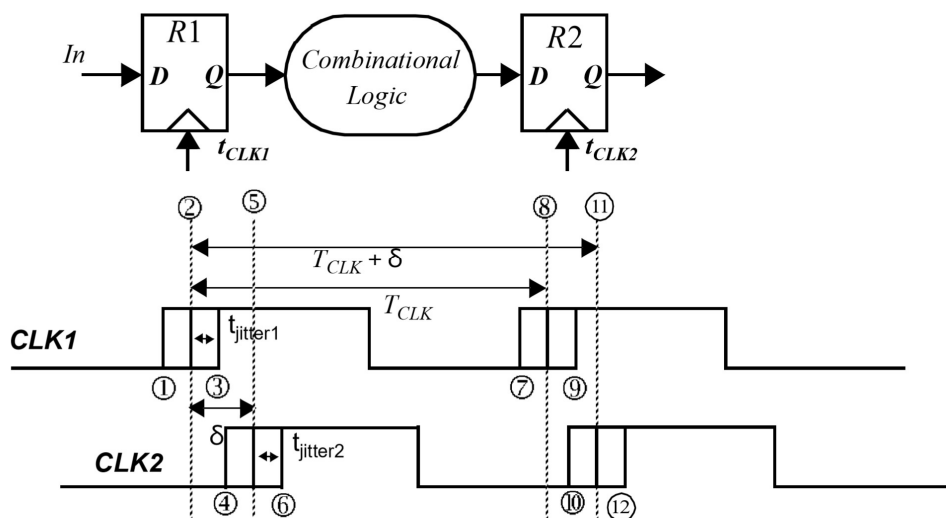


图 10.77 研究偏差与抖动对边沿触发系统影响的时序电路（正时钟偏差）

假设由于时钟分布的原因，两个寄存器上的时钟信号之间存在一个静态偏差 δ （假设 $\delta > 0$ ）。而且，两个时钟都有抖动 t_{jitter} 。为了确定对最小时钟周期的限制，需要先看一下完成所要求的计算可用的最小时间是多少。最坏情况发生在 $CLK1$ 的当前时钟周期的上升沿推迟出现（边沿③）而 $CLK2$ 的下一个时钟周期的上升沿提前出现（边沿⑩）时。由此得到了下列约束条件：

$$T_{CLK} + \delta - 2t_{jitter} \geq t_{c-q} + t_{logic} + t_{su}$$

或

$$T_{CLK} \geq t_{c-q} + t_{logic} + t_{su} - \delta + 2t_{jitter}$$

这一公式表明，正偏差（ $\delta > 0$ ）可以带来性能上的好处，而抖动总是对最小时钟周期有负面影响。

为了推导最小延时约束的公式，考虑 $CLK1$ 周期的上升沿提前到达（边沿①），而

CLK2 当前周期的上升沿推迟到达（边沿⑥）的情形。边沿①和边沿⑥之间的间隔时间应当小于经过整个电路的最小延时。由此得到：

$$\delta + t_{\text{hold}} + 2t_{\text{jitter}} < t_{\text{c-q,cd}} + t_{\text{logic,cd}}$$

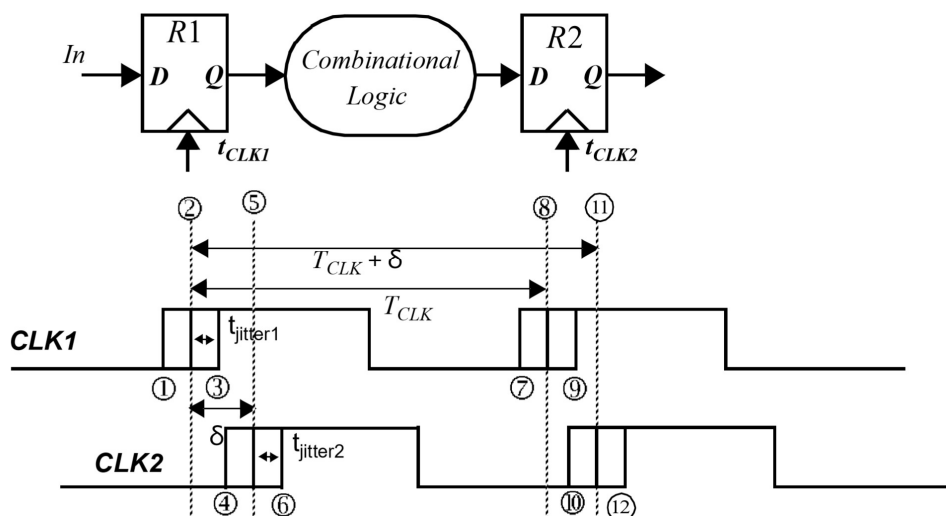
或

$$\delta < t_{c-q,cd} + t_{logic,cd} - t_{hold} - 2t_{jitter}$$

这一关系表明两个信号的抖动降低了可接受的时钟偏差值。

(不过需要注意:这一分析完全是针对最坏情形,它假设在源端与终端节点上的抖动值是独立的统计变量。实际上,涉及到维持时间分析的时钟边沿是由同一个时钟边沿推导出来的,因此它们在统计学上相关。当考虑这一相关性时会显著降低对时序的约束条件。)

下面考虑负偏差 ($\delta < 0$) 的情况, 如下图 10.78 所示。假设 $|\delta| > t_{\text{jitter}}$ 。可以证明最坏情形下的时序与前面所分析的情况完全一致, 只是 δ 为负值。因此负偏差会降低性能。

图 10.78 具有负时钟偏差 (δ) 的时序电路 (假设偏差大于抖动)

10.7.4 偏差和抖动的来源*

一个理想的时钟定义为同时触发芯片上各种存储元件的周期性信号。然而由于制造工艺和环境变化的不同，时钟并不那么理想。为了说明偏差和抖动的来源，考虑一个简化的典型时钟产生和分布网络，如图 10.79 所示。一个高频时钟可以由片外提供，也可以在片上产生。时钟从中心点出发，利用多条匹配路径向低层的时序元件分布，图中显示了两条路径。时钟路径包括导线以及相关的分布缓冲器，用以驱动互连线和负载。实现时钟分布时应当了解的关键点是经过时钟分布路径的绝对延时并不重要，重要的是它们到达每一条路径末端寄存器处的相对时间。只要所有的时钟同时到达芯片上所有的寄存器，那么时钟信号经过多个周期从中心分配点到达低层的寄存器是完全可以接受的。

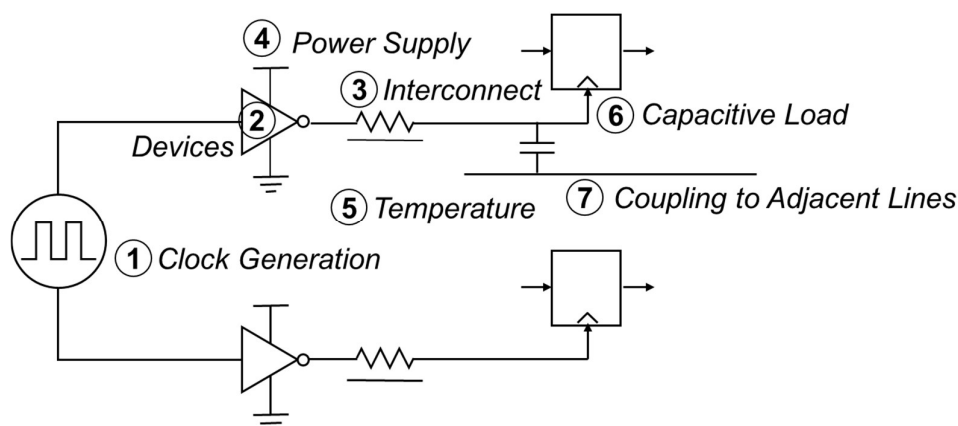


图 10.79 同步时钟分布中的偏差和抖动的来源

有许多原因使这两条并行路径不能有完全相同的延时。时钟不确定性的来源可以按几种方式来分类。首先，出错可以分为两类：系统错误和随机错误。系统错误名义上在不同的芯片之间是完全一样的并且是可以预见的（如每条时钟路径上的总负载电容不同）。从理论上讲，只要有足够好的模型和模拟工具，这类错误可以在设计阶段进行模拟并予以纠正。简言之，系统错误可以通过测试一组芯片发现，然后通过调整设计来进行弥补。而随机错误是由于制造中的变化产生的，所以很难模拟和消除（例如掺杂变化引起阈值变化）。

时钟不一致（未对准）也可以分为静态的和时变的。例如，一个芯片上的温度梯度会在毫秒大小的时间上发生变化。虽然一个经一次校准的时钟网络易受由温度梯度变化引起的时变失配（时钟不一致）的影响。但热的变化对于一个带宽为几兆赫兹的反馈电路来说却还是显得相当稳定。另一个例子就是电源噪声。时钟网络是芯片上最大的信号网络，因此时钟驱动器的同时翻转会在电源上引起噪声。但这一高速的影响并不会引起时变失配，因为它对每个时钟周期都相同，并且以相同的方式影响每个时钟上升沿。当然这一电源上的毛刺（*glitch*）如果在芯片各处不相同则仍然会引起静态失配。下面分别讨论图 10.79 中所示的偏差和抖动的各种来源。

时钟信号的产生（1）

时钟信号的产生本身就会引起抖动。一个典型的片上时钟发生器以一个低频参照时钟信号为输入产生一个处理器所需要的高频全局参照信号。这样一个发生器的核心是一个电压控制振荡器（*voltage-controlled oscillator, VCO*）。这是一个模拟电路，它对器件的本征噪声和电源电压的变化很敏感。它的主要问题是来自周围数字电路的噪声通过衬底的耦合。这在现代制造工艺中更是一个问题，因为这些工艺采用在重掺杂衬底上的轻掺杂外延层（以防止闩锁），这会使衬底噪声在芯片上传递一段很长的距离。这些噪声源会引起时钟信号的暂时改变，且不过滤地通过时钟驱动器传播到触发器，从而引起周期至周期的时钟周期变化。

器件制造中的偏差（2）

分布在电路中的缓冲器是整个时钟分布网络的一部分。它们必须同时驱动寄存器负载以及全局和本地互连线。在多条时钟路径上缓冲器器件的匹配对于减小时序的不确定性非常关键。遗憾的是，由于工艺的偏差，使在不同路径上缓冲器的器件参数也不尽相同，从而引起静态时钟偏差。偏差有许多来源，例如氧化层的变化（影响到增益和阈值）、掺杂的变化以及横向尺寸（长和宽）的变化。掺杂的变化会影响结深和掺杂的分布形态，从而造成器件电气参数（如器件的阈值和寄生电容）的变化。

多晶硅的走向对器件参数也有一定影响。因此使整个芯片上时钟驱动器保持同一取向非常关键。多晶硅关键尺寸的变化影响特别大，因为它直接影响 MOS 晶体管的沟道长度，从而影响驱动电流和开关特性。时钟在空间上的变化包括圆片级（即圆片上不同芯片之间）的变化和芯片级（即同一芯片内不同位置）的变化。这些变化至少有一部分属于系统级的，因此可以进行模拟和弥补。然而随机变化最终限制了时钟的匹配（一致性）以及可以达到的最小时钟偏差。

互连偏差 (3)

互连线垂直（厚度）和横向（宽度）尺度的偏差造成芯片上互连线电容和电阻的不同。由于这些差别是静态的，所以会造成不同路径间的时钟偏差。互连线偏差的一个重要来源是层间电介质（Inter-layer Dielectric, ILD）厚度的不均。例如，在形成铝互连线时二氧化硅层夹在互连图形的金属层之间，氧化物淀积在已形成图形的金属层上时，一般都会留下一些台阶高度或表面形貌。化学机械抛光（chemical-mechanical-polishing, CMP）用来“平整”表面并除去淀积和刻蚀所造成的形貌，虽然在特定范围内（即在一条单独的金属线上）CMP 能够达到很高的平整度，但从整个芯片范围上来讲平整度就很有有限了，这主要是由于抛光速度有所不同，而抛光速度又与电路版图的密度以及图形的影响有关。如图 10.80 显示了这一影响在空间密度较低的区域抛光速度较高，因而使电介质的厚度较小，电容较大。

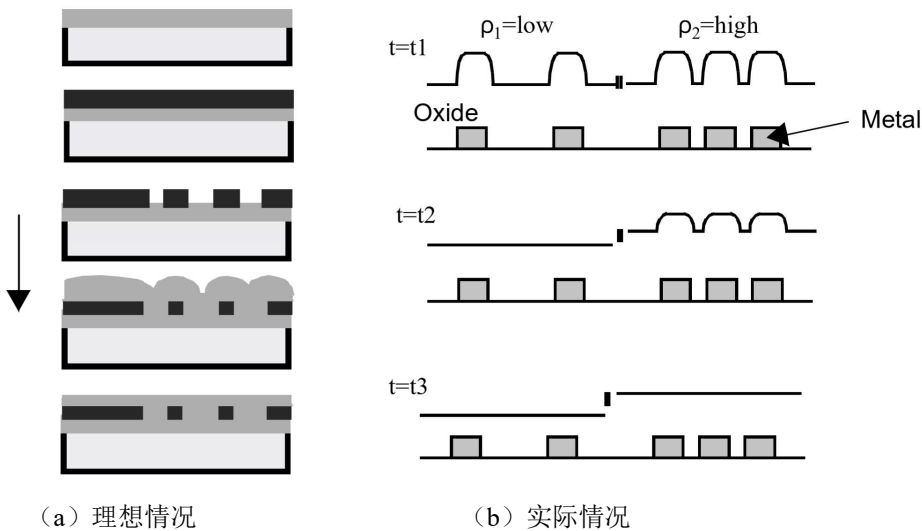


图 10.80 层间电介质(ILD)厚度因空间密度而改变

确定并控制这些变化对半导体工艺的发展和制造至关重要。在开发根据空间密度估计 ILD 厚度变化的解析模型方面已经有了显著的进展。由于这部分经常可以根据版图来预测,所以实际上有可能在设计阶段就对这一系统性的偏差部分予以纠正(例如通过适当增加延时或通过加入“虚设填料”而使密度一致)。

互连线的其他变化包括导线宽度和线间距离的变化,这与光刻和刻蚀过程有关。在多层金属结构的较低金属层上,光刻的影响比较重要;而在较高的金属层上,依赖于线宽和版图的刻蚀影响起主要作用。线宽是一个关键参数,因为它直接影响导线的电阻,而线间距离则影响线间电容。近来的处理器采用铜互连线,由于 CMP 形成凹坑和磨损的作用,铜线的厚度也表现出与图形密切相关。

环境变化(4 和 5)

环境变化可能是引起时钟偏差和抖动的最重要因素,环境变化的两个主要来源是温度和电源。芯片上的温度梯度起因于芯片上各处的功耗不同,随着时钟门控选通(clock gating)的出现温度差异已成为一个重要问题,因为时钟门控选通使芯片上的某些部分不工作,而其余部分则满负荷工作。近年来时钟门控选通作为使不工作模块功耗最小的一种方法已变得越来越普遍,关断部分芯片时钟会导致温度有很大差别。由于器件参数(如阈值和迁移率)与温度密切相关,所以一个时钟分布网络中不同路径上的缓冲器延时可以有很大的不同。更重要的是,这部分差别是时变的,因为温度随电路的逻辑活动而改变。因此仅模拟最差温度情形下的时钟网络是不够的,温度差别最大的情况也必须进行模拟。一个有趣的问题是温度差异是否对时钟偏差或抖动起作用。显然,温度差异随时间而变化,但这些变化很慢(一般温度变化的时间常数在毫秒的范围内)。因此通常把温度差异看做是引起时钟偏差的原因,并且把它作为最坏情况的条件来使用。幸运的是,利用反馈电路可以校准温度影响并对它进行补偿。

另一方面,供电变化则是时钟分布网络中抖动的主要原因。经过缓冲器的延时与电源电压密切相关,因为它直接影响晶体管的驱动能力。与温度一样,电源电压对开关的活动性也非常敏感,因此不同路径上缓冲器的延时会有很大的不同。电源电压的变化可以分为慢的(或称静态的)变化和高频率的变化。静态电源电压变化可以来自从不同模块汲取固定电流,而高频率的变化则是由于开关活动的起伏引起电源网络上瞬时的 IR 压降造成的。电感对电源电压的影响也是一个重要因素,因为它会引起电压的波动。同样,时钟门控选通使这一问题变得更加严重,因为逻辑在闲置与工作状态间的切换会造成电源供电电流的重大变化。由于供电电压可以快速变化,使时钟信号周期也在各个不同的周期上变化,从而造成了抖动。两个不同时钟点的抖动有可能相关也可能不相关,这取决于供电网络的结构和切换方式的分布形态。遗憾的是,高频率的供电变化即便采用反馈技术也很难补偿。因此,电源噪声基本上限制了时钟网络的性能。为了减少供电变化,高性能的设计都在主要时钟驱动器的周围加上去耦电容。

电容耦合(6 和 7)

电容负载的变化也会影响到时序的不确定性。电容负载变化有两个主要来源:时钟

线与相邻信号线之间的耦合，以及栅电容的变化。时钟网络同时包括互连线电容和锁存器及寄存器的栅电容。时钟线与相邻信号之间的任何耦合都会引起时序的不确定性。由于相邻信号可以在任意时间向任意方向变化，所以与时钟网络的确切耦合在不同的时钟周期是不固定的，因而造成了抖动。时钟不确定性的另一个主要来源是所连时序元件栅电容的变化。负载电容是高度非线性的并取决于所加的电压。对于许多锁存器和寄存器来说，时钟负载与锁存器/寄存器的当前状态（即存储于电路内部节点上的值）以及下一个状态有关。这就造成了通过时钟缓冲器的延时在不同的周期不一样，这也会造成抖动。下面通过一个例子来说明这种原因造成的抖动。

例 10.2 与数据有关的抖动

考虑下图 10.81 的电路，图中一个最小尺寸的局部时钟缓冲器驱动一个寄存器（实际上每个时钟缓冲器通常要驱动四个寄存器，但这里只显示了一个）。模拟显示了 CKb ，即第一个反相器输出的四种可能的变化（ $0 \rightarrow 0$ ， $0 \rightarrow 1$ ， $1 \rightarrow 0$ ， $1 \rightarrow 1$ ）。它说明了因电容与数据有关而引起的时钟抖动。一般来说，解决这一问题的惟一方法是采用负载变化与数据关系不大的寄存器。

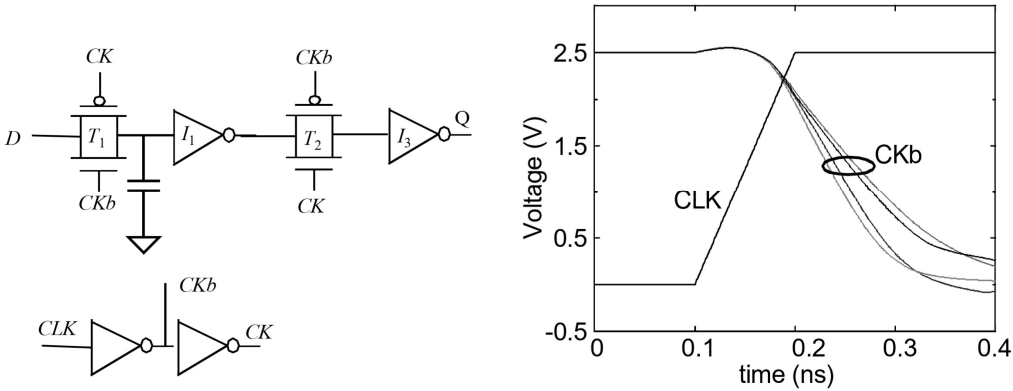


图 10.81 传输门寄存器中与数据相关时钟负载对时钟抖动的的影响